Numerisk Lineær Algebra

Andrew Swann

Institut for Matematik
Aarhus Universitet

Forår 2024

Sidst ændret: 14. maj 2024.
Versionskode: fbc42ca.

Indhold

Indhold iii

Figurer ix

Tabeller xii

Forord xiii

Notesættets opsætning xv

1 Indledning og første betragtninger 1
1.1 Tal og fejl 2
1.2 Flydende-komma repræsentation 5
1.3 Fejl i flydende-komma tal 9

2 Plangeometri: vektorer og matricer 13
2.1 Vektorer, norm og vinkelbestemmelse 13
2.2 Indre produkt og matrixtransformationer 15
2.3 Første tegninger 22

3 Matricer og vektorer 27
3.1 Matricer 27
3.2 Matricer i python 28
3.3 Heatmap plot 30
3.4 Multiplikation med en skalar og matrixsum 33
3.5 Transponering 36
3.6 Vektorer 36

4 Matrix multiplikation **39**
4.1 Række-søjleprodukt 39
4.2 Matrixprodukt 42
4.3 Regneregler for matrixprodukt 47
4.4 Identitetsmatricer 49
4.5 Ydre produkt 50
4.6 Tegning i tre dimensioner 54

5 Numerisk håndtering af matricer **57**
5.1 Omkostninger ved matrixberegning 57
5.2 Pythons for-løkke 59

6 Lineære ligningssystemer **65**
6.1 Elementære rækkeoperationer 65
6.2 Rækkeoperationer i python 68
6.3 Echelonform 70
6.4 Løsning via rækkeoperationer 72

7 Matrixinvers **77**
7.1 Egenskaber af den inverse matrix 77
7.2 Eksistens af den inverse 81
7.3 Beregning af den inverse 83

8 Ortogonalitet og projektioner **85**
8.1 Standard indre produkt 85
8.2 Vinkel mellem vektorer 88
8.3 Projektion på en linje 93
8.4 Ortogonalitet 96

9 Ortogonale matricer **107**
9.1 Definition og første eksempler 107
9.2 Egenskaber 108
9.3 Ortonormale vektorer og ortogonale matricer 110
9.4 Householdermatrix 112
9.5 Udvidelse til ortonormal basis 118
9.6 Plan geometri: trigonometriske identiteter 119

10 Singulærværdidekomponering **125**
10.1 Singulærværdier i dimension 2 126
10.2 SVD generelt 129
10.3 Eksempler på SVD 130
10.3.1 Punkter i planen 130
10.3.2 Billede med støj 133
10.3.3 Komprimering af et billede 135
10.4 Eksistens af SVD 139

11 Konditionstal **143**
11.1 Konditionstal for differentiable funktioner 143
11.2 Matrixnorm 146
11.3 Konditionstal og lineære ligningssystemer 149

12 Generelle vektorrum **155**
12.1 Vektorrum 155
12.2 Andre skalarer 157
12.3 Underrum 162

13 Indre produkter **167**
13.1 Definitioner og eksempler 167
13.2 Ortogonale samlinger og tilnærmelse 172
13.3 Numerisk integration 176

14 Ortogonale samlinger: klassisk Gram-Schmidt **183**
14.1 Den klassiske Gram-Schmidt proces 183
14.2 Klassisk Gram-Schmidt i $\mathbb{R}^n$ 190

15 Forbedring af Gram-Schmidt processen **193**
15.1 Numeriske problemer for klassisk Gram-Schmidt 193
15.2 Den forbedrede Gram-Schmidt proces 196
15.3 Klassisk kontra forbedret Gram-Schmidt 198
15.4 Flops 203

16 Mindste kvadraters metode **205**
16.1 Problemformulering 205
16.2 Løsning via QR-dekomponering 206
16.3 Løsning via singulærværdidekomponering 207
16.4 Polynomier 209

16.5 Løsning via normalligninger 217

17 Mindste kvadrater: konditionstal og Householder 219
17.1 Konditionstal for mindste kvadraters problemer 219
17.2 Et eksempel 221
17.3 Householdertriangulering 225
17.4 Householdermetoden 233

18 Lineære afbildninger og matricer 237
18.1 Lineære afbildninger 237
18.2 Standardmatricen for en lineær transformation 241
18.3 Kombinationer af lineære afbildninger 242
18.4 Kerne og billedmængde 243

19 Koordinater 245
19.1 Basis 247
19.2 Koordinater 249
19.3 Koordinatskift 252
19.4 Baser for nul- og søjlerum af en matrix 254
19.5 Dimension 255

20 Lineære transformationer og koordinatskift 257
20.1 Generelle matrixrepræsentationer 257
20.2 Basisskift 260
20.3 Afbildning på ét vektorrum 261

21 Egenværdier og egenvektorer 267
21.1 Et første eksempel 267
21.2 Definitioner og første regnemetoder 268
21.3 Andre eksempler 272
21.4 Determinanter 275
21.5 Et andet eksempel 277
21.6 Gentagende egenværdier 279

22 Differentialligninger 283
22.1 Førsteordenssystemer 283
22.2 Komplekse egenværdier 290
22.3 Højere ordens lineære differentialligninger 295

23 Matrixfaktorisering fra egenværdier **299**
23.1 Eksistens af egenværdier 299
23.2 Determinant og egenværdier 300
23.3 Unitære matricer og Schurdekomponeringen 301
23.4 Spektralsætningen 304
23.5 Egenskaber af determinanter 309

24 Potensmetoden og inverspotensmetoden for egenværdier **317**
24.1 Potensmetoden 317
24.2 Page rank 324
24.3 Rayleighs kvotient 329
24.4 Potensmetoden for den største singulærværdi 329
24.5 Inverspotensmetoden 333

25 Hessenberg- og tridiagonalform **339**
25.1 Reduktion til Hessenbergform 342
25.2 Reduktion til tridiagonalform 346
25.3 Reduktion til bidiagonalform 349

26 Egendekomponering via QR-metoden **355**
26.1 QR-metoden: første version 355
26.2 Ortogonal iteration 358
26.3 Ækvivalens af metoderne 362
26.4 QR-dekomponering af tridiagonale matricer 364
26.5 Praktisk QR-metode 366

27 Principalkomponent analyse **373**
27.1 Principalkomponent analyse 373
27.2 Singulærværdidekomponering og symmetriske matricer 379

28 LU-dekomponering **385**
28.1 LU-dekomponering og Gausseliminering 385

A Det græske alfabet **399**

B Python **401**

Bibliografi **403**

Python indeks **405**

Indeks **409**

Figurer

2.1 Afstand og vinkel i planen 13
2.2 Vinklen mellem to vektorer 16
2.3 Skalering 18
2.4 Rotation 18
2.5 Spejling 20
2.6 Projektion 21
2.7 Skævvridning 21
2.8 En første tegning 23
2.9 Tegning flyttet via en matrix 24

3.1 En første heatmap plot 31
3.2 Heatmap med colorbar 32
3.3 Heatmap med justeret farveskala 32
3.4 Heatmap for sum af to matricer 35
3.5 Vektorsum af $u = (2, 1)$ og $v = (1, 3)$ 37

4.1 Linjen $x + 3y = 2$ 41
4.2 Elektrisk kredsløb 46
4.3 Første eksempler på et ydre produkt 51
4.4 Ydre produkter med vektorer fra `linspace` 52
4.5 Ydre produkt af tilfældige vektorer 53
4.6 Projektion af rotationer i tre dimensioner 56

5.1 Oprindelig figur og dens rotationer 63

6.1 Rundkørsel 72
6.2 To linjer i planen, som skærer 75
6.3 Tre linjer i planen, som har intet fællespunkt 75
6.4 To parallelle linjer i planen har intet fællespunkt 76

8.1 Projektion på en linje 94
8.2 Ortogonal samling af polynomier 103
8.3 Numerisk approksimation af eksponentialfunktionen 104
8.4 Eksponentialfunktionen og dens Taylor udvikling 105

9.1 Effekten af en Householdermatrix 113
9.2 Spejling til $-\varepsilon e_0$ 115
9.3 Cosinus og sinus for en enhedsvektor i planen 120
9.4 Enhedsvektoren u drejet igennem vinkel φ 121
9.5 Spejling i x-aksen 124

10.1 Cirkel efter matrixmultiplikation 126
10.2 SVD af punktsamling 133
10.3 Cifret 0, plus støj 134
10.4 SVD af billede med støj 136
10.5 Uppsala domkirke 137
10.6 Singulærværdier for billedet af domkirken 138
10.7 SVD komprimering 139

11.1 Trekantsuligheden 146

13.1 Fourier cosinus tilnærmelse for $1 - x$ 175
13.2 Integral 177
13.3 Rektanglet fra den venstre funktionsværdi 177
13.4 Venstre- og højre-tilnærmelse til integral 178
13.5 Trapez-tilnærmelse til integral 179

14.1 Legendre og Chebychev polynomier 189

15.1 Klassisk kontra forbedret Gram-Schmidt 202

16.1 Andengradspolynomium igennem 3 datapunkter 211
16.2 Lineær tilnærmelse af data 214
16.3 Højere grad 215

17.1 Mindste kvadraters problemstilling 220

19.1 Standardkoordinater 245
19.2 Drejet ortogonal koordinatsystem 246
19.3 Generel lineært koordinatsystem 246

21.1 Tredjegradspolynomium 278

22.1 Beholdere forbudne med rør 286
22.2 Saltmængden i beholder A mod beholder B 289
22.3 Elektrisk kredsløb 292
22.4 Løsningen til kredsløb 295
22.5 System med klods og fjedre 296

24.1 Værdier af λ i løbet af potensmetoden. 319
24.2 Numeriske værdier af komponenterne af w i løbet af potensmetoden. 321
24.3 En samling websider med links 325

26.1 En tridiagonal matrix 365
26.2 Q og R faktorer for en tridiagonal matrix 366
26.3 QR-trin på en tridiagonal matrix 367

27.1 Plot af data og den centrede version 377
27.2 Principale komponenter, uden normalisering 378
27.3 Principale komponenter, normaliserede 380

Tabeller

1.1 Python regneoperationer på tal 3
1.2 Fejl i beregninger til 3 decimaler 5

5.1 Omkostninger ved vektor- og matrixberegninger 59

6.1 Rækkeoperationer i python 68

17.1 Løsningsmetoder for mindste kvadraters problemer 234

27.1 Vægt og højde for nogle personer 373

Forord

Disse noter er skrevet for kurset Numerisk Lineær Algebra ved Aarhus Universitet, som kørte første gang i foråret af 2020. Denne version er redigeret ved kurset i 2024.

Lineær algebra et smukt matematisk emne og et stærkt værktøj, der anvendes i mange fag i forbindelse med matematiske modeller, og som er grundlæggende for mange teorier i naturvidenskabelige fag. Et område, hvor der er voksende fokus, er datavidenskab og analyse af store datamængde. I disse sammenhæng og ingeniørfag er det nødvendigt at behandle data på en computer og udfører operationer via et programmeringssprog.

Kurset har til formål både at indføre de studerende i sådanne beregningsmetode og at forklare og begrunde den relevante teori. Programmeringssproget python anvendes igennem kurset, som eksempelsprog. Tidligere programmeringserfaring er ikke påkrævet, og den matematiske baggrund er begrænset til viden opnået på gymnasiet og i matematikkurser der ligger i det første semester af universitetsuddannelser.

Der er mange bøger, der giver en god indledning til lineær algebra og dens anvendelser. En, der kan anbefales i forbindelse med dette kursus, er Leon (2015); andre bøger med lignende titler er lige så relevant. Men det er en vigtig pointe at de matematiske metoder der præsenteres i standard kurser og tekster om lineær algebra er ikke velegnede til numerisk behandling af tal på en computer. Derfor lægges der vægt i dette kursus på at beskrive metoder, der har gode egenskaber i forhold til styring af fejl i beregninger. Der er gode kilder for sådanne materiale, jeg vil især nævne bogen Trefethen og Bau (1997), men disse tekster forventer at læseren har allerede gennemført et matematisk kursus i lineær algebra. Sådan et krav stilles ikke til læseren af disse noter. Vi indfører teorien samtidig med de varianter der er mest relevant i en numerisk kontekst, og lægger vægt på delene af teorien, som er bedst egenet til disse metoder og deres anvendelse.

I løse form følger disse noter en kombination af de to ovennævnte kilder. Efter gennemførsel af kurset, kan bogen af Trefethen og Bau (1997) godt anbefales til viderelæsning. For dem der vil læse endnu dybere om den numeriske side af faget er der et stor og uomgåelig værk, nemlig Golub og Van Loan (2013), som har også været en god inspirationskilde for mig.

Dette notesæt er skrevet i LaTeX ved hjælp af PythonTeX, Poore (2015).

Jeg takker alle de studerende og instruktorer, som er kommet med kommentarer og rettelser til notesættet, og høre gerne om ydeligere forslag til forbedringer og emne valg.

Andrew Swann

Notesættets opsætning

Hovedresultater er »sætning«'er og »proposition«'er, med sætninger, som resultater der har lidt mere tyngde en propositioner.

Sætning 0.1. *En hovedsætning.*

Disse ofte begrundes med et »bevis«, hvis afslutning markeres med en firkant.

Bevis. Sætning gælder, da... □

Mindre hjælperesulteter betegnes som »lemma«'er.

Der gives flere eksempler, som vises som nedenfor, og hvis afslutning markeres med en trekant.

Eksempel 0.2. Et eksempel er... △

Teksten indeholder en del python code, enten i stilen fra en terminal window

```
>>> x = 3
>>> print('Python kode', x)
Python kode 3
```

eller med koden efterfuldt af dens output

```
x = 3
print('Python kode', x)
```

```
Python kode 3
```

Flere begreber specificeres i »definition«'er med begrebet fremhævet

Definition 0.3. En *norm* er…

For at navigere notesættet kan man bruge indholdsfortegnelsen, lige i begyndelsen af sættet, og/eller en af de to stikords registrer i slutningen af sættet: indeks og Python indeks. Flere af dokumentets elementer også fungerer som hyperlinks til andre steder i notesættet.

Kapitel 1

Indledning og første betragtninger

Numerisk lineær algebra er studiet af metoder til behandling af beregningsproblemer indenfor matricer og vektorer. Det er et hjørnesten i computerberegninger baseret på matematiske modeller for problemstillinger fra alle videnskaber. Faget har udviklet sig hastigt og er under stadig forandring. Grundlaget er teorien for vektorrum og lineære transformationer. Det praktiske arbejde med tal betyder dog at disse teorier skal vinkles og forfines for at behandle problemer baserede på eksperimentelle eller observationsbaserede data og for at udføre beregninger på en computer. Moderne anvendelser kræver ofte store mængder data, og det kan medføre at en direkte tilgang til implementering er ikke tilstrækkelig effektiv.

Overraskende mange problemstillinger kan udtrykkes via matrixligninger, og ofte er den bedste metode numerisk at finde og anvende gode faktoriseringer af disse matricer. Af disse problemstilling falder de fleste indfor variationer over en af de følgende to:

(a) Givet en matrix A og en vektor b bestem alle vektorer x, som opfylder

$$Ax = b.$$

(b) Givet en kvadratisk matrix A bestem alle vektorer v og alle tal λ således at

$$Av = \lambda v.$$

For databehandling og numerisk anvendelser har det sidste problem en variant, der er særligt vigtig:

(c) Givet en matrix A bestem alle vektorer u og v og alle tal σ således at

$$Av = \sigma u \quad \text{og} \quad A^T u = \sigma v.$$

I dette første kapitel, kikker vi på nogle af de grundlæggende betragtninger omkring tal og deres brug på en computer.

1.1 Tal og fejl

Tal i vores data er yderst sjælden præcis. Måler man et lufttryk til at være 1005 hPa er der usikkerhed omkring det sidste tal; har man et computerbilled er den røde farve i et given punkt typisk gemt som et heltal mellem 0 og 255, og så kan ikke afspejle virkeligheden eksakt. Det er kun når vi tæller objekter 1, 2, 3, … at vi får et eksakt tal at arbejde med; men også her hvis der er mange objekter eller objekterne er svære at tælle, som f.eks. ulve i Jylland, er tallet ikke nødvendigvis korrekt.

Selvom man har et eksakt tal opstår der også praktiske problemer når en computer skal opbevare og behandle den. Medmindre vi arbejder symbolsk, er tal typisk opbevaret i form der svarer til visse rationelle tal a/b. Det betyder at tal som $\sqrt{2} \approx 1{,}414\,21$, $e \approx 2{,}718\,28$, $\pi \approx 3{,}141\,59$ har ikke eksakte repræsentationer. Desuden er nævneren b typisk en potens af 2 (binærbrøk). Som konsekvens kan hverken $1/3 \approx 0{,}333\,33$ eller $1/7 \approx 0{,}142\,86$ behandles præcist.

Dette forværres af at computeren også giver os rige mulighed for at lave beregninger med mange tal. En computer vil sagtens kunne beregne en sum for et datasæt bestående af en million tal, men så risikerer man også at fejleffekterne bidrager en million gange.

Selv for heltal er der problemer. Computerhukommelse bliver billigere og billigere, men det er alligevel dyrt at arbejde med større heltal. Et heltal med mange cifre beslaglægger en del hukommelsesplads, og beregninger bliver langsommere.

I dette kursus har vi programmeringssproget python som eksempelsprog. Hvad et python program kan behandle er ikke fuldstændigt fastlagt og kan variere, afhængig af hvilke version og implementering der er tale om, samt egenskaber af den underliggende computer den køres på.

I python skal man skelner mellem heltal og decimalbrøk: skriver man `143` har man et heltal, ønskes derimod en decimalbrøk skrives `143.0` i stedet. Når vi bruger decimalbrøk, viser nogle af de ovennævnte problemer sig ret hurtig:

python	matematik	navn
`x + y`	$x+y$	sum
`x - y`	$x-y$	differens
`x * y`	xy	produkt
`x / y`	x/y	kvotient
`x ** y`	x^y	potens

Tabel 1.1: Python regneoperationer på tal.

```
>>> 1.1 + 1.2
2.3
>>> 1.9 + 1.2
3.0999999999999996
>>> 1.1 - 1.2
-0.09999999999999987
>>> 2.1 * 2.7
5.670000000000001
>>> 2.1 / 2.0
1.05
>>> 2.1 / 2.5
0.8400000000000001
```

hvor vi har brugt pythons regneoperationer som i tabel 1.1. For reelle tal ville alle svar ovenfor haft højst 2 decimaler. Desuden har vi

```
>>> 1.1 + (1.3 + 1.5)
3.9
>>> (1.1 + 1.3) + 1.5
3.9000000000000004
```

selvom begge sum er det samme for reelle tal. Når vi betragter regneoperationer, har manglende præcision nogle konsekvenser. Som eksempel lad os arbejder med decimalbrøk med kun 3 cifre efter kommaet. Så vil $a' = 10{,}153$ blive nødt til at repræsentere et hvilken som helst tal a mellem $a_{\text{min}} = 10{,}1525$ og $a_{\text{max}} = 10{,}1535$.

Hvis et reelt tal x er repræsenteret af tallet x' vil vi kalde

$$x' - x$$

for *fejlen*. Vi vil også kalde

$$\delta = \frac{x' - x}{x}$$

for den *relative fejl*, så længe x er ikke 0. Den opfylder

$$x' = x(1 + \delta). \tag{1.1}$$

Ofte udtrykker vi den relative fejl i procent, ved at gange δ med 100.

For a mellem $a_{\min}$ og $a_{\max}$ ovenfor har vi så, at fejlen $a' - a$ ligger mellem $a' - a_{\max} = -0{,}0005$ og $a' - a_{\min} = 0{,}0005$, samt at den relative fejl ligger mellem

$$\frac{a' - a_{\max}}{a_{\max}} = -0{,}000\,049\,244\,1 \approx -0{,}004\,92\,\%$$

og

$$\frac{a' - a_{\min}}{a_{\min}} = 0{,}000\,049\,249\,0 \approx 0{,}004\,92\,\%.$$

Betragt nu $b' = 9{,}986$, repræsenterende et reelt tal b mellem $b_{\min} = 9{,}9855$ og $b_{\max} = 9{,}9865$. Fejlen for b' er den samme som for a', mens den relative fejl er $\pm 0{,}005\,01\,\%$.

Summen forventes repræsenteret af $a' + b' = 20{,}139$, men $a + b$ ligger mellem $a_{\min} + b_{\min} = 20{,}138$ og $a_{\max} + b_{\max} = 20{,}140$. Fejlen er $\pm 0{,}001$, mens den relative fejl er mellem $\pm 0{,}004\,97\,\%$. For den plus $+$ operation, ser vi at fejlen er dobbelt så stort som for a, mens i dette eksempel er den relative fejl er næsten uændret. For de andre regneoperationer difference $-$, produkt $\times$ og kvotient $/$, kan fejlen regnes på tilsvarende måde. Resultaterne gives i tabel 1.2. Her ser vi en del udsving i fejl og relativ fejl. Mest bekymrende er den store relativ fejl for difference operationen, som afspejler færre betydende cifre i resultatet.

Generelle regneregler for disse type uligheder er givet ved:

Proposition 1.1. *For reelle tal a og b med $a_{\min} \leqslant a \leqslant a_{\max}$ og $b_{\min} \leqslant b \leqslant b_{max}$ gælder*

(a) $a_{\min} + b_{\min} \leqslant a + b \leqslant a_{\max} + b_{\max}$,

(b) $a_{\min} - b_{\max} \leqslant a - b \leqslant a_{\max} - b_{\min}$.

Hvis yderligere $a_{\min}, b_{\min} > 0$, så har vi

(c) $a_{\min} b_{\min} \leqslant ab \leqslant a_{\max} b_{\max}$,

(d) $a_{\min}/b_{\max} \leqslant a/b \leqslant a_{\max}/b_{\min}$. □

	beregnet	min værdi	max værdi	fejl	relativ fejl %
+	20,139	20,138 000	20,140 000	0,0010	0,004 97
−	0,167	0,166 000	0,168 000	0,0010	0,602 41
×	101,388	101,377 789	101,397 928	0,0102	0,010 07
/	1,017	1,016 622	1,016 824	0,0002	0,017 27

Tabel 1.2: Fejl i beregninger til 3 decimaler ved brug af repræsentanter $a' = 10{,}153$ og $b' = 9{,}986$.

1.2 Flydende-komma repræsentation

Et par eksempler på tal i flydende-komma form, eller tal i videnskabelig notation, er

$$1{,}639\,01 \cdot 10^2$$
$$-2{,}704\,62 \cdot 10^{-3}$$
$$5{,}672\,91 \cdot 10^{100}$$

Disse skrives i python som

```
>>> 1.63901e2
163.901
>>> -2.70462e-3
-0.00270462
>>> 5.67291e100
5.67291e+100
```

og som set vil tal, der er ikke for store, bliver skrevet ud som en almindelig decimalbrøk. Her benyttes grundtallet 10.

Generelt er en *flydende-komma repræsentation* med grundtal 10 et reelt tal x af formen

$$x = \pm m \cdot 10^k$$

hvor

$$m = \text{“}d_0{,}d_1 \ldots d''_{r-1} = d_0 + \frac{d_1}{10} + \cdots + \frac{d_{r-1}}{10^{r-1}}$$

er en decimalbrøk med r cifre, $d_i \in \{0, 1, 2, \ldots, 9\}$, $d_0 \neq 0$ når $x \neq 0$, og k er et heltal. Decimaltallet m kaldes *mantissen* og k kaldes *eksponenten*. Så tallet $-2{,}704\,62 \cdot 10^{-3}$ har fortegn -1, mantisse 2,704 62 og eksponent -3.

I python repræsenteres alle decimaltal internt i en tilsvarende flydende-komma form, men med grundtallet 2 i stedet for 10, dvs. en binærbrøk ganget med en potens som 2^{17}. Denne type kaldes for `float`:

```
>>> type(143)
<class 'int'>
>>> type(143.0)
<class 'float'>
>>> type(-2.70462e-3)
<class 'float'>
```

Der afsættes en fast mængde hukommelse til enhver float. Vi vil arbejde med NumPy pakken. På min maskine fortæller

```
>>> import numpy as np
>>> np.finfo(float).bits
64
```

at der afsættes 64 bit = 8 byte for hver `float`. Din computer råder sikkert over hukommelse med adskillelige gigabyte = 2^{30} byte = 1 073 741 824 byte. I hver gigabyte er der så plads til at opbevare $134\,217\,728 \approx 1{,}3 \cdot 10^8$ tal i `float` form.

Da der afsættes 64 bit til en `float`, er der højst

$$2^{64} = 18\,446\,744\,073\,709\,551\,616 \approx 1{,}8 \cdot 10^{19}$$

forskellige reelle tal der kan bruges som repræsentanter i `float` form. Dette står i kontrast til at der er uendelige mange reelle tal, og har nogle væsentlige konsekvenser

(a) Der er et største tal $\text{float}_{\text{max}}$ og et mindste tal $\text{float}_{\text{min}}$ af typen `float`:

```
>>> np.finfo(float).max
1.7976931348623157e+308
>>> np.finfo(float).min
-1.7976931348623157e+308
```

(b) Der er et mindste tal $\epsilon_{\text{machine}} > 0$, *machine epsilon* eller *nøjagtigheden*, af type `float` således at $1 + \epsilon_{\text{machine}}$ er igen en `float` og $1 + \epsilon_{\text{machine}} > 1$:

```
>>> np.finfo(float).eps
2.220446049250313e-16
```

(c) Der er et mindste tal $\text{float}_{\text{smallest}}$ af type `float` som er skarpt større end 0:

```
>>> np.nextafter(0.0, 1)
5e-324
>>> np.finfo(float).smallest_subnormal
5e-324
```

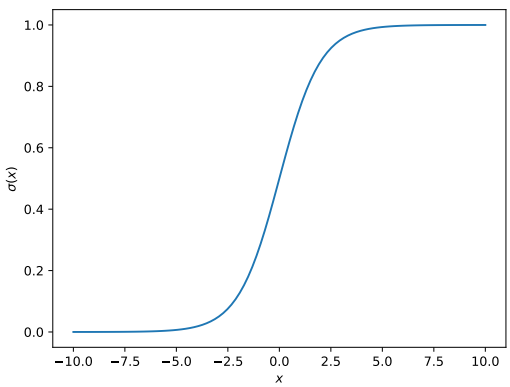
(d) Resultater af regneoperationer med tal af type `float` skal generelt afrundes for at give et svar der igen er af type `float`.

(e) Resultater der er for store kan ikke repræsenteres som `float`; dette kaldes *overflow*.

(f) Resultater der er for tæt på 0 riskærer at blive afrundet til `0.0`; dette kaldes *underflow*.

Eksempel 1.2. I machine learning er der ofte brug for sigmoidfunktionen

$$\sigma(x) = \frac{e^x}{1 + e^x},$$



som er en differentiabel funktion med værdier skarpt mellem 0 og 1. Beregning af $\sigma(x)$ indebærer udregning af eksponentialfunktionen e^x, som hurtigt antage meget store værdier for x positivt, og værdier meget tæt på 0 for x negativt. Eksponentialfunktionen er tilgængelig i `numpy` pakken som `np.exp`

```
>>> import numpy as np
>>> np.exp(1.75)
```

```
5.754602676005731
>>> np.exp(700.0)
1.0142320547350045e+304
>>> np.exp(-700.0)
9.85967654375977e-305
```

Prøver man en lidt større værdi, får man et svar der er ikke et `float` tal

```
>>> np.exp(750.0)
inf
```

som standard ledsaget af en advarsel

```
<stdin>:1: RuntimeWarning: overflow encountered in exp
```

Hvis vi forsørger at beregne $\sigma(750)$, får vi

```
>>> np.exp(750.0) / (1 + np.exp(750.0))
nan
```

```
<stdin>:1: RuntimeWarning: invalid value encountered in
↪  double_scalars
```

Men vi kan omskrive udtrykket for σ til

$$\sigma(x) = \frac{e^x}{1+e^x} = \frac{e^x}{1+e^x} \cdot \frac{e^{-x}}{e^{-x}} = \frac{1}{1+e^{-x}}$$

Nu vil beregning af $\sigma(750)$ bruger `np.exp(-750.0)`, som afrundes til 0.0, uden nogen advarsel, og beregning af sigmoidfunktionen giver værdien

```
>>> 1 / (1 + np.exp(-750.0))
1.0
```

som kan være mere brugbart. Der er en anden omskrivning

$$\sigma(x) = \frac{e^x}{1+e^x} = \frac{e^x}{1+e^x} \cdot \frac{e^{-x/2}}{e^{-x/2}} = \frac{e^{x/2}}{e^{x/2}+e^{-x/2}}$$

som undgår både overflow og underflow ved denne værdi

```
>>> np.exp(750.0/2) / (np.exp(750.0/2) + np.exp(-750.0/2))
1.0
```

men det hjælper ikke med udregning af $\sigma(1500)$. △

Bemærkning 1.3. Det er ikke nødvendigvis godt, at overflow eller underflow opstår ubemærket. De er tegn på at noget er gået galt. Python kan tvinges til at stoppe ved sådan en fejl i `float`-beregninger via

```
>>> import numpy as np
>>> save_err = np.seterr(all='raise')
```

Man kan derefter komme tilbage til standardinstillinger via

```
>>> np.seterr(**save_err)
{'divide': 'raise', 'over': 'raise', 'under': 'raise',
↪  'invalid': 'raise'}
```

Det der vises her som output er indstillinger *inden* `np.seterr` blev kørt. ◇

1.3 Fejl i flydende-komma tal

Moderne computer plejer at følge en standard (IEEE Computer Society 2019) fastlagt af the Institute of Electrical and Electronics Engineers (IEEE) for tal af type `float`. Ignorerer vi problemer med overflow og underflow, kan man generelt forvente af en ideal implementering af float at

(F1) for ethvert reel tal $x \in \mathbb{R}$ findes der et tal $\mathrm{float}(x)$ af type `float`, der repræsenterer x inden for en relativ fejl af højst $\epsilon_{\text{machine}}/2$, og

(F2) operationerne $+$, $-$, $\times$, $/$ og $\sqrt{\cdot}$ er implementeret på tal af type `float` således at svar er korrekt inden for en relative fejl af højst $\epsilon_{\text{machine}}/2$.

Punkt (F1) siger, at for alle reelle tal x har vi

$$\mathrm{float}(x) = x(1 + \delta_x)$$

hvor den relative fejl δ_x opfylder $|\delta_x| \leqslant \epsilon_{\text{machine}}/2$. Dette er det samme, som at sige

$$|\mathrm{float}(x) - x| \leqslant |x|\epsilon_{\text{machine}}/2.$$

For python `float`, vi kan forventer at mantissen afrundet til de første

```
>>> np.finfo(float).precision
15
```

cifre er korrekt.

Punkt (F2) siger tilsvarende, at for a og b af type `float` gælder

$$a +_{\text{float}} b = (a+b)(1+\delta_1), \quad a -_{\text{float}} b = (a-b)(1+\delta_2),$$
$$a *_{\text{float}} b = ab(1+\delta_3), \quad a /_{\text{float}} b = (a/b)(1+\delta_4),$$
$$\texttt{sqrt}(a) = \sqrt{a}\,(1+\delta_5),$$

med $|\delta_i| \leqslant \epsilon_{\text{machine}}/2$ for alle relative fejl δ_i. Disse fejl δ_i varierer med a og b, men er altid kontrolleret af det samme tal, $\epsilon_{\text{machine}}/2$.

På trods af $\epsilon_{\text{machine}}$ er ret lille er vi stadigvæk plaget af problemerne i afsnit 1.1, især med hensyn til differenceoperationen og tab af betydende cifre.

Eksempel 1.4. Betragt andengradsligningen

$$2x^2 + 98x + \frac{1}{4} = 0.$$

Vi husker at $ax^2 + bx + c = 0$ har rødder

$$q = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{og} \quad r = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

I vores tilfælde er $b = 98$ og kvadratroden af diskriminaten er $\sqrt{b^2 - 4ac} = \sqrt{9602.0} \approx 97{,}990$. Dette betyder at når vi regner q, hvor skal vi tage differencen af b og $\sqrt{b^2 - 4ac}$, kan vi miste flere betydende cifre. Selvom vi bruger `float` kan det give fejl i den beregnede værdi for q. Vi kan delvis se dette ved at sætte q tilbage i polynomiet:

```
>>> import numpy as np
>>> a = 2.0  #kommentar: sæt 'a' til værdien 2.0
>>> b = 98.0
>>> c = 1.0 / 4.0
>>> sqrt_discriminant = np.sqrt(b**2 - 4*a*c)
>>> q = (-b + sqrt_discriminant) / (2*a)
>>> q
```

```
-0.0025511532323037045
>>> a*q**2 + b*q + c
-1.3367085216486885e-13
```

Tilgengæld er udregning af r uden dette problem

```
>>> r = (-b - sqrt_discriminant) / (2*a)
>>> r
-48.997448846767696
>>> a*r**2 + b*r + c
0.0
```

Husker vi at q og r opfylder

$$\begin{aligned} ax^2 + bx + c &= a(x-q)(x-r) \\ &= ax^2 - a(q+r)x + aqr \end{aligned}$$

kan vi sammenligne koefficienter af de forskellige potenser af x, som giver

$$b = -a(q+r) \quad \text{og} \quad c = aqr.$$

Den sidste kan løses for q:

$$q = \frac{c}{ar} = \frac{2c}{-b - \sqrt{b^2 - 4ac}}.$$

Bruges denne formel for q, får vi et bedre resultat:

```
>>> q_new = (2*c) / (-b - sqrt_discriminant)
>>> q_new
-0.0025511532323023406
>>> a*q_new**2 + b*q_new + c
0.0
```

Den relative fejl i q er så

```
>>> (q - q_new) / q_new
5.346312858502275e-13
```

som er væsentlig større end machine epsilon. △

Generelt afhænger valget af beregningsmetoden af fortegnet $\operatorname{sgn}(b)$ af b.

Proposition 1.5. *Givet*

$$\operatorname{sgn}(b) = \begin{cases} 1, & \textit{for } b \geqslant 0, \\ -1, & \textit{for } b < 0, \end{cases}$$

er rødderne r_1, r_2 *af*

$$ax^2 + bx + c = 0 \qquad (a \neq 0,\ (b,c) \neq (0,0))$$

givet ved

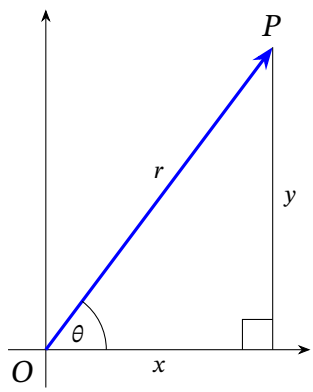
$$r_1 = \frac{-b - \operatorname{sgn}(b)\sqrt{b^2 - 4ac}}{2a}, \quad r_2 = \frac{2c}{-b - \operatorname{sgn}(b)\sqrt{b^2 - 4ac}}. \qquad \square$$

Kapitel 2

Plangeometri: vektorer og matricer

2.1 Vektorer, norm og vinkelbestemmelse

Vi er vandt til at repræsentere punkter P i planen via deres koordinater: $P = (x, y)$. F.eks. $P = (3, 4)$ er punktet med x-koordinat 3 og y-koordinat 4. Sådan et koordinatpar kan også betragtes som endepunkt for en pil $\overrightarrow{OP}$ fra origo $O = (0, 0)$ til punktet $P = (x, y)$. Denne pil kan ses som den længste side i en retvinklet trekant med kateter af længde hhv. x og y, se figur 2.1. Den



Figur 2.1: Afstand og vinkel i planen.

pythagoræiske læresætning fortæller os at $\overrightarrow{OP}$ har længde $r = \sqrt{x^2 + y^2}$:

$$|\overrightarrow{OP}| = r = \sqrt{x^2 + y^2}.$$

Desuden kan vinklen θ bestemmes fra de trigonometriske relationer

$$c = \cos(\theta) = \frac{x}{\sqrt{x^2 + y^2}}, \quad s = \sin(\theta) = \frac{y}{\sqrt{x^2 + y^2}}.$$

Oftest har vi ikke bruge for at kende selve værdien af θ, men kan nøjes med parret (c, s).

Et koordinatpar (x, y) giver anledning til en *vektor*

$$v = \begin{bmatrix} x \\ y \end{bmatrix}.$$

Vi indfører *normen* af denne vektor til at være længde af $\overrightarrow{OP}$:

$$\|v\|_2 = \left\| \begin{bmatrix} x \\ y \end{bmatrix} \right\|_2 = \sqrt{x^2 + y^2} = r.$$

Skaleres figur 2.1 med faktoren $1/\|v\|_2$, fås en ny vektor

$$w = \frac{1}{\|v\|_2} v = \begin{bmatrix} x/\sqrt{x^2 + y^2} \\ y/\sqrt{x^2 + y^2} \end{bmatrix} = \begin{bmatrix} c \\ s \end{bmatrix}. \tag{2.1}$$

Denne vektor w har norm 1, da længden af kateteret i den nye retvinklet trekant er 1. Dette kan bekræftes med regnestykket

$$\|w\|_2^2 = \left(\frac{x}{\sqrt{x^2 + y^2}} \right)^2 + \left(\frac{y}{\sqrt{x^2 + y^2}} \right)^2 = \frac{x^2}{x^2 + y^2} + \frac{y^2}{x^2 + y^2} = 1.$$

Vektorer af norm 1 kaldes for *enhedsvektorer*. Relationen (2.1) kan bruges til at bestemme vektoren v ud fra længde r og parret (c, s) ved

$$v = \|v\|_2 w = r \begin{bmatrix} c \\ s \end{bmatrix} = \begin{bmatrix} rc \\ rs \end{bmatrix}. \tag{2.2}$$

Givet $r \geqslant 0$ og (c, s) med $c^2 + s^2 = 1$, fås fra (2.2) vektoren $v = \left[\begin{smallmatrix} rc \\ rs \end{smallmatrix}\right]$ med $\|v\|_2 = r$ og enhedsvektor $\frac{1}{\|v\|_2} v = \left[\begin{smallmatrix} c \\ s \end{smallmatrix}\right]$.

Bemærk at vektorerne e_0 og e_1 givet ved

$$e_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \text{og} \quad e_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

er enhedsvektorer, som peger langs koordinatakserne.

2.2 Indre produkt og matrixtransformationer

Givet to vektorer $u = \left[\begin{smallmatrix} a \\ b \end{smallmatrix}\right]$ og $v = \left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$ sætter vi

$$\langle u, v\rangle = \left\langle \begin{bmatrix} a \\ b \end{bmatrix}, \begin{bmatrix} x \\ y \end{bmatrix} \right\rangle = ax + by.$$

Tallet $\langle u, v\rangle$ kaldes det *indre produkt* mellem u og v. Nogle steder skrives dette som $u \cdot v$, og dermed får det også navnet *prikproduktet*.

Vi får straks at

$$\langle v, v\rangle = \left\langle \begin{bmatrix} x \\ y \end{bmatrix}, \begin{bmatrix} x \\ y \end{bmatrix} \right\rangle = x^2 + y^2 = \|v\|_2^2,$$

dvs.

$$\|v\|_2 = \sqrt{\langle v, v\rangle}.$$

Med hensyn til parret $(c, s) = (\cos(\theta), \sin(\theta))$, der bestemmer vinklen θ, bemærker at vi har

$$\langle e_0, w\rangle = \left\langle \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} c \\ s \end{bmatrix} \right\rangle = 1.c + 0.s = c.$$

Givet to enhedsvektorer $w = \left[\begin{smallmatrix} \cos(\theta) \\ \sin(\theta) \end{smallmatrix}\right]$, $z = \left[\begin{smallmatrix} \cos(\varphi) \\ \sin(\varphi) \end{smallmatrix}\right]$ har vi

$$\begin{aligned} \langle w, z\rangle &= \left\langle \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}, \begin{bmatrix} \cos(\varphi) \\ \sin(\varphi) \end{bmatrix} \right\rangle \\ &= \cos(\theta)\cos(\varphi) + \sin(\theta)\sin(\varphi) \\ &= \cos(\varphi - \theta) \end{aligned}$$

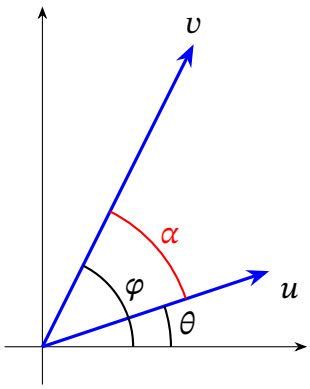
når vi bruger trigonometriske identiteter. Generelt har vi

$$\langle u, v\rangle = \|u\|_2 \|v\|_2 \cos(\alpha)$$

hvor α er vinklen mellem u og v, se figur 2.2.

Det indre produkt kan skrives på en anden måde hvis indfører transponerings operation. For en vektor $u = \left[\begin{smallmatrix} a \\ b \end{smallmatrix}\right]$ er den *transponerede*

$$u^T = \begin{bmatrix} a & b \end{bmatrix},$$



Figur 2.2: Vinklen mellem to vektorer.

en *rækkevektor*. Så sætter vi

$$u^T v = \begin{bmatrix} a & b \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = ax + by = \langle u, v \rangle.$$

Denne notation har den fordel, at vi kan skrive mange indre produkter på en gang: Givet en til vektor $\tilde{u} = \left[\begin{smallmatrix} c \\ d \end{smallmatrix}\right]$ kan vi stable u^T og $\tilde{u}^T$ ovenpå hinanden og danne matricen

$$A = \begin{bmatrix} u^T \\ \tilde{u}^T \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Vi kan nu stable deres indre produkter med v:

$$Av = \begin{bmatrix} u^T \\ \tilde{u}^T \end{bmatrix} v = \begin{bmatrix} u^T v \\ \tilde{u}^T v \end{bmatrix}, \tag{2.3}$$

dvs.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}. \tag{2.4}$$

Operationen $v \mapsto Av$ tager så et punkt i planen (x, y) og transformerer den til et nyt punkt $(ax + by, cx + dy)$. Dette kaldes en *matrixtransformation*. Forskellige matricer A giver forskellige transformationer.

For at forstå disse transformationer, er det en ide at kikke på hvad transformationer gør med flere punkter. Givet vektorer

$$v_0 = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}, \quad \ldots, \quad v_{n-1} = \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}$$

kan disse samles i en matrix

$$\left[v_0 \mid v_1 \mid v_2 \mid \dots \mid v_{n-1}\right] = \begin{bmatrix} x_0 & x_1 & x_2 & \dots & x_{n-1} \\ y_0 & y_1 & y_2 & \dots & y_{n-1} \end{bmatrix}.$$

Så kan deres indre produkter med u samles til en rækkevektor

$$\begin{aligned} & u^T\left[v_0 \mid v_1 \mid v_2 \mid \dots \mid v_{n-1}\right] \\ &= \begin{bmatrix} u^T v_0 & u^T v_1 & u^T v_2 & \dots & u^T v_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} a & b \end{bmatrix} \begin{bmatrix} x_0 & x_1 & x_2 & \dots & x_{n-1} \\ y_0 & y_1 & y_2 & \dots & y_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} ax_0 + by_0 & ax_1 + by_1 & ax_2 + by_2 & \dots & ax_{n-1} + by_{n-1} \end{bmatrix}. \end{aligned}$$

Tilsvarende kan vi samle effekten af transformation $v \mapsto Av$ på $v_0, \dots, v_{n-1}$ til en matrix med søjler $Av_0, \dots, Av_{n-1}$

$$A\left[v_0 \mid v_1 \mid v_2 \mid \dots \mid v_{n-1}\right] = \left[Av_0 \mid Av_1 \mid Av_2 \mid \dots \mid Av_{n-1}\right] \tag{2.5}$$

dvs.

$$\begin{aligned} & \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_0 & x_1 & x_2 & \dots & x_{n-1} \\ y_0 & y_1 & y_2 & \dots & y_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} ax_0 + by_0 & ax_1 + by_1 & ax_2 + by_2 & \dots & ax_{n-1} + by_{n-1} \\ cx_0 + dy_0 & cx_1 + dy_1 & cx_2 + dy_2 & \dots & cx_{n-1} + dy_{n-1} \end{bmatrix}. \end{aligned}$$

Lad os nu giver nogle eksempler på sådanne transformationer.

Skalering

$$S = \begin{bmatrix} r & 0 \\ 0 & r \end{bmatrix}, \qquad r > 0.$$

Dette sender $\left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$ til $\left[\begin{smallmatrix} rx \\ ry \end{smallmatrix}\right] = r\left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$, se figur 2.3.

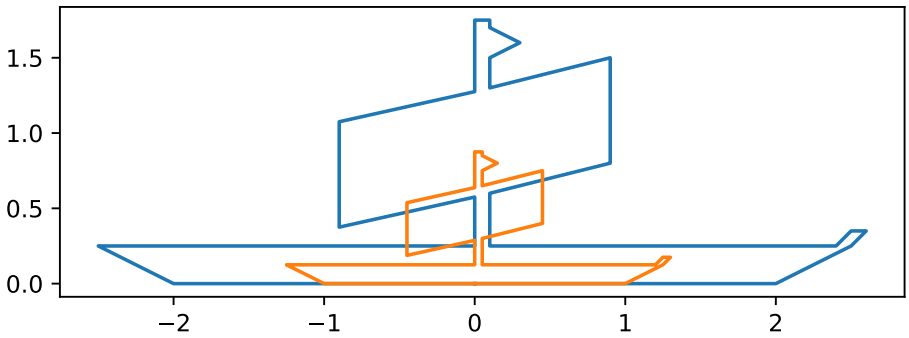
Skalering med faktor $r = 1$ ændrer ikke på punktet (x, y); $\left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right] = \left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$. Matricen

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

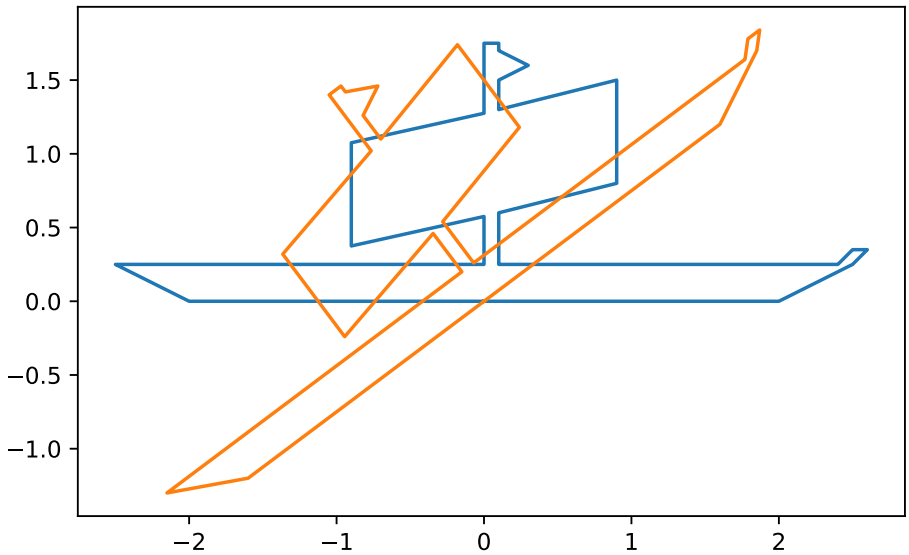
kaldes *identitetsmatricen*.

Ønsker man skalering med forskellige faktorer, r_1 i x-retningen og r_2 i y-retningen, bruger man matricen

$$\begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix}.$$



Figur 2.3: Skalering med faktor $r = 0{,}5$.



Figur 2.4: Rotation med $c = 0{,}8$, $s = \sqrt{1 - c^2} = 0{,}6$.

Rotation

$$R = \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \qquad c^2 + s^2 = 1$$

Dette sender e_0 til (c, s), og e_1 til $(-s, c)$. Det er en rotation igennem vinklen θ, som opfylder $\cos(\theta) = c$, $\sin(\theta) = s$, se figur 2.4. I mange anvendelser har vi ikke brug for at beregne vinklen θ.

Bemærkning 2.1. Hvis man absolut skal bestemme vinklen θ, så kan funktionen `np.arctan2` fra NumPy pakken bruges. For en vektor $v = \left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right]$ er vinklen θ givet via `np.arctan2(y,x)`, pas på rækkefølgen!

```
>>> import numpy as np
>>> np.arctan2(0,1)
0.0
>>> np.arctan2(1,0)
1.5707963267948966
>>> np.arctan2(1,1)
0.7853981633974483
```

Svaret er i radianer. Ønsker man at vide hvilke multiplum af π det er skal man dele med `np.pi`. F.eks.

```
>>> np.pi
3.141592653589793
>>> np.arctan2(1,0) / np.pi
0.5
>>> np.arctan2(1,1) / np.pi
0.25
```

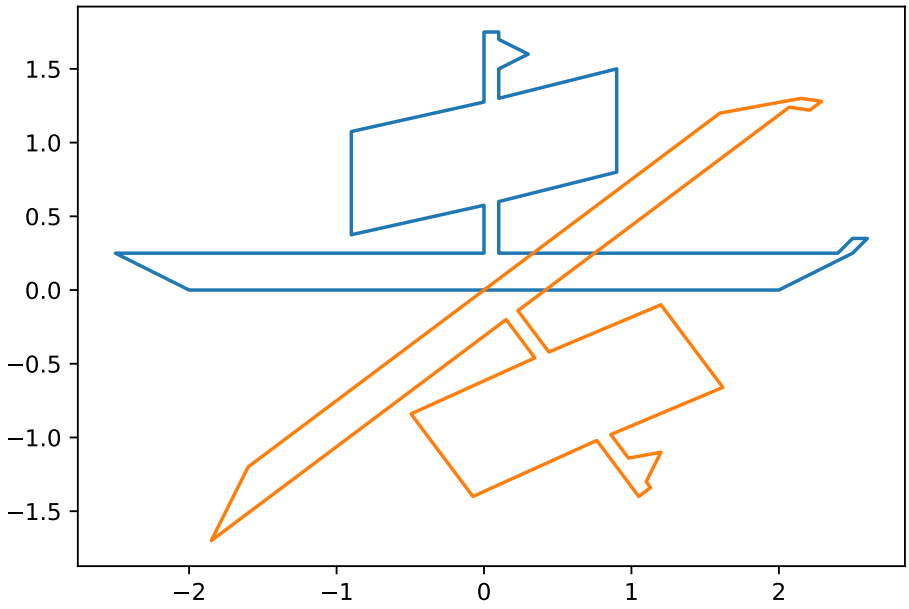
svarende til at $e_1 = (0,1)$ har vinkel $\pi/2$ fra e_0 og at $(1,1)$ har vinkel $\pi/4$. Ønsker man svaret i grader kan man gange svaret i radian med `180/np.pi` eller bruge `np.degrees`:

```
>>> np.degrees(np.pi)
180.0
>>> np.degrees(np.arctan2(1,0))
90.0
>>> np.degrees(np.arctan2(1,1))
45.0
```

◇

Spejling

$$M = \begin{bmatrix} c & s \\ s & -c \end{bmatrix}, \qquad c^2 + s^2 = 1.$$



Figur 2.5: Spejling med $c = 0{,}8$, $s = \sqrt{1 - c^2} = 0{,}6$.

Bemærk fortegnsforskel fra rotationsmatricen; her ligger minusfortegnet på et element på diagonalen. Hvis $(c, s) = (\cos\theta, \sin\theta)$, så giver M en spejling i linjer igennem origo med vinkel $\theta/2$ til x-aksen, se figur 2.5.

Projektion

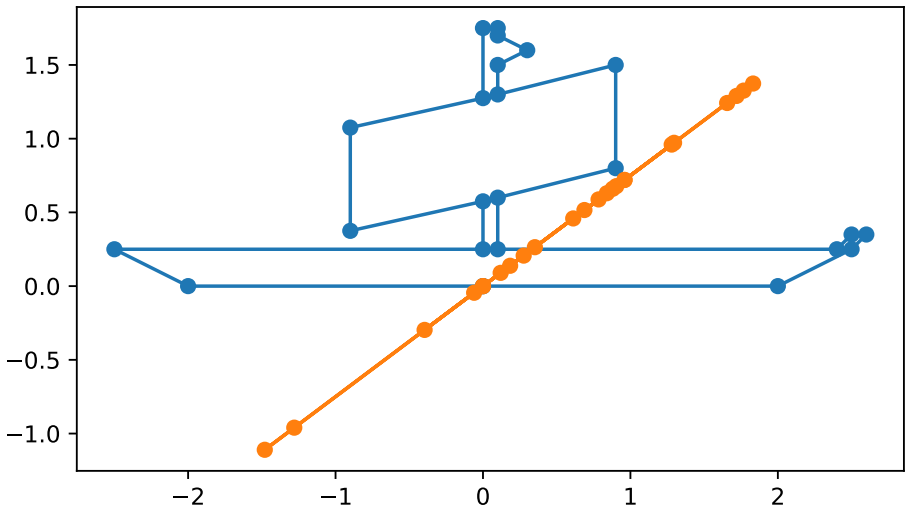
$$P = \begin{bmatrix} p^2 & pq \\ pq & q^2 \end{bmatrix}, \qquad p^2 + q^2 = 1.$$

Dette sender alle punkter til punkter på linjen igennem origo og (p, q), se figur 2.6. For $q = 0$, er $P = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right]$ og $P\left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right] = \left[\begin{smallmatrix} x \\ 0 \end{smallmatrix}\right]$, så y-koordinatet sættes til 0, i dette tilfælde.

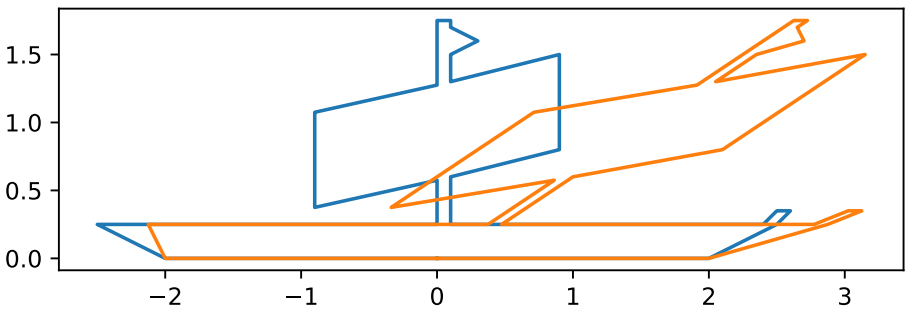
Skævvridning

$$K = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

forskubber punkter til højre ved en størrelse bestemt af y-koordinatet: $K\left[\begin{smallmatrix} x \\ y \end{smallmatrix}\right] = \left[\begin{smallmatrix} x+ty \\ y \end{smallmatrix}\right]$, se figur 2.6.



Figur 2.6: Projektion med $p = 0{,}8$, $q = \sqrt{1 - p^2} = 0{,}6$.



Figur 2.7: Skævvridning med $t = 1{,}5$.

2.3 Første tegninger

Lad os se hvordan man kan lave tegninger som ovenfor med python. Vi bruger pakken `matplotlib.pyplot` samt `numpy`:

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
```

Lad os begynder med nogle koordinatpar, som f.eks.

$$\begin{bmatrix} x_0 & x_1 & x_2 & x_3 \\ y_0 & y_1 & y_2 & y_3 \end{bmatrix} = \begin{bmatrix} 0{,}2 & 0{,}2 & -0{,}2 & 0{,}3 \\ -0{,}2 & 0{,}8 & 0{,}1 & 0{,}1 \end{bmatrix}.$$

Vi kan gemme x og y koordinater i to `np.array`:

```
>>> x = np.array([ 0.2, 0.2, -0.2, 0.3])
>>> y = np.array([-0.2, 0.8,  0.1, 0.1])
```

Nu tegner vi linjen, som begynder i (x_0, y_0) og kører videre igennem (x_1, y_1), (x_2, y_2) og hen til (x_3, y_3).

```
>>> fig, ax = plt.subplots()
>>> ax.set_aspect('equal')
>>> ax.plot(x, y)
[<matplotlib.lines.Line2D object at 0x106c5fef0>]
```

Afhængig af opsætningen, får man enten vist billedet straks eller skal man bede om at få den vist via

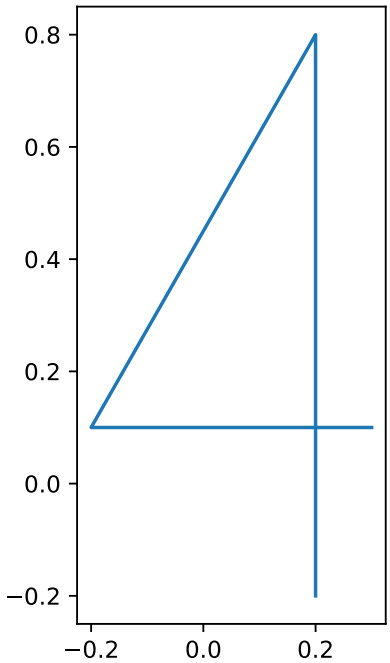
```
fig.show()
```

Du burde få noget der ligner figur 2.8.

Lad os nu flytter vores tegning via en matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1{,}0 & 0{,}5 \\ 0{,}0 & 1{,}0 \end{bmatrix}.$$

Vi kan bruge formlen (2.4) næsten direkte; vores nye x-koordinater er $ax + by =$ `a*x+b*y`, og den nye y-koordinater er $cx + dy =$ `c*x+d*y`:



Figur 2.8: En første tegning.

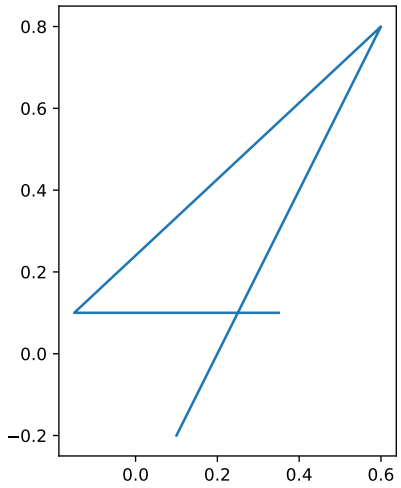
```
>>> a = 1.0
>>> b = 0.5
>>> c = 0.0
>>> d = 1.0
>>> fig, ax = plt.subplots()
>>> ax.set_aspect('equal')
>>> ax.plot(a*x+b*y, c*x+d*y)
[<matplotlib.lines.Line2D object at 0x111029520>]
```

som vises frem på samme måde som ovenfor, se figur 2.9.

Figuren kan gemmes i en `pdf` fil via

```
fig.savefig('figure-name.pdf')
```

Man kan give tegningen en anden farve og/eller markere punkter ved f.eks.



Figur 2.9: Tegning flyttet via en matrix.

```
ax.plot(a*x+b*y, c*x+d*y, color='r')  #r = rød
ax.plot(a*x+b*y, c*x+d*y, marker='o') #o = cirkler/prikker
```

Plotgrænserne kan angives med

```
ax.set_xlim(-3.0, 3.0)
ax.set_ylim(-2.0, 4.0)
```

Figurens størrelse kan bestemmes med `figsize`

```
fig, ax = plt.subplots(figsize=(3, 3))
```

hvor tallene angives i tommer. Hvis man vil bruge centimeter, kan man konvertere via en passende faktor

```
cm = 1 / 2.54
fig, ax = plt.subplots(figsize=(6*cm, 6*cm))
```

Bemærkning 2.2. I `jupyter lab` kan man søge for at figurer generes i `pdf` formatet, i stedet for `svg`, på den følgende måde

```
from matplotlib_inline.backend_inline import
↪  set_matplotlib_formats
set_matplotlib_formats('pdf')
```

(Her viser den lille pil ↪ at linjen fortætter.)

Dette kan hjælpe med eksportering af notebook til pdf. ◇

Kapitel 3

Matricer og vektorer

3.1 Matricer

En *matrix* er et rektangulært skema af tal, som for eksempel

$$A = \begin{bmatrix} 3{,}1 & -2{,}1 & 4{,}0 & -1{,}6 \\ 7{,}2 & 3{,}6 & -2{,}7 & 11{,}3 \\ -5{,}6 & -1{,}2 & 3{,}5 & -17{,}2 \end{bmatrix}.$$

En matrix med m rækker og n søjler kaldes en *$(m \times n)$-matrix*. Tallene, der kan bruges i matricen, kaldes *skalarer*. Tallet, der er placeret i række i, søjle j, kaldes den *(i, j)te indgang* i matricen; for en matrix A skriver vi a_{ij} eller $A_{[i,j]}$ for denne indgang. Mht. konventionerne i python begynder nummerering ved $i = 0$, $j = 0$. Så i eksemplet overfor er A en (3×4)-matrix, med $a_{00} = 3{,}1$ og $a_{13} = 11{,}3$. Generelt, har vi

$$A = (a_{ij}) = \begin{bmatrix} a_{00} & a_{01} & \dots & a_{0,n-1} \\ a_{10} & a_{11} & \dots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \dots & a_{m-1,n-1} \end{bmatrix}.$$

Mængden af alle $(m \times n)$-matricer med reelle indgange skrives $\mathbb{R}^{m \times n}$.

To matricer A, B er *lige med hinanden* hvis de har samme størrelse $m \times n$ og samme indgange, dvs. $a_{ij} = b_{ij}$ for alle $i = 0, \dots, m-1$, $j = 0, \dots, n-1$.

3.2 Matricer i python

Pakken NumPy har en fundamental indbygget objekt `ndarray` for matricer (og vektorer og tensorer). Matricer oprettes med kommandoen `np.array`

```
>>> import numpy as np
>>> a = np.array([[3.1, -2.1, 4.0, -1.6],
...               [7.2, 3.6, -2.7, 11.3],
...               [-5.6, -1.2, 3.5, -17.2]])
>>> a
array([[  3.1,  -2.1,   4. ,  -1.6],
       [  7.2,   3.6,  -2.7,  11.3],
       [ -5.6,  -1.2,   3.5, -17.2]])
```

(prikkerne `...` i begyndelsen af linjen skrives ikke ind, de viser bare at input-linjen begyndende med `>>>` fortsætter). Dette er en 3×4 matrix

```
>>> a.shape
(3, 4)
```

og dens type er `np.ndarray`

```
>>> type(a)
<class 'numpy.ndarray'>
```

I python siger man at objektet `a` har 2 *akser*. Antallet af akser fås fra `ndim`

```
>>> a.ndim
2
```

Vi får adgang til enkelte indgange i `a` via `a[2,3]` osv. Så de overnævnte indgange er

```
>>> a[0,0]
3.1
>>> a[1,3]
11.3
```

En hel søjle af `a` fås via `a[:, [2]]`, en række fås via `a[[1], :]`

```
>>> a[:, [2]] #søjle
array([[ 4. ],
       [-2.7],
       [ 3.5]])
>>> a[[1], :] #række
array([[ 7.2,  3.6, -2.7, 11.3]])
```

Bemærk at der bruges ekstra parentes `[...]`; glemmes disse får man resultater med kun én akse

```
>>> a[:, 2] #indgange i en søjle
array([ 4. , -2.7,  3.5])
>>> a[:, 2].ndim
1
>>> a[1, :] #indgange i en række
array([ 7.2,  3.6, -2.7, 11.3])
>>> a[1, :].ndim
1
```

Indekserne begynder ved `0` og tæller op ad, men negative indekser kan bruges til at tælle ned fra den største værdi

```
>>> a[[-1], :] #sidste række
array([[ -5.6,  -1.2,   3.5, -17.2]])
>>> a[:, [-2]] #næstsidste søjle
array([[ 4. ],
       [-2.7],
       [ 3.5]])
```

Elementerne i en `ndarray` har alle den samme datatype, `dtype`. Dette betyder at hvert element tager lige meget plads i computerens hukommelse, og gøre det nemmere og hurtigere for computeren til at finde et givent element. Vi har

```
>>> a.dtype
dtype('float64')
```

Denne datatype bruges automatisk af python når mindst en indgang i a er givet som decimalbrøk. For at sikre, man får den rigtig datatype, kan man angive datatypen når matricen oprettes

```
>>> b = np.array([[1, 2],
...               [3, 7]], dtype='float')
>>> b
array([[1., 2.],
       [3., 7.]])
>>> b.dtype
dtype('float64')
```

En liste over forskellige datatyper NumPy kender som standard findes i afsnittet om Data types under Fundamentals and usage i NumPy user guide fra *NumPy documentation* af the NumPy community 2023, og mere uddybende i afsnittet Scalars under Array objects i NumPy API reference fra samme sted. For det meste holder vi os til `float` typen. Lidt senere vil vi have brug for `complex` typen.

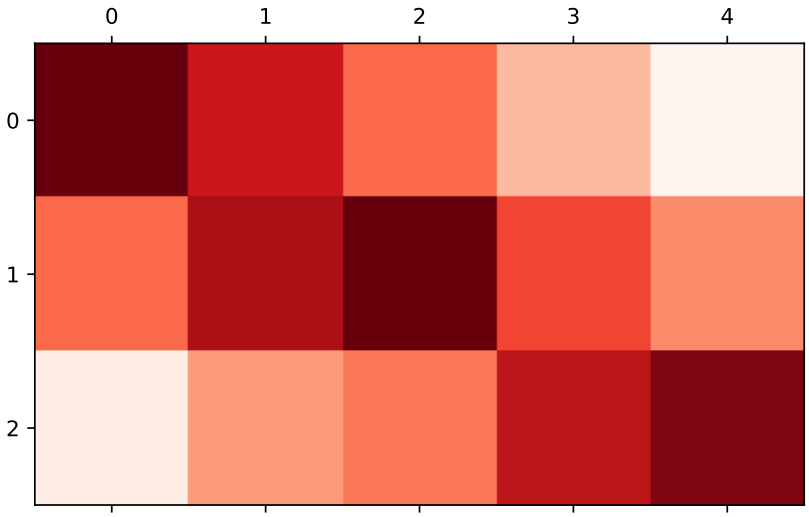
3.3 Heatmap plot

En nyttig metode for at visualisere en matrix er at farvelægge hver indgang med en farve bestemt af dens værdi. For dette kan man med fordel bruge `matshow` fra `matplotlib`.

```
import matplotlib.pyplot as plt
import numpy as np

a = np.array([[1.0, 0.5, 0.0, -0.5, -1.0],
              [0.0, 0.7, 1.0, 0.2, -0.2],
              [-0.9, -0.3, -0.1, 0.6, 0.9]])

fig, ax = plt.subplots()
ax.matshow(a, cmap='Reds')
```

Som giver figur 3.1.

Argumentet `cmap` angiver hvilke farve skala der skal anvendes. Navne som `'Reds'`, `'Blues'` eller `'Greys'` bruge en stærk farve for høje værdier og hvid



Figur 3.1: En første heatmap plot.

for de lavste værdier. Det kan være nyttig at tilføje oplysning om hvilke farver svarer til hvilke værdi, og det vises fint med en `colorbar`.

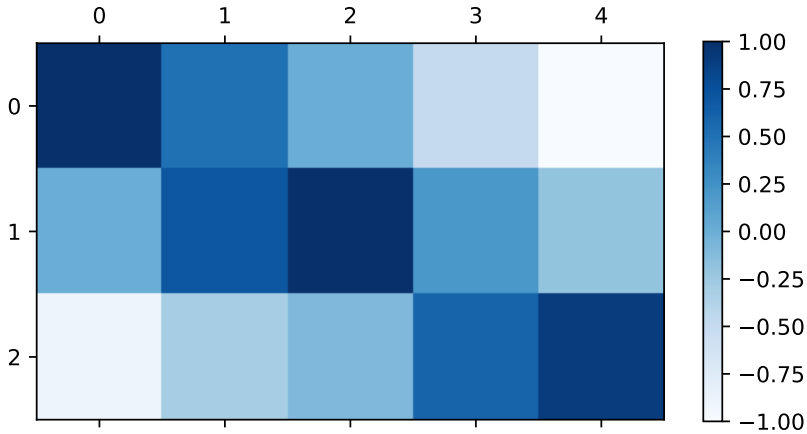
```
fig, ax = plt.subplots()
im = ax.matshow(a, cmap='Blues')
fig.colorbar(im, shrink=0.65)
```

Størrelsen på `colorbar` styres af argumentet `shrink`, som skalere bjælken med den givne faktor.

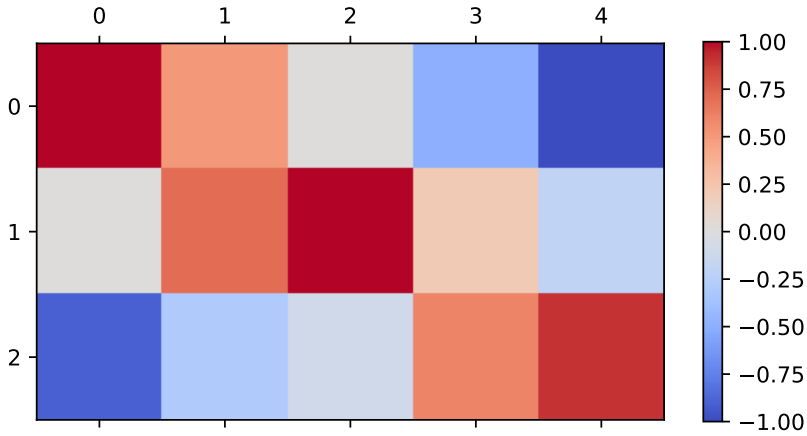
Man kan justere hvilke værdier dækkes af farveskalaen, ved at sætte `clim`, f.eks. `clim = (-1.0,1.0)` siger at farveskalaen skal bruges til at dække værdierne fra −1,0 til 1,0.

```
fig, ax = plt.subplots()
im = ax.matshow(a, cmap='coolwarm', clim=(-1.0,1.0))
fig.colorbar(im, shrink=0.65)
```

Her har vi brugt en farveskala der går fra blå til rød. Ved at bestemme grænser `clim=(-1.0,1.0)` som er symmetrisk omkring 0,0, opnås at negative værdier får en blå nuance, og positive værdier er rødlig nuance.



Figur 3.2: Heatmap med colorbar.



Figur 3.3: Heatmap med justeret farveskala.

3.4 Multiplikation med en skalar og matrixsum

Givet en matrix $A \in \mathbb{R}^{m\times n}$ som i afsnit 3.1 og en skalar $s \in \mathbb{R}$ defineres *multiplikation af A med en skalar s* som $(m \times n)$-matricen sA hvor s er ganget ind på alle indgange i A enkeltvis:

$$sA = s(a_{ij}) := (sa_{ij}) = \begin{bmatrix} sa_{00} & sa_{01} & \dots & sa_{0,n-1} \\ sa_{10} & sa_{11} & \dots & sa_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ sa_{m-1,0} & sa_{m-1,1} & \dots & sa_{m-1,n-1} \end{bmatrix}.$$

I python er operationen givet via operatoren `*`

```
>>> a
array([[  3.1,  -2.1,   4. ,  -1.6],
       [  7.2,   3.6,  -2.7,  11.3],
       [ -5.6,  -1.2,   3.5, -17.2]])
>>> s = 2.0
>>> s * a
array([[  6.2,  -4.2,   8. ,  -3.2],
       [ 14.4,   7.2,  -5.4,  22.6],
       [-11.2,  -2.4,   7. , -34.4]])
```

I tilfældet hvor $s = 0$, er $sA = 0A = 0_{m\times n}$ *nulmatricen* i $\mathbb{R}^{m\times n}$, hvor alle indgange er 0. Hvis størrelsen af nulmatricen er underordnet, skriver vi blot 0 i stedet for $0_{m\times n}$. Funktionen `np.zeros()` giver en nulmatrix af en given størrelse; størrelsen specificeres som tupel, som f.eks. `(2,3)`

```
>>> np.zeros((2,3))
array([[0., 0., 0.],
       [0., 0., 0.]])
```

Givet to matricer A, B af samme størrelse $m \times n$, defineres deres *matrixsum* $A + B \in \mathbb{R}^{m\times n}$ ved at dens (i, j)-indgang er summen af de tilsvarende (i, j)-

indgange i A og B:

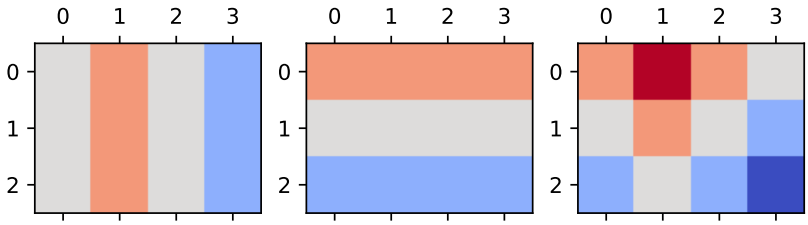
$$
\begin{aligned}
A + B &= (a_{ij}) + (b_{ij}) \coloneqq (a_{ij} + b_{ij}) \\
&= \begin{bmatrix} a_{00} + b_{00} & a_{01} + b_{01} & \dots & a_{0,n-1} + b_{0,n-1} \\ a_{10} + b_{10} & a_{11} + b_{11} & \dots & a_{1,n-1} + b_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} + b_{m-1,0} & a_{m-1,1} + b_{m-1,1} & \dots & a_{m-1,n-1} + b_{m-1,n-1} \end{bmatrix}.
\end{aligned}
$$

Dette gøres med almindelig plus operatoren + i python

```
>>> a
array([[  3.1,  -2.1,   4. ,  -1.6],
       [  7.2,   3.6,  -2.7,  11.3],
       [ -5.6,  -1.2,   3.5, -17.2]])
>>> b = np.array([[2.0, 1.0, 0.0, -1.0],
...               [2.0, 1.0, 0.0, -1.0],
...               [2.0, 1.0, 0.0, -1.0]])
>>> b
array([[ 2.,  1.,  0., -1.],
       [ 2.,  1.,  0., -1.],
       [ 2.,  1.,  0., -1.]])
>>> a + b
array([[  5.1,  -1.1,   4. ,  -2.6],
       [  9.2,   4.6,  -2.7,  10.3],
       [ -3.6,  -0.2,   3.5, -18.2]])
```

Vi kan lave heatmaps, der illustrerer summen på følgende måde:

```
import matplotlib.pyplot as plt
import numpy as np

a = np.array([[0.0, 1.0, 0.0, -1.0],
              [0.0, 1.0, 0.0, -1.0],
              [0.0, 1.0, 0.0, -1.0]])
b = np.array([[ 1.0,  1.0,  1.0,  1.0],
              [ 0.0,  0.0,  0.0,  0.0],
              [-1.0, -1.0, -1.0, -1.0]])
```



Figur 3.4: Heatmap for A, B og deres sum $A + B$.

```
fig, axs = plt.subplots(1, 3)  #én rækker med tre billeder
axs[0].matshow(a, cmap='coolwarm', clim=(-2.0, 2.0))
axs[1].matshow(b, cmap='coolwarm', clim=(-2.0, 2.0))
axs[2].matshow(a + b, cmap='coolwarm', clim=(-2.0, 2.0))
```

Resultatet ses i figur 3.4. Tilsvarende er *differencen* af to $(m \times n)$-matricer

$$A - B = (a_{ij}) - (b_{ij}) \coloneqq (a_{ij} - b_{ij})$$

og i python bruges almindelig minus `-`.

Proposition 3.1. *Skalar multiplikation og matrixsum har følgende regneregler*:

(a) $0A = 0_{m\times n}$ *og* $1A = A$, *for* $0, 1 \in \mathbb{R}$, $A \in \mathbb{R}^{m\times n}$,
(b) $A + 0_{m\times n} = A$, *for* $A, 0_{m\times n} \in \mathbb{R}^{m\times n}$,
(c) $(s + t)A = sA + tA$, *for* $s, t \in \mathbb{R}$, $A \in \mathbb{R}^{m\times n}$,
(d) $s(A + B) = sA + sB$, *for* $s \in \mathbb{R}$, $A, B \in \mathbb{R}^{m\times n}$,
(e) $A + B = B + A$, *for* $A, B \in \mathbb{R}^{m\times n}$,
(f) $A - B = A + (-1)B$, *for* $A, B \in \mathbb{R}^{m\times n}$,
(g) $A + (B + C) = (A + B) + C$, *for* $A, B, C \in \mathbb{R}^{m\times n}$.

Bevis. Alle resultater tjekkes ved at sammenligne tilsvarende indgange på begge sider. □

Skalarmultiplikation fra højre side defineres ved

$$As = (a_{ij})s \coloneqq (a_{ij}s).$$

Da $a_{ij}s = sa_{ij}$, har vi

$$As = sA.$$

3.5 Transponering

Transponering er operationen $A \to A^T$ på matricer der ombytter rækker og søjler. Hvis $A \in \mathbb{R}^{m\times n}$, så er $A^T \in \mathbb{R}^{n\times m}$

```
>>> a
array([[  3.1,  -2.1,   4. ,  -1.6],
       [  7.2,   3.6,  -2.7,  11.3],
       [ -5.6,  -1.2,   3.5, -17.2]])
>>> a.T
array([[  3.1,   7.2,  -5.6],
       [ -2.1,   3.6,  -1.2],
       [  4. ,  -2.7,   3.5],
       [ -1.6,  11.3, -17.2]])
```

Mere visuelt har vi følgende

$$A = \begin{bmatrix} 3{,}1 & -2{,}1 & 4{,}0 & -1{,}6 \\ 7{,}2 & 3{,}6 & -2{,}7 & 11{,}3 \\ -5{,}6 & -1{,}2 & 3{,}5 & -17{,}2 \end{bmatrix} \mapsto \begin{bmatrix} 3{,}1 & 7{,}2 & -5{,}6 \\ -2{,}1 & 3{,}6 & -1{,}2 \\ 4{,}0 & -2{,}7 & 3{,}5 \\ -1{,}6 & 11{,}3 & -17{,}2 \end{bmatrix} = A^T$$

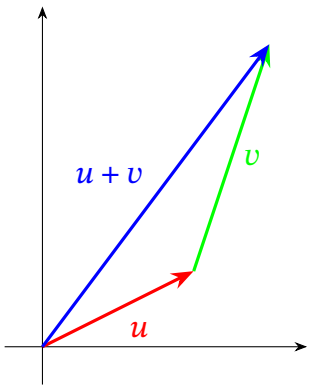
Matricen A bliver spejlet i diagonalen, som består af de grønne indgange. I indeksnotation har vi

$$A = (a_{ij}) \mapsto A^T = (a_{ji}).$$

3.6 Vektorer

En *søjlevektor* $v \in \mathbb{R}^n$ er en $(n \times 1)$-matrix, som vi skriver

$$v = (v_i) = \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1} \end{bmatrix}.$$



Figur 3.5: Vektorsum af $u = (2, 1)$ og $v = (1, 3)$.

I tekst skriver vi $v = (v_0, v_1, \ldots, v_{n-1})$. Da v er en $(n \times 1)$-matrix, kan vi bruge operationerne i afsnit 3.4 til at få skalarmultiplikation og vektorsum

$$sv = s(v_i) := (sv_i) = \begin{bmatrix} sv_0 \\ sv_1 \\ \vdots \\ sv_{n-1} \end{bmatrix},$$

$$u + v = (u_i) + (v_i) := (u_i + v_i) = \begin{bmatrix} u_0 + v_0 \\ u_1 + v_1 \\ \vdots \\ u_{n-1} + v_{n-1} \end{bmatrix}.$$

Som i kapitel 2 kan en vektor tænkes som en pil fra origo til punktet med koordinator $(v_0, v_1, \ldots, v_{n-1})$. Vektorsum svarer til at sætte to piler efter hinanden, som i figur 3.5.

Der er to forskellige måde til at skrive søjlevektorer ind i python. For den første kan vi bruge den samme metode, som for en $(n \times 1)$-matrix:

```
>>> v = np.array([[3.0], [4.0], [-0.7], [6.2]])
>>> v
array([[ 3. ],
       [ 4. ],
       [-0.7],
       [ 6.2]])
```

Den anden metode er at omforme en `np.array` med én akse til en søjlevektor ved hjælp af `np.newaxis`

```
>>> np.array([3.0, 4.0, -0.7, 6.2])[:, np.newaxis]
array([[ 3. ],
       [ 4. ],
       [-0.7],
       [ 6.2]])
```

Denne metode er oftest mest praktisk, og vil være den, der bruges i resten af kurset.

Transponering af en søjlevektor $v \in \mathbb{R}^n$ giver en *rækkevektor* $v^T \in \mathbb{R}^{1\times n}$:

```
>>> v
array([[ 3. ],
       [ 4. ],
       [-0.7],
       [ 6.2]])
>>> v.T
array([[ 3. ,  4. , -0.7,  6.2]])
```

Kapitel 4

Matrix multiplikation

I afsnit 2.2 så vi hvordan man kan bruge en (2×2)-matrix A til at flytte på vektorer v i $\mathbb{R}^2$, via $v \mapsto Av$, se ligninger (2.3) og (2.4). Ved at stille flere vektorer op efter hinanden fik så vi også (2.5) hvordan A flytter en $(2 \times n)$-matrix $\left[v_0 \mid v_1 \mid v_2 \mid \dots \mid v_{n-1}\right]$ til $\left[Av_0 \mid Av_1 \mid Av_2 \mid \dots \mid Av_{n-1}\right]$. Lad os nu udvider disse processer til matricer af vilkårlig størrelse.

4.1 Række-søjleprodukt

Vi begynder med et produkt for en rækkevektor med en søjlevektor. Hvis $u^T \in \mathbb{R}^{1\times n}$ og $v \in \mathbb{R}^{n\times 1}$ er en rækkevektor og en søjlevektor *med det samme antal indgange* så er deres *række-søjleprodukt* givet ved

$$u^T v = \begin{bmatrix} u_0 & u_1 & \dots & u_{n-1} \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1} \end{bmatrix} = u_0 v_0 + u_1 v_1 + \cdots + u_{n-1} v_{n-1}.$$

Eksempel 4.1. Et eksempel med $n = 3$ er

$$\begin{aligned} \begin{bmatrix} 3 & -1 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 5 \\ -4 \end{bmatrix} &= 3 \times 6 + (-1) \times 5 + 2 \times (-4) \\ &= 18 - 5 - 8 = 5. \end{aligned}$$

△

Eksempel 4.2. For $u^T = \begin{bmatrix} 1 & 1 & \dots & 1 \end{bmatrix} \in \mathbb{R}^{1\times n}$ er

$$\begin{aligned} u^T v &= \begin{bmatrix} 1 & 1 & \dots & 1 \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1} \end{bmatrix} = 1 \times v_0 + 1 \times v_1 + \cdots + 1 \times v_{n-1} \\ &= v_0 + v_1 + \cdots + v_{n-1} \end{aligned}$$

summen af indgangene i v. Ligeledes, for

$$w^T = (1/n)u^T = \begin{bmatrix} 1/n & 1/n & \dots & 1/n \end{bmatrix} \in \mathbb{R}^{1\times n}$$

er

$$w^T v = \frac{1}{n}(v_0 + v_1 + \cdots + v_{n-1}),$$

som er middelværdien af indgangene i v. △

Eksempel 4.3. Betragt et polynomium

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}.$$

Vi gemmer koefficienterne a_i i en rækkevektor u^T:

$$u^T = \begin{bmatrix} a_0 & a_1 & \dots & a_{n-1} \end{bmatrix} \in \mathbb{R}^{1\times n}.$$

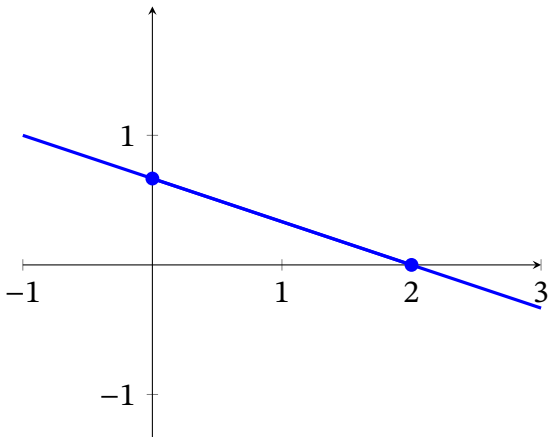
Givet et tal x_0 kan vi betragte Vandermondevektoren

$$v = \begin{bmatrix} 1 \\ x_0 \\ x_0^2 \\ \vdots \\ x_0^{n-1} \end{bmatrix} \in \mathbb{R}^{n\times 1}.$$

Række-søjleproduktet af u^T med v er så

$$\begin{aligned} u^T v &= \begin{bmatrix} a_0 & a_1 & \dots & a_{n-1} \end{bmatrix} \begin{bmatrix} 1 \\ x_0 \\ x_0^2 \\ \vdots \\ x_0^{n-1} \end{bmatrix} \\ &= a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_{n-1} x_0^{n-1} = p(x_0) \end{aligned}$$

evaluering af $p(x)$ i $x = x_0$. △



Figur 4.1: Linjen $x + 3y = 2$.

Eksempel 4.4. Givet en rækkevektor $u^T = \begin{bmatrix} a & b \end{bmatrix} \in \mathbb{R}^{1\times 2}$ som er ikke nul, og en værdi $c \in \mathbb{R}$, kan vi lede efter punkterne (x, y) således at vektoren $v = \begin{bmatrix} x \\ y \end{bmatrix}$ opfylder

$$u^T v = c.$$

Skrives denne ligning ud, får vi

$$ax + by = c,$$

som er ligningen for en ret linje i $\mathbb{R}^2$. F.eks. for $(a, b) = (1, 3)$ og $c = 2$ har vi

$$x + 3y = 2.$$

Sættes $y = 0$, fås $x = 2$, så denne linje skær x-aksen i punktet $(2, 0)$. Tilsvarende hvis vi sætter $x = 0$ fås $y = 2/3$, så linjen går også igennem $(0, 2/3)$, se figur 4.1.

Tilsvarende for $u^T = \begin{bmatrix} a & b & c \end{bmatrix} \in \mathbb{R}^{1\times 3}$, $d \in \mathbb{R}$ og $v = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \in \mathbb{R}^{3\times 1}$, er $u^T v = d$ planen

$$ax + by + cz = d.$$

△

4.2 Matrixprodukt

Definition 4.5. For to matricer $A \in \mathbb{R}^{m\times n}$ og $B \in \mathbb{R}^{n\times r}$ defineres deres *matrixprodukt* til at være matricen $AB \in \mathbb{R}^{m\times r}$ hvis (i, j)-indgang er række-søjleproduktet af række i fra A med søjle j fra B. Dvs. $C = AB$ har (i, j)-indgang

$$c_{ij} = a_{i0}b_{0j} + a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{i,n-1}b_{n-1,j}.$$

Eksempel 4.6. For et eksempel med $m = 2$, $n = 3$, $r = 4$ tager vi

$$A = \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3} \quad \text{og} \quad B = \begin{bmatrix} 6 & 1 & 0 & 0 \\ 5 & 1 & 1 & 2 \\ -4 & 1 & 0 & 2 \end{bmatrix} \in \mathbb{R}^{3\times 4}.$$

Så er produktet AB givet ved

$$AB = \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 6 & 1 & 0 & 0 \\ 5 & 1 & 1 & 2 \\ -4 & 1 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 5 & 4 & -1 & 2 \\ 1 & 2 & 1 & 4 \end{bmatrix}.$$

F.eks. indgangen nederst til højre er række-søjleproduktet

$$\begin{bmatrix} 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix} = 0 \times 0 + 1 \times 2 + 1 \times 2 = 4.$$

Bemærk at for disse matricer er produktet BA, i den omvendte rækkefølge, ikke defineret: da $B \in \mathbb{R}^{3\times 4}$ og $A \in \mathbb{R}^{2\times 3}$, men $4 \neq 2$. △

I python er der naturligvis en funktion der vil beregne matrixproduktet for os, givet ved operatoren `@`. Så eksemplet ovenfor er

```
>>> import numpy as np
>>> a = np.array([[3.0, -1.0, 2.0],
...               [0.0,  1.0, 1.0]])
>>> b = np.array([[6.0, 1.0, 0.0, 0.0],
...               [5.0, 1.0, 1.0, 2.0],
...               [-4.0, 1.0, 0.0, 2.0]])
>>> a @ b
array([[ 5.,  4., -1.,  2.],
       [ 1.,  2.,  1.,  4.]])
```

Python er også enig at produktet BA er ikke defineret for disse to matricer

```
>>> b @ a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: matmul: Input operand 1 has a mismatch in its
↪  core dimension 0, with gufunc signature
↪  (n?,k),(k,m?)->(n?,m?) (size 2 is different from 4)
```

Advarsel 4.7. Matrix multiplikation er *ikke* givet ved `*`, man skal bruge pythons `@` operator `a @ b`. Desuden giver nogle gange `a * b` ingen fejlmeddelelser og udfører en anden operation end matrixmultiplikation. !

Matrixproduktet kan fortolkes på forskellige måde. Bemærk først at hvis vi deler $A \in \mathbb{R}^{m\times n}$ op i dens m rækker, og $B \in \mathbb{R}^{n\times r}$ op i dens r søjler har vi tre forskellige måde at se på produktet:

$$
\begin{aligned}
AB &= \begin{bmatrix} a_0^T \\ \hline a_1^T \\ \hline a_2^T \\ \hline \vdots \\ \hline a_{m-1}^T \end{bmatrix} \left[b_0 \;\middle|\; b_1 \;\middle|\; b_2 \;\middle|\; \dots \;\middle|\; b_{r-1} \right] \\
&= \begin{bmatrix} a_0^T b_0 & a_0^T b_1 & a_0^T b_2 & \dots & a_0^T b_{r-1} \\ a_1^T b_0 & a_1^T b_1 & a_1^T b_2 & \dots & a_1^T b_{r-1} \\ a_2^T b_0 & a_2^T b_1 & a_2^T b_2 & \dots & a_2^T b_{r-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1}^T b_0 & a_{m-1}^T b_1 & a_{m-1}^T b_2 & \dots & a_{m-1}^T b_{r-1} \end{bmatrix} && (4.1) \\
&= \left[Ab_0 \;\middle|\; Ab_1 \;\middle|\; Ab_2 \;\middle|\; \dots \;\middle|\; Ab_{r-1} \right] && (4.2)
\end{aligned}
$$

$$= \begin{bmatrix} a_0^T B \\ \hline a_1^T B \\ \hline a_2^T B \\ \hline \vdots \\ \hline a_{m-1}^T B \end{bmatrix}. \tag{4.3}$$

Udtrykket (4.1) er selve definitionen på matrixprodukt, som en matrix af række-søjleprodukter.

I (4.2), ser vi at søjlerne af AB består af A ganget på søjlerne af B. Hvert søjle b_i af B er et punkt i $\mathbb{R}^n$, og Ab_i tager dette punkt og giver et nyt punkt i $\mathbb{R}^m$. Produktet AB er så effekten af transformationen $v \mapsto Av$, $\mathbb{R}^n \to \mathbb{R}^m$, på søjlerne af B. For $m = n = 2$, er det dette syn vi havde på matrixprodukt i afsnit 2.2.

Her er et par eksempler i denne stil.

Eksempel 4.8. Lad

$$R_0 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix} \qquad \text{med } c^2 + s^2 = 1.$$

Så er R_0 en rotationsmatrix udvidet til en (3×3)-matrix. På et punkt $v = (x, y, z) \in \mathbb{R}^3$ er

$$R_0 v = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ cy - sz \\ sy + cz \end{bmatrix},$$

som er igen et punkt i $\mathbb{R}^3$. Transformationen holder x-koordinatet fast, og udfører en rotation i (y, z)-planen.

Tilsvarende er

$$R_2 = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

en rotation i (x, y)-planen og

$$R_1 = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix}$$

en rotation i (x, z)-planen. △

Eksempel 4.9. Betragt matricen

$$P = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

P er en (2×3)-matrix, så giver en transformation fra $\mathbb{R}^3$ til $\mathbb{R}^2$. Mere præcist har vi for $v = (x, y, z) \in \mathbb{R}^3$ at

$$Pv = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix},$$

dvs. at Pv består kun af de første to koordinater fra v. Se afsnit 4.6 for eksempel hvordan sådan en matrix kan bruges for 3d tegninger. △

Ligeledes beskriver udtrykket (4.3) rækkerne i matrixproduktet AB som effekten af en transformation $u^T \mapsto u^T B$, $\mathbb{R}^{1\times n} \to \mathbb{R}^{1\times r}$.

Eksempel 4.10. Følgende matrixprodukt

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \dots & 0 \\ 1 & 1 & 0 & 0 & \dots & 0 \\ 1 & 1 & 1 & 0 & \dots & 0 \\ \vdots & & & \ddots & & \\ 1 & 1 & 1 & 1 & \dots & 1 \end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ \vdots \\ v_{n-1} \end{bmatrix} = \begin{bmatrix} v_0 \\ v_0 + v_1 \\ v_0 + v_1 + v_2 \\ \vdots \\ v_0 + v_1 + v_2 + \cdots + v_{n-1} \end{bmatrix}$$

giver en løbende sum af indgangene i v. △

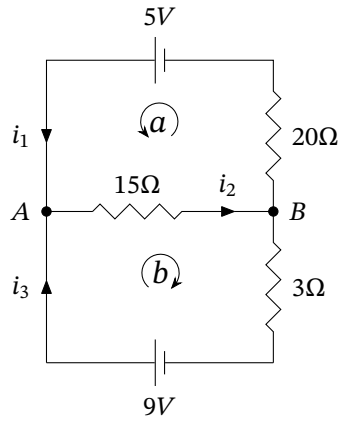
Især for matrix-vektorprodukter $Ax \in \mathbb{R}^m$, af $A \in \mathbb{R}^{m\times n}$ og $x \in \mathbb{R}^n$ er der to yderligere måde at anskue produktet. For det første, hvis vi betragter indgangene i x som variabler og vælger et fast $b \in \mathbb{R}^m$, så er

$$Ax = b$$

et lineært ligningssystem

$$\begin{aligned} a_{00}x_0 + \quad a_{01}x_1 + \cdots + \quad a_{0,n-1}x_{n-1} &= b_0, \\ a_{10}x_0 + \quad a_{11}x_1 + \cdots + \quad a_{1,n-1}x_{n-1} &= b_1, \\ \vdots \qquad \vdots \qquad \vdots \quad \vdots \\ a_{m-1,0}x_0 + a_{m-1,1}x_1 + \cdots + a_{m-1,,n-1}x_{n-1} &= b_{m-1}, \end{aligned}$$

bestående af m ligninger i de n variabler $x_0, x_1, \dots, x_{n-1}$. Sådanne systemer optræder i mange forskellige sammenhæng.



Figur 4.2: Elektrisk kredsløb.

Eksempel 4.11. Betragt det elektriske kredsløb givet i figur 4.2. Det består af to batterier og tre elektriske modstander forbundet som vist. Pilene angiver den elektriske strøm i ledningen.

Den første lov af Kirchhoff fortæller at summen af de elektriske strøm regnet med fortegn mod et knudepunkt er nul. Vores kreds har to knudepunkter markeret A og B. Kirchhoffs lov giver så to ligninger

$$\begin{aligned} \text{knudepunkt } A: &\qquad i_1 - i_2 + i_3 = 0, \\ \text{knudepunkt } B: &\qquad -i_1 + i_2 - i_3 = 0. \end{aligned}$$

Kirchhoffs anden lov siger at sum af spændings fald over komponenter i hver delløkke af kredset er lige med spændingen fra batteriet, eller nul hvis der er ingen strømkilde i delkredsen. Spændingsfaldet V over en elektrisk modstand R er givet ved Ohms lov $V = IR$, hvor I er den elektriske strøm igennem komponenten. Vores figur har to lukkede delkredse, angivet som a og b, og vi har så

$$\begin{aligned} \text{løkke } a: &\qquad 20i_1 + 15i_2 = 5, \\ \text{løkke } b: &\qquad 15i_2 + 3i_3 = 9. \end{aligned}$$

Vi har i alt 4 lineære ligninger i de 3 ubekendte i_1, i_2, i_3. Disse 4 ligninger

kan skrives som matrixligningen

$$\begin{bmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 20 & 15 & 0 \\ 0 & 15 & 3 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 5 \\ 9 \end{bmatrix}.$$

△

En anden måde at anskue produktet på, især når vi har en matrix-vektor-produkt $Ax \in \mathbb{R}^m$, af $A \in \mathbb{R}^{m\times n}$ og $x \in \mathbb{R}^n$, er at dele A op i søjler og skrive x ud i koordinater

$$Ax = \left[v_0 \mid v_1 \mid v_2 \mid \dots \mid v_{n-1} \right] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}.$$

Dette er en *lineær kombination* af søjlerne $v_0, v_1, \dots, v_{n-1}$ af A.

Eksempel 4.12. For

$$A = \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix} \quad \text{og} \quad x = \begin{bmatrix} -2 \\ 5 \\ 7 \end{bmatrix}$$

er

$$Ax = \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} -2 \\ 5 \\ 7 \end{bmatrix} = (-2) \begin{bmatrix} 3 \\ 0 \end{bmatrix} + 5 \begin{bmatrix} -1 \\ 1 \end{bmatrix} + 7 \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

△

4.3 Regneregler for matrixprodukt

Proposition 4.13. *Matrixproduktet opfylder følgende regneregler i forhold til sum og skalarmultiplikation*

(a) $A(B + C) = AB + AC$, *for* $A \in \mathbb{R}^{m\times n}$, $B, C \in \mathbb{R}^{n\times r}$,
(b) $(A + B)C = AC + BC$, *for* $A, B \in \mathbb{R}^{m\times n}$, $C \in \mathbb{R}^{n\times r}$,
(c) $A(sB) = s(AB) = (sA)B$, *for* $s \in \mathbb{R}$, $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times r}$. □

Især vigtigt er at

Sætning 4.14. *For $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times r}$, $C \in \mathbb{R}^{r\times p}$, gælder*

$$A(BC) = (AB)C.$$

Bevis. Skriv $A = (a_{ij})$, $B = (b_{jk})$, $C = (c_{k\ell})$ for $i \in \{0,\dots,m-1\}$, $j \in \{0,\dots,n-1\}$, $k \in \{0,\dots,r-1\}$, $\ell \in \{0,\dots,p-1\}$. Så er den (i,ℓ)te indgang af $A(BC)$ lige med

$$\sum_{j=0}^{n-1} a_{ij}\left(\sum_{k=0}^{r-1} b_{jk}c_{k\ell}\right).$$

Men dette udtryk er lige med

$$\sum_{j=0}^{n-1}\sum_{k=0}^{r-1} a_{ij}(b_{jk}c_{k\ell}) = \sum_{j=0}^{n-1}\sum_{k=0}^{r-1} a_{ij}b_{jk}c_{k\ell} = \sum_{k=0}^{r-1}\sum_{j=0}^{n-1} a_{ij}b_{jk}c_{k\ell} = \sum_{k=0}^{r-1}\left(\sum_{j=0}^{n-1} a_{ij}b_{jk}\right)c_{k\ell},$$

som er netop den (i,ℓ)te indgang af $(AB)C$. □

Specielt for $v \in \mathbb{R}^r$ har vi

$$A(Bv) = (AB)v,$$

som siger at transformationen $v \mapsto (AB)v$ er det samme som at transformere v under B og derefter transformere resultatet med A.

Fra de overnævnte egenskaber ser det ud til at matrix multiplikation ligner meget produkt operationen for almindelig tal, men der er nogle væsentlige forskelle.

Advarsel 4.15. Produkterne AB og BA er generelt ikke ens. !

Vi så endda i eksempel 4.6 at selvom AB er defineret, kan det være at BA ikke giver mening. Selv når begge produkter eksisterer er de oftest forskellige. Dette kan vi nemt se i python, og det er ikke et problem der skyldes unøjagtighed.

```
>>> import numpy as np
>>> a = np.array([[1.0, 2.0],
...               [0.0, 1.0]])
>>> b = np.array([[1.0, 0.0],
...               [1.0, 2.0]])
>>> a @ b
array([[3., 4.],
```

```
       [1., 2.]])
>>> b @ a
array([[1., 2.],
       [1., 4.]])
```

Advarsel 4.16. $AB = 0$ kan forekomme selvom A og B er ikke nulmatricer. !

```
>>> a = np.array([[1.0, 1.0],
...               [2.0, 2.0]])
>>> b = np.array([[1.0, 2.0],
...               [-1.0, -2.0]])
>>> a @ b
array([[0., 0.],
       [0., 0.]])
```

Som konsekvens har vi

Advarsel 4.17. $CA = CB$ medfører *ikke* at A er lig med B, selvom C er ikke en nulmatrix. !

4.4 Identitetsmatricer

Definition 4.18. Den *$(n \times n)$-identitetsmatrix* er

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & & \ddots & \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix} \in \mathbb{R}^{n \times n}.$$

Den (i, j)te indgang af I_n er 1 for $i = j$, og 0 for $i \neq j$.

Proposition 4.19. *For $A \in \mathbb{R}^{m \times n}$ er*

$$I_m A = A = A I_n. \qquad \square$$

I python er identitetsmatricer givet ved `np.eye`:

```
>>> import numpy as np
>>> a = np.array([[1.3, -3.0, 0.7, 1.4],
...               [2.0, 4.2, -5.6, 6.2],
...               [1.5, -2.5, 3.2, 5.4]])
>>> np.eye(3) @ a
array([[ 1.3, -3. ,  0.7,  1.4],
       [ 2. ,  4.2, -5.6,  6.2],
       [ 1.5, -2.5,  3.2,  5.4]])
>>> a @ np.eye(4)
array([[ 1.3, -3. ,  0.7,  1.4],
       [ 2. ,  4.2, -5.6,  6.2],
       [ 1.5, -2.5,  3.2,  5.4]])
```

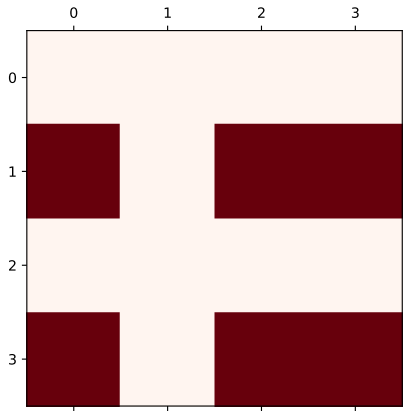
4.5 Ydre produkt

Vi begyndte kapitlet med række-søjleproduktet, som giver en skalar. Omvendt hvis vi har en søjle vektor $v \in \mathbb{R}^{m\times 1}$ og en rækkevektor $w^T \in \mathbb{R}^{1\times n}$ kan vi danne matrixproduktet $vw^T \in \mathbb{R}^{m\times n}$. Resultatet kaldes det *ydre produkt* af v med w^T. I komponenter har vi

$$
\begin{aligned}
vw^T &= \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{m-1} \end{bmatrix} \begin{bmatrix} w_0 & w_1 & \dots & w_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} v_0 w_0 & v_0 w_1 & \dots & v_0 w_{n-1} \\ v_1 w_0 & v_1 w_1 & \dots & v_1 w_{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{m-1} w_0 & v_{m-1} w_1 & \dots & v_{m-1} w_{n-1} \end{bmatrix} \in \mathbb{R}^{m\times n}.
\end{aligned}
$$

Vi kan bruge `np.matshow` til at plotte nogle eksempler på ydre produkter. Først kikker vi på nogle vektorer der kun indeholde `1.0` og `0.0`. Dette gælder også for deres ydre produkt

```
import matplotlib.pyplot as plt
import numpy as np
```



Figur 4.3: Første eksempler på et ydre produkt.

```
v = np.array([0.0, 1.0, 0.0, 1.0])[:, np.newaxis]
wt = np.array([[1.0, 0.0, 1.0, 1.0]])
print(v @ wt)
```

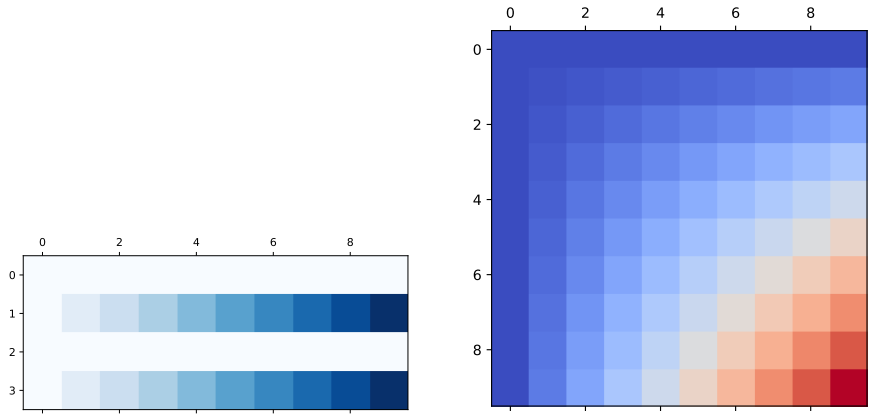
```
[[0. 0. 0. 0.]
 [1. 0. 1. 1.]
 [0. 0. 0. 0.]
 [1. 0. 1. 1.]]
```

og vi får figur 4.3, som heatmap.

```
fig, ax = plt.subplots()
ax.matshow(v @ wt, cmap='Reds')
```

Vi kan bruge `linspace` til at producere vektorer med jævnt fordelt tal fra et interval. F.eks. giver

```
w2t = np.linspace(0, 1, 10)[np.newaxis, :]
print(w2t)
```



Figur 4.4: Ydre produkter med vektorer fra `linspace`.

```
[[0.         0.11111111 0.22222222 0.33333333 0.44444444
  0.55555556 0.66666667 0.77777778 0.88888889 1.        ]]
```

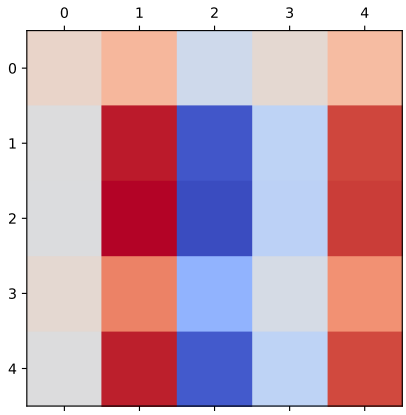
`10` tal fra `0.0` til `1.0`. Vi giver et par eksempler på ydre produkter ved en eller flere vektorer dannet af `linspace` fra nedenstående kode i figur 4.4.

```
fig, ax = plt.subplots()
ax.matshow(v @ w2t, cmap='Blues')
```

```
v2 = np.linspace(0, 1, 10)[:, np.newaxis]
fig, ax = plt.subplots()
ax.matshow(v2 @ w2t, cmap='coolwarm')
```

For det sidste eksempel bruger vi nogle tilfældige vektorer. Disse frembringes ved hjælp en tilfældighedsgenerator `rng = np.random.default_rng()` og derefter en funktionen som `rng.random()`, for en uniform fordeling på $[0, 1)$, eller `rng.standard_normal()`, for en standard normalfordeling:

```
# opstil en tilfældighedsgenerator
rng = np.random.default_rng()
# 5 tilfældige tal i [0.0, 1.0)
vr = rng.random(5)[:, np.newaxis]
print(vr)
```



Figur 4.5: Ydre produkt af tilfældige vektorer.

```
[[0.21173182]
 [0.92570366]
 [0.97017388]
 [0.52435034]
 [0.91676995]]
```

```
# 5 tilfældige tal normalfordelt med middelværdi 0
# og varians 1
wrt = rng.standard_normal(5)[np.newaxis, :]
print(wrt)
```

```
[[-0.33487529  1.88237622 -2.50102618 -0.80008783
   1.60517885]]
```

Deres ydre produkt vises i figur 4.5.

```
fig, ax = plt.subplots()
ax.matshow(vr @ wrt, cmap='coolwarm')
```

4.6 Tegning i tre dimensioner

Vi kan kombinere eksempel 4.8 og en version af eksempel 4.9 til at lave simple tegninger i tre dimensioner. Vi indlæser vores standard pakker og opstille en samling punkter i $\mathbb{R}^3$, som søjler i en matrix `punkter` i $\mathbb{R}^{3\times r}$.

```
import matplotlib.pyplot as plt
import numpy as np

punkter = np.array([[1.0, 0.0,-1.0, 0.0, 0.0, 0.0, 1.5, 0.0],
                    [0.0, 1.0, 0.0,-1.0, 0.0, 0.0, 1.5, 0.0],
                    [0.0, 0.0, 0.0, 0.0, 0.0, 5.0, 4.5, 4.0]])
```

Nu kan vi vælge at lave en objekt ved at forbinder disse punkter i en bestemt rækkefølge ved rette linjer

```
objekt = punkter[:, [0, 1, 2, 3, 0, 4, 5, 6, 7]]
```

Lad os vælge at plotte y og z koordinaterne. Så vi bruger

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

som har

$$Pv = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} y \\ z \end{bmatrix}.$$

```
p = np.array([[0.0, 1.0, 0.0],
              [0.0, 0.0, 1.0]])

fig, ax = plt.subplots()
ax.set_xlim(-5.4, 5.4)
ax.set_ylim(-5.4, 5.4)
ax.plot(*(p @ objekt))
```

Dette giver figuren øverst til venstre i figur 4.6 på side side 56. Bemærk at `p @ objekt` er en $(2 \times k)$-matrix og at `*(p @ objekt)` pakker den ud til dens to rækker.

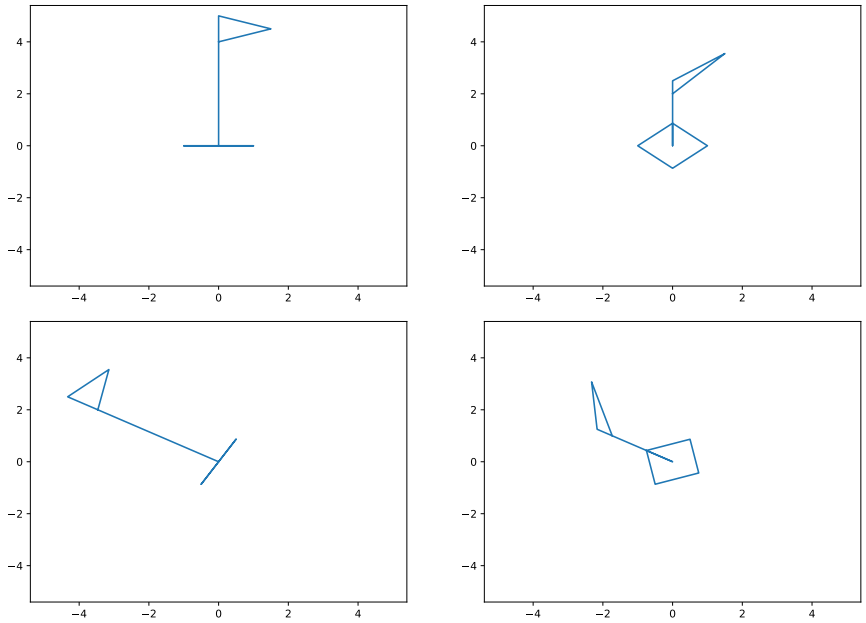
Lad os nu dreje denne objekt. `r_zero` drejer omkring x-aksen i tre dimensioner, `r_one` drejer omkring y.

```
vinkel = np.pi / 3
c = np.cos(vinkel)
s = np.sin(vinkel)

r_zero = np.array([[1.0, 0.0, 0.0],
                   [0.0,   c,  -s],
                   [0.0,   s,   c]])

r_one = np.array([[  c, 0.0,  -s],
                  [0.0, 1.0, 0.0],
                  [  s, 0.0,   c]])

fig, ax = plt.subplots()
ax.set_xlim(-5.4, 5.4)
ax.set_ylim(-5.4, 5.4)
ax.plot(*(p @ r_one @ objekt))
```

```
fig, ax = plt.subplots()
ax.set_xlim(-5.4, 5.4)
ax.set_ylim(-5.4, 5.4)
ax.plot(*(p @ r_zero @ objekt))
```

```
fig, ax = plt.subplots()
ax.set_xlim(-5.4, 5.4)
ax.set_ylim(-5.4, 5.4)
ax.plot(*(p @ r_zero @ r_one @ objekt))
```

Øverst til højre i figur 4.6 ser vi effekten af `r_one`, nederst til venstre har vi `r_zero`, og nederst til højre er effekten af `r_zero @ r_one`.



Figur 4.6: Projektion af rotationer i tre dimensioner.

Kapitel 5

Numerisk håndtering af matricer

5.1 Omkostninger ved matrixberegning

Matrixberegninger dækker over mange små aritmetiske operationer på indgangene. Det er derfor nødvendigt at have en idé om hvor mange ressourcer de beslaglægger ved computeren, og hvilke metoder er mindre ressourcekrævende end andre. Desuden har vi også set at hver operation med `float` risikerer at tilføje usikkerhed til resultatet, så det er godt at vide hvilke metoder bruger færre af disse operationer.

En simpel, men anvendelig, model, siger at hver af operationerne `+`, `-`, `*` og `/` på `float`s bruger den samme mængde tid eller ressource, kaldet en *flop*. Omkostninger ved en beregning gives så som et vist antal flops.

Som første eksempel lad os kikke på multiplikation af en vektor $v \in \mathbb{R}^{n\times 1}$ med en skalar $s \in \mathbb{R}$. Ved beregning af $u = sv$ udføres følgende operationer:

SKALAR-VEKTORPRODUKT(s, v)
1 **for** $i \in \{0, 1, \ldots, n-1\}$**:**
2 $\quad u_i = s * v_i$
3 **return** u

Dvs. for hvert heltal i fra 0 til $n-1$, udføres én `float` operation `*`. Da der er n tal i $\{0, 1, \ldots, n-1\}$, koster SKALAR-VEKTORPRODUKT(s, v) $n \times 1 = n$ flops.

For et række-søjleprodukt $u^T v$, hvor $u^T \in \mathbb{R}^{1\times n}$ og $v \in \mathbb{R}^{n\times 1}$ har vi tilsvarende

```
RÆKKE-SØJLEPRODUKT(u, v)
1 c = 0
2 for i ∈ {0, 1, ..., n − 1}:
3     c = c + u_i * v_i
4 return c
```

Denne gang er der to `float`-operationer i linje 3: en gang + og en gang *. Vi får så at RÆKKE-SØJLEPRODUKT(u, v) koster $2n$ flops.

Et matrixprodukt AB, for $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times r}$ består blot af $m \times r$ række-søjleprodukter:

```
MATRIXPRODUKT(A, B)
for i ∈ {0, 1, ..., m − 1}:
    for j ∈ {0, 1, ..., r − 1}:
        c_ij = RÆKKE-SØJLEPRODUKT(A_[i,:], B_[:,j])
return (c_ij)
```

Omkostningen er så $mr \times 2n = 2mnr$ flops.

Bemærkning 5.1. Antag at $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times r}$ og $C \in \mathbb{R}^{r\times p}$. Vi har set at matematisk er $(AB)C = A(BC)$. Men hvis vi kikker på omkostningerne ved beregning af de to udtryk ser vi en forskel. Beregningen af $(AB)C$ koster

$$2mnr + 2mrp = 2(n + p)mr \text{ flops},$$

hvorimod beregningen af $A(BC)$ koster

$$2nrp + 2mnp = 2(m + r)np \text{ flops}.$$

Da disse to udtryk er ikke ens; der kan være stor forskel i antallet af flops. ◇

Eksempel 5.2. For $A \in \mathbb{R}^{2000\times 2}$, $B \in \mathbb{R}^{2\times 1000}$, $C \in \mathbb{R}^{1000\times 100}$ har vi

$$\begin{aligned} (AB)C: &\quad 2 \times (2 + 100) \times 2000 \times 1000 = 4{,}08 \cdot 10^8 \text{ flops}, \\ A(BC): &\quad 2 \times (2000 + 1000) \times 2 \times 100 = 1{,}2 \cdot 10^6 \text{ flops}, \end{aligned}$$

så der er en faktor 400 til forskel. △

Tabel 5.1 giver et overblik over omkostninger ved vektor- og matrixregneoperationer. Det skal noteres at disse vurderinger kan reduceres i nogle situationer, især hvis matricerne eller vektorerne har en bestemt struktur, f.eks. med mange indgange lige med 0.

operation			flops
vektorsum	$u + v$	$u, v \in \mathbb{R}^n$	n
skalar-vektorprodukt	sv	$s \in \mathbb{R}, v \in \mathbb{R}^n$	n
række-søjleprodukt	$u^T v$	$u^T \in \mathbb{R}^{1\times n}, v \in \mathbb{R}^{n\times 1}$	$2n$
ydre produkt	vw^T	$v \in \mathbb{R}^{m\times 1}, w^T \in \mathbb{R}^{1\times n}$	mn
matrixsum	$A + B$	$A, B \in \mathbb{R}^{m\times n}$	mn
skalar-matrixprodukt	sA	$s \in \mathbb{R}, A \in \mathbb{R}^{m\times n}$	mn
matrix-vektorprodukt	Av	$A \in \mathbb{R}^{m\times n}, v \in \mathbb{R}^{n\times 1}$	$2mn$
matrixprodukt	AB	$A \in \mathbb{R}^{m\times n}, B \in \mathbb{R}^{n\times r}$	$2mnr$

Tabel 5.1: Omkostninger ved vektor- og matrixberegninger.

Omvendt kommer andre betragtninger i spil for meget store vektorer og matricer. En computer har typisk en begrænset mængde hukommelse, som er hurtigt tilgængeligt (cache og RAM). Dette betyder at større matricer skal nødvendigvis gemmes på en disk eller SSD. Men indhentning af data fra en disk eller SSD er væsentlige langsommere (f.eks. en ved faktor 1000) end fra cache eller RAM. Så dette vil også påvirker hvor hurtigt en beregning kan udføres.

Ud fra flops-perspektivet findes der overraskende nok hurtigere måde at gange matricer sammen end MATRIXPRODUKT ovenfor. For $A, B \in \mathbb{R}^{n\times n}$ koster MATRIXPRODUKT(A, B) $2n^3$ flops. I 1969 fandt Volker Strassen en beregningsmetode, der bruger højst $c \times n^{2,807}$ flops, hvor c er en konstant uafhængigt af n (Strassen 1969). Da der er taler om et mindre potens af n, er Strassens metode hurtigere en MATRIXPRODUKT på tilstrækkelige stor matricer, og metoden kan bruges effektivt i nogle situationer. Der findes desuden andre nyere metoder hvis omkostning afhænger af en endnu mindre potens af n, f.eks. $c \times n^{2,371\,552}$ (Williams m.fl. 2023), men de har mest teoretisk interesse.

Pythons matrixprodukt bygger på et standard bibliotek BLAS (Basic Linear Algebra Subprograms, `http://www.netlib.org/blas/`), der bruges af mange software pakker for lineær algebra i forskellige programmeringssprog.

5.2 Pythons for-løkke

I de overstående eksempler har vi ikke skrevet algoritmerne i python med i generisk kode. Syntaksen er dog alligevel tæt på en konstruktion i python, som har den generelle form

```
for variabel in objekt:
    udfør forskellige trin
    der muligvis bruger variabel
```

Mellemrum i koden ovenpå er væsentlig. Disse vises eksplicit nedenfor

```
1 for␣variabel␣in␣objekt:
2 ␣␣␣␣udfør␣forskellige␣trin
3 ␣␣␣␣der␣muligvis␣bruger␣variabel
```

Det vigtige er at linjerne 2, 3, . . ., der skal udføres i løkken, er rykket 4 mellemrum til højre i forhold til selve `for ... in ... :` linjen 1. Ligeledes er skrifttegnet `:` ved slutningen af linje 1 nødvendig.

Objektet der typisk bruges i `for`-løkker er `range(...)`. For eksempel, skriver man `range(stop)` med `stop` et positivt heltal, fås rækken af heltal fra `0` til `stop-1`:

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

Man kan også skrive `range(start, stop)` for at angive en anden begyndelsesværdi end `0`:

```
>>> for i in range(5, 10):
...     print(i)
...
5
6
7
8
9
```

Desuden kan man given en anden trinstørrelse via et tredje argument, som `range(start, stop, step)`:

```
>>> for i in range(0, 10, 2):
...     print(i)
...
0
2
4
6
8
```

Nu kan vi udregne række-søjleproduktet ovenfor med den følgende python kode

```
import numpy as np
rng = np.random.default_rng()
n = 20
ut = rng.random((1, n))
v = rng.random((n, 1))
c = 0
for i in range(n):
    c = c + ut[0, i] * v[i, 0]
print(c)
```

som giver

```
4.540319907840884
```

NumPys egen `@`-operation er skrevet specielt til at gøre den type beregning mere effektivt, med mere kompakt syntaks, og kan tage højde for korrekt brug af `float`-operationer. Vi kan lige tjekke at vores svar er nogenlunde rigtig

```
c_numpy = (ut @ v)[0,0]
print(c_numpy)
```

som giver

```
4.540319907840883
```

med forskel

```
print(c - c_numpy)
```

```
8.881784197001252e-16
```

Generelt skal **for**-løkker ikke bruges til at erstatte operationer tilgængelig fra NumPy, men det kan være nyttigt når man afprøver nye beregningsmetode og i forbindelse med plot af figurer.

Som eksempel lad os illustrere effekten af gentagende brug af en rotationsmatrix på en simple figur. Vi begynder med en figur i planen angivet ved søjlerne af en (2×4)-matrix `points`

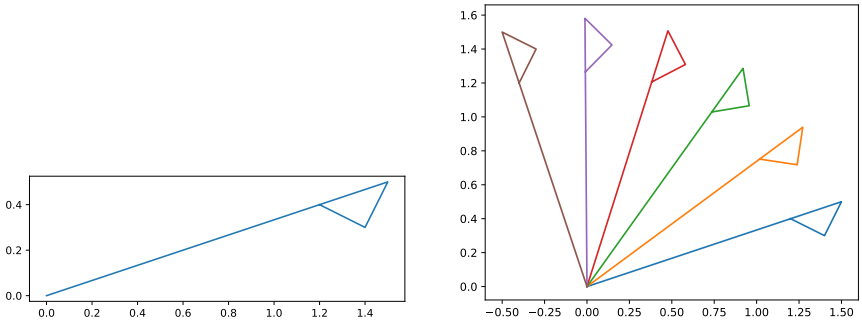
```
import matplotlib.pyplot as plt
import numpy as np

points = np.array([[0.0, 1.5, 1.4, 1.2],
                   [0.0, 0.5, 0.3, 0.4]])
fig, ax = plt.subplots()
ax.set_aspect('equal')
ax.plot(*points)
```

som giver det første billede i figur 5.1.

Bemærk at vi har brugt lidt andet syntaks en tidligere, hvor en plot var typisk `ax.plot(x, y)`. I vores tilfælde er `x` og `y` rækkerne `points[0]` og `points[1]`, og `ax.plot(*points)` er en kort form for `ax.plot(points[0], points[1])`. Dvs. `*points` pakker `points` ud til `points[0], points[1]`.

Nu stiller vi en rotationsmatrix op og bruger den til at dreje figuren 5 gange i en **for**-løkke:

```
c = np.cos(np.pi / 10)
s = np.sin(np.pi / 10)
R = np.array([[c, -s],
              [s,  c]])

fig, ax = plt.subplots()
ax.set_aspect('equal')
ax.plot(*points)
```



Figur 5.1: Oprindelig figur og dens rotationer.

```
for i in range(5):
    points = R @ points
    ax.plot(*points)
```

Resultatet er det andet billede i figur 5.1.

Kapitel 6

Lineære ligningssystemer

Vi har set tidligere hvordan en simpel elektrisk kreds giver anledning til et lineært ligningssystem. I dette kapitel kikker vi på nogle matematiske metoder for at forstå og løse sådanne systemer eksakt. Det skal bemærkes at umiddelbart er disse metoder ikke så hensigt mæssig for numerisk beregning med `float`-tal, men der et par udsagn, som har væsentlig betydning for metoderne der bruges senere og som skal være på plads inden vi kan komme til det numeriske.

6.1 Elementære rækkeoperationer

Betragt det lineære ligningssystem

$$2x + 3y - 4z = 7, \tag{6.1a}$$
$$3x - 4y + z = -2, \tag{6.1b}$$
$$x + y + 2z = 3. \tag{6.1c}$$

Dette er et system af 3 lineære ligninger i de 3 ubekendte x, y og z. Vi er interesseret i at finde alle løsninger (x, y, z) til systemet.

Vi kunne løse systemet eksempelvis på den følgende måde: Først isolerer vi variablen z i (6.1c). Derefter indsætter vi dette udtryk for z i (6.1a) og (6.1b). Vi får $z = 3/2 - x/2 - y/2$ og (6.1a)–(6.1b) bliver

$$2x + 3y - 4\left(\frac{3}{2} - \frac{1}{2}x - \frac{1}{2}y\right) = 7,$$
$$3x - 4y + \left(\frac{3}{2} - \frac{1}{2}x - \frac{1}{2}y\right) = -2,$$

som forkortes og omskrives til

$$4x + 5y = 13, \tag{6.2a}$$
$$\frac{5}{2}x - \frac{7}{2}y = -\frac{7}{2}. \tag{6.2b}$$

Processen kan så gentages ved at isolere y i (6.2b), og indsætte udtrykket i (6.2a). Der fås så én ligning med kun variablen x, som kan løses for x, og derefter kan man finde først y og så z ved at indsætte denne x-værdi.

En mere systematisk fremgangsmåde baserer sig på den følgende observation. Der er tre simple operationer vi kan udføre på systemet uden at ændre mængden af løsninger, nemlig

(I) byt om på rækkefølgen af ligningerne,
(II) gang en ligning igennem med en skalar som er ikke 0, eller
(III) tilføj et multiplum af én ligning til en anden ligning.

Lad os skifte til matrixnotation for at lette arbejdet. Systemet (6.1a)–(6.1c) er ækvivalent med matrixligningen

$$\begin{bmatrix} 2 & 3 & -4 \\ 3 & -4 & 1 \\ 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 7 \\ -2 \\ 3 \end{bmatrix}.$$

Vi kan samle koefficienterne i en *udvidet matrix*

$$\left[\begin{array}{ccc|c} 2 & 3 & -4 & 7 \\ 3 & -4 & 1 & -2 \\ 1 & 1 & 2 & 3 \end{array}\right] \tag{6.3}$$

De tre elementære ligningsoperationer ovenfor er nu det samme som de følgende tre *elementære rækkeoperationer* på den udvidede matrix:

(I)	$R_i \leftrightarrow R_j$		byt række i med række j,
(II)	$R_i \longrightarrow sR_i$	$s \neq 0$	skalér R_i med en faktor s,
(III)	$R_i \longrightarrow R_i + tR_j$	$j \neq i$	læg t gange række j til række i.

Disse operationer kan bruges på (6.3), f.eks. på den følgende måde. Vores mål

er at have 0-tal under diagonalen.

$$\begin{aligned}
&\left[\begin{array}{ccc|c} 2 & 3 & -4 & 7 \\ 3 & -4 & 1 & -2 \\ 1 & 1 & 2 & 3 \end{array}\right] \\
&\sim_{R_0 \leftrightarrow R_2} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 3 & -4 & 1 & -2 \\ 2 & 3 & -4 & 7 \end{array}\right] \sim_{R_1 \to R_1 - 3R_0} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & -7 & -5 & -11 \\ 2 & 3 & -4 & 7 \end{array}\right] \\
&\sim_{R_2 \to R_2 - 2R_0} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & -7 & -5 & -11 \\ 0 & 1 & -8 & 1 \end{array}\right] \sim_{R_1 \leftrightarrow R_2} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & 1 & -8 & 1 \\ 0 & -7 & -5 & -11 \end{array}\right] \\
&\sim_{R_2 \to R_2 + 7R_1} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & 1 & -8 & 1 \\ 0 & 0 & -61 & -4 \end{array}\right]
\end{aligned} \tag{6.4}$$

Bemærk den sidste udvidede matrix svarer til ligningssystemet

$$\begin{aligned} x + y + \ \ 2z &= 3, \\ y - \ \ 8z &= 1, \\ -61z &= -4. \end{aligned}$$

Dette reduceret system kan nu løsers via *back substitution*: den sidste ligning siger $-61z = -4$, så $z = 4/61$, den anden ligning giver $y = 1+8z = (61+32)/61 = 93/61$ og nu den første fører til $x = 3 - y - 2z = (183 - 93 - 8)/61 = 82/61$. Dvs.

$$x = \frac{82}{61}, \quad y = \frac{93}{61}, \quad z = \frac{4}{61}. \tag{6.5}$$

Som alternativ kan udføre denne back substitution ved at forsætte med (6.4), og bruge yderligere rækkeoperationer til at få 1-tal på diagonalen og 0-tal i de andre indgange i koefficientmatricen:

$$\begin{aligned}
&\left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & 1 & -8 & 1 \\ 0 & 0 & -61 & -4 \end{array}\right] \\
&\sim_{R_2 \to -R_2/61} \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 0 & 1 & -8 & 1 \\ 0 & 0 & 1 & 4/61 \end{array}\right] \sim_{R_0 \to R_0 - R_1} \left[\begin{array}{ccc|c} 1 & 0 & 10 & 2 \\ 0 & 1 & -8 & 1 \\ 0 & 0 & 1 & 4/61 \end{array}\right] \\
&\sim_{R_0 \to R_0 - 10R_2} \left[\begin{array}{ccc|c} 1 & 0 & 0 & 82/61 \\ 0 & 1 & -8 & 1 \\ 0 & 0 & 1 & 4/61 \end{array}\right] \sim_{R_1 \to R_1 + 8R_2} \left[\begin{array}{ccc|c} 1 & 0 & 0 & 82/61 \\ 0 & 1 & 0 & 93/61 \\ 0 & 0 & 1 & 4/61 \end{array}\right]
\end{aligned} \tag{6.6}$$

	operation	python
(I)	$R_i \leftrightarrow R_j$	`a[ [i,j], :] = a[ [j,i], :]`
(II)	$R_i \to sR_i$ $(s \neq 0)$	`a[i, :] *= s`
(III)	$R_i \to R_i + tR_j$ $(j \neq i)$	`a[i, :] += t * a[j, :]`

Tabel 6.1: Rækkeoperationer i python.

Fra den sidste udvidede matrix kan man direkte aflæse løsningerne (6.5).

6.2 Rækkeoperationer i python

Lad os se hvordan sådanne rækkeoperationer kan udføres i python. Husk at `a[i, :]` er den $i$te række i `a`. Først kan rækkeoperation (II) $R_i \to sR_i$ udføres via `a[i, :] = s * a[i, :]`. Men der er en nyttig forkortelse `*=`, som også hjælper med at undgå slåfejl: generelt i python er `x *= s` er det samme som `x = x * s`. Vi kan derfor lave rækkeoperation (II), som

```
a[i, :] *= s
```

Tilsvarende for operation (III) kan vi bruge `x += y` som forkortelse af `x = x + y`, så operation (III) $R_i \to R_i + tR_j$ udføres via

```
a[i, :] += t * a[j, :]
```

For rækkeoperation (I) er der en nyttig udvidet indekserings notation: `a[ [i,j], :]` giver nemlig rækker `i` og `j` fra `aub` i den givne rækkefølge. Så operation (I) $R_i \leftrightarrow R_j$ kan laves via

```
a[ [i,j], :] = a[ [j,i], :]
```

Disse rækkeoperationer er opsummeret i tabel 6.1.

Vi kan bruge disse operationer til at lave reduktionen (6.3) i python. Først lad os se hvordan man danner den udvidede matrix ved hjælpe af `np.hstack`. Kommandoen `np.hstack([a, b])` giver en ny `array` med søjler fra `a` efterfulgt af søljer fra `b`.

```
>>> import numpy as np
>>> a = np.array([[2.0,  3.0, -4.0],
...               [3.0, -4.0,  1.0],
...               [1.0,  1.0,  2.0]])
>>> a
array([[ 2.,  3., -4.],
       [ 3., -4.,  1.],
       [ 1.,  1.,  2.]])
>>> b = np.array([7.0, -2.0, 3.0])[:, np.newaxis]
>>> b
array([[ 7.],
       [-2.],
       [ 3.]])
>>> aub = np.hstack([a, b])
>>> aub
array([[ 2.,  3., -4.,  7.],
       [ 3., -4.,  1., -2.],
       [ 1.,  1.,  2.,  3.]])
```

Derefter bruger vi rækkeoperationerne fra tabel 6.1:

```
>>> aub
array([[ 2.,  3., -4.,  7.],
       [ 3., -4.,  1., -2.],
       [ 1.,  1.,  2.,  3.]])
>>> aub[ [0,2], :] = aub[ [2,0], :]
>>> aub
array([[ 1.,  1.,  2.,  3.],
       [ 3., -4.,  1., -2.],
       [ 2.,  3., -4.,  7.]])
>>> aub[1, :] += -3*aub[0, :]
>>> aub
array([[  1.,   1.,   2.,   3.],
       [  0.,  -7.,  -5., -11.],
       [  2.,   3.,  -4.,   7.]])
>>> aub[2, :] += -2*aub[0, :]
>>> aub
```

```
array([[  1.,   1.,   2.,   3.],
       [  0.,  -7.,  -5., -11.],
       [  0.,   1.,  -8.,   1.]])
>>> aub[ [1,2], :] = aub [ [2,1], :]
>>> aub
array([[  1.,   1.,   2.,   3.],
       [  0.,   1.,  -8.,   1.],
       [  0.,  -7.,  -5., -11.]])
>>> aub[2, :] += 7*aub[1, :]
>>> aub
array([[  1.,   1.,   2.,   3.],
       [  0.,   1.,  -8.,   1.],
       [  0.,   0., -61.,  -4.]])
```

Man kan forsætte i samme stil hvis man vil udføre operationerne i (6.6).

6.3 Echelonform

Generelt kan man ikke reducere en koefficientmatrix til en form med 1-tal på diagonalen, men man kan komme tæt på. Følgende matrix er i det der kaldes »echelonform«:

$$\begin{bmatrix} 1 & 0 & 2{,}3 & -1{,}2 & 0{,}5 \\ 0 & 0 & 1 & 3{,}2 & -2{,}2 \\ 0 & 0 & 0 & 1 & 0{,}3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \tag{6.7}$$

Definition 6.1. En matrix er i *echelonform* hvis alle de følgende tre krav er opfyldte:

(a) Den første ikke nul indgang i hver række er lige med 1. Indgangen kaldes et *pivotelement*.
(b) Pivot elementer ligger længere til højre i efterfølgende rækker.
(c) Eventuelle nulrækker kommer til sidst.

I vores matrix er pivotelementerne fremhævet nedenfor

$$\begin{bmatrix} 1 & 0 & 2{,}3 & -1{,}2 & 0{,}5 \\ 0 & 0 & 1 & 3{,}2 & -2{,}2 \\ 0 & 0 & 0 & 1 & 0{,}3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

og vi ser at alle indgange under den blå trappesti dannet af pivotelementerne er 0.

Proposition 6.2. *Enhver matrix kan reduceres til echelonform via rækkeoperationer.*

Bevis. Givet en matrix $A \in \mathbb{R}^{m\times n}$, find den første søjle som er ikke $0_{m\times 1}$ og vælg et element i denne søjle som er forskellig fra 0. Brug rækkeoperation (I) til at flytte dette element til den første række. Så har vi en matrix af formen

$$\begin{bmatrix} 0 & \dots & 0 & \times & * & \dots & * \\ 0 & \dots & 0 & * & * & \dots & * \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ 0 & \dots & 0 & * & * & \dots & * \end{bmatrix},$$

hvor $\times$ betegner et element forskelligt fra 0, og $*$ angiver vilkårlige tal.

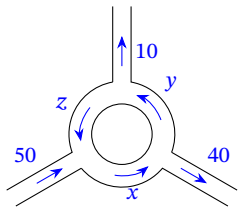
Brug rækkeoperation (II) til at få et 1-tal i position $\times$. Hvis $m = 1$ har vi nu en matrix i echelonform og vi er færdig.

Hvis $m > 1$, bruge nu rækkeoperation (III) op til $(m-1)$-gange til at få en ny matrix med alle elementer i søjlen under 1-tallet lige med 0:

$$\begin{bmatrix} 0 & \dots & 0 & 1 & * & \dots & * \\ \hdashline 0 & \dots & 0 & 0 & * & \dots & * \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ 0 & \dots & 0 & 0 & * & \dots & * \end{bmatrix} = \begin{bmatrix} u^T \\ \hdashline B \end{bmatrix}. \tag{6.8}$$

Matricen B under den punkterede linje har $B \in \mathbb{R}^{(m-1)\times n}$, så har færre rækker end A. Gentages den overstående proces på B kan vi rækkereducerer B til echelonform. Bemærk at pivotelementer fra B ligger til højre fra pivotelementet 1 i rækken u^T i (6.8). Så ved at reducere B til echelonform får vi en echelonform for A. □

Bemærk at denne proces er næste entydig. Vi har blot truffet et valg af ikke-nul element i den første ikke-nul søjle Vælges elementet men rækkeindeks mindst muligt, fås et entydigt valg. For `float` beregninger er det dog nok mere fornuftigt at vælge elementet i søjlen er der størst numerisk.



Figur 6.1: Rundkørsel.

6.4 Løsning af lineære ligningssystemer via rækkeoperationer

For matricen (6.7) i echelonform, er det tilsvarende system af lineære ligninger af formen

$$\begin{aligned} x_0 + 2{,}3x_2 - 1{,}2x_3 + 0{,}5x_4 &= 1{,}2, \\ x_2 + 3{,}2x_3 - 2{,}2x_4 &= 0{,}7, \\ x_3 + 0{,}3x_4 &= -0{,}2, \\ 0 &= 0, \end{aligned}$$

hvor vi har indsat nogle tilfældige værdier på den højre side. I dette system ser vi at hvis vi giver x_4, og det usynlige x_1, vilkårlige værdier, så kan vi bestemme x_3, x_2 og til sidst x_0 via back substitution. Vi siger at variablerne x_1, x_4 er *frie*, og at x_0, x_2, x_3 er *bundne*. Systemet har så uendelig mange løsninger $(x_0, x_1, x_2, x_3, x_4)$, da x_1, x_4 kan tage hvilket som helst reelle værdier. Bemærk at de bundne variabler svar netop til pivotsøjlerne, så

(a) antallet af bundne variabler er antallet af pivotelementer, og
(b) antallet af frie variabler er antallet af søjler i koefficientmatricen minus antallet af pivotelementer.

Eksempel 6.3. Betragt rundkørslen i figur 6.1. På de ensrettede tilsluttende vejer er antallet af biler per time målt som angivet. Antallet x, y, z af biler per time på de forskellige deler af selve rundkørslen opfylder

$$z + 50 = x, \quad y + 40 = x \quad \text{og} \quad z + 10 = y.$$

Dette omskrives til systemet

$$\begin{aligned} x \qquad - z &= 50, \\ x - y \qquad &= 40, \\ y - z &= 10. \end{aligned}$$

Den tilsvarende udvidede matrix er

$$\left[\begin{array}{ccc|c} 1 & 0 & -1 & 50 \\ 1 & -1 & 0 & 40 \\ 0 & 1 & -1 & 10 \end{array}\right] \sim_{R_1 \to R_1 - R_0} \left[\begin{array}{ccc|c} 1 & 0 & -1 & 50 \\ 0 & -1 & 1 & -10 \\ 0 & 1 & -1 & 10 \end{array}\right]$$

$$\sim_{R_2 \to R_2 + R_1} \left[\begin{array}{ccc|c} 1 & 0 & -1 & 50 \\ 0 & -1 & 1 & -10 \\ 0 & 0 & 0 & 0 \end{array}\right] \sim_{R_1 \to -R_1} \left[\begin{array}{ccc|c} 1 & 0 & -1 & 50 \\ 0 & 1 & -1 & 10 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

Systemet er konsistent. Pivotelementer er søjler 0 og 1, så z er fri, og x, y er bundne. Den generelle løsning til systemet er

$$x = 50 + z, \quad y = 10 + z, \qquad z \in \mathbb{R}.$$

Realistiske løsninger skal dog være positive så vi må begrænse os til $z \geqslant 0$. $\triangle$

Generelt givet et lineært ligningssystem

$$\begin{aligned} a_{00}x_0 + a_{01}x_1 + \cdots + a_{0,n-1}x_{n-1} &= b_0, \\ a_{10}x_0 + a_{11}x_1 + \cdots + a_{1,n-1}x_{n-1} &= b_1, \\ &\vdots \\ a_{m-1,0}x_0 + a_{m-1,1}x_1 + \cdots + a_{m-1,n-1}x_{n-1} &= b_{m-1}, \end{aligned}$$

skriver vi det i matrixform

$$Ax = b$$

med $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ givne, og $x \in \mathbb{R}^n$ ukendt. Vi danner så den udvidede matrix

$$\left[A \mid b\right]$$

og reducerer til echelonform

$$\left[C \mid d\right]$$

Vores oprindelig lineære ligningssystem har de samme løsninger som systemet

$$Cx = d. \tag{6.9}$$

For et generelt system er der tre muligheder

(a) $\left[C \mid d\right]$ har en række af formen

$$\left[0 \quad 0 \quad \dots \quad 0 \mid 1\right]$$

Dette betyder at den tilsvarende ligning siger $0 = 1$, som er en modstrid. Så har systemet $Ax = b$ ingen løsninger. Systemet siges at være *inkonsistent*.

(b) Systemet er konsistent og et af de følgende to tilfælde gælder:
   (i) Antallet af pivotelementer er lige med antallet af søjler. Så er der ingen frie variabler, alle variabler er bundne og systemet har en entydig løsning.
   (ii) Der er strengt færre pivotelementer end søjler. Så giver de andre søjler frie variabler og der er uendelige mange løsninger til system. Systemet er *underbestemt*.

Vi har derfor vist at:

Sætning 6.4. *Et lineært ligningssystem enten*
(a) *er en inkonsistent, eller*
(b) (i) *har præcis én løsninger, eller*
   (ii) *har uendelig mange løsninger bestemt af frie variabler.* □

Bemærkning 6.5. For et konsistent system af m ligninger i n variabler, med $m < n$, er der altid frie variabler og så uendelig mange løsninger. ◇

Eksempel 6.6. For systemer i to variabler kan vi tegne os frem til de forskellige udfald. En lineær ligning $ax + by = c$ beskriver er en linje i planen, som vi så i eksempel 4.4. Løsningen på et system af sådanne ligninger er koordinaterne på punkter der ligger på alle de beskrevne linjer i planen, dvs. koordinater på fælles skæringspunkter.

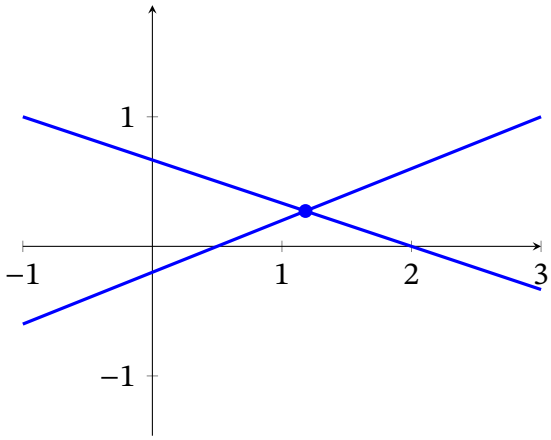
Systemet

$$\begin{aligned} x+3y &= 2, \\ 2x-5y &= 1 \end{aligned}$$

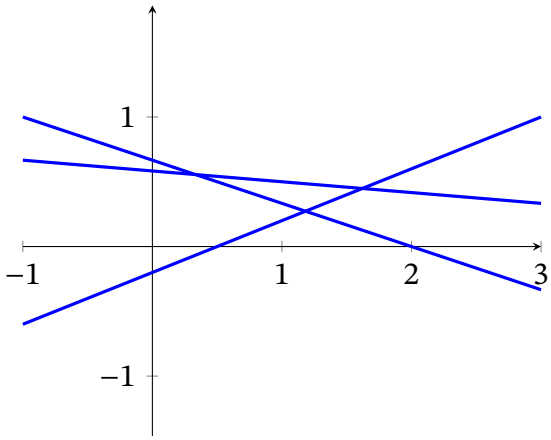
vises i figur 6.2, og det ses et der er netop ét skæringspunkt, dvs. en entydig løsning på systemet.

Derimod for systemet

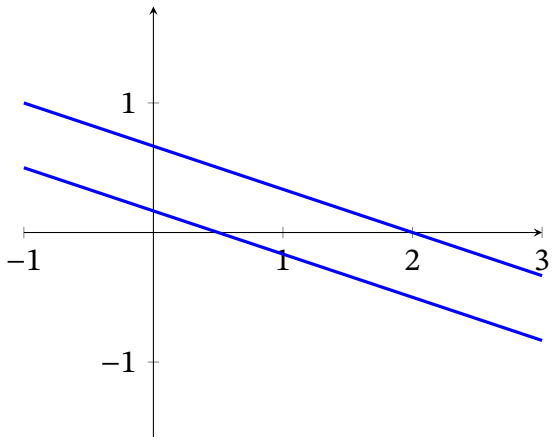
$$\begin{aligned} x+3y &= 2, \\ 2x-5y &= 1, \\ x+6y &= 5, \end{aligned}$$



Figur 6.2: To linjer i planen, som skærer.



Figur 6.3: Tre linjer i planen, som har intet fællespunkt.



Figur 6.4: To parallelle linjer i planen har intet fællespunkt.

er tegningen, som i figur 6.3. Vi ser at de tre linjer har intet fællespunkt, systemet har ingen løsning og er inkonsistent.

En anden situation uden fællesløsninger er

$$\begin{aligned} x+3y &= 2, \\ 2x+6y &= 1. \end{aligned}$$

Som det ses i figur 6.4, er de to linjer parallelle, så har ingen skæringspunkt. Her er systemet inkonsistent. Men hvis vi justerer konstanterne på højre siden til

$$\begin{aligned} x+3y &= 2, \\ 2x+6y &= 4, \end{aligned}$$

så beskriver begge ligninger den samme linje i planen, og alle punkter på linjen er løsninger på systemet. Dvs. dette er et systemet med uendelige mange løsninger. △

Kapitel 7

Matrixinvers

For reelle tal er det meget nyttigt at man kan dele et tal med et andet tal s, som er ikke nul. Dette er det sammen som at gange med den reciprokke $1/s$. For matricer er der en tilsvarende matrix, den inverse, som er især nyttig i eksakte regnestykker. Men modsat reelle tal er der mange matricer der ikke tillader en invers. For det første skal matricen være kvadratisk (definition 7.1 nedenfor), og for kvadratiske matricer er der ydeligere krav, der skal opfyldes.

7.1 Egenskaber af den inverse matrix

Definition 7.1. En matrix A er *kvadratisk* hvis den har det samme antal rækker som søjler. Dvs. en kvadratisk matrix er et $A \in \mathbb{R}^{n\times n}$.

Bemærk at identitetsmatricen I_n er kvadratisk og for alle kvadratiske matricer $A \in \mathbb{R}^{n\times n}$ af samme størrelse som I_n har vi

$$I_n A = A = A I_n,$$

som i proposition 4.19.

Sætning 7.2. *Lad $A \in \mathbb{R}^{n\times n}$ være en kvadratisk matrix. Hvis der findes en matrix $A^{-1} \in \mathbb{R}^{n\times n}$ således at*

$$A^{-1}A = I_n \tag{7.1}$$

så

(a) *er A invertibel,*

(b) *er A^{-1} den eneste matrix der opfylder (7.1), og*

(c) *gælder der*

$$AA^{-1} = I_n. \tag{7.2}$$

Omvendt, hvis en matrix A^{-1} opfylder (7.2), så er den entydigt bestemt og opfylder (7.1).

Definition 7.3. Når den eksisterer kalder vi matricen A^{-1} i sætning 7.2 den *inverse* til A.

Beviset for sætning 7.2 gives senere i kapitlet, se afsnit 7.2. Her giver vi nogle eksempler og fokuserer på nogle egenskaberne for den inverse A^{-1}.

Bemærkning 7.4. For $A \in \mathbb{R}^{2\times 2}$ er der en simpel formel for A^{-1}. Matricen

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

har invers

$$A^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \qquad \text{når } ad - bc \neq 0.$$

Dette kan bekræftes direkte ved beregning af matrixproduktet (7.1):

$$\begin{aligned} A^{-1}A &= \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \frac{1}{ad - bc}\begin{bmatrix} da - bc & db - bd \\ -ca + ac & -cb + ad \end{bmatrix} \\ &= \frac{1}{ad - bc}\begin{bmatrix} ad - bc & 0 \\ 0 & ad - bc \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2. \end{aligned}$$

Ligningen (7.2) kan bekræftes ved en tilsvarende udregning. Bemærk betingelsen $ad - bc \neq 0$ for eksistens af den inverse i disse tilfælde. Tallet $ad - bc$ kaldes determinanten af $A \in \mathbb{R}^{2\times 2}$. ◇

Givet et lineært ligningssystem

$$Ax = b$$

med $A \in \mathbb{R}^{n\times n}$ kvadratisk og invertibel, kan man bruge den inverse til at løse systemet. Vi ganger begge sider igennem med A^{-1}, til at få

$$A^{-1}Ax = A^{-1}b. \tag{7.3}$$

Men $A^{-1}A = I_n$, så $A^{-1}Ax = I_n x = x$, og ligning (7.3) er blot

$$x = A^{-1}b.$$

Dvs. vi har fundet den entydige løsning til systemet. Dette er ofte af teoretisk interesse; i en numerisk sammenhæng er det generelt bedst at undgå den inverse matrix og bruge andre metoder for sådanne systemer, som ikke er så følsom over for unøjagtigheder.

Eksempel 7.5. Betragt systemet

$$\begin{aligned} 2x + 3y &= 1, \\ x + 2y &= 2. \end{aligned}$$

I matrixform er dette

$$\begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Koefficientmatricen har $ad - bc = 2 \times 2 - 3 \times 1 = 1$, som er forskellig fra nul, så matricen er invertibel. Vi får så

$$\begin{aligned} \begin{bmatrix} x \\ y \end{bmatrix} &= \begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix}^{-1} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{2 \times 2 - 3 \times 1} \begin{bmatrix} 2 & -3 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{1} \begin{bmatrix} 2 \times 1 - 3 \times 2 \\ -1 \times 1 + 2 \times 2 \end{bmatrix} \\ &= \begin{bmatrix} -4 \\ 3 \end{bmatrix}. \end{aligned}$$

Dvs. $x = -4$, $y = 3$. △

Transformationen $b \mapsto A^{-1}b$ er den omvendte transformation af $x \mapsto Ax$. Så f.eks. den inverse til en rotation i $\mathbb{R}^2$ er den samme rotation i den modsatte retning. Som matricer har rotationsmatricen

$$R = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}, \qquad c^2 + s^2 = 1$$

invers

$$R^{-1} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}.$$

Bemærk at i dette tilfælde fås R^{-1} fra R via simpel ombytning af indgange. Generelt er beregning af den inverse væsentlig mere kompliceret, og resultatet kan være svært at kontrollere. Senere vil vi have særlig interesse for matricer hvor den inverse er nemt tilgængelig.

Der er en simpel formel for den inverse af et produkt af matricer:

Proposition 7.6. *Hvis $A, B \in \mathbb{R}^{n \times n}$ er invertibel, så er AB invertibel med invers*

$$(AB)^{-1} = B^{-1}A^{-1}.$$

Bevis. Det er nok at regne

$$(B^{-1}A^{-1})(AB) = B^{-1}(A^{-1}A)B = B^{-1}I_nB = B^{-1}B = I_n,$$

så $B^{-1}A^{-1}$ er den inverse til AB. □

Matrixinvers kan beregnes i python via `np.lingalg.inv`. Her kan vi begynde at ane nogle af de numeriske problemer med den inverse.

```
>>> import numpy as np

>>> a = np.array([[1.0,          1.0],
...                [1.000000001, 1.0]])
>>> a
array([[1., 1.],
       [1., 1.]])
>>> np.linalg.inv(a)
array([[-9.99999916e+08,  9.99999916e+08],
       [ 9.99999917e+08, -9.99999916e+08]])
```

Vi ser at selvom matricen `a` har alle indgange af størrelse 1, har den inverse indgange af størrelsesorden 10^9. Dette stammer fra at i beregningen af den inverse kommer vi til at gange igennem med

$$\frac{1}{ad-bc} = \frac{1}{1.0 - 1.000000001} = \frac{1}{-0.000000001} = -1{,}0 \cdot 10^9.$$

Et eksempel af en anden type er

```
>>> n = 100
>>> a = np.triu(2*np.eye(n) - np.ones((n,n)))
>>> a
array([[ 1., -1., -1., ..., -1., -1., -1.],
       [ 0.,  1., -1., ..., -1., -1., -1.],
       [ 0.,  0.,  1., ..., -1., -1., -1.],
       ...,
       [ 0.,  0.,  0., ...,  1., -1., -1.],
       [ 0.,  0.,  0., ...,  0.,  1., -1.],
       [ 0.,  0.,  0., ...,  0.,  0.,  1.]])
>>> np.linalg.inv(a)[0,-1]
3.1691265005705735e+29
```

Matricen a har -1 i alle pladser over diagonalen, 1 på diagonalen, og 0 under diagonalen. Så alle indgange har størrelse højst 1. Men den inverse har en indgang der er størrelsesorden 10^{29}. I dette eksempel indeholder den inverse matrix faktisk elementer med alle størrelsesordner mellem 1 og 10^{29}.

7.2 Eksistens af den inverse

Definition 7.7. En kvadratisk matrix $A \in \mathbb{R}^{n\times n}$ er *invertibel* hvis der findes et $B \in \mathbb{R}^{n\times n}$ således at

$$BA = I_n = AB. \tag{7.4}$$

Lemma 7.8. *Matricen B i (7.4) er entydig.*

Bevis. Antag at $C \in \mathbb{R}^{n\times n}$ opfylder (7.4) og

$$CA = I_n = AC.$$

Så har vi

$$B = BI_n = B(AC) = (BA)C = I_nC = C,$$

dvs. $C = B$. Så B er entydig. □

Når vi har dette resultat, giver det god mening at skrive $A^{-1} = B$ og at kalde A^{-1} den *inverse* til A. Hvis A er ikke invertibel siges vi at A er *singulær*.

Lemma 7.9. *Hvis $A \in \mathbb{R}^{n\times n}$ er invertibel, så har det lineære ligningssystem*

$$Ax = 0, \qquad x \in \mathbb{R}^n \tag{7.5}$$

kun én løsning, nemlig $x = 0 \in \mathbb{R}^n$.

Bevis. Bemærk først at $x = 0$ er en løsning til ligningen, da $A0 = 0$.

Nu lad os gange A^{-1} på (7.5) fra den venstre side. Vi får

$$A^{-1}Ax = 0.$$

Men $A^{-1}A = I_n$, så $0 = A^{-1}Ax = I_nx = x$. Dvs. $x = 0 \in \mathbb{R}^n$ er den eneste løsning. □

Bemærkning 7.10. Beviset for lemma 7.9 bruger kun ligningen $A^{-1}A = I_n$. Så for vilkårlig A hvis B er en matrix således at $BA = I_n$, kan vi også konkludere at ligningen $Ax = 0$ har kun $x = 0$ som løsning. ◇

Hvis ligningssystemet $Ax = 0$ har kun én løsning, så indeholder hver søjle ét pivotelement. For $A \in \mathbb{R}^{n\times n}$, betyder dette at A har echelonform

$$\begin{bmatrix} 1 & * & * & \cdots & * \\ 0 & 1 & * & \cdots & * \\ 0 & 0 & 1 & & * \\ \vdots & \vdots & & \ddots & \\ 0 & 0 & 0 & & 1 \end{bmatrix}. \tag{7.6}$$

Nu kan vi bruge yderlige rækkeoperationer af type (III) på (7.6) til at sætte alle elementer $*$ til 0, og dermed reducerer (7.6) til identitetsmatricen I_n. I andre ord, A kan rækkereduceres til I_n.

En bemærkelsesværdig ting er at rækkeoperationer kan realises via matrix-multiplikation via bestemte matricer.

Definition 7.11. En *elementær matrix* E er resultatet af en elementær rækkeoperation på I_n.

Eksempel 7.12. For $n = 2$, lad os kikke på et eksempel på hver type rækkeoperation.

Type (I): er den eneste mulighed $R_0 \leftrightarrow R_1$. Den giver den elementære matrix

$$E = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Type (II): rækkeoperationen $R_1 \to sR_1$ har elementær matrix

$$E = \begin{bmatrix} 1 & 0 \\ 0 & s \end{bmatrix}.$$

Type (III): rækkeoperationen $R_1 \to R_1 + tR_0$ svarer til matricen

$$E = \begin{bmatrix} 1 & 0 \\ t & 1 \end{bmatrix}.$$

△

Proposition 7.13. *At udføre en elementære rækkeoperation på en $(m \times n)$-matrix A er det samme som at tage matrixproduktet EA, hvor $E \in \mathbb{R}^{m\times m}$ er den elementære matrix svarende til den pågældende rækkeoperation.*

Bevis. Hvis $C \in \mathbb{R}^{r\times m}$ har rækker $u_0^T, u_1^T, \ldots, u_{r-1}^T$, så har produktet CA rækker $u_0^T A, u_1^T A, \ldots, u_{r-1}^T A$. Det følger at udførelse af en elementær rækkeoperation på CA er det samme som at udføre den samme elementær rækkeoperation på C og derefter gange fra højre med A: for type (I) er der intet at vise; for type (II) har vi $R_i \to sR_i$, som skifter $u_i^T A$ ud med $su_i^T A = (su_i^T)A$; og for type (III) skifter operationen $R_i \to R_i + tR_j$ rækken $u_i^T A$ ud med $u_i^T A + tu_j^T A = (u_i + tu_j)^T A$. Ved at tage $C = I_m$, fås resultatet. □

Bemærk at elementære matricer er invertible: deres invers er blot den elementære matrix svarende til den omvendte rækkeoperation.

Bevis for sætning 7.2. Hvis $A^{-1}A = I_n$, så giver lemma 7.9 at det lineære ligningssystem $Ax = 0$ har kun én løsning, nemlig $x = 0$. Det følger at A kan rækkereduceres til I_n. Men det er det samme som at sige at der findes elementære matricer $E_0, E_1, \ldots, E_{r-2}, E_{r-1}$ således at

$$E_{r-1}E_{r-2}\ldots E_1E_0A = I_n.$$

Ganges denne ligning fra venstre med først E_{r-1}^{-1}, så E_{r-2}^{-1}, osv., fås

$$A = E_0^{-1}E_1^{-1}\ldots E_{r-2}^{-1}E_{r-1}^{-1}.$$

Men den højre side er invertibel med invers $E_{r-1}E_{r-2}\ldots E_1E_0$. Så A selv er en invertibel matrix, og $AA^{-1} = I_n$.

For det omvendte resultat, bruger vi bare at $AA^{-1} = I_n$ fortæller at matricen $B = A^{-1}$ har en matrix $B^{-1} = A$ således at $B^{-1}B = I_n$. Det overstående argument viser at B er invertibel, med invers $B^{-1} = A$. Det siger at $B^{-1}B = I_n = BB^{-1}$, som er $AA^{-1} = I_n = A^{-1}A$, og vi har at A er invertibel. □

7.3 Beregning af den inverse

Beviset for sætning 7.2 fører også til en metode for at beregne A^{-1} via elementære rækkeoperationer. Vi så at $A^{-1} = E_{r-1}E_{r-2}\ldots E_1E_0$, men dette er resultatet af at udføre på I_n de rækkeoperationer, som reducerer A til I_n. Vi kan så beregne A^{-1} ved at starte med den udvidede matrix

$$\left[A \mid I_n\right]$$

og derefter rækkereducerer den til

$$\left[I_n \mid A^{-1}\right].$$

Eksempel 7.14. For matricen

$$A = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

beregner vi den inverse

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ -1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 2 & 0 & 0 & 1 \end{array}\right]$$

$$\sim_{R_1 \to R_1 + R_0} \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 3 & 4 & 1 & 1 & 0 \\ 1 & 0 & 2 & 0 & 0 & 1 \end{array}\right] \sim_{R_2 \to R_2 - R_0} \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 3 & 4 & 1 & 1 & 0 \\ 0 & -2 & -1 & -1 & 0 & 1 \end{array}\right]$$

$$\sim_{R_1 \to R_1 + R_2} \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 3 & 0 & 1 & 1 \\ 0 & -2 & -1 & -1 & 0 & 1 \end{array}\right] \sim_{R_2 \to R_2 + 2R_1} \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 3 & 0 & 1 & 1 \\ 0 & 0 & 5 & -1 & 2 & 3 \end{array}\right]$$

$$\sim_{R_0 \to R_0 - 2R_1} \left[\begin{array}{ccc|ccc} 1 & 0 & -3 & 1 & -2 & -2 \\ 0 & 1 & 3 & 0 & 1 & 1 \\ 0 & 0 & 5 & -1 & 2 & 3 \end{array}\right] \sim_{R_2 \to \frac{1}{5} R_2} \left[\begin{array}{ccc|ccc} 1 & 0 & -3 & 1 & -2 & -2 \\ 0 & 1 & 3 & 0 & 1 & 1 \\ 0 & 0 & 1 & -\frac{1}{5} & \frac{2}{5} & \frac{3}{5} \end{array}\right]$$

$$\sim_{R_0 \to R_0 + 3R_2} \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{2}{5} & -\frac{4}{5} & \frac{1}{5} \\ 0 & 1 & 3 & 0 & 1 & 1 \\ 0 & 0 & 1 & -\frac{1}{5} & \frac{2}{5} & \frac{3}{5} \end{array}\right] \sim_{R_1 \to R_1 - 3R_2} \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{2}{5} & -\frac{4}{5} & -\frac{1}{5} \\ 0 & 1 & 0 & \frac{3}{5} & -\frac{1}{5} & -\frac{4}{5} \\ 0 & 0 & 1 & -\frac{1}{5} & \frac{2}{5} & \frac{3}{5} \end{array}\right].$$

Fra dette aflæser vi at

$$A^{-1} = \begin{bmatrix} \frac{2}{5} & -\frac{4}{5} & -\frac{1}{5} \\ \frac{3}{5} & -\frac{1}{5} & -\frac{4}{5} \\ -\frac{1}{5} & \frac{2}{5} & \frac{3}{5} \end{bmatrix}.$$

△

Kapitel 8

Ortogonalitet og projektioner

8.1 Standard indre produkt

For vektorer $u, v \in \mathbb{R}^n$, definerer vi deres standard *indre produkt*til at være

$$\begin{aligned}\langle u, v\rangle &= u^T v \\ &= \begin{bmatrix} u_0 & u_1 & \dots & u_{n-1}\end{bmatrix} \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{n-1}\end{bmatrix} \\ &= u_0 v_0 + u_1 v_1 + \cdots + u_{n-1} v_{n-1}.\end{aligned} \tag{8.1}$$

Vi siger at u, v er *ortogonal*, eller at v står *vinkelret* på u, og skriver $u \perp v$, hvis

$$\langle u, v\rangle = 0.$$

Eksempel 8.1. For $u = (1, -2, 3)$ og $v = (0, 5, 7)$ har vi

$$\langle u, v\rangle = 1 \times 0 + (-2) \times 5 + 3 \times 7 = 0 - 10 + 21 = 11,$$

så de står ikke vinkelret på hinanden. Tilgengæld har vi for $w = (2, 1, 0)$ og $w' = (3, 0, -1)$ er

$$\begin{aligned}\langle u, w\rangle &= 1 \times 2 + (-2) \times 1 + 3 \times 0 = 0, \\ \langle u, w'\rangle &= 1 \times 3 + (-2) \times 0 + 3 \times 1 = 0,\end{aligned}$$

så w og w' står vinkelret på u. Bemærk at w' er ikke proportionelt med w, så der kan være forskellige retninger der er vinkelret på en given vektor u. △

I python kan disse regnes i formen $u^T v$ via `@`, men det giver en (1×1)-matrix:

```
>>> import numpy as np
>>> u = np.array([1.0, -2.0, 3.0])[:, np.newaxis]
>>> v = np.array([0.0,  5.0, 7.0])[:, np.newaxis]
>>> u.T @ v
array([[11.]])
```

Så for at få en skalar skal vi tage indgangen `[0,0]`:

```
>>> (u.T @ v)[0,0]
11.0
```

Som praktisk alternativ er der funktionen `np.vdot`, som giver direkte en skalar og kræver ingen transponering:

```
>>> np.vdot(u, v)
11.0
```

Lemma 8.2. *Det indre produkt har de følgende regneegenskaber, for $u, v, w \in \mathbb{R}^n$ og $a, b \in \mathbb{R}$:*

(a) $\langle u, v\rangle = \langle v, u\rangle$,
(b) $\langle au + bw, v\rangle = a\langle u, v\rangle + b\langle w, v\rangle$,
(c) $\langle u, av + bw\rangle = a\langle u, v\rangle + b\langle u, w\rangle$,
(d) $\langle v, v\rangle \geqslant 0$,
(e) *for* $v \neq 0$ *er* $\langle v, v\rangle > 0$.

Bevis. For del (a), bruger vi slutformlen i (8.1) til at få

$$\begin{aligned}\langle u, v\rangle &= u_0v_0 + u_1v_1 + \cdots + u_{n-1}v_{n-1}\\ &= v_0u_0 + v_1u_1 + \cdots + v_{n-1}u_{n-1}\\ &= \langle v, u\rangle,\end{aligned}$$

da $u_0v_0 = v_0u_0$, osv.

For dele (b) og (c) brug bare at $\langle u, v\rangle = u^T v$ og de tilsvarende relationer for matrixmultiplikation.

For del (d), beregner vi

$$\langle v, v\rangle = v_0^2 + v_1^2 + \cdots + v_{n-1}^2.$$

Men $v_k^2 \geqslant 0$, så $\langle v, v\rangle$ er sum af led større end eller lige med 0, og dermed er $\langle v, v\rangle \geqslant 0$. For del (e), hvis $v \neq 0$, så er der mindst en indgang v_k med $v_k \neq 0$. Men så har vi $v_k^2 > 0$, og $\langle v, v\rangle \geqslant v_k^2 > 0$, som påstået. □

Vi sætter

$$\|v\|_2 = \sqrt{\langle v, v\rangle} = \sqrt{v_0^2 + v_1^2 + \cdots + v_{n-1}^2} \tag{8.2}$$

til at være *2-normen* af $v \in \mathbb{R}^n$, og siger at v er en *enhedsvektor* hvis $\|v\|_2 = 1$. Vi siger også at v har *længden* $\|v\|_2$, så enhedsvektorer er vektorer af længde 1.

For two vektorer $u, v \in \mathbb{R}^n$ definerer vi deres *afstand* til at være $\|u - v\|_2$, som er udtrykket

$$\|u - v\|_2 = \sqrt{(u_0 - v_0)^2 + (u_1 - v_1)^2 + \cdots + (u_{n-1} - v_{n-1})^2}.$$

I python kan 2-normen $\|v\|_2$ beregnes via `np.linalg.norm(v)`:

```
>>> import numpy as np
>>> v = np.array([1.0, -2.0, 3.0])[:, np.newaxis]
>>> np.linalg.norm(v)
3.7416573867739413
>>> np.sqrt(v[0]**2 + v[1]**2 + v[2]**2)
array([3.74165739])
```

Proposition 8.3 (Pythagoras sætning)**.** *For u, v ortogonal er*

$$\|u + v\|_2^2 = \|u\|_2^2 + \|v\|_2^2.$$

Bevis. Vi beregner dette kun ved brug af regnereglerne fra lemma 8.2.

$$\begin{aligned}
\|u + v\|_2^2 &= \langle u + v, u + v\rangle = \langle u, u + v\rangle + \langle v, u + v\rangle && \text{lemma 8.2(b)},\\
&= \langle u, u\rangle + \langle u, v\rangle + \langle v, u\rangle + \langle v, v\rangle && \text{lemma 8.2(c)},\\
&= \|u\|_2^2 + 2\langle u, v\rangle + \|v\|_2^2 && \text{lemma 8.2(a)}.
\end{aligned}$$

Men u, v er ortogonal, så $\langle u, v\rangle = 0$, og vi har resultatet. □

Lemma 8.4. *Hvis v er en vektor in $\mathbb{R}^n$ med $v \neq 0$, så er*

$$w = \frac{1}{\|v\|_2} v$$

en enhedsvektor.

Bevis. Vi begregner

$$\|w\|_2^2 = \langle w, w\rangle = \frac{1}{\|v\|_2^2}\langle v, v\rangle = 1,$$

da $\langle v, v\rangle = \|v\|_2^2$. □

8.2 Vinkel mellem vektorer

Definition 8.5. For $u, v \in \mathbb{R}^n$ forskellig fra 0 siger vi at *vinklen* mellem u og v er $0 \leqslant \theta \leqslant \pi$ med

$$\cos\theta = \frac{\langle u, v\rangle}{\|u\|_2\|v\|_2}.$$

For $u \perp v$ har vi $\langle u, v\rangle = 0$, så $\theta = \pi/2 = 90°$, som stemmer overens med vores almindelig vinkelmål.

Vi siger at u og v er *parallelle* hvis der findes et $s \in \mathbb{R}$ således at $u = sv$ eller $v = su$.

Eksempel 8.6. Vektorerne $u = (1, -2, 3)$ og $v = (0, 5, 7)$ fra eksempel 8.1 er ikke parallelle. De har $\langle u, v\rangle = 11$, $\|u\|_2 = \sqrt{1^2 + (-2)^2 + 3^2} = \sqrt{14}$, $\|v\|_2 = \sqrt{0^2 + 5^2 + 7^2} = \sqrt{74}$. Så

$$\cos\theta = \frac{\langle u, v\rangle}{\|u\|_2\|v\|_2} = \frac{11}{\sqrt{14}\sqrt{74}} \approx \frac{11}{32{,}2} = 0{,}342,$$

som giver $\theta \approx \cos^{-1}(0{,}342) = 1{,}22\,\text{rad} = 70{,}0°$. I python kan man bruge `np.arccos` og `np.degrees` for de sidste regnesykker

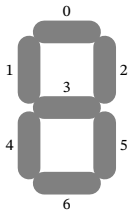
```
>>> import numpy as np

>>> u = np.array([1.0, -2.0, 3.0])[:, np.newaxis]
>>> v = np.array([0.0,  5.0, 7.0])[:, np.newaxis]
>>> cos_theta = ( np.vdot(u, v) / np.linalg.norm(u)
...               / np.linalg.norm(v) )
>>> cos_theta
0.3417533713006601
>>> theta = np.arccos(cos_theta)
>>> theta
```

```
1.2220143545935722
>>> theta_grader = np.degrees(theta)
>>> theta_grader
70.01626502261492
```

△

Lad os bemærke at for $u \neq 0 \neq v$ har vi at u, v er parallelle kun hvis $u/\|u\|_2 = \pm v/\|v\|_2$. Det sidste er det samme som at vinklen θ er enten 0 eller π.

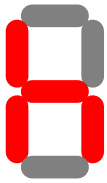
Eksempel 8.7. Et digitalt ur viser cifre ved at oplyse nogle elementer, der er arrangeret som nedenfor:



Standard cifre kan gemmes i en vektor som har et 1-tal for hvert segment der er oplyst, og 0-tal ellers. F.eks.

$$v_0 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} \quad v_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} \quad \ldots$$

En dag viser uret



(8.3)

Hvilke cifre ligner dette mest?

Vi kan danne en matrix med søjler repræsenterende cifre 0 til 9:

```
>>> import numpy as np
>>> v = np.empty((7,10), dtype=float)
>>> v[:,0] = np.array([1,1,1,0,1,1,1])
>>> v[:,1] = np.array([0,0,1,0,0,1,0])
>>> v[:,2] = np.array([1,0,1,1,1,0,1])
>>> v[:,3] = np.array([1,0,1,1,0,1,1])
>>> v[:,4] = np.array([0,1,1,1,1,0,0])
>>> v[:,5] = np.array([1,1,0,1,0,1,1])
>>> v[:,6] = np.array([0,1,0,1,1,1,1])
>>> v[:,7] = np.array([1,0,1,0,0,1,0])
>>> v[:,8] = np.array([1,1,1,1,1,1,1])
>>> v[:,9] = np.array([1,1,1,1,0,1,0])
>>> v
array([[1., 0., 1., 1., 0., 1., 0., 1., 1., 1.],
       [1., 0., 0., 0., 1., 1., 1., 0., 1., 1.],
       [1., 1., 1., 1., 1., 0., 0., 1., 1., 1.],
       [0., 0., 1., 1., 1., 1., 1., 0., 1., 1.],
       [1., 0., 1., 0., 1., 0., 1., 0., 1., 0.],
       [1., 1., 0., 1., 0., 1., 1., 1., 1., 1.],
       [1., 0., 1., 1., 0., 1., 1., 0., 1., 0.]])
```

(8.3) repræsenteres så af vektoren

```
>>> u = np.array([0,1,0,1,1,1,0], dtype=float)[:, np.newaxis]
>>> u
array([[0.],
       [1.],
       [0.],
       [1.],
       [1.],
       [1.],
       [0.]])
```

Vi kan beregne indre produkterne mellem søjlerne af v og u ved

```
>>> v.T @ u
array([[3.],
```

```
       [1.],
       [2.],
       [2.],
       [3.],
       [3.],
       [4.],
       [1.],
       [4.],
       [3.]])
```

Dette tæller hvor mange tændte elementer `u` har til fælles med hvert cifre, og vi ser den har 4 tændte elementer til fælles med 6 og 8.

Hvis vi skalerer søjlerne af v og vektoren u til enhedsvektor, kan vi i stedet beregne cosinus til vinklerne mellem u og hvert søjle via matrix-vektorprodukt

```
>>> v_normaliseret = np.empty_like(v)
>>> for i in range(10):
...     v_normaliseret[:, i] = (
...         v[:, i] / np.linalg.norm(v[:, i])
...         )
...
>>> u_normaliseret = u / np.linalg.norm(u)

>>> cosines = v_normaliseret.T @ u_normaliseret
>>> cosines
array([[0.61237244],
       [0.35355339],
       [0.4472136 ],
       [0.4472136 ],
       [0.75      ],
       [0.67082039],
       [0.89442719],
       [0.28867513],
       [0.75592895],
       [0.67082039]])
```

Her bruges `np.empty_like(v)` til at danne en ny matrix med samme antal rækker og søjler som `v`. Fra resultatet ser vi at cosinus til vinklen til 6-tallet er størst

```
>>> cosines.max()
0.8944271909999159
>>> np.argwhere(cosines == cosines.max())
array([[6, 0]])
```

Her giver funktionen `cosines.max()` den største indgang i `cosines`, og funktionen `np.argwhere(...)` fortæller hvilke indgange har denne værdi. Resultatet er at indgangen i plads $(6, 0)$ er størst, så vi konkluderer at 6 er den bedste kandidat for det korrekte cifre.

Denne metode for at sammenligne vektorer kaldes *cosine similarity* på engelsk. △

For at definitionen på vinklen giver mening har vi brug for at udtrykket for $\cos\theta$ ligger mellem -1 og 1. Men dette følger fra:

Sætning 8.8 (Cauchy-Schwarz ulighed)**.** *Givet $u, v \in \mathbb{R}^n$, gælder*

$$|\langle u, v\rangle| \leqslant \|u\|_2\|v\|_2, \tag{8.4}$$

med lighed hvis og kun hvis u, v er parallelle.

Bevis. Bemærk først at hvis $u = 0$ eller $v = 0$, så er begge sidder af uligheden lige med 0, så der er intet at vise. Vi kikker derfor på tilfældet hvor $u \neq 0 \neq v$. For $a, b \in \mathbb{R}$ betragter vi $\|au + bv\|_2^2$. Vi har

$$\begin{aligned} 0 \leqslant \|au + bv\|_2^2 &= \langle au + bv, au + bv\rangle \\ &= a^2\|u\|_2^2 + 2ab\langle u, v\rangle + b^2\|v\|_2^2. \end{aligned} \tag{8.5}$$

Det følger at

$$-2ab\langle u, v\rangle \leqslant a^2\|u\|_2^2 + b^2\|v\|_2^2. \tag{8.6}$$

Vælg nu $\varepsilon \in \{\pm 1\}$, $a = -\varepsilon\|v\|_2$ og $b = \|u\|_2$. Så bliver (8.6) til

$$2\varepsilon\|u\|_2\|v\|_2\langle u, v\rangle \leqslant 2\|u\|_2^2\|v\|_2^2.$$

Vi deler med $2\|u\|_2\|v\|_2$, som er strengt positivt, da $u \neq 0 \neq v$, til at få

$$\varepsilon\langle u, v\rangle \leqslant \|u\|_2\|v\|_2 \qquad \text{for både } \varepsilon = +1 \text{ og } \varepsilon = -1.$$

Dette er det samme som (8.4).

Hvornår får vi lighed? Hvis $u = 0$ eller $v = 0$, er der intet at vise: u, v er parallelle og vi har lighed. For $u \neq 0 \neq v$, får vi lighed netop når vi har lighed i (8.5) for et af vores særlig valg af a og b, dvs. netop når

$$0 = \left\| -\varepsilon \|v\|_2 u + \|u\|_2 v \right\|_2^2$$

for et $\varepsilon \in \{\pm 1\}$. Men dette sker kun for $-\varepsilon\|v\|_2 u + \|u\|_2 v = 0$, dvs. $u/\|u\|_2 = \varepsilon v/\|v\|_2$, som siger at u, v er parallelle. □

8.3 Projektion på en linje

Givet en vektor $v \in \mathbb{R}^n$, som er ikke nul, kan vi betragte den rette linje L igennem origo i retningen v:

$$L = \{sv \mid s \in \mathbb{R}\}.$$

Givet en anden vektor $u \in \mathbb{R}^n$, kan vi spørge hvilket punkt på L er tættest på u. Afstanden mellem u og sv er $\|u - sv\|_2$. At minimere $\|u - sv\|_2$ er det sammen som at minimere $\|u - sv\|_2^2$. Vi har

$$\|u - sv\|_2^2 = \langle u - sv, u - sv\rangle = \|u\|_2^2 - 2s\langle u, v\rangle + s^2\|v\|_2^2 =: p(s),$$

som er et andengradspolynomium $p(s)$ i s. Toppunktet bestemmes via

$$0 = p'(s) = -2\langle u, v\rangle + 2s\|v\|_2^2,$$

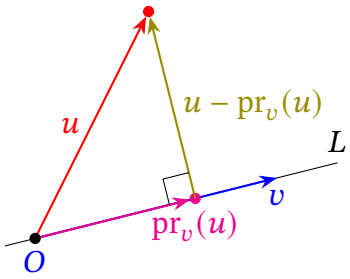
så ligger ved $s = \langle u, v\rangle/\|v\|_2^2$. Dvs. punktet på L, som ligger tættest på u, er

$$\mathrm{pr}_v(u) = \frac{\langle u, v\rangle}{\|v\|_2^2} v. \tag{8.7}$$

Vi kalder $\mathrm{pr}_v(u)$ *projektionen* af u på v.

Det skal bemærkes at differensen $u - \mathrm{pr}_v(u)$ er vinkelret på v:

$$\begin{aligned}
\langle u - \mathrm{pr}_v u, v\rangle &= \langle u, v\rangle - \langle \mathrm{pr}_v u, v\rangle = \langle u, v\rangle - \left\langle \frac{\langle u, v\rangle}{\|v\|_2^2} v, v \right\rangle \\
&= \langle u, v\rangle - \frac{\langle u, v\rangle}{\|v\|_2^2}\langle v, v\rangle = \langle u, v\rangle - \langle u, v\rangle \\
&= 0,
\end{aligned}$$



Figur 8.1: Projektion på en linje.

da $\langle v, v\rangle = \|v\|_2^2$. Det betyder vi får figur 8.1 for projektionen af u på v. Det betyder at vi kan anvende Pythagoras sætning til at få

$$\|u\|_2^2 = \|\mathrm{pr}_v(u)\|_2^2 + \|u - \mathrm{pr}_v(u)\|_2^2,$$

så afstanden fra u til dens projektion er

$$\|u - \mathrm{pr}_v(u)\|_2 = \sqrt{\|u\|_2^2 - \|\mathrm{pr}_v(u)\|_2^2}.$$

Vi kalder dette også for afstanden fra u til linjen L udspændt af v.

Eksempel 8.9. Betragt $v = (1, 2) \in \mathbb{R}^2$ og $u = (2, 3)$. Vi har $\langle u, v\rangle = 2\times1+3\times2 = 8$, $\|v\|_2^2 = 1^2 + 2^2 = 5$, giver

$$\mathrm{pr}_v(u) = \frac{8}{5}\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}8/5\\16/5\end{bmatrix}.$$

Afstanden fra u til L er så

$$\begin{aligned}\sqrt{\|u\|_2^2 - \|\mathrm{pr}_v(u)\|_2^2} &= \sqrt{(2^2 + 3^2) - ((8/5)^2 + (16/5)^2)}\\ &= \sqrt{13 - (64/5)} = \sqrt{1/5}.\end{aligned}$$

△

Observér også at for et punkt sv på L er projektionen punktet selv:

$$\mathrm{pr}_v(sv) = sv,$$

og derved har vi

$$\mathrm{pr}_v(\mathrm{pr}_v(u)) = \mathrm{pr}_v(u). \tag{8.8}$$

Projektion kan skrives som en matrix multiplikation

$$\mathrm{pr}_v(u) = \frac{1}{\|v\|_2^2} v\langle v, u\rangle = \frac{1}{\|v\|_2^2} vv^T u = Pu$$

for matricen

$$P = \frac{1}{\|v\|_2^2} vv^T.$$

Ligning (8.8) siger at

$$P^2 = P.$$

Vi har desuden at

$$P^T = P,$$

dvs. at P er også en *symmetrisk* matrix.

Eksempel 8.10. For $v = (1, 2) \in \mathbb{R}^2$, har vi

$$P = \frac{1}{5}\begin{bmatrix}1\\2\end{bmatrix}\begin{bmatrix}1 & 2\end{bmatrix} = \frac{1}{5}\begin{bmatrix}1 & 2\\2 & 4\end{bmatrix}.$$

△

Som eksempel i python har vi

```
>>> import numpy as np
>>> v = np.array([1.0, -2.0, 3.0])[:, np.newaxis]
>>> normsq = np.vdot(v, v)
>>> p = (1/normsq) * v @ v.T
>>> p
array([[ 0.07142857, -0.14285714,  0.21428571],
       [-0.14285714,  0.28571429, -0.42857143],
       [ 0.21428571, -0.42857143,  0.64285714]])
```

som er symmetrisk

```
>>> np.all(p.T == p)
True
```

og opfylder $P^2 = P$ inden for machine epsilon

```
>>> machine_epsilon = np.finfo(float).eps
>>> np.allclose(p @ p, p,
...             rtol = machine_epsilon,
...             atol = machine_epsilon)
True
```

Funktionen `np.allclose(a,b,rtol=...,atol=...)` giver **True** hvis alle tilsvarende indgange a, b af `a`, `b` opfylder

$$|a - b| \leqslant \texttt{atol} + \texttt{rtol} * |b|.$$

Som standard vælger numpy værdier for `rtol` og `atol` som er væsentlige større end $\epsilon_{\text{machine}}$.

8.4 Ortogonalitet

Lad os nu kikke på et større antal vektorer.

Definition 8.11. En samling vektorer $v_0, v_1, \ldots, v_{k-1}$ er *ortogonal* hvis

$$\langle v_i, v_j \rangle = 0, \qquad \text{for alle } i \neq j.$$

Hvis alle vektorerne $v_0, v_1, \ldots, v_{k-1}$ er desuden enhedsvektor, så siger vi at samlingen er *ortonormal*. Ortonormal er det sammen som

$$\langle v_i, v_j \rangle = \begin{cases} 1, & \text{for } i = j, \\ 0, & \text{for } i \neq j. \end{cases}$$

Eksempel 8.12. Samlingen

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 2 \\ -2 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} -1 \\ -1 \\ 2 \end{bmatrix}$$

er ortogonal, men ingen af vektorerne har længde 1, så samlingen er ikke ortonormal. △

Eksempel 8.13. Standardvektorerne $e_0, e_1, \ldots, e_{n-1}$ i $\mathbb{R}^n$ er givet ved

$$e_i = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

hvor ettallet ligger i den $i$te række. Dvs. $e_i = (v_0, \ldots, v_{n-1})$ med $v_k = 1$ for $k = i$ og $v_k = 0$ ellers. Dette kaldes også nogle gange en *one hot vector* på engelsk.

Når vi tager standard indre produkt af e_i med e_j får vi kun noget der er forskelligt fra nul hvis ettallet i begge vektorer ligger ved den samme plads, dvs. kun hvis i er lige med j. I dette tilfælde er deres indre produkt lige med 1. Så samlingen $e_0, e_1, \ldots, e_n$ er ortonormal. △

Eksempel 8.14. For $c^2 + s^2 = 1$ er vektorerne

$$\begin{bmatrix} c \\ s \end{bmatrix}, \quad \begin{bmatrix} -s \\ c \end{bmatrix}$$

ortonormal. Det samme gælder parret

$$\begin{bmatrix} c \\ s \end{bmatrix}, \quad \begin{bmatrix} s \\ -c \end{bmatrix}.$$

△

Bemærk betingelsen for at $v_0, v_1, \ldots, v_{k-1}$ er ortonormal er det samme som

$$\langle v_i, v_j \rangle = \begin{cases} 1, & \text{for } i = j, \\ 0, & \text{for } i \neq j. \end{cases}$$

Ofte er det nemmere at finde vektorer der er ortogonal end ortonormal. Men så forskelligt er de to begreber ikke:

Lemma 8.15. *Hvis $v_0, v_1, \ldots, v_{k-1}$ er en ortogonal samling og alle vektorer er forskellige fra nul, så er*

$$\frac{1}{\|v_0\|_2} v_0, \frac{1}{\|v_1\|_2} v_1, \ldots, \frac{1}{\|v_{k-1}\|_2} v_{k-1}$$

en ortonormal samling.

Bevis. Vi har $\langle v_i, v_j \rangle = 0$ for $i \neq j$. Desuden er $\langle v_i, v_i \rangle = \|v_i\|_2^2 > 0$, da $v_i \neq 0$. Resultatet følger nu fra lemma 8.4, og

$$\left\langle \frac{1}{\|v_i\|_2} v_i, \frac{1}{\|v_j\|_2} v_j \right\rangle = \frac{1}{\|v_i\|_2 \|v_j\|_2} \langle v_i, v_j \rangle = 0, \qquad \text{for } i \neq j.$$

□

Eksempel 8.16. Vektorerne $u = (1, -2, 3), w = (3, 0, -1)$ er ortogonal, som beregnet i eksempel 8.1. Men

$$\langle u, u \rangle = 1^2 + (-2)^2 + 3^2 = 14 \neq 1,$$

så de er ikke ortonormal. Til gengæld får vi en ortonormal samling ved at skalere disse vektorer til enhedsvektorer, dvs.

$$\frac{u}{\|u\|_2} = \frac{1}{\sqrt{14}} \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix}, \qquad \frac{w}{\|w\|_2} = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ 0 \\ -1 \end{bmatrix}$$

er ortonormal. △

Lad $v_0, v_1, \ldots, v_{k-1}$ være en ortogonal samling af vektorer. En styrke ved sådan en samling er det er nemt at dekomponere andre vektor som kombinationer af $v_0, v_1, \ldots, v_{k-i}$, uden at løse lineære ligningssytemer.

Proposition 8.17. *Hvis*

$$u = x_0 v_0 + x_1 v_1 + \cdots + x_{k-1} v_{k-1} \tag{8.9}$$

hvor $v_0, v_1, \ldots, v_{k-1}$ er ortogonal, og ingen er 0, så er

$$x_0 = \frac{\langle u, v_0 \rangle}{\|v_0\|_2^2}, \; x_1 = \frac{\langle u, v_1 \rangle}{\|v_1\|_2^2}, \; \ldots, \; x_{k-1} = \frac{\langle u, v_{k-1} \rangle}{\|v_{k-1}\|_2^2}. \tag{8.10}$$

Desuden er

$$\|u\|_2^2 = x_0^2 \|v_0\|_2^2 + x_1^2 \|v_1\|_2^2 + \cdots + x_{k-1}^2 \|v_{k-1}\|_2^2. \tag{8.11}$$

Den sidste ligning kendes som Parsevals identitet.

Bevis. Lad os tage det indre produkt af (8.9) med v_i. Så får vi

$$\begin{aligned} \langle u, v_i \rangle &= \langle x_0 v_0 + x_1 v_1 + \cdots + x_{k-1} v_{k-1}, v_i \rangle \\ &= x_0 \langle v_0, v_i \rangle + x_1 \langle v_1, v_i \rangle + \cdots + x_{k-1} \langle v_{k-1}, v_i \rangle. \end{aligned}$$

Det eneste led på den højre side, der er ikke nul, er leddet $x_i\langle v_i, v_i\rangle = x_i\|v_i\|_2^2$. Det følger at $x_i = \langle u, v_i\rangle/\|v_i\|_2^2$, som påstået.

Tilsvarende har vi

$$\begin{aligned}\|u\|_2^2 &= \langle x_0v_0 + \cdots + x_{k-1}v_{k-1}, x_0v_0 + \cdots + x_{k-1}v_{k-1}\rangle \\ &= \sum_{i,j=0}^{k-1} x_ix_j\langle v_i, v_j\rangle.\end{aligned}$$

Men $\langle v_i, v_j\rangle$ er kun forskellig fra nul for $i = j$ og dens værdi er $\|v_i\|_2^2$. Så vi har

$$\|u\|_2^2 = \sum_{i=0}^{k-1} x_i^2\|v_i\|_2^2,$$

som ønsket. □

Eksempel 8.18. Lad v_0, v_1, v_2 være vektorerne fra eksempel 8.12. Vi har $\|v_0\|_2^2 = 3$, $\|v_1\|_2^2 = 8$ og $\|v_2\|_2^2 = 6$. For $u = (0, 1, 0)$, har vi så

$$\begin{aligned}u = \begin{bmatrix}0\\1\\0\end{bmatrix} &= \frac{1}{3}\begin{bmatrix}1\\1\\1\end{bmatrix} + \frac{-2}{8}\begin{bmatrix}2\\-2\\0\end{bmatrix} + \frac{-1}{6}\begin{bmatrix}-1\\-1\\2\end{bmatrix} \\ &= \frac{1}{3}\begin{bmatrix}1\\1\\1\end{bmatrix} - \frac{1}{4}\begin{bmatrix}2\\-2\\0\end{bmatrix} - \frac{1}{6}\begin{bmatrix}-1\\-1\\2\end{bmatrix}.\end{aligned}$$

△

Givet (8.9) og (8.10) med v_i, $i = 0, \ldots, k-1$, ortogonal og ingen 0, kan vi skrive den højre side af (8.9), som

$$\mathrm{pr}_{v_0}(u) + \mathrm{pr}_{v_1}(u) + \cdots + \mathrm{pr}_{v_{k-1}}(u).$$

Husk at $\mathrm{pr}_v(u) = Pu$ for $P = (1/\|v\|_2^2)vv^T$. Vi har så at den højre side af (8.9) er

$$\begin{aligned}&\frac{\langle v_0, u\rangle}{\|v_0\|_2^2}v_0 + \frac{\langle v_1, u\rangle}{\|v_1\|_2^2}v_1 + \cdots + \frac{\langle v_{k-1}, u\rangle}{\|v_{k-1}\|_2^2}v_{k-1} \\ &\quad = \left(\frac{1}{\|v_0\|_2^2}v_0v_0^T + \frac{1}{\|v_1\|_2^2}v_1v_1^T + \cdots + \frac{1}{\|v_{k-1}\|_2^2}v_{k-1}v_{k-1}^T\right)u.\end{aligned}$$

Definition 8.19. For $v_0, v_1, \ldots, v_{k-1}$ ortogonal, med alle $v_i \neq 0$, er *projektionen af u på samlingen $v_0, v_1, \ldots, v_{k-1}$* givet ved Pu, hvor P er matricen

$$P = \frac{1}{\|v_0\|_2^2} v_0 v_0^T + \frac{1}{\|v_1\|_2^2} v_1 v_1^T + \cdots + \frac{1}{\|v_{k-1}\|_2^2} v_{k-1} v_{k-1}^T. \tag{8.12}$$

Bemærkning 8.20. Når man beregne projektionen i python, gør man det typisk uden at opbygge selve projektionsmatricen. Oftes ønsker man blot at beregne Pw for en vektor w, og dette gives ved

$$Pw = \mathrm{pr}_{v_0}(w) + \mathrm{pr}_{v_1}(w) + \cdots + \mathrm{pr}_{v_{k-1}}(w),$$

med hver enkelt led som i (8.7). Se senere eksempler. ◇

Proposition 8.21. *For P som i definition 8.19 gælder*
(a) $P^2 = P$ *og*
(b) $P^T = P$.
Desuden for vilkårlig $w \in \mathbb{R}^n$ har vi
(c) $w - Pw \perp v_i$ *for alle i, og*
(d) $\|w\|_2^2 = \|Pw\|_2^2 + \|w - Pw\|_2^2$.
Det følger at $u = Pw$ er vektoren af formen (8.9), som er tættest på w.

Bevis. For at vise $P^2 = P$ er det nok at bemærke at

$$\begin{aligned}(v_i v_i^T)(v_j v_j^T) &= v_i (v_i^T v_j) v_j^T \\ &= \langle v_i, v_j \rangle v_i v_j^T = \begin{cases} 0, & \text{for } i \neq j, \\ \|v_i\|_2^2 v_i v_i^T, & \text{for } i = j. \end{cases}\end{aligned}$$

Så følger resultatet for del (a) ved at gange P^2 ud.

Ligningen $P^T = P$ i del (b) følger fra bemærkningen at $v_i v_i^T$ er symmetrisk.

For del (c) observér at $\langle Pw, v_i \rangle = \langle v_i, Pw \rangle = v_i^T Pw$ og at $v_i^T v_j v_j^T = \langle v_i, v_j \rangle v_j^T$. Så har vi

$$\begin{aligned}\langle w - Pw, v_i \rangle &= \langle w, v_i \rangle - v_i^T Pw = \langle w, v_i \rangle - \frac{1}{\|v_i\|_2^2} \|v_i\|_2^2 v_i^T w \\ &= \langle w, v_i \rangle - \langle v_i, w \rangle = 0.\end{aligned}$$

Identiteten i del (d) for $\|w\|_2^2$ følger nu fra Pythagoras sætning.

Vi skal til sidst vise at $u = Pw$ er vektoren på formen (8.9), som er tættest på w. Betragt et $y = y_0 v_0 + \cdots + y_{k-1} v_{k-1}$ på formen (8.9), som er ikke lige med

Pw. Så er $Pw - y \neq 0$ og lemma 8.2(e) giver $\|Pw - y\|_2^2 > 0$. Da $w - Pw$ er vinkelret på hver v_i, er $w - Pw$ vinkelret på både y og Pw, og så vinkelret på $Pw - y$. Det følger fra Pythagoras at

$$\begin{aligned}\|w - y\|_2^2 &= \|(w - Pw) + (Pw - y)\|_2^2 = \|w - Pw\|_2^2 + \|Pw - y\|_2^2 \\ &> \|w - Pw\|_2^2.\end{aligned}$$

Dvs. at afstanden fra w til y er større end afstanden fra w til Pw. □

Eksempel 8.22. Lad $v_0 = (1, -2, 3)$, $v_1 = (3, 0, -1)$ være det ortogonale par u, w fra eksempel 8.1. Vi kan beregne

$$P\begin{bmatrix}1\\1\\0\end{bmatrix} = \frac{-1}{14}\begin{bmatrix}1\\-2\\3\end{bmatrix} + \frac{3}{10}\begin{bmatrix}3\\0\\-1\end{bmatrix} = \frac{1}{35}\begin{bmatrix}29\\5\\-3\end{bmatrix}.$$

△

Eksempel 8.23. Lad os forsøge at approksimere funktionen $y = e^x$ for $-1 \leqslant x \leqslant 1$ med polynomier af grad højst 2. Vi vil gøre dette numerisk i python.

```
import matplotlib.pyplot as plt
import numpy as np
```

Vi start med funktionerne 1, x og x^2 evalueret ved 100 punkter jævnt fordelt over $[-1, 1]$:

```
x = np.linspace(-1, 1, 100)

# funktion 1
v0 = np.ones(100)[:, np.newaxis]
# funktion x
v1 = x[:, np.newaxis]
# funktion x**2
u2 = (x**2)[:, np.newaxis]
```

`v0` svarer til 1, `v1` til x og `u2` til x^2. Vi ser at `v0` er stort set vinkelret på `v1`

```
print(np.vdot(v0, v1))
```

```
7.771561172376096e-15
```

og at v1 er næsten vinkelret på u2

```
print(np.vdot(v1, u2))
```

```
7.66053886991358e-15
```

men u2 er ikke vinkelret på v0

```
print(np.vdot(v0, u2))
```

```
34.006734006734014
```

Vi ønsker så at erstatte u2 med restvektoren v2 efter projektion på v0, v1. Så $\mathtt{v2} = \mathtt{u2} - \mathrm{pr}_{\mathtt{v0}}(\mathtt{u2}) - \mathrm{pr}_{\mathtt{v1}}(\mathtt{u2})$. Men u2 er vinkelret på v1, så den sidste led er 0 og vi kan nøjes med

```
v2 = u2 - np.vdot(u2, v0) / np.vdot(v0, v0) * v0
```

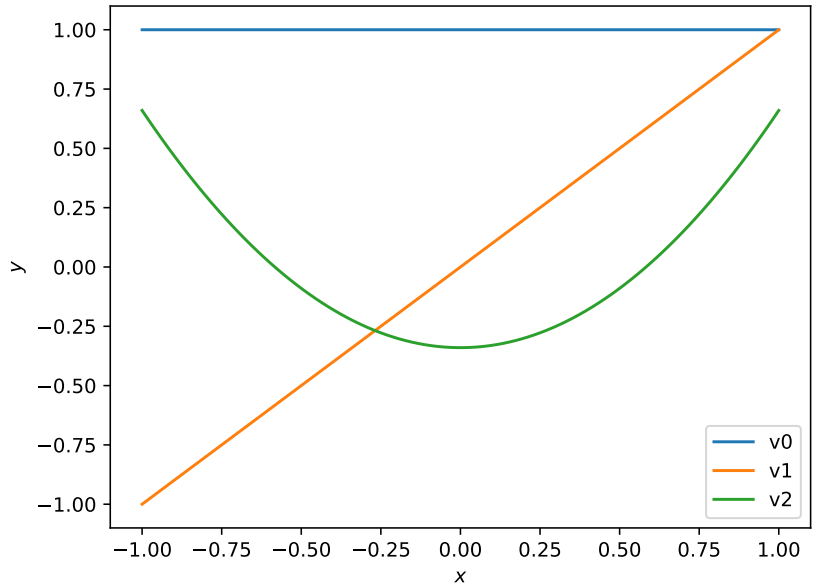
Så er v2 stort set vinkelret på v0 og v1

```
print(np.vdot(v0, v2))
print(np.vdot(v1, v2))
```

```
-1.1102230246251565e-15
4.884981308350689e-15
```

og vi kan bruge v0, v1, v2 som ortogonal samling. Disse funktioner er tegnet i figur 8.2.

```
fig, ax = plt.subplots()
ax.plot(x, v0[:, 0], label='v0')
ax.plot(x, v1[:, 0], label='v1')
ax.plot(x, v2[:, 0], label='v2')
```



Figur 8.2: Den ortogonale samling `v0`, `v1`, `v2` af polynomier af højst anden grad.

```
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.legend()
```

Lad `u` være repræsentationen af eksponentialfunktionen

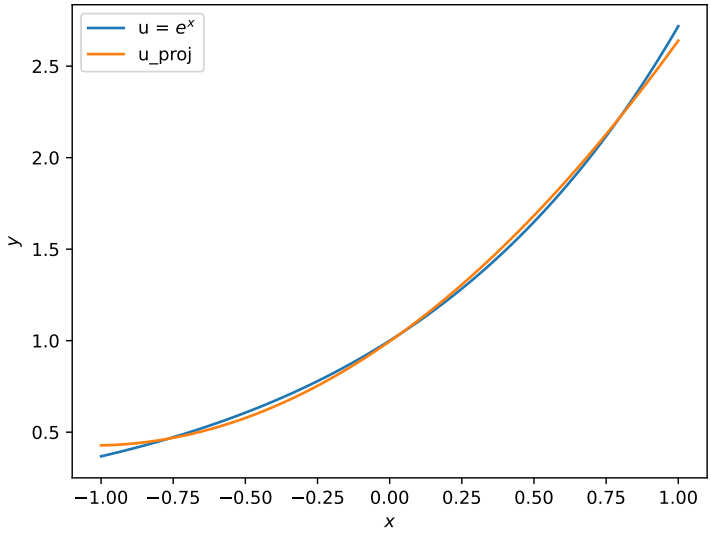
```
u = np.exp(x)[:, np.newaxis]
```

Dens projektion er så

```
u_proj = (1/np.vdot(v0, v0) * v0 @ (v0.T @ u)
          + 1/np.vdot(v1, v1) * v1 @ (v1.T @ u)
          + 1/np.vdot(v2, v2) * v2 @ (v2.T @ u))
```

En plot af `u` og `u_proj` giver figur 8.3. Vi ser at `u_proj` ligger meget tæt på `u` over hele intervallet.

```
fig, ax = plt.subplots()
```



Figur 8.3: Numerisk approksimation af $y = e^x$, $-1 \leqslant x \leqslant 1$.

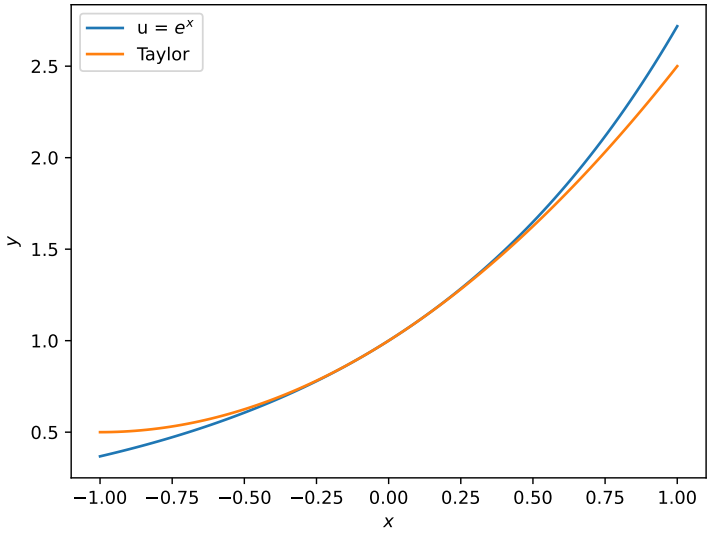
```
ax.plot(x, u[:, 0], label='u = $e^{x}$')
ax.plot(x, u_proj[:, 0], label='u_proj')

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.legend()
```

Dette er i modsætning til den anden ordens Taylor udviklingen

$$e^x \approx 1 + x + \frac{1}{2}x^2$$

som kun er en god tilnærmelse til u tæt på $x = 0$, se figur 8.4.

```
fig, ax = plt.subplots()
ax.plot(x, u[:, 0], label='u = $e^{x}$')
ax.plot(x, (1. + x + (1/2.)*x**2), label='Taylor')
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
ax.legend()
```



Figur 8.4: Approksimation af $y = e^x$, $-1 \leqslant x \leqslant 1$, via dens anden ordens Taylor udvikling er kun godt tæt på $x = 0$.

$\triangle$

Kapitel 9

Ortogonale matricer

Ortogonale og ortonormale samlinger af vektorer er tæt forbundne med ortogonale matricer. Desuden er brug af ortogonale matricer det foretrukne værktøj for numerisk arbejde. Dette er af forskellige grunde. Blandt andet bevarer de længde af vektorer, så øger de ikke fejl i de vektorer de ganges på. Desuden har de en invers der fås via simpel ombytning af matrixindgange. I praksis betyder dette at den inverse kan bruges uden tab af nøjagtighed, uden brug af flops og uden brug af ekstra computerhukommelse.

9.1 Definition og første eksempler

Definition 9.1. En kvadratisk matrix $A \in \mathbb{R}^{n\times n}$ er *ortogonal* hvis

$$A^T A = I_n.$$

Eksempel 9.2. For $n = 2$ er enhver rotationsmatrix ortogonal. Nemlig, for $c^2 + s^2 = 1$ har vi

$$R^T R = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}^T \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} c^2 + s^2 & -cs + sc \\ -sc + cs & s^2 + c^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Tilsvarende gælder for spejlingsmatricer

$$M^T M = \begin{bmatrix} c & s \\ s & -c \end{bmatrix} \begin{bmatrix} c & s \\ s & -c \end{bmatrix} = \begin{bmatrix} c^2 + s^2 & 0 \\ 0 & c^2 + s^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

△

Eksempel 9.3. Lad $u \in \mathbb{R}^n$ være en enhedsvektor. Så er

$$A = I_n - 2uu^T$$

er en ortogonal matrix: det ydre produkt uu^T er symmetrisk, og vi har

$$\begin{aligned} A^T A &= (I_n - 2uu^T)(I_n - 2uu^T) = I_n - 4uu^T + 4(uu^T)(uu^T) \\ &= I_n - 4uu^T + 4u(u^T u)u^T = I_n, \end{aligned}$$

da $u^T u = \langle u, u \rangle = 1$.

For eksempel for $n = 4$, er $u = (1/2, -1/2, 1/2, -1/2)$ en enhedsvektor, så

$$\begin{aligned} A &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} - 2 \begin{bmatrix} 1/2 \\ -1/2 \\ 1/2 \\ -1/2 \end{bmatrix} \begin{bmatrix} 1/2 & -1/2 & 1/2 & -1/2 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1/2 & -1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & -1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & -1/2 & 1/2 \end{bmatrix} \\ &= \begin{bmatrix} 1/2 & 1/2 & -1/2 & 1/2 \\ 1/2 & 1/2 & 1/2 & -1/2 \\ -1/2 & 1/2 & 1/2 & 1/2 \\ 1/2 & -1/2 & 1/2 & 1/2 \end{bmatrix} \end{aligned}$$

en ortogonal matrix. △

9.2 Egenskaber

Vi vil se om lidt at ortogonale matricer er netop dem, der bevarer det indre produkt. Dette medfører at ortogonale matricer også bevarer norm, afstand og vinkel, da disse er defineret ud fra det indre produkt. Så enhver figur bevarer dens form og størrelse under en transformation givet ved en ortogonal matrix.

For at vise at ortogonale matricer bevarer det indre produkt, har vi brug for at vide hvordan transponering påvirker matrixprodukter.

Proposition 9.4. *For $B \in \mathbb{R}^{m \times n}$ og $C \in \mathbb{R}^{n \times p}$ har vi*

$$(BC)^T = C^T B^T. \tag{9.1}$$

Observér at $C^T \in \mathbb{R}^{p\times n}$, og $B^T \in \mathbb{R}^{n\times m}$, så $C^T B^T$ er det eneste produkt vi kan danne generelt ud fra C^T og B^T. Desuden er $(BC)^T$ og $C^T B^T$ begge matricer i $\mathbb{R}^{p\times m}$.

Bevis. Husk at BC har (i,j)-indgang $\sum_{k=0}^{n-1} b_{ik}c_{kj}$. Så dette er den (j,i)te indgang af $(BC)^T$. Men $C^T B^T$ har (j,i)-indgang

$$\sum_{k=0}^{n-1}(C^T)_{jk}(B^T)_{ki} = \sum_{k=0}^{n-1} c_{kj}b_{ik} = \sum_{k=0}^{n-1} b_{ik}c_{kj} = (BC)_{ij} = ((BC)^T)_{ji},$$

som påstået. □

Proposition 9.5. *For $A \in \mathbb{R}^{n\times n}$ ortogonal gælder*

$$\langle Au, Av\rangle = \langle u, v\rangle, \qquad \text{for alle } u, v \in \mathbb{R}^n. \tag{9.2}$$

Specielt gælder $\|Au\|_2 = \|u\|_2$ for alle $u \in \mathbb{R}^n$.

Omvendt, hvis $A \in \mathbb{R}^{n\times n}$ opfylder (9.2), så er A en ortogonal matrix.

Bevis. For A ortogonal har vi, ved brug af (9.1), at

$$\langle Au, Av\rangle = (Au)^T(Av) = u^T A^T Av = u^T I_n v = u^T v = \langle u, v\rangle,$$

så (9.2) holder.

Omvendt antag at (9.2) gælder. Bemærk at for en vilkårlig matrix B, har vi at den (i,j)te indgang af B er givet ved $e_i^T B e_j$, hvor e_k er standardvektorer, cf. eksempel 8.13. Vi har nu at den (i,j)te indgang af $A^T A$ er givet ved

$$\begin{aligned}(A^T A)_{ij} &= e_i^T(A^T A)e_j = e_i^T A^T A e_j = (Ae_i)^T(Ae_j)\\ &= \langle Ae_i, Ae_j\rangle = \langle e_i, e_j\rangle = \begin{cases}1, & \text{for } i = j,\\ 0, & \text{for } i \neq j,\end{cases}\end{aligned}$$

som er den (i,j)te indgang af I_n. Så $A^T A = I_n$ og A er en ortogonal matrix. □

Bemærkning 9.6. Numerisk er ortogonale matricer meget nyttig. Antag at en vektor $v \in \mathbb{R}^n$ har en fejl givet ved en vektor $w \in \mathbb{R}^n$ med $\|w\|_2 < c$, dvs. v repræsenteres af $v+w$. For A ortogonal har vi at Av repræsenteres af $A(v+w) = Av + Aw$, og fejlen er så Aw. Da A er ortogonal har vi $\|Aw\|_2 = \|w\|_2$, så fejlen i Av opfylder den samme estimering $\|Aw\|_2 < c$, som den oprindelige fejl w. ◇

Desuden har ortogonale matricer andre pæne egenskaber. Specielt kan deres invers beregnes ved simpel ombytning af indgangene.

Proposition 9.7. (a) *Hvis A er ortogonal, så er A invertibel med invers A^T.*
(b) *For $A, B \in \mathbb{R}^{n\times n}$ ortogonale, er AB ortogonal.*

Bevis. Del (a) følger direkte fra $A^T A = I_n$ og sætning 7.2.

For del (b), bruger vi (9.1):

$$(AB)^T(AB) = (B^T A^T)(AB) = B^T(A^T A)B = B^T I_n B = B^T B = I_n,$$

så AB er ortogonal. □

9.3 Ortonormale vektorer og ortogonale matricer

Givet en samling vektorer $v_0, v_1, \ldots, v_{k-1}$ i $\mathbb{R}^n$, kan vi danne en matrix V med disse vektorer som søjler

$$V = \left[v_0 \mid v_1 \mid \ldots \mid v_{k-1}\right] \in \mathbb{R}^{n\times k}.$$

Matricen

$$G = V^T V \in \mathbb{R}^{k\times k}$$

kaldes *Grammatricen* for $v_0, \ldots, v_{k-1}$, efter Jørgen Pedersen Gram (1850–1916). Vi har

$$\begin{aligned} G := V^T V &= \begin{bmatrix} v_0^T \\ v_1^T \\ \vdots \\ v_{k-1}^T \end{bmatrix} \left[v_0 \mid v_1 \mid \ldots \mid v_{k-1}\right] \\ &= \begin{bmatrix} v_0^T v_0 & v_0^T v_1 & \ldots & v_0^T v_{k-1} \\ v_1^T v_0 & v_1^T v_1 & \ldots & v_1^T v_{k-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_{k-1}^T v_0 & v_{k-1}^T v_1 & \ldots & v_{k-1}^T v_{k-1} \end{bmatrix}. \end{aligned}$$

Dette sige at den (i, j)te indgang i G er det indre produkt mellem v_i og v_j:

$$G = (g_{ij}) = (v_i^T v_j) = \big(\langle v_i, v_j\rangle\big).$$

Som direkte konsekvens har det følgende resultat.

Lemma 9.8. *En samling vektorer $v_0, v_1, \ldots, v_{k-1}$ i $\mathbb{R}^n$ er ortonormal hvis og kun hvis Grammatricen $G = V^T V \in \mathbb{R}^{k\times k}$ er lige med identitetsmatricen I_k:*

$$G = V^T V = I_k. \qquad \square$$

Sætning 9.9. *En ortonormal samling $v_0, v_1, \ldots, v_{k-1}$ i $\mathbb{R}^n$ består af højst $k = n$ vektorer.*

Bevis. Hvis $k \geqslant n$, så betragter vi de første n vektorer og deres tilhørende matrix

$$A = \left[v_0 \;\middle|\; v_1 \;\middle|\; \ldots \;\middle|\; v_{n-1}\right] \in \mathbb{R}^{n\times n}.$$

Matricen A er kvadratisk, og lemma 9.8 giver $A^T A = I_n$, så A er ortogonal. Det følger fra proposition 9.7(a) at A er invertibel med invers A^T.

Lad $u \in \mathbb{R}^n$ være en vektor, som står vinkelret på alle de n vektorer $v_0, v_1, \ldots, v_{n-1}$. Dette betyder at $v_i^T u = 0$ for $i = 0, \ldots, n-1$. Da er

$$A^T u = \begin{bmatrix} v_0^T \\ v_1^T \\ \vdots \\ v_{n-1}^T \end{bmatrix} u = \begin{bmatrix} v_0^T u \\ v_1^T u \\ \vdots \\ v_{n-1}^T u \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Men A^T er invertibel (med invers A), så lemma 7.9 giver at ligningssystemet $A^T u = 0$ har kun én løsning, nemlig $u = 0$. Vi konkluderer at $u = 0$, og dermed kan den ortonormale samling $v_0, v_1, \ldots, v_{n-1}$ ikke tilføjes flere enhedsvektorer. $\square$

Definition 9.10. En ortonormal samling i $\mathbb{R}^n$ med netop n vektorer kaldes en *ortonormal basis* for $\mathbb{R}^n$.

Lemma 9.8 giver straks det følgende resultat.

Proposition 9.11. *Søjlerne af en ortogonal matrix $A \in \mathbb{R}^{n\times n}$ udgør en ortonormal basis for $\mathbb{R}^n$.* $\square$

Eksempel 9.12. En rotationsmatrix $R = \left[\begin{smallmatrix} c & -s \\ s & c \end{smallmatrix}\right]$, $c^2 + s^2 = 1$, er en ortogonal matrix. Det følger at dens søjler

$$\begin{bmatrix} c \\ s \end{bmatrix}, \quad \begin{bmatrix} -s \\ c \end{bmatrix}$$

er en ortonormal basis for $\mathbb{R}^2$. $\triangle$

Eksempel 9.13. Den konkrete ortogonale matrix $A \in \mathbb{R}^{4\times}$ i slutningen af eksempel 9.3 har søjler

$$\begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \\ 1/2 \end{bmatrix}, \quad \begin{bmatrix} 1/2 \\ 1/2 \\ 1/2 \\ -1/2 \end{bmatrix}, \quad \begin{bmatrix} -1/2 \\ 1/2 \\ 1/2 \\ 1/2 \end{bmatrix}, \quad \begin{bmatrix} 1/2 \\ -1/2 \\ 1/2 \\ 1/2 \end{bmatrix},$$

som er dermed en ortonormal basis for $\mathbb{R}^4$. △

9.4 Householdermatrix

Lad os skrive eksempel 9.3 om til noget lidt mere generelt.

Definition 9.14. En *Householdermatrix* er en kvadratisk matrix $H \in \mathbb{R}^{n\times n}$, som er ortogonal og har formen

$$H = I_n - svv^T \tag{9.3}$$

for et $v \in \mathbb{R}^n$ og et $s \in \mathbb{R}$. Vektoren v kaldes en *Householdervektor* for H.

Lemma 9.15. *Enhver matrix af formen* (9.3) *er symmetrisk.*

Formlen (9.3) *med* $s \neq 0$ *og* $v \neq 0$ *giver en ortogonalmatrix, og dermed en Householdermatrix, hvis og kun hvis*

$$s = \frac{2}{\|v\|_2^2}. \tag{9.4}$$

Bevis. For symmetri har vi

$$H^T = (I_n - svv^T)^T = I_n^T - s(vv^T)^T = I_n - s(v^T)^T v^T = I_n - svv^T = H.$$

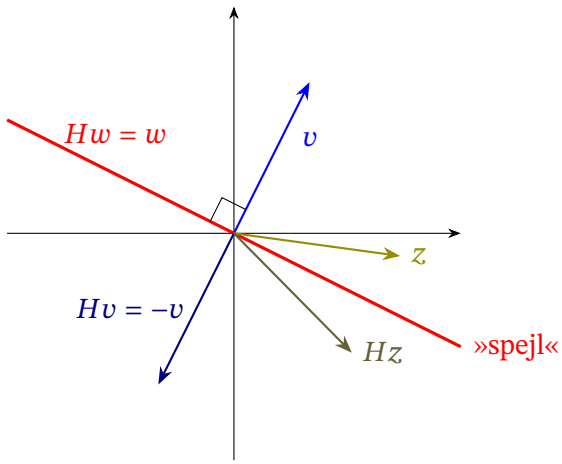
For ortogonalitet betragt

$$\begin{aligned} H^T H &= (I_n - svv^T)(I_n - svv^T) = I_n - 2svv^T + s^2 v(v^T v)v^T \\ &= I_n + s\big(-2 + s\|v\|_2^2\big)vv^T. \end{aligned}$$

Vi ser at $H^T H = I_n$ hvis og kun hvis

$$s\big(-2 + s\|v\|_2^2\big)vv^T = 0. \tag{9.5}$$

Men for $v \neq 0$ er $vv^T \neq 0$, og med antagelsen $s \neq 0$, har vi (9.5) kun når skalaren $(-2 + s\|v\|_2^2)$ er 0. Dette siger at $s\|v\|_2^2 = 2$, og dermed har vi ligning (9.4), som ønsket. □



Figur 9.1: Effekten af en Householdermatrix i $\mathbb{R}^n$. Det røde »spejl« har dimension $n - 1$.

Proposition 9.16. *Enhver Householdermatrix $H = I_n - svv^T$ med Householdervektor v opfylder*

(a) $H^T = H$,
(b) $H^2 = I_n$,
(c) $Hv = -v$, *og*
(d) $Hw = w$ *for alle* $w \perp v$.

Denne proposition fortæller os at når $v \neq 0$ og $s \neq 0$ er H en spejlingsmatrix, som sender v til $-v$ og fastholder alle vektorer w, som står vinkelret på v. Se figur 9.1, hvor den røde »linje« er rummet af vektorer w vinkelret på v. Dette rum er »spejlet«, som H spejler vektorer i.

Bevis. Vi har allerede vist at $H^T = H$, så vi har del (a). Ved at bruge dette får vi $H^2 = H^T H$, som er lige med I_n, da H er en ortogonal matrix. Så del (b) holder. For de sidste to dele, har vi først at

$$Hv = (I_n - svv^T)v = v - sv(v^T v) = v - s\|v\|_2^2 v = v - 2v = -v,$$

som er del (c). For $w \perp v$ har vi $v^T w = 0$, så

$$Hw = w - sv(v^T w) = w - 0 = w,$$

som påstået i del (d). □

I mange situationer, f.eks. løsning af ligningssystemer, vil vi gerne flytte en given vektor til en med mange indgange lige med 0. Householdermatricer er rigtig til god til dette. Faktisk kan vi flytte til et multiplum af standardvektoren $e_0 = (1, 0, \ldots, 0)$. For numerisk sikkerhed er det bedst at være forsigtig med fortegnet af dette multiplum. I det næste resultat træffer vi det rigtige valg.

Proposition 9.17. *Givet en enhedsvektor $u = (u_0, u_1, \ldots, u_{n-1}) \in \mathbb{R}^n$, sæt*

$$\varepsilon = \begin{cases} +1, & \text{hvis } u_0 \geqslant 0, \\ -1, & \text{hvis } u_0 < 0, \end{cases}$$

$$s = 1 + |u_0| \quad \text{og} \quad v = \frac{1}{s}(e_0 + \varepsilon u).$$

Så er $H = I_n - svv^T$ en Householdermatrix med

$$Hu = -\varepsilon e_0 = -\varepsilon \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} -\varepsilon \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{og} \quad He_0 = -\varepsilon u. \tag{9.6}$$

Desuden har Householdervektoren v formen

$$v = \begin{bmatrix} 1 \\ v_1 \\ \vdots \\ v_{n-1} \end{bmatrix}. \tag{9.7}$$

Bevis. Bemærk at $\varepsilon u_0 = |u_0|$ og at $s = 1 + |u_0| \geqslant 1$, så i det følgende må vi gerne dele med s. Skrives $v = (v_0, v_1, \ldots, v_{n-1})$ har vi at den 0te indgang i v er

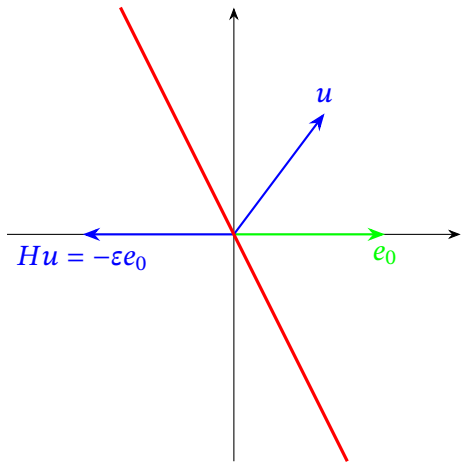
$$v_0 = \frac{1}{s}(1 + \varepsilon u_0) = \frac{1 + |u_0|}{1 + |u_0|} = 1,$$

så (9.7) gælder.

Fra $1 = v_0 = v^T e_0$, får vi at

$$He_0 = (I_n - svv^T)e_0 = I_n e_0 - svv^T e_0 = e_0 - sv = e_0 - (e_0 + \varepsilon u) = -\varepsilon u,$$

som er den sidste ønskede relation i (9.6).



Figur 9.2: Spejling af en enhedsvektor u til $-\varepsilon e_0$ via en Householdermatrix.

For vise at H er en Householdermatrix mangler vi blot at ligning (9.4) er opfyldt. Først noterer vi at

$$(e_0 + \varepsilon u)^T u = e_0^T u + \varepsilon u^T u = u_0 + \varepsilon,$$

da $u^T u = \|u\|_2^2 = 1$. Dette giver

$$\varepsilon v^T u = \frac{\varepsilon}{s}(e_0 + \varepsilon u)^T u = \frac{\varepsilon}{s}(u_0 + \varepsilon) = \frac{1}{s}(|u_0| + 1) = +1,$$

og så

$$s\|v\|_2^2 = s v^T v = v^T (e_0 + \varepsilon u) = v_0 + \varepsilon v^T u = 1 + 1 = 2,$$

som ønsket.

Da H er Householdermatrix, har vi $Hu = -\varepsilon H(He_0) = -\varepsilon H^2 e_0 = -\varepsilon e_0$, så den resterende del af (9.6) holder. □

Eksempel 9.18. Vi ønsker at bestemme en Householdermatrix H der flytter $x = (1, 2, 3)$ til et multiplum af e_0.

Sæt

$$u = \frac{x}{\|x\|_2} = \frac{x}{\sqrt{14}}.$$

Så i proposition 9.17 har vi

$$u_0 = \frac{1}{\sqrt{14}}, \quad \varepsilon = +1, \quad s = 1 + \frac{1}{\sqrt{14}} = \frac{1 + \sqrt{14}}{\sqrt{14}}$$

og

$$
\begin{aligned}
v = \frac{1}{s}(e_0 + \varepsilon u) &= \frac{\sqrt{14}}{1+\sqrt{14}}\left(\begin{bmatrix}1\\0\\0\end{bmatrix} + \frac{1}{\sqrt{14}}\begin{bmatrix}1\\2\\3\end{bmatrix}\right) \\
&= \frac{1}{1+\sqrt{14}}\begin{bmatrix}\sqrt{14}+1\\2\\3\end{bmatrix} = \begin{bmatrix}1\\2/(1+\sqrt{14})\\3/(1+\sqrt{14})\end{bmatrix},
\end{aligned}
$$

som rigtig nok har 0te indgang lige med 1. Vi kan lave denne beregning i python og bekræfter effekten af den frembragte Householdertransformation:

```
>>> import numpy as np
>>> x = np.array([1.0, 2.0, 3.0])[:, np.newaxis]
>>> u = x / np.linalg.norm(x)
>>> u[0,0]
0.2672612419124244
>>> eps = 1 if u[0,0] >=0 else -1
>>> eps
1
>>> s = 1 + np.abs(u[0,0])
>>> s
1.2672612419124243
>>> v = (eps/s) * u
>>> v[0,0] = 1
>>> v
array([[1.        ],
       [0.42179344],
       [0.63269017]])
>>> Hx = x - s * v @ (v.T @ x)
>>> Hx
array([[-3.74165739e+00],
       [-4.44089210e-16],
       [-4.44089210e-16]])
>>> float(np.linalg.norm(x))
3.7416573867739413
```

△

Bemærk at vi implementer multiplikation med $H = I_n - svv^T$ på et x, som

$$Hx = x - sv(v^T x)$$

for at spare på antallet af flops. Desuden behøver vi hverken at konstruere eller gemme matricen H, men kan nøjes med at gemme skalaren s og vektoren v. (Da $v_0 = 1$ kan dette plads i v bruges til at gemme s.) Dette sparer computerhukommelse. Det kan være nyttigt at oprette en funktion, som udfører Householdertransformationen givet af s og v på en anden vektor w.

```
import numpy as np

def house_transformation(s, v, w):
    return w - s * v @ (v.T @ w)
```

Dette bruges på følgende måde

```
rng = np.random.default_rng()
v = rng.random((10,1))
s = 2 / np.vdot(v,v)

w = rng.random((10,1))
Hw = house_transformation(s, v, w)
print(Hw)
```

```
[[-0.94111005]
 [ 0.24828972]
 [ 0.70600171]
 [-0.26613446]
 [-0.57674306]
 [-0.45892994]
 [-0.11580324]
 [-0.4712503 ]
 [-0.86028159]
 [-0.32701089]]
```

Vi kan bekræfte at $H^2 w = w$ og at H afbilder v til $-v$ indenfor to gange machine epsilon:

```
machine_epsilon = np.finfo(float).eps
print(np.allclose(house_transformation(s, v, Hw), w,
                  rtol = 2*machine_epsilon,
                  atol = 2*machine_epsilon))
```

```
True
```

```
print(np.allclose(house_transformation(s, v, v), -v,
                  rtol = 2*machine_epsilon,
                  atol = 2*machine_epsilon))
```

```
True
```

Bemærkning 9.19. Valgene i proposition 9.17 kan forklares på følgende vis. Betingelsen (9.7) at $v^T e_0 = v_0 = 1$, giver $He_0 = e_0 - sv(v^T e_0) = e_0 - sv$. Så $He_0 = -\varepsilon u$ medfører $e_0 - sv = -\varepsilon u$, som er det samme som $sv = e_0 + \varepsilon u$. Den 0te indgang af denne ligning er $s = 1 + \varepsilon u_0$. Valget af ε sikrer $s = 1 + |u_0| \geqslant 1$, som er numerisk bedre at dele med end $1 - |u_0|$. Dette valg med $s \geqslant 1$, giver også at $\|v\|_2^2 = 2/s \leqslant 2$, så der er god kontrol over indgangene i v. ◇

9.5 Udvidelse til ortonormal basis

Vi skal senere i kurset bruge Householdermatricer som praktisk numerisk værktøj. Men de har også fine teoretiske anvendelser, herunder de følgende to resultater, som vil være vigtig i forbindelse med arbejdet med singulærværdidekomponering i det næste kapitel.

Sætning 9.20. *Givet en enhedsvektor $u \in \mathbb{R}^n$, så findes der en ortonormal basis $v_0, v_1, \ldots, v_{n-1}$ for $\mathbb{R}^n$ med $v_0 = u$.*

Bevis. Definér ε og H som i proposition 9.17. Da er $He_0 = -\varepsilon u$ og H er en ortogonal matrix. Matricen $A = -\varepsilon H$ er også ortogonal, da $\varepsilon^2 = 1$, og har u som 0te søjle. Fra proposition 9.11 får vi nu vores ortonormal basis som søjlerne $v_0, \ldots, v_{n-1}$ af A . □

Korollar 9.21. *For $u_0, u_1, \ldots, u_{k-1}$ ortonormal i $\mathbb{R}^n$ kan den udvides til en ortonormal basis $u_0, \ldots, u_{k-1}, \ldots, u_{n-1}$ for $\mathbb{R}^n$.*

Bevis. Da ortonormale baser er det sammen som søjlerne af en ortogonal matrix, se proposition 9.11, er det nok at finde en ortogonal matrix A med søjler 0 til $k-1$ givet ved de givne vektorer $u_0, \ldots, u_{k-1}$.

Brug proposition 9.17 til at finde ε og en Householdermatrix H med $He_0 = -\varepsilon u_0$. Sæt $B = -\varepsilon H$, som er ortogonal og har u_0 som 0te søjle. Vi har $u_0 = Be_0$ og $e_0 = Bu_0$. For $i > 0$ har vi også at $\langle Bu_i, e_0\rangle = \langle Bu_i, Bu_0\rangle = \langle u_i, u_0\rangle = 0$. Dette siger at den 0te indgang i Bu_i er 0, så

$$Bu_i = \left[\begin{array}{c} 0 \\ \hline w_i \end{array}\right], \qquad i = 1, \ldots, k-1$$

for vektorer $w_1, \ldots, w_{k-1} \in \mathbb{R}^{n-1}$. Da $\langle w_i, w_j\rangle = \langle Bu_i, Bu_j\rangle = \langle u_i, u_j\rangle$ og $u_1, \ldots, u_{k-1}$ er ortonormal, får vi at $w_1, \ldots, w_{k-1}$ er ortonormal i $\mathbb{R}^{n-1}$. Per induktion, findes der en ortogonal matrix $C \in \mathbb{R}^{(n-1)\times(n-1)}$ hvis første $k-1$ søjler er $w_1, \ldots, w_{k-1}$. Dannes matricen

$$D = \left[\begin{array}{c|c} 1 & 0 \\ \hline 0 & C \end{array}\right] \in \mathbb{R}^{n\times n}$$

er D ortogonal, og

$$\begin{aligned} D &= \left[e_0 \mid Bu_1 \mid \ldots \mid Bu_{k-1} \mid \ldots\right] \\ &= \left[Bu_0 \mid Bu_1 \mid \ldots \mid Bu_{k-1} \mid \ldots\right] \\ &= B\left[u_0 \mid u_1 \mid \ldots \mid u_{k-1} \mid \ldots\right]. \end{aligned}$$

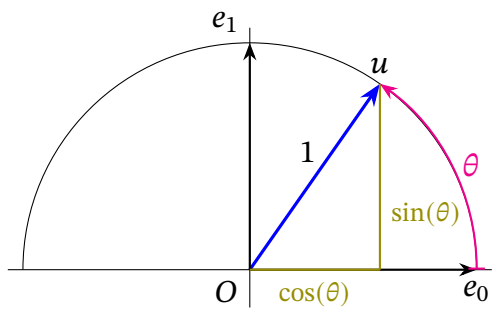
Matricen $A = B^T D$ er produktet af to ortogonale matricer, så er ortogonal og vi har

$$\left[u_0 \mid u_1 \mid \ldots \mid u_{k-1} \mid \ldots\right] = B^T D = A.$$

Søjlerne af A udgør derfor en ortonormal basis indeholder $u_0, \ldots, u_{k-1}$. □

9.6 Plan geometri: trigonometriske identiteter

Ortogonale matricer af størrelse 2×2 giver os nogle bestemte transformationer af planen og disse kan bruges til at vise nogle trigonometriske identiteter. Allerede i kapitel 2 brugte vi en formlen for cosinus af en vinkelsum, dog uden bevis. I



Figur 9.3: Cosinus og sinus for en enhedsvektor i planen.

dette afsnit vil vi vise de følgende to identiteter for cosinus og sinus af summen af vinkler.

$$\cos(\theta + \varphi) = \cos(\theta)\cos(\varphi) - \sin(\theta)\sin(\varphi), \tag{9.8}$$
$$\sin(\theta + \varphi) = \cos(\theta)\sin(\varphi) + \sin(\theta)\cos(\varphi). \tag{9.9}$$

Andre nyttige identiteter følger fra disse, som vi skal se om lidt.

For at dette skal give mening, har vi først brug for at definere sinusfunktionen. Vi arbejder i planen $\mathbb{R}^2$. Givet to enhedsvektorer u og v, har vi defineret

$$\cos(\theta) = \langle u, v \rangle,$$

se definition 8.5. Når vi arbejder i planen vi vil tillade at θ ligger i intervallet $[0, 2\pi)$, intervallet $[-\pi, \pi)$ eller i $\mathbb{R}$.

Betragt nu matricen

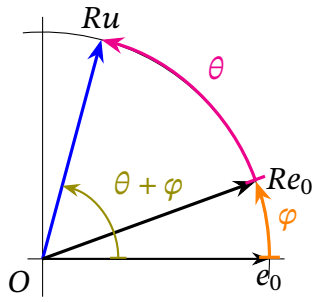
$$J_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

Vi sætter

$$\sin(\theta) = \langle u, J_2 v \rangle$$

til at være vores *sinusfunktion*.

For $v = e_0$ er $Jv = e_1$, og $\sin(\theta) = \langle u, Je_0 \rangle$ er dermed y-koordinaten af u. Dette stemmer overens med den sædvanlige definition på sinus, se figur 9.3. Bemærk at selve vinklen θ mellem u og e_0 er længden af buestykket langs enhedscirklen fra e_0 til u, målt med positiv omløbsretning.



Figur 9.4: Enhedsvektoren u drejet igennem vinkel φ.

Hvis $A \in \mathbb{R}^{2\times 2}$ er en ortogonal matrix, så er $\cos(\theta)$ det samme for Au, Av som for u, v. For sinus har vi

$$\sin(\theta) = \langle u, J_2 v\rangle = \langle Au, AJ_2 v\rangle,$$

så vi får den samme sinusværdi for Au, Av hvis

$$AJ_2 = J_2 A. \tag{9.10}$$

En ortogonal matrix, der opfylder (9.10), siges at være *speciel*. Vi har så at specielle ortogonale matricer bevarer både $\cos(\theta)$ og $\sin(\theta)$.

Eksempel 9.22. Lad $c, s \in \mathbb{R}$ opfylder $c^2 + s^2 = 1$. Vi påstår at rotationsmatricen

$$R = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$$

er en speciel ortogonal matrix. Vi så i eksempel 9.2 at R er en ortogonal matrix. For den specielle betingelse beregner vi

$$RJ_2 = \begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} -s & -c \\ c & -s \end{bmatrix}$$

og ser at

$$J_2 R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} -s & -c \\ c & -s \end{bmatrix} = RJ_2,$$

som ønsket. △

Proposition 9.23. *Lad u og v være enhedsvektorer $\mathbb{R}^2$. Med de overstående definitioner på cosinus og sinus gælder*

$$\cos(\theta)^2 + \sin(\theta)^2 = 1. \tag{9.11}$$

Bevis. Da $v = (v_0, v_1)$ er en enhedsvektor har vi at matricen R_v givet ved

$$R_v = \begin{bmatrix} v_0 & -v_1 \\ v_1 & v_0 \end{bmatrix}$$

end en speciel ortogonal matrix. Bemærk den sidste søjle af R_v er

$$\begin{bmatrix} -v_1 \\ v_0 \end{bmatrix} = J_2 v,$$

så

$$R_v = \begin{bmatrix} v \mid J_2 v \end{bmatrix}.$$

Vi har dermed at

$$\begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix} = \begin{bmatrix} \langle u, v \rangle \\ \langle u, J_2 v \rangle \end{bmatrix} = \begin{bmatrix} \langle v, u \rangle \\ \langle J_2 v, u \rangle \end{bmatrix} = \begin{bmatrix} v^T u \\ (J_2 v)^T u \end{bmatrix} = R_v^T u.$$

Da R_v er en ortogonal matrix, er $(R_v)^T$ også en ortogonal matrix. Dette giver

$$\cos(\theta)^2 + \sin(\theta)^2 = \left\langle \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}, \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix} \right\rangle = \left\langle R_v^T u, R_v^T u \right\rangle = \langle u, u \rangle = 1,$$

da u er en enhedsvektor. Vi har så det ønskede identitet. □

Lad os betragte den specielle ortogonale matrix

$$R = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) \\ \sin(\varphi) & \cos(\varphi) \end{bmatrix}.$$

Bemærk at

$$Re_0 = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) \\ \sin(\varphi) & \cos(\varphi) \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(\varphi) \\ \sin(\varphi) \end{bmatrix},$$

så R flytter e_0 til enhedsvektoren, som har vinkel φ til e_0

Lad $u = (\cos(\theta), \sin(\theta))$, som har vinkel θ til e_0-aksen. Da R er en speciel ortogonal matrix, er vinklen mellem Ru og Re_0 også θ. Dette illustreres i figur 9.4, hvor man kan også se at vinklen mellem Ru og e_0 er $\theta + \varphi$. Dette giver

$$\begin{aligned}\begin{bmatrix}\cos(\theta+\varphi)\\ \sin(\theta+\varphi)\end{bmatrix} = Ru &= \begin{bmatrix}\cos(\varphi) & -\sin(\varphi)\\ \sin(\varphi) & \cos(\varphi)\end{bmatrix}\begin{bmatrix}\cos(\theta)\\ \sin(\theta)\end{bmatrix}\\ &= \begin{bmatrix}\cos(\theta)\cos(\varphi) - \sin(\theta)\sin(\varphi)\\ \cos(\theta)\sin(\varphi) + \sin(\theta)\cos(\varphi)\end{bmatrix}.\end{aligned}$$

De 0te og 1te koordinater i denne relation er netop ligninger (9.8) og (9.9), som ønsket.

For nogle ydeligere identiteter bemærk at den ortogonale matrix

$$S = \begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix}$$

sender $u = (\cos(\theta), \sin(\theta))$ til

$$Su = \begin{bmatrix}\cos(\theta)\\ -\sin(\theta)\end{bmatrix}.$$

Men Su har vinkel $-\theta$ til e_0, se figur 9.5: buelængden er det sammen men vil skal tilføje et minus fortegn, da vi har den modsatte omløbsretning. Dette give at

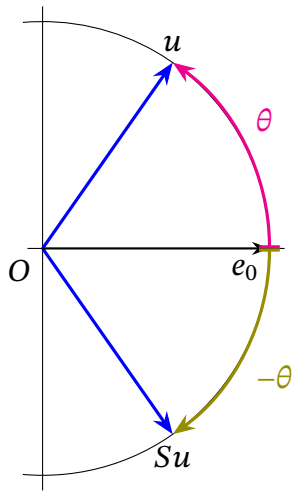
$$Su = \begin{bmatrix}\cos(-\theta)\\ \sin(-\theta)\end{bmatrix}.$$

Sammenlignes disse to udtryk for Su, fås

$$\cos(-\theta) = \cos(\theta) \quad \text{og} \quad \sin(-\theta) = -\sin(\theta). \tag{9.12}$$

Som konsekvens har vi de følgende nyttige relationer.

$$2\cos(\alpha)\cos(\beta) = \cos(\alpha+\beta) + \cos(\alpha-\beta), \tag{9.13}$$
$$2\cos(\alpha)\sin(\beta) = \sin(\alpha+\beta) - \sin(\alpha-\beta), \tag{9.14}$$
$$2\sin(\alpha)\sin(\beta) = -\cos(\alpha+\beta) + \cos(\alpha-\beta). \tag{9.15}$$



Figur 9.5: Spejling i x-aksen.

Disse kan alle vises ved at begynde med den højre side, anvende (9.8) og (9.9) og derefter bruge (9.12). F.eks.

$$\begin{aligned}
\cos(\alpha+\beta)+\cos(\alpha-\beta) &= \cos(\alpha+\beta)+\cos(\alpha+(-\beta)) \\
&= \cos(\alpha)\cos(\beta)-\sin(\alpha)\sin(\beta) \\
&\qquad +\cos(\alpha)\cos(-\beta)-\sin(\alpha)\sin(-\beta) \\
&= \cos(\alpha)\cos(\beta)-\sin(\alpha)\sin(\beta) \\
&\qquad +\cos(\alpha)\cos(\beta)+\sin(\alpha)\sin(\beta) \\
&= 2\cos(\alpha)\cos(\beta).
\end{aligned}$$

Kapitel 10

Singulærværdidekomponering

En vektor $v \in \mathbb{R}^2$ i planen beskrives fint af dens længde $\|v\|_2$ og vinklen θ fra x-aksen (så længe $v \neq 0$). Som vi har set før, kan denne vinkeloplysning erstattes af enhedsvektoren (c, s), $c^2 + s^2 = 1$, med $c = \cos(\theta)$, $s = \sin(\theta)$. Husk at (c, s) er blot enhedsvektoren $v/\|v\|_2$.

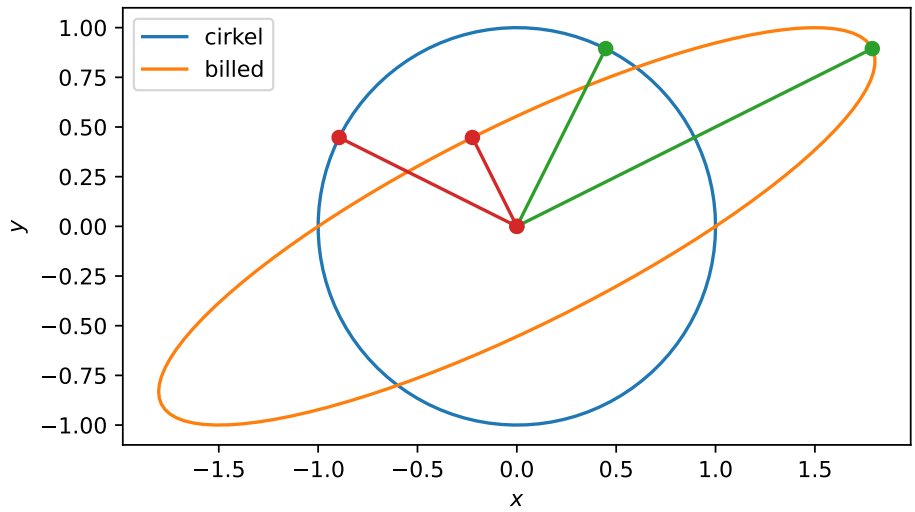
I højere dimensioner, $v \in \mathbb{R}^n$, $v \neq 0$, beskrives tilsvarende ved dens længde $\sigma = \|v\|_2$ og enhedsvektoren $u = v/\|v\|_2$. Betragtes v som en matrix $v \in \mathbb{R}^{n\times 1}$, har vi

$$v = \sigma u = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{n-1} \end{bmatrix} \begin{bmatrix} \sigma \end{bmatrix} \begin{bmatrix} 1 \end{bmatrix}$$

hvor $\begin{bmatrix} \sigma \end{bmatrix}$ og $\begin{bmatrix} 1 \end{bmatrix}$ er (1×1)-matricer. Dette er eksempel på en (tynd) singulærværdidekomponering (SVD) af en matrix. Tallet σ er en singulærværdi; vektorerne $u \in \mathbb{R}^n$ og $\begin{bmatrix} 1 \end{bmatrix} \in \mathbb{R}^1$ venstre- og højresingulærvektorer.

En tilsvarende dekomponering findes for vilkårlige matricer $A \in \mathbb{R}^{m\times n}$, og giver os god oplysning om deres struktur. Desuden findes der effektive og præcise metoder til at beregne singulærværdidekomponering, så det er blevet til et vigtigt numerisk værktøj.

Singulærværdierne gemmer på oplysning om A der er relevant for at vurdere pålideligheden af f.eks. løsninger af lineære ligningssystemer $Ax = b$, og dekomponeringen kan bruges til approksimering af A i komprimeret form. Brug af singulærværdier og singulærvektorer er også meget relevant for analyse af datamængder: et kraftigt ofte brugt dataværktøj er principalkomponent



Figur 10.1: Billedet af en cirkel efter matrixmultiplikation med $A \in \mathbb{R}^{2\times 2}$.

analyse; singulærværdidekomponering udføre dette analyse, og mere, på en numerisk pålidelig måde.

10.1 Singulærværdier i dimension 2

Lad os begynde med at betragte en (2×2)-matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Vi kan tegne billedet af enhedscirklen $(\cos(\theta), \sin(\theta))$ under transformationen $x \mapsto Ax$. Et typisk resultat vises i figur 10.1.

Vi ser at dette billede af cirklen er altid ellipseformet, uanset hvilke matrix A vi bruger. (Nogle gange vil denne ellipse dog være mast sammen til et linjestykke eller et enkelt punkt).

En ellipse har to akser: en stor akse og en lille akse. Halvdelen af længden af disse to akser er netop det vi kalde singulærværdierne σ_0 og σ_1 for A. Det to akser står vinkelret på hinanden og peger i retninger u_0 og u_1, hvor vi tager u_i til at være enhedsvektorer.

Det er et overraskende faktum at der findes enhedsvektorer v_0 og v_1 i $\mathbb{R}^2$, som står vinkelret på hinanden, således at

$$Av_0 = \sigma_0 u_0, \quad Av_1 = \sigma_1 u_1. \tag{10.1}$$

Dette er en uventet da en vilkårlig matrix A bevarer ikke vinkel mellem tilfældige vektorer.

Lad os samler u_0, u_1 og v_0, v_1 til (2×2)-matricer

$$U = \left[u_0 \mid u_1\right] \quad \text{og} \quad V = \left[v_0 \mid v_1\right].$$

Så er U og V ortogonale matricer, da deres søjler er enhedsvektorer, der står vinkelret på hinanden. Sættes

$$u_0 = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}, \quad u_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}.$$

kan vi omskrive (10.1) til

$$\begin{aligned} AV &= \left[Av_0 \mid Av_1\right] = \left[\sigma_0 u_0 \mid \sigma_1 u_1\right] = \begin{bmatrix} x_0\sigma_0 & x_1\sigma_1 \\ y_0\sigma_0 & y_1\sigma_1 \end{bmatrix} \\ &= \begin{bmatrix} x_0 & x_1 \\ y_0 & y_1 \end{bmatrix} \begin{bmatrix} \sigma_0 & 0 \\ 0 & \sigma_1 \end{bmatrix} = U\Sigma, \end{aligned} \tag{10.2}$$

hvor

$$\Sigma = \begin{bmatrix} \sigma_0 & 0 \\ 0 & \sigma_1 \end{bmatrix}.$$

Da V er en ortogonal matrix, er den invertibel med invers V^T. Ganges (10.2) med $V^{-1} = V^T$ fra den højre side, fås

$$A = U\Sigma V^T. \tag{10.3}$$

Dette kaldes singulærværdidekomponeringen (SVD) af A.

Denne SVD kan beregnes i python via `np.linalg.svd`. Denne python funktionen giver tre objekter: (a) matricen U, (b) en `ndarray` med kun én akse, som indholder singulærværdierne σ_0, σ_1, og (c) den transponerede matrix V^T.

```
>>> import numpy as np

>>> a = np.array([[1.0,  7.0],
```

```
...                 [1.0, -1.0]])

>>> u, s, vt = np.linalg.svd(a)
>>> s
array([7.12310563, 1.12310563])
>>> u
array([[-0.99250756,  0.12218326],
       [ 0.12218326,  0.99250756]])
>>> vt
array([[-0.12218326, -0.99250756],
       [ 0.99250756, -0.12218326]])
```

Disse kan bruges til at rekonstruere `a`

```
>>> u @ np.diag(s) @ vt
array([[ 1.,  7.],
       [ 1., -1.]])
```

Her danner `np.diag(s)` en diagonal matrix med indgange fra `s` sat langs diagonalen.

(10.3) kan også skrives via ydre produkter. Sæt

$$v_0 = \begin{bmatrix} a_0 \\ b_0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} a_1 \\ b_1 \end{bmatrix}.$$

Så er (10.3)

$$\begin{aligned}
U\Sigma V^T &= \left[\sigma_0 u_0 \mid \sigma_1 u_1\right] \begin{bmatrix} v_0^T \\ v_1^T \end{bmatrix} \\
&= \begin{bmatrix} x_0\sigma_0 & x_1\sigma_1 \\ y_0\sigma_0 & y_1\sigma_1 \end{bmatrix} \begin{bmatrix} a_0 & b_0 \\ a_1 & b_1 \end{bmatrix} = \begin{bmatrix} x_0\sigma_0 a_0 + x_1\sigma_1 a_1 & x_0\sigma_0 b_0 + x_1\sigma_1 b_1 \\ y_0\sigma_0 a_0 + y_1\sigma_1 a_1 & y_0\sigma_0 b_0 + y_1\sigma_1 b_1 \end{bmatrix} \\
&= \sigma_0 \begin{bmatrix} x_0 a_0 & x_0 b_0 \\ y_0 a_0 & y_0 b_0 \end{bmatrix} + \sigma_1 \begin{bmatrix} x_1 a_1 & x_1 b_1 \\ y_1 a_1 & y_1 b_1 \end{bmatrix} \\
&= \sigma_0 \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \begin{bmatrix} a_0 & b_0 \end{bmatrix} + \sigma_1 \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \begin{bmatrix} a_1 & b_1 \end{bmatrix} \\
&= \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T.
\end{aligned} \tag{10.4}$$

Bemærkning 10.1. Som supplement til (10.1), har vi

$$A^T u_0 = \sigma_0 v_0, \quad A^T u_1 = \sigma_1 u_1.$$

Dette kan ses fra (10.3), da

$$A^T = (U\Sigma V^T)^T = V\Sigma^T U^T$$

eller ved at transponere (10.4). ◇

10.2 SVD generelt

Sætning 10.2. *Enhver matrix $A \in \mathbb{R}^{m\times n}$ har en* singulærværdidekomponering

$$A = U\Sigma V^T \tag{10.5}$$

med $U \in \mathbb{R}^{m\times m}$ og $V \in \mathbb{R}^{n\times n}$ ortogonale matricer, med $\Sigma \in \mathbb{R}^{m\times n}$ en diagonal matrix

$$\Sigma = \operatorname{diag}(\sigma_0, \dots, \sigma_{k-1}) = \begin{bmatrix} \sigma_0 & 0 & 0 & \dots \\ 0 & \sigma_1 & 0 & \dots \\ \vdots & & \ddots & \vdots \end{bmatrix}, \qquad k = \min\{m, n\}, \tag{10.6}$$

og med $\sigma_0 \geqslant \sigma_1 \geqslant \cdots \geqslant \sigma_{k-1} \geqslant 0$.

Tallene σ_i kaldes *singulærværdier* for A. Søjlerne af U er *venstresingulærvektorer*; søjlerne af V er *højresingulærvektorer*.

Beviset gives i afsnit 10.4. Her vil vi fokusere på fortolkningen og anvendelser.

Eksempel 10.3. For $A \in \mathbb{R}^{2\times 3}$ har dens SVD formen

$$A = \begin{bmatrix} u_0 \mid u_1 \end{bmatrix} \begin{bmatrix} \sigma_0 & 0 & 0 \\ 0 & \sigma_1 & 0 \end{bmatrix} \begin{bmatrix} v_0^T \\ v_1^T \\ v_2^T \end{bmatrix}, \qquad u_i \in \mathbb{R}^{2\times 1}, v_j \in \mathbb{R}^{3\times 1}.$$

Dette kan skrives via ydre produkter, som vi gjorde for (2×2)-matricer i (10.4), og giver

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T.$$

Bemærk at dette form tager ikke vektoren v_2 i brug. Det svarer til matrixproduktet

$$A = \begin{bmatrix} u_0 \mid u_1 \end{bmatrix} \begin{bmatrix} \sigma_0 & 0 \\ 0 & \sigma_1 \end{bmatrix} \begin{bmatrix} v_0^T \\ v_1^T \end{bmatrix},$$

som kaldes en tynd SVD. △

Generelt er (10.5) det samme som

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{k-1} u_{k-1} v_{k-1}^T, \tag{10.7}$$

hvor igen $k = \min\{m, n\}$. Der er sørget for at u_i og v_j er enhedsvektorer, og at faktorerne σ_i er aftagende. Derfor er de led, der bidrager mest til A, er dem der kommer først i (10.7). Sagt med andre ord, er den største del af oplysningen i A indeholdt i de første led af (10.7).

I en del applikationer kan man nøjes med at approksimere A med de første r led i (10.7), $r \leqslant k$. Vi kalder

$$\sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{r-1} u_{r-1} v_{r-1}^T$$

en *forkortet SVD* med r led.

10.3 Eksempler på SVD

10.3.1 Punkter i planen

Betragt 1000 punkter i $\mathbb{R}^2$, som til venstre i Figur 10.2.

```
import matplotlib.pyplot as plt
import numpy as np

rng = np.random.default_rng()

n = 1000
a = (np.array([[1.0, 5.0], [1.0, -1.0]])
     @ rng.standard_normal((2, n)))

print(np.abs(a).max())
```

```
16.20771551521247
```

```
fig, ax = plt.subplots()

ax.set_xlim(-20, 20)
ax.set_ylim(-20, 20)
ax.set_aspect('equal')

ax.plot(*a, 'o', color='cyan', markersize = 2)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Hvis vi beregner en fuld SVD, får vi et V^T, som er meget stor.

```
u, s, vt = np.linalg.svd(a)
print(u.shape, s.shape, vt.shape)
```

```
(2, 2) (2,) (1000, 1000)
```

I stedet kan vi nøjes med at beregne en tynd SVD ved hjælp af tilføjelsen

```
full_matrices=False
```

til `np.linalg.svd`:

```
u, s, vt = np.linalg.svd(a, full_matrices=False)
print(u.shape, s.shape, vt.shape)
```

```
(2, 2) (2,) (2, 1000)
```

Vi ser at den første singulærværdi er væsentlig større end den anden

```
print(s)
```

```
[161.33809905  36.26369701]
```

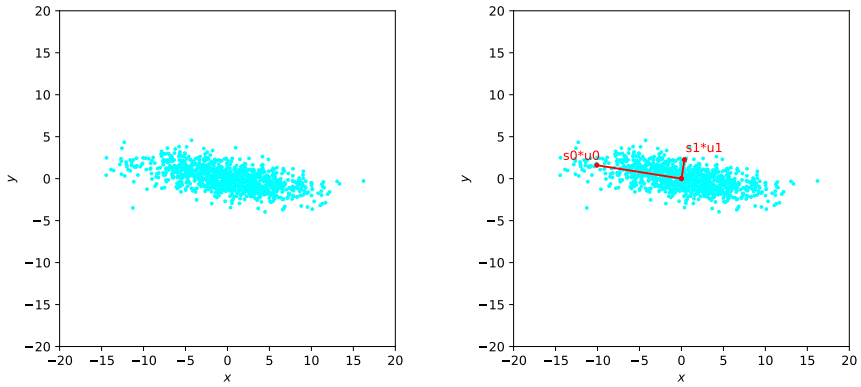
Denne singulærværdi har venstresingulærvektor, som er den først søjle af u

```
print(u[:, [0]])
```

```
[[-0.98766723]
 [ 0.15656772]]
```

```
origo = np.zeros((2, 1))
scale = 2 / np.sqrt(n)
tscale = 1.4 * scale

fig, ax = plt.subplots()

ax.set_xlim(-20, 20)
ax.set_ylim(-20, 20)
ax.set_aspect('equal')

ax.plot(*a, 'o', markersize = 2, color='cyan')

ax.plot(*(np.hstack([origo, u[:, [0]]*s[0]*scale])),
        color='red', marker='.')
ax.text(*u[:, [0]]*s[0]*tscale, 's0*u0', color='red')

ax.plot(*(np.hstack([origo, u[:, [1]]*s[1]*scale])),
        color='red', marker='.')
ax.text(*u[:, [1]]*s[1]*tscale, 's1*u1', color='red')

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Figur 10.2 demonstrerer at de venstresingulærvektorer giver retningerne hvor variationen af punkterne er hhv. størst og mindst.



Figur 10.2: SVD af punktsamling i planen.

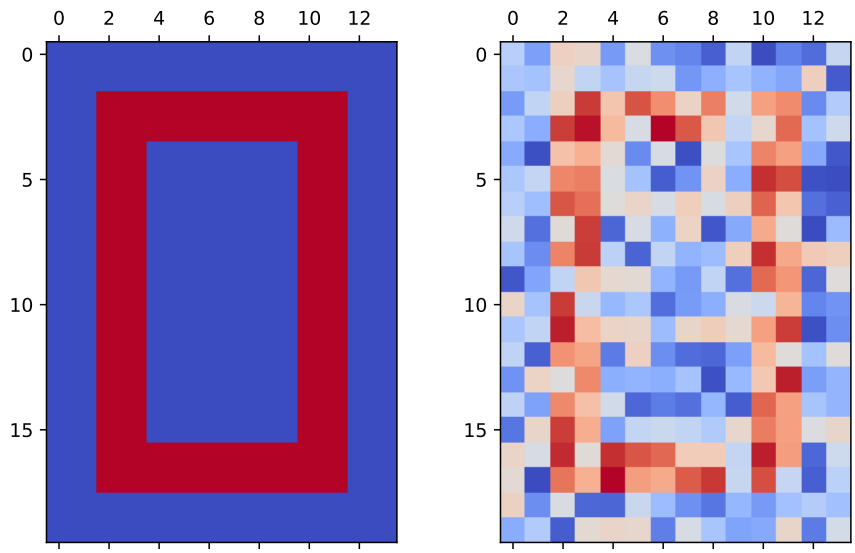
10.3.2 Billede med støj

For det næste eksempel, lad os danne en et simpelt billede af cifret 0:

```
import matplotlib.pyplot as plt
import numpy as np

a = np.zeros((20, 14))
a[2, 2:12] = 1
a[3, 2:12] = 1
a[16, 2:12] = 1
a[17, 2:12] = 1
a[2:18, 2] = 1
a[2:18, 3] = 1
a[2:18, 10] = 1
a[2:18, 11] = 1

fig, ax = plt.subplots()
ax.matshow(a, cmap='coolwarm')
```

Dette giver billedet til venstre i figur 10.3.

Lad os tilføje støj til dette billede. Bemærk at støjniveauet er ret højt i forhold til den oprindelige matrix, se billedet til højre i figur 10.3.



Figur 10.3: Billedet af cifret 0 samt en version med tilføjet støj.

```
rng = np.random.default_rng()
a += 1.5 * rng.random(a.shape)

fig, ax = plt.subplots()
ax.matshow(a, cmap='coolwarm')
```

Vi beregner en tynd SVD og kikker på singulærværdierne

```
u, s, vt = np.linalg.svd(a, full_matrices=False)
np.set_printoptions(linewidth = 60)
print(np.round(s,3))
```

```
[20.164  4.129  3.232  2.699  2.495  2.204  2.094  1.513
  1.44   1.372  1.15   0.943  0.901  0.473]
```

Vi ser at de første to til tre værdier er en del større end resten, så vi kan forvente bedste støjreduktion ved at bruge kun disse værdier. Lad os danne en funktion som angiver SVD for `a` forkortet til r led.

```
def svdapprox(u, s, vt, r):
    "Givet svd u, s, vt angiv forkortelsen til niveau r."
    return u[:, :r] @ np.diag(s[:r]) @ vt[:r, :]
```

Vi kan nu plotte alle disse forkortede SVD. I det følgende løber `axs.flat` over alle delfigurer, og `enumerate` tæller dem. Optælling begynder ved `1` i stedet for `0`, da vi skriver `enumerate(..., start=1)`. Desuden sørger kommandoen `ax.set_axis_off()` for at fjerne akserne omkring hver delfigurer, og `fig.tight_layout()` pakker figurerne tættere sammen end ellers.

```
fig, axs = plt.subplots(3, 4)
for r, ax in enumerate(axs.flat, start=1):
    ax.matshow(svdapprox(u, s, vt, r), cmap='coolwarm')
    ax.set_axis_off()
fig.tight_layout()
```

Som kan ses i figur 10.4 giver den anden og tredje forkortede SVD gode gengivelse af det oprindelige billede

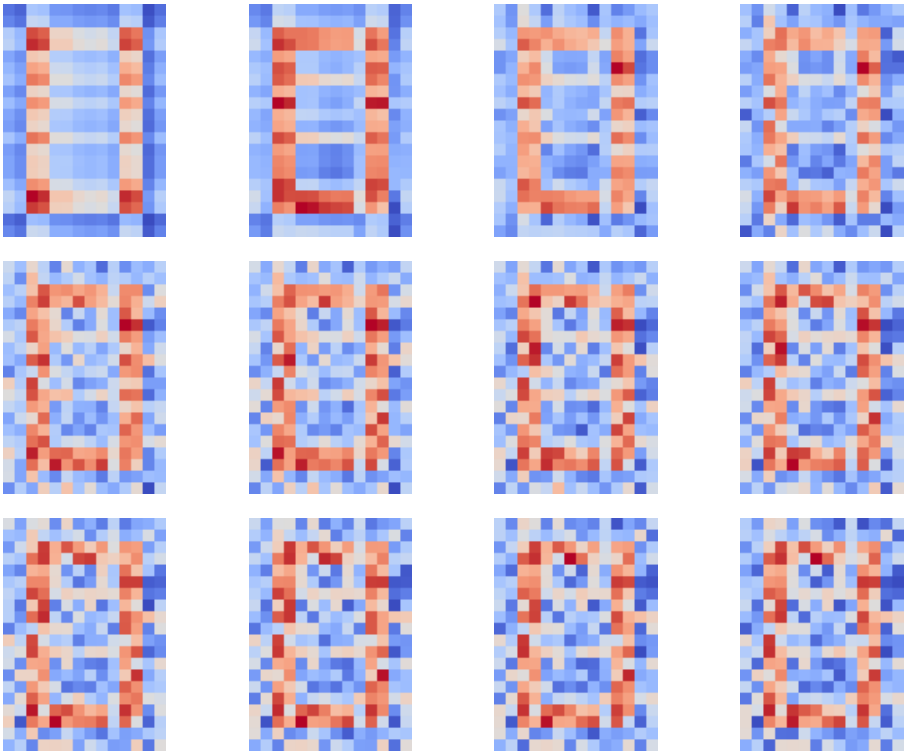
10.3.3 Komprimering af et billede

Lad os arbejde med et billede taget med mobiltelefonen, se figur 10.5, af domkirken i Uppsala. Billedfiler fra mobiltelefon er typisk i `.jpg` format, men man kan også arbejder med andre formater, som `.png`.

Når vi har en `.jpg` kan billedfilen læses ind via `plt.imread` og vises via `imshow`:

```
import matplotlib.pyplot as plt
import numpy as np

a = plt.imread('uppsala-img.jpg')

fig, ax = plt.subplots()
ax.imshow(a)
```

Dette giver en `ndarray` med `shape`



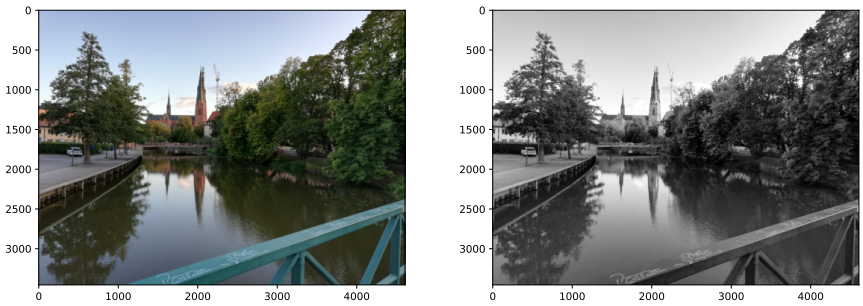
Figur 10.4: SVD af billede med støj.

```
print(a.shape)
```

```
(3456, 4608, 3)
```

Den sidste indeks svarer til tal for rød, grøn og blå farver, i denne rækkefølger. (For en fil i `.png` format vil der også være et sidste tal for gennemsigtigheden.) Billedet her har $3456 \times 4608 = 15925248$ pixels, dvs. $3456 \times 4608/2^{20} = 15.1875$ megapixels.

Typisk får vi nok med at kun kikke på den ene farve. Her vælger vi rød og får en almindelig matrix `b`



Figur 10.5: Uppsala domkirke.

```
b = a[:, :, 0]
print(b.shape)
```

```
(3456, 4608)
```

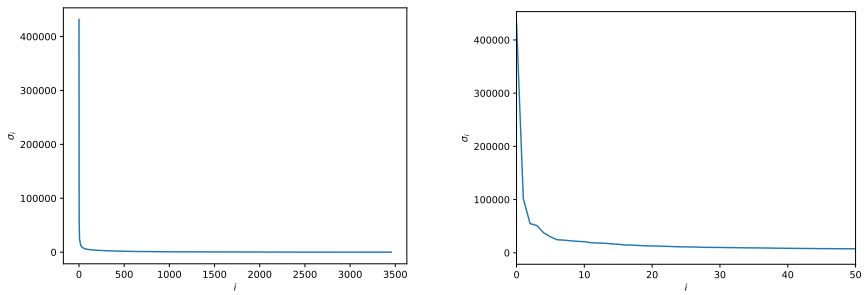
som kan plottes i sort/hvid.

```
fig, ax = plt.subplots()
ax.imshow(b, cmap='Greys_r')
```

Lad os nu beregne den tynde SVD af b. Dette tager lidt tid, men mindre end 1 min på min maskine. Vi kan derefter plotte singulærværdierne, se figur 10.6.

```
u, s, vt = np.linalg.svd(b, full_matrices=False)

fig, ax = plt.subplots()
ax.set_xlabel('$i$')
ax.set_ylabel('$\sigma_{i}$')
ax.plot(s)
```

Vi ser at mange singulærværdier er meget tæt på 0. Vi kikker lidt nærmere på en plot af de første værdier.



Figur 10.6: Singulærværdier for billedet af domkirken.

```
fig, ax = plt.subplots()
ax.set_xlim(0, 50)
ax.set_xlabel('$i$')
ax.set_ylabel('$\sigma_{i}$')
ax.plot(s)
```

Til orientering kan vi se på nogle konkrete singulærværdier

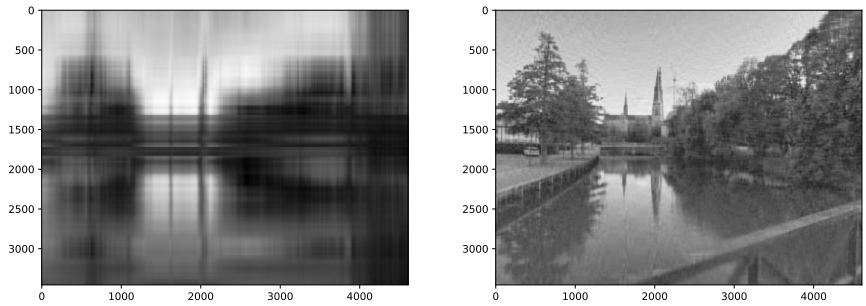
```
print(np.round(s[[0, 2, 10, 50, 100]], 3))
```

```
[431654.686  54864.817  20826.709   7494.8     5049.833]
```

Vi kan forkorte SVD ved forskellige niveauer, og to af disse vises i figur 10.7.

```
def svdapprox(u, s, vt, r):
    "Givet svd u, s, vt angiv trunkering til niveau r."
    return u[:, :r] @ np.diag(s[:r]) @ vt[:r, :]

fig, ax = plt.subplots()
ax.imshow(svdapprox(u, s, vt, 3), cmap='Greys_r')
```

```
fig, ax = plt.subplots()
ax.imshow(svdapprox(u, s, vt, 50), cmap='Greys_r')
```



Figur 10.7: SVD komprimeringer til hhv. 3 og 50 led.

En forkortning med kun 50 singulærværdier giver en god gengivelse af originalen. Dette er også en komprimering.

```
print('Bytes i a: ', a.nbytes)
print('Bytes i b: ', b.nbytes)
komprimeret_size = (u[:, :50].nbytes + s[:50].nbytes +
                    vt[:50, :].nbytes)
print('Bytes for det komprimerede billed: ',
      komprimeret_size)
print('Relativ størrelse af det komprimerede til b')
print('i procent: ', komprimeret_size / b.nbytes * 100)
```

```
Bytes i a:  47775744
Bytes i b:  15925248
Bytes for det komprimerede billed:  3226000
Relativ størrelse af det komprimerede til b
i procent:  20.257141364454732
```

Data for den forkortede SVD med 50 led fylder kun 2,5 % af størrelsen af det oprindelige billede

10.4 Eksistens af SVD

Bevis for sætning 10.2. Idet V er ortogonal er (10.5) det samme som

$$AV = U\Sigma.$$

Hvis V har søjler $v_0, \dots, v_{n-1}$, U har søjler $u_0, \dots, u_{m-1}$ og Σ er som i (10.6), er denne ligning det samme som

$$\big[Av_0 \mid Av_1 \mid \dots\big] = \big[\sigma_0 u_0 \mid \sigma_1 u_1 \mid \dots\big]. \tag{10.8}$$

Så vores opgave er at finde ortonormale samlinger $v_0, \dots, v_{n-1}$ og $u_0, \dots, u_{m-1}$ samt skalarer $\sigma_0, \dots, \sigma_{k-1}$ således at (10.8) gælder.

Betragt funktionen

$$f_0(v) = \|Av\|_2^2, \qquad \text{for } \|v\|_2 = 1.$$

Dette har en maksimumsværdi $s_0 = \|Av_0\|_2^2$, der opnås ved en enhedsvektor v_0. Da $s_0 \geqslant 0$, kan vi skrive $\sigma_0 = \sqrt{s_0} = \|Av_0\|_2 \geqslant 0$. Hvis $\sigma_0 = 0$, har vi $A = 0$, og kan tage $\Sigma = 0$, $U = I_m$, $V = I_n$. Når $\sigma_0 > 0$ sætter vi $u_0 = Av_0/\sigma_0 = Av_0/\|Av_0\|_2$, som er en enhedsvektor. Så har vi

$$Av_0 = \sigma_0 u_0,$$

som er den ønskede ligning for den 0te søjle i (10.8).

Vi vil nu gerne gentage dette argument, men kun med vektorer ortogonale på v_0. For at gøre dette har vi brug for den følgende observation:

> v_0 er et maksimumspunkt for f_0 på enhedsvektorer medfører at $\langle u_0, Av\rangle = 0$ for alle v ortogonal på v_0.

Dette kan ses på følgende måde. Det er nok at bekræfte udsagnet for v, der er en enhedsvektorer ortogonal på v_0. Så er

$$w(t) = \cos(t)v_0 + \sin(t)v$$

en enhedsvektor for alle $t \in \mathbb{R}$. Funktionen $h(t) = f_0(w(t))$, opfylder at $h(0) = f_0(v_0) = s_0$, så $h(0) \geqslant h(t)$ for alle t, og specielt er $h'(0) = 0$. Men

$$\begin{aligned} h(t) = f_0(w(t)) &= \|\cos(t)Av_0 + \sin(t)Av\|_2^2 \\ &= \cos^2(t)\|Av_0\|_2^2 + 2\cos(t)\sin(t)\langle Av_0, Av\rangle + \sin^2(t)\|Av\|_2^2. \end{aligned}$$

Så

$$\begin{aligned} 0 &= h'(0) \\ &= \big(-2\cos(t)\sin(t)\|Av_0\|_2^2 + 2(\cos^2(t) - \sin^2(t))\langle Av_0, Av\rangle \\ &\qquad + 2\sin(t)\cos(t)\|Av\|_2^2\big)\big|_{t=0} \\ &= 2\langle Av_0, Av\rangle. \end{aligned}$$

Derfor er $\langle Av_0, Av\rangle = 0$ for alle v ortogonal på v_0. Da u_0 er proportionelt med Av_0, har vi så vist at

$$Av \perp u_0, \qquad \text{for alle } v \perp v_0.$$

Vi kan nu gentage argumenterne ovenfor ved at kikke på maksimumsværdierne af

$$f_1(v) = \|Av\|_2^2, \qquad \text{for } v \perp v_0 \text{ med } \|v\|_2 = 1,$$

og derefter

$$f_2(v) = \|Av\|_2^2, \qquad \text{for } v \perp v_0, v_1 \text{ med } \|v\|_2 = 1,$$

osv. og stopper når f_r er identisk 0. Vi får $v_0, \ldots, v_{r-1}, u_0, \ldots, u_{r-1}$ og $\sigma_0 \geqslant \sigma_1 \geqslant \cdots \geqslant \sigma_{r-1} > 0$ med

$$Av_j = \sigma_j u_j$$

samt

> $\sigma_j^2 = \|Av_j\|_2^2$ er maksimalværdien af $\|Av\|_2^2$ over enhedsvektorer v ortogonal på $v_0, \ldots, v_{j-1}$

for $j = 0, \ldots, r-1$. Nu udvid til ortonormale baser $v_0, \ldots, v_{n-1}, u_0, \ldots, u_{m-1}$, ved hjælp af korollar 9.21, og sæt $\sigma_r = \cdots = \sigma_{k-1} = 0$, til at få (10.8) og dermed (10.5), som ønsket. □

Kapitel 11

Konditionstal

Vi har set at fejl kan opstå fra forskellige kilder: fejl i den oprindelige data, fejl ved repræsentation på computeren, fejl i beregningsoperationer. Konditionstal giver et praj om om hvor meget fejl sendes videre igennem beregninger og om hvor følsomt problemet er ovenfor fejl.

11.1 Konditionstal for differentiable funktioner

Betragt funktionen der beregner $f(x) = x^2$. Hvis x ændres til $x + h$, så ændres $f(x)$ til

$$f(x+h) = (x+h)^2 = x^2 + 2xh + h^2.$$

Ændringen i f er så

$$f(x+h) - f(x) = 2xh + h^2 = h(2x+h).$$

Vi er interesseret i hvordan denne differens opfører sig for h tæt på 0. Man definerer det *absolutte konditionstal* til at være

$$\hat{\kappa}(x) = \lim_{h \to 0} \frac{|f(x+h) - f(x)|}{|h|} = |f'(x)|.$$

Så for vores funktion $f(x) = x^2$ er

$$\hat{\kappa}(x) = |2x| = 2|x|.$$

Dette siger at en lille ændring i x fører til en ændring i $f(x)$, som er $2|x|$ større.

Vi har set tidligere at især i forbindelse med `float` er det relevant at beregne relative fejl. Tilsvarende kan vi definere det *relative konditionstal*, eller blot *konditionstallet* $\kappa(x)$ til at være forholdet mellem de relative ændringer af $f(x)$ og x. Dette giver følgende definition for konditionstallet:

$$\begin{aligned}\kappa(x) &= \lim_{h\to 0}\left(\frac{|f(x+h)-f(x)|}{|f(x)|} \Big/ \frac{|h|}{|x|}\right)\\ &= \lim_{h\to 0}\left(\frac{|f(x+h)-f(x)|}{|h|}\frac{|x|}{|f(x)|}\right)\\ &= \frac{|x||f'(x)|}{|f(x)|},\end{aligned}$$

forudsat at $f(x) \neq 0 \neq x$.

For funktionen $f(x) = x^2$ har vi $\kappa(x) = |x||2x|/|x^2| = 2$. Dvs. en lille relative fejl i x afspejles i en relativ fejl, der er dobbelt så stort, i $x^2 = f(x)$.

Eksempel 11.1. $f(x) = \sqrt{x}$, $x > 0$, har konditionstal

$$\kappa(x) = |x|\left|\frac{d}{dx}\sqrt{x}\right| \Big/ |\sqrt{x}| = |x|\left|\frac{1}{2\sqrt{x}}\right| \Big/ |\sqrt{x}| = \left|\frac{\sqrt{x}}{2}\right| \Big/ |\sqrt{x}| = \frac{1}{2}.$$

Det betyder at en relativ fejl i x, giver en relativ fejl i $f(x) = \sqrt{x}$, som er halvt så stort. △

Eksempel 11.2. $f(x) = e^x$ har konditionstal

$$\kappa(x) = |x|\left|\frac{d}{dx}e^x\right| \Big/ |e^x| = |x|e^x \;/\; e^x = |x|.$$

Så beregning af e^x for x stort kan forventes at have en stor relativ fejl, uanset beregningsmetoden. △

Konditionstal af størrelsesorden 1 eller 10 er til at tåle; vi mister få betydende cifre i beregning af svaret. Omvendt betyder et konditionstal af størrelsesorden 10^6, at en relativ fejl i x forstørres voldsomt, og dette er ofte uacceptabel.

Når vi arbejder med vektorer, skal vi justere lidt på de overnævnte definitioner. Først skal vi erstatte numerisk værdi med normen af vektoren. For $f\colon \mathbb{R}^n \to \mathbb{R}$ erstattes den afledede med gradienten

$$(\nabla f)_x = \left(\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_{n-1}}\right).$$

For en differentiabel $f\colon \mathbb{R}^n \to \mathbb{R}$, defineres *konditionstallet* i x til at være

$$\kappa(x) = \frac{\|x\|_2 \|(\nabla f)_x\|_2}{|f(x)|}.$$

Eksempel 11.3. Betragt funktionen $f\colon \mathbb{R}^2 \to \mathbb{R}$, som beregner differensen,

$$f(x_0, x_1) = x_1 - x_0$$

af tallene x_0 og x_1. Gradienten af f er

$$(\nabla f)_x = \left(\frac{\partial(x_1 - x_0)}{\partial x_0}, \frac{\partial(x_1 - x_0)}{\partial x_1} \right) = (-1, 1).$$

Konditionstallet for f beregnes til at være

$$\kappa(x) = \frac{\|x\|_2 \|(-1,1)\|_2}{|x_1 - x_0|} = \sqrt{2} \frac{\sqrt{x_0^2 + x_1^2}}{|x_1 - x_0|}. \tag{11.1}$$

Vi ser at konditionstallet er stort for x_0 tæt på x_1. Dette understøtter det vi har tidligere set (f.eks. eksempel 1.4) at differenceoperationen kan føre til stor fejl når tallene er tæt på hinanden.

Lad os kikke lidt nærmere på dette eksempel, for at understøtte definitionen på $\kappa(x)$.

Hvis (x_0, x_1) ændres med en vektor (h_0, h_1), er ændringen i f:

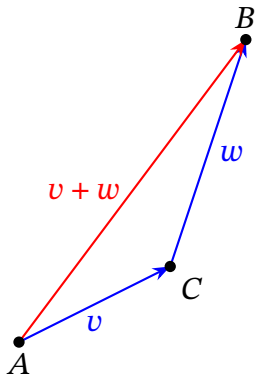
$$\begin{aligned} f(x_0 + h_0, x_1 + h_1) - f(x_0, x_1) &= ((x_1 + h_1) - (x_0 + h_0)) - (x_1 - x_0) \\ &= h_1 - h_0. \end{aligned}$$

Vi kan skrive det sidste som et indre produkt

$$f(x_0 + h_0, x_1 + h_1) - f(x_0, x_1) = h_1 - h_0 = \left\langle \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \begin{bmatrix} h_0 \\ h_1 \end{bmatrix} \right\rangle.$$

Bemærk at vektoren $(-1, 1)$ er $(\nabla f)_x$. Nu giver Cauchy-Schwarz uligheden at den absolutte fejl i f ved x er

$$\begin{aligned} &\left| f(x_0 + h_0, x_1 + h_1) - f(x_0, x_1) \right| \\ &\quad = \left| \left\langle \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \begin{bmatrix} h_0 \\ h_1 \end{bmatrix} \right\rangle \right| \leqslant \left\| \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right\|_2 \left\| \begin{bmatrix} h_0 \\ h_1 \end{bmatrix} \right\|_2 = \sqrt{2} \|h\|_2. \end{aligned}$$



Figur 11.1: Trekantsuligheden.

Bemærk at Cauchy-Schwarz uligheden sikrer også at der er lighed for visse h: nemlig de h, som er er parallel med $(-1,1)$. Så den størst mulige fejl er givet ved den højre side $\sqrt{2}\|h\|_2$. Beregner vi forholdet mellem den relative fejl i f og den relative ændring i x, får vi

$$\frac{|f(x+h)-f(x)|}{|f(x)|} \Big/ \frac{\|h\|_2}{\|x\|_2} = \frac{|f(x+h)-f(x)|}{\|h\|_2}\frac{\|x\|_2}{|f(x)|} \leqslant \sqrt{2}\frac{\sqrt{x_0^2+x_1^2}}{|x_1-x_0|},$$

og denne grænse opnås for h parallel med $(-1,1)$. Denne maksimal værdi er netop konditionstallet $\kappa(x)$ i (11.1). △

11.2 Matrixnorm

Vi ønsker at udforske tilsvarende ændringer i relation til lineære ligningssystemer $Ax = b$. Det kræver dog at vi har et norm-begreb for matricen A.

Definition 11.4. En *norm* på $\mathbb{R}^n$ er en funktion $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ med

(N1) $\|v\| \geqslant 0$ for alle $v \in \mathbb{R}^n$, og $\|v\| = 0$ kun for $v = 0$,
(N2) $\|sv\| = |s|\|v\|$ for alle $s \in \mathbb{R}$,
(N3) $\|v+w\| \leqslant \|v\| + \|w\|$ for alle $v, w \in \mathbb{R}^n$.

Vi har set at 2-normen $\|\cdot\|_2 = \sqrt{\langle\,\cdot\,,\,\cdot\,\rangle}$ opfylder (N1) og (N2). Betingelse (N3) kaldes *trekantsuligheden*, da den siger at i figur 11.1 er vejen direkte fra A til B kortere end turen langs de andre to sider af trekanten via punktet C.

Trekantsuligheden gælder for $\|\cdot\|_2$, da

$$\begin{aligned}\|v+w\|_2^2 &= \langle v+w, v+w\rangle = \langle v,v\rangle + \langle v,w\rangle + \langle w,v\rangle + \langle w,w\rangle \\ &= \|v\|_2^2 + 2\langle v,w\rangle + \|w\|_2^2 \\ &\leqslant \|v\|_2^2 + 2\|v\|_2\|w\|_2 + \|w\|_2^2 \\ &= (\|v\|_2 + \|w\|_2)^2,\end{aligned}$$

hvor uligheden mellem linjer to og tre stammer fra Cauchy-Schwarz uligheden (8.4).

Der findes andre normer på $\mathbb{R}^n$. To første eksempler er *1-normen*

$$\|v\|_1 = \|(v_0, v_1, \ldots, v_{n-1})\|_1 = |v_0| + |v_1| + \cdots + |v_{n-1}|$$

og maksimumsnormen eller *∞-normen*

$$\|v\|_\infty = \|(v_0, v_1, \ldots, v_{n-1})\|_\infty = \max\{|v_0|, |v_1|, \ldots, |v_{n-1}|\}.$$

Numerisk kan det være hurtigere at beregne $\|v\|_1$ eller $\|v\|_\infty$ end $\|v\|_2$. Desuden indeholder disse normer sammenlignelig oplysning om v, da for $v \in \mathbb{R}^n$ gælder

$$\|v\|_\infty \leqslant \|v\|_2 \leqslant \|v\|_1 \leqslant \sqrt{n}\|v\|_2 \leqslant n\|v\|_\infty.$$

Som navnene antyder er de overnævnte normer en del af en større familie. For hvert tal p med $1 \leqslant p \leqslant \infty$ er *p-normen* givet ved

$$\|v\|_p = \|(v_0, v_1, \ldots, v_{n-1})\|_p = \left(\sum_{i=0}^{n-1} |v_i|^p\right)^{1/p}$$

For matricer $A \in \mathbb{R}^{m\times n}$ indfører vi en matrix eller operator *2-norm* på den følgende måde

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2. \tag{11.2}$$

Dvs. $\|A\|_2$ måler den største skalering af længden af x når vi ganger med A. At (11.2) definerer en norm, følger direkte fra egenskaberne af $\|\cdot\|_2$ for vektorer. Ligning (11.2) er det samme som

$$\|A\|_2 = \max_{x\neq 0} \frac{\|Ax\|_2}{\|x\|_2}, \tag{11.3}$$

som ofte gives som definition. Dette ækvivalens gælder, da

$$\frac{\|Ax\|_2}{\|x\|_2} = \frac{1}{\|x\|_2}\|Ax\|_2 = \left\|\frac{1}{\|x\|_2}Ax\right\|_2 = \left\|A\left(\frac{1}{\|x\|_2}x\right)\right\|_2 = \left\|A\left(\frac{x}{\|x\|_2}\right)\right\|_2$$

og $x/\|x\|_2$ er en enhedsvektor. Omskrivningen (11.3) giver den nyttige ulighed

$$\|Ax\|_2 \leqslant \|A\|_2\|x\|_2, \qquad \text{for alle } x \in \mathbb{R}^n. \tag{11.4}$$

Man kan definere en tilsvarende p-norm for A ved $\|A\|_p = \max_{\|x\|_p=1}\|Ax\|_p$, men vi vil holdes os mest til $\|A\|_2$, som har de pæneste egenskaber.

Lemma 11.5. *For $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times p}$ gælder*

$$\|AB\|_2 \leqslant \|A\|_2\|B\|_2.$$

Bevis. Det følger fra (11.4) at

$$\|ABx\|_2 = \|A(Bx)\|_2 \leqslant \|A\|_2\|Bx\|_2 \leqslant \|A\|_2\|B\|_2\|x\|_2.$$

Så for $\|x\|_2 = 1$, gælder

$$\|ABx\|_2 \leqslant \|A\|_2\|B\|_2.$$

Vi har nu at maksimumsværdien for $\|ABx\|_2$ over $\|x\|_2 = 1$ er højst $\|A\|_2\|B\|_2$. Men det er netop udsagnet $\|AB\|_2 \leqslant \|A\|_2\|B\|_2$, der ønskes. □

Dette 2-norm for A er faktisk noget vi har set før:

Lemma 11.6. *For $A \in \mathbb{R}^{m\times n}$, lad $A = U\Sigma V^T$ være en singulærværdidekomponering for A, med singulærværdier $\sigma_0 \geqslant \cdots \geqslant \sigma_{k-1} \geqslant 0$, hvor $k = \min\{m, n\}$. Så er 2-normen af A givet ved*

$$\|A\|_2 = \sigma_0,$$

hvor σ_0 er den største singulærværdi.

Bevis. Matricen $U \in \mathbb{R}^{m\times m}$ er ortogonal, så $\|Uy\|_2 = \|y\|_2$ for alle $y \in \mathbb{R}^m$, se proposition 9.5. Vi har så at

$$\|Ax\|_2 = \|U\Sigma V^T x\|_2 = \|\Sigma V^T x\|_2. \tag{11.5}$$

For at bestemme $\|A\|_2$ skal vi så

$$\text{maksimere (11.5) over alle } x \text{ med } \|x\|_2 = 1. \tag{11.6}$$

Men V^T er også ortogonal, så bevarer $\|\cdot\|_2$ og er invertibel. Ved at skrive $w = V^T x$, får vi alle w med $\|w\|_2 = 1$, fra de forskellige x med $\|x\|_2 = 1$. Det giver at (11.6) er det sammen som at

$$\text{maksimere } \|\Sigma w\|_2 \text{ over alle } w \text{ med } \|w\|_2 = 1. \tag{11.7}$$

Bemærk nu at da $\sigma_i \leqslant \sigma_0$ for alle i, har vi for $w = (w_0, \ldots, w_{n-1})$ at

$$\begin{aligned}\|\Sigma w\|_2 &= \sqrt{\sigma_0^2 w_0^2 + \cdots + \sigma_{k-1}^2 w_{k-1}^2} \\ &\leqslant \sqrt{\sigma_0^2 (w_0^2 + \cdots + w_{n-1}^2)} = \sigma_0 \|w\|_2\end{aligned}$$

og at vi har lighed for $w = e_0$. Det følger at

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \max_{\|w\|_2=1} \|\Sigma w\|_2 = \sigma_0,$$

som ønsket. □

11.3 Konditionstal og lineære ligningssystemer

Lad os nu betragter et lineært ligningssystem med det samme antal ligninger, som variabler:

$$Ax = b$$

for $x, b \in \mathbb{R}^n$, $A \in \mathbb{R}^{n\times n}$. Lad os desuden antage at den kvadratiske matrix A er invertibel, så for hvert b er der en entydig løsning $x = A^{-1}b$. Der er flere forskellige måder at beregne et konditionstal for systemet, med det viser sig at alle kontrolleres af *konditionstallet for A*

$$\kappa(A) = \|A\|_2 \|A^{-1}\|_2.$$

Proposition 11.7. *Konditionstallet for en invertibel matrix $A \in \mathbb{R}^{n\times n}$, er givet ved*

$$\kappa(A) = \sigma_0/\sigma_{n-1} \tag{11.8}$$

hvor $\sigma_0 \geqslant \cdots \geqslant \sigma_{n-1} > 0$ er singulærværdierne for A.

Bevis. Lemma 11.6 viser netop at $\|A\|_2 = \sigma_0$. Skriver vi A's singulærværdidekomponering, som $A = U\Sigma V^T$, har vi for A invertibel at $A^{-1} = V\Sigma^{-1}U^T$. Men

$$\Sigma^{-1} = \begin{bmatrix} \sigma_0 & 0 & \dots & 0 \\ 0 & \sigma_1 & & 0 \\ \vdots & & \ddots & \\ 0 & 0 & & \sigma_{n-1} \end{bmatrix}^{-1} = \begin{bmatrix} 1/\sigma_0 & 0 & \dots & 0 \\ 0 & 1/\sigma_1 & & 0 \\ \vdots & & \ddots & \\ 0 & 0 & & 1/\sigma_{n-1} \end{bmatrix}.$$

Det følger at A^{-1} har singulærværdier $1/\sigma_{n-1} \geqslant \cdots \geqslant 1/\sigma_0$, og den singulærværdidekomponering af A^{-1} er

$$A^{-1} = \left[v_{n-1} \mid \dots \mid v_1 \mid v_0 \right] \begin{bmatrix} 1/\sigma_{n-1} & 0 & \dots & 0 \\ & \ddots & & \vdots \\ 0 & & 1/\sigma_1 & 0 \\ 0 & \dots & 0 & 1/\sigma_0 \end{bmatrix} \begin{bmatrix} u_{n-1}^T \\ \vdots \\ u_1^T \\ u_0^T \end{bmatrix}$$

hvor u_i og v_j er søjlerne af U hhv. V. Vi har så at $\|A^{-1}\|_2 = 1/\sigma_{n-1}$, og resultatet følger. □

Lad os først betragte funktionen $f\colon \mathbb{R}^n \to \mathbb{R}^n$

$$f(x) = Ax,$$

for en given invertibel matrix A. Ændrer vi x til $x + h$, ændres $f(x)$ til

$$f(x+h) = A(x+h) = Ax + Ah.$$

Den relative ændring i $f(x)$ i forhold til den for x har størrelse

$$\begin{aligned} \frac{\|f(x+h) - f(x)\|_2}{\|f(x)\|_2} \Bigg/ \frac{\|h\|_2}{\|x\|_2} &= \frac{\|Ah\|_2}{\|Ax\|_2} \Bigg/ \frac{\|h\|_2}{\|x\|_2} \\ &= \frac{\|Ah\|_2}{\|h\|_2} \frac{\|x\|_2}{\|Ax\|_2}. \end{aligned}$$

Dens maksimal værdi når vi varierer h er så

$$\kappa(x) = \|A\|_2 \frac{\|x\|_2}{\|Ax\|_2},$$

som opnås for h parallelt med v_0.

Betragt nu ændringer i x. Vi kan skrive $b = Ax$, så $x = A^{-1}b$, og har

$$\frac{\|x\|_2}{\|Ax\|_2} = \frac{\|A^{-1}b\|_2}{\|b\|_2}$$

som har maksimumsværdi $\|A^{-1}\|_2$. Det følger at

$$\max_{x \neq 0} \kappa(x) = \|A\|_2 \|A^{-1}\|_2 = \kappa(A), \tag{11.9}$$

med maksimum opnået for x parallelt med v_{n-1}.

Eksempel 11.8. Betragt ligningssystemet

$$\begin{aligned} 7x_0 + 3x_1 &= 5, \\ 9x_0 + 4x_1 &= -1. \end{aligned}$$

Den tilhørende matrix er

$$A = \begin{bmatrix} 7 & 3 \\ 9 & 4 \end{bmatrix},$$

hvis konditionstal kan beregnes i python:

```
>>> import numpy as np
>>> a = np.array([[7.0, 3.0],
...               [9.0, 4.0]])

>>> u, s, vt = np.linalg.svd(a)
>>> s
array([12.44964048,  0.0803236 ])
>>> # konditionstal for a
>>> s[0] / s[-1]
154.99354811853624
```

Tages x til at være den sidste vektor $v_1 = v_{n-1}$ fra SVD for $A = U\Sigma V^T$, $V = \begin{bmatrix} v_0 \mid v_1 \end{bmatrix}$ og h til at være proportionelt med den første vektor v_0, ser vi at ændringen fra Ax til $A(x+h)$ er ret stort

```
>>> x = vt[[1], :].T
>>> x
array([[-0.40157455],
```

```
       [ 0.91582634]])
>>> a @ x
array([[-0.06354287],
       [ 0.04913435]])
>>> h = 0.1 * vt[[0], :].T
>>> h
array([[-0.09158263],
       [-0.04015746]])
>>> a @ (x+h)
array([[-0.82509367],
       [-0.93573917]])
>>> ((np.linalg.norm(a @ h)/np.linalg.norm(h))
...  * (np.linalg.norm(x)/np.linalg.norm(a @ x)))
154.99354811853738
```

Dette er nemlig det størst mulige relativ fejl. Vælges x og h tilfældigt er fejlen typisk mindre

```
>>> rng = np.random.default_rng()
>>> x = rng.standard_normal((a.shape[1], 1))
>>> a @ x
array([[10.3987852 ],
       [13.37142441]])
>>> h = rng.normal(0.0, 0.1, (a.shape[1], 1))
>>> a @ (x+h)
array([[10.78283649],
       [13.88192976]])
>>> a @ (x+h)
array([[10.78283649],
       [13.88192976]])
>>> ((np.linalg.norm(a @ h) / np.linalg.norm(h))
...  * (np.linalg.norm(x) / np.linalg.norm(a @ x)))
0.4766659456458515
```

Faktisk er fejlen mindst for x parallelt med v_0 og h parallelt med $v_1 = v_{n-1}$. $\triangle$

Hvis vi er interesseret i hvordan løsninger x til $Ax = b$ ændrer sig når b varierer, kan vi bruge den samme analyse, men for funktionen $b \mapsto x = A^{-1}b$.

Dvs. A udskiftes med A^{-1} ovenfor. Men så er resultatet det samme udtryk (11.9); hvor rollerne af v_0 og v_{n-1} spilles nu af u_{n-1} og u_0.

Desuden hvis man holder b fast, og lad A varierer, dvs. betragter funktionen $A \mapsto x = A^{-1}b$, kan man også vise at ændringerne styres også af (11.9).

Motiveret af (11.8) definerer man *konditionstallet for $A \in \mathbb{R}^{m\times n}$* til at være

$$\kappa(A) = \frac{\sigma_0}{\sigma_{k-1}}, \tag{11.10}$$

hvor $k = \min\{m, n\}$. Når $A \neq 0$, betragtes dette $\kappa(A)$ som et tal i $[0, \infty]$. Nogle af de overnævnte resultater kan så overføres til denne situation.

Kapitel 12

Generelle vektorrum

12.1 Vektorrum

Indtil videre har vi kun betragtet vektorer som elementer i $\mathbb{R}^n$. Men i det forrige kapitel, så vi at matricer i $\mathbb{R}^{m\times n}$ kan også håndteres på en lignende måde. Både $\mathbb{R}^n$ og $\mathbb{R}^{m\times n}$ er eksempler på vektorrum, men der er andre objekter der kan med fordel behandles på en tilsvarende måde.

Definition 12.1. Et *vektorrum* består af en mængde V, hvis elementer vi kalder *vektorer*. Der skal være to operationer defineret. Den første er en *sumoperation* $u + v \in V$ for alle $u, v \in V$, som opfylder

(V1) $u + v = v + u$, for alle $u, v \in V$,
(V2) $(u + v) + w = u + (v + w)$, for alle $u, v, w \in V$,
(V3) der findes $0 \in V$ således at $v + 0 = v$ for alle $v \in V$,
(V4) for hver $v \in V$ findes der $-v \in V$ således at $v + (-v) = 0$.

Den anden operation er *multiplikation med en skalar* $sv \in V$ defineret for alle skalar s og alle $v \in V$, med følgende egenskaber

(V5) $s(u + v) = su + sv$,
(V6) $(s + t)v = sv + tv$,
(V7) $(st)v = s(tv)$,
(V8) $1v = v$,

for alle skalarer s og t, og alle $u, v \in V$.

Eksempel 12.2. $V = \mathbb{R}^n = \mathbb{R}^{n\times 1}$ med vektorsum og skalar multiplikation som vi har brugt tidligere opfylder dette. Den eneste der skal bemærkes er elementet 0

i del (V3) er vektoren

$$0 = 0_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in \mathbb{R}^n.$$

△

Eksempel 12.3. Rummet af matricer $V = \mathbb{R}^{m\times n}$ er ligeledes et vektorrum. Denne gang er nulelementet matricen $0_{m\times n}$. Bemærk at rummet af matricer har yderlige matrixmultiplikation, som en ekstra operation ud over de to operationer i definition 12.1. △

Eksempel 12.4. Lad V består af alle polynomier af grad højst 2. Et typisk element i V er

$$p(x) = a_0 + a_1x + a_2x^2$$

med $a_0, a_1, a_2 \in \mathbb{R}$. Vi kan definere en sumoperation for $p(x) = a_0 + a_1x + a_2x^2$ og $q(x) = b_0 + b_1x + b_2x^2$ ved

$$(p + q)(x) = (a_0 + b_0) + (a_1 + b_1)x + (a_2 + b_2)x^2.$$

Så kan man tjekke at dele (V1) til (V4) er opfyldt med nulpolynomiet givet ved

$$0(x) = 0 + 0x + 0x^2.$$

Med skalarmultiplikation givet ved

$$(sp)(x) = sa_0 + (sa_1)x + (sa_2)x^2,$$

er de resterende punkter i definition 12.1 opfyldt, og vi får et vektorrum. △

Eksempel 12.5. Eksempel 12.4 kan udvides til polynomier af grad højst n, eller til alle polynomier, på tilsvarende måde. △

Lemma 12.6. *Lad V være et vektorrum og lad $v \in V$.*
(a) *Så kan nulelementet beregnes som $0 = 0v$.*
(b) *Elementet $-v$ er givet som $-v = (-1)v$.*
Begge elementer er entydigt bestemt.

Bevis. Først er 0 entydig: hvis $v + u = v$, så har vi $0 = v - v = (v + u) - v = u$, dvs. $u = 0$. Nu gælder $v + 0v = 1v + 0v = (1 + 0)v = 1v = v$, så $0v = 0$.

Elementet $-v$ er entydig, da $v + w = 0$ giver $-v = -v + 0 = -v + (v + w) = (-v + v) + w = 0 + w = w$, så $w = -v$. Vi beregner nu $v + (-1)v = 1v + (-1)v = (1 - 1)v = 0v = 0$, så $(-1)v = -v$. □

Eksempel 12.7. Lad V består af alle differentiable funktioner $f\colon [a,b] \to \mathbb{R}$. Vi sætter $f+g$ til at være funktionen givet ved

$$(f+g)(x) = f(x) + g(x), \qquad \text{for alle } x \in [a,b],$$

og definerer skalarmultiplikation ved

$$(sf)(x) = s(f(x)), \qquad \text{for alle } x \in [a,b].$$

Bemærk at både $f+g$ og sf er differentiable funktioner. Vi får et vektorrum, med nulvektor funktionen $0(x) = 0$, for alle $x \in [a,b]$. △

12.2 Andre skalarer

Andre eksempler på vektorrum gives ved at bruge andre typer skalarer. Indtil videre har vores skalarer været reelle tal $s \in \mathbb{R}$. Men vektorrum kan dannes med andre koefficienter. Tænkes der på hvordan vi har løst lineære ligningssystemer, er det væsentlig at vi kan tage sum og differens af skalarer og at vi kan gange og dele med dem. Vigtigst for os vil være at vi kan bruge komplekse tal.

De *komplekse tal* betegnes $\mathbb{C}$. Et element $z \in \mathbb{C}$ skrives

$$z = x + iy$$

hvor $x, y \in \mathbb{R}$ og i er et symbol, med $i^2 = -1$. Vi kalder $x = \mathrm{Re}(z)$ den *reelle* del af z, og $y = \mathrm{Im}(z)$ den *imaginære* del. Beregning med komplekse tal udføres netop som for reelle tal blot med den ekstra relation at $i^2 = -1$.

Eksempel 12.8. For $z = 1 + 2i$, $w = -2 + 3i$ har vi

$$\begin{aligned} z + w &= (1+2i) + (-2+3i) \\ &= 1 + (-2) + i(2+3) \\ &= -1 + 5i \end{aligned}$$

og

$$\begin{aligned} zw &= (1+2i)(-2+3i) \\ &= 1 \times (-2) + 1 \times 3i + 2i \times (-2) + (2i) \times (3i) \\ &= -2 + 3i - 4i + 6i^2 \\ &= -2 - i - 6 \\ &= -8 - i. \end{aligned}$$

△

For $z = x + iy$, $w = s + it$, med $x, y, s, t \in \mathbb{R}$, får vi så

$$z + w = (x + iy) + (s + it) = (x + s) + i(y + t), \tag{12.1}$$
$$zw = (x + iy)(s + it) = (xs - yt) + i(xt + ys). \tag{12.2}$$

(12.1) er bare vektorsum i $\mathbb{R}^2$ oversat til denne ny notation. (12.2) kommer fra at gange $(x + iy)(s + it)$ ud på almindeligvis samt reglen $i^2 = -1$ som i eksemplet.

Vi kan godt regne med komplekse tal i python (her er python en del bedre end andre programmeringssprog). Vi skal dog gå opmærksom på at python bruger `j` for tallet i, i tråd med god traditioner for elektriske ingeniører:

```
>>> import numpy as np
>>> z = 1.0 + 2.0j
>>> type(z)
<class 'complex'>
>>> np.real(z)
1.0
>>> np.imag(z)
2.0
>>> w = -2.0 + 3.0j
>>> z + w
(-1+5j)
>>> z * w
(-8-1j)
```

Man skal være opmærksom på at der skal stå et tal direkte foran `j`, dvs. uden mellemrum og uden `*`. F.eks. $1 - i$ skrives `1-1j` eller `1.0 - 1.0j`.

Man kan også dele med komplekse tal, som er forskellig fra $0 = 0 + 0i$. Opskriften er

$$\frac{1}{z} = \frac{1}{x + iy} = \frac{x - iy}{x^2 + y^2}. \tag{12.3}$$

Ved direkte udregning kan man tjekke at $z(1/z) = 1$:

$$\begin{aligned} z\frac{1}{z} &= (x + iy)\frac{x - iy}{x^2 + y^2} = \frac{1}{x^2 + y^2}(x + iy)(x - iy) \\ &= \frac{1}{x^2 + y^2}(x^2 - ixy + ixy - i^2 y) = \frac{x^2 + y^2}{x^2 + y^2} = 1. \end{aligned}$$

Eksempel 12.9. For $z = 1 + 2i$ har vi

$$\frac{1}{z} = \frac{1}{1+2i} = \frac{1-2i}{1^2+2^2} = \frac{1-2i}{5} = \frac{1}{5} - \frac{2}{5}i,$$

som python kan bekræfte

```
>>> z = 1.0 + 2.0j
>>> 1/z
(0.2-0.4j)
>>> z * (1/z)
(1+0j)
```

△

Vi kan også arbejde med lineære ligningssystemer hvis koefficienter er komplekse tal på præcis den samme måde med rækkeoperationer, som vi gjorde tidlige.

Eksempel 12.10. Systemet

$$\begin{aligned} 2x + \quad 3y - \quad 4z &= 7 - i, \\ 3ix - (4+i)y + \quad z &= -2, \\ x + \quad y + (2+i)z &= 3 + i \end{aligned}$$

kan løses via rækkeoperationer. F.eks. i python

```
>>> import numpy as np
>>> a = np.array([[2.0, 3.0,          -4.0     ],
...               [3.0j, -(4.0+1.0j), 1.0      ],
...               [1.0, 1.0,           2.0+1.0j]])
>>> a.dtype
dtype('complex128')
>>> b = np.array([7.0-1.0j, -2.0, 3.0+1.0j])[:, np.newaxis]
>>> aub = np.hstack([a, b])
>>> aub
array([[ 2.+0.j,  3.+0.j, -4.+0.j,  7.-1.j],
       [ 0.+3.j, -4.-1.j,  1.+0.j, -2.+0.j],
       [ 1.+0.j,  1.+0.j,  2.+1.j,  3.+1.j]])
>>> aub[ [0,2], :] = aub[ [2,0], :]
```

```
>>> aub[1, :] += -3.0j * aub[0, :]
>>> aub[2, :] += -2.0 * aub[0, :]
>>> aub
array([[ 1.+0.j,  1.+0.j,  2.+1.j,  3.+1.j],
       [ 0.+0.j, -4.-4.j,  4.-6.j,  1.-9.j],
       [ 0.+0.j,  1.+0.j, -8.-2.j,  1.-3.j]])
>>> aub[ [1,2], :] = aub[ [2,1], :]
>>> aub[0, :] += - aub[1, :]
>>> aub[2, :] += (4.0 + 4.0j) * aub[1, :]
>>> aub
array([[  1. +0.j,   0. +0.j,  10. +3.j,   2. +4.j],
       [  0. +0.j,   1. +0.j,  -8. -2.j,   1. -3.j],
       [  0. +0.j,   0. +0.j, -20.-46.j,  17.-17.j]])
>>> aub[2, :] *= 1/(-20.0-46.0j)
>>> aub[0, :] += -(10.0+3.0j) * aub[2, :]
>>> aub[1, :] += (8.0+2.0j) * aub[2, :]
>>> aub
array([[ 1.        +0.j        ,  0.        +0.j        ,
         0.        +0.j        ,  1.58108108-0.98648649j],
       [ 0.        +0.j        ,  1.        +0.j        ,
         0.        +0.j        ,  1.51351351+0.91891892j],
       [-0.        +0.j        , -0.        +0.j        ,
         1.        +0.j        ,  0.17567568+0.44594595j]])
>>> print('x = ', aub[0, -1])
x =  (1.581081081081081-0.9864864864864868j)
>>> print('y = ', aub[1, -1])
y =  (1.5135135135135136+0.9189189189189189j)
>>> print('z = ', aub[2, -1])
z =  (0.17567567567567569+0.44594594594594594j)
>>> # Tjek af løsning
>>> v = aub[:, [-1]]
>>> a @ v - b
array([[-8.88178420e-16-8.88178420e-16j],
       [ 1.11022302e-15-5.55111512e-16j],
       [ 0.00000000e+00-4.44089210e-16j]])
>>> machine_epsilon = np.finfo(float).eps
>>> np.allclose(a @ v, b, atol = machine_epsilon)
```

```
True
```

△

Formlen (12.3) indeholder to væsentlig operationer på $z = x + iy$. Den første er den *konjugerede*

$$\overline{z} = \overline{x + iy} = x - iy.$$

```
>>> z = 1.0 + 2.0j
>>> np.conj(z)
(1-2j)
```

Den anden den *numeriske værdi*

$$|z| = |x + iy| = \sqrt{x^2 + y^2},$$

som er blot normen af vektoren (x, y) i $\mathbb{R}^2$.

```
>>> import numpy as np
>>> z = 1.0 + 2.0j
>>> np.abs(z)
2.23606797749979
>>> np.sqrt(np.real(z)**2+np.imag(z)**2)
2.23606797749979
```

Vi kan så omskrive formlen (12.3), som

$$\frac{1}{z} = \frac{\overline{z}}{|z|^2}$$

Bemærk at $|z|^2 = x^2 + y^2 = (x + iy)(x - iy) = z\overline{z}$. Vi har også

$$|zw| = |z||w|$$

for alle komplekse tal z og w. Konjugation opfylder

$$\overline{z + w} = \overline{z} + \overline{w}, \quad \overline{zw} = \overline{z}\,\overline{w}.$$

Bemærkning 12.11. Vi vil udlukkende brug $\mathbb{R}$ og $\mathbb{C}$ i vores vektorrum. Men der skal noteres at der to andre valg, der kan være nyttige.

(a) *Rationelle tal* $\mathbb{Q}$ er tal af formen $x = a/b$, med a, b heltal og $b \neq 0$. Når vi arbejder med lineære ligningssystemer, hvis koefficienter er heltal, så vil elementære rækkeoperationer, fører til koefficienter der er rationelle tal, men ikke mere generelle reelle tal.
(b) Når p er et primtal, så har $\mathbb{Z}/p\mathbb{Z}$, restklasser af heltal modulo p, passende egenskaber for at være skalarer i et vektorrum. Disse tal er ofte nyttig i kryptering.

◇

12.3 Underrum

En nem og væsentlig kilde til vektorrum er visse delmængde af kendte vektorrum.

Definition 12.12. Lad V være et vektorrum. En ikke-tom delmængde $W \subset V$ er et *underrum* hvis sum og skalarmultiplikation sender elementer af W til elementer af W. Dvs.
(a) W er ikke tom,
(b) $v, w \in W$ medfører $v + w \in W$,
(c) $w \in W$ og s en skalar medfører $sw \in W$.

Bemærk at nulvektoren 0 må nødvendigvis ligge i W, da for et $w \in W$ har vi $0 = 0w \in W$. Det kan godt tjekkes at et hvert underrum opfylder kravene i definition 12.1, så selv et vektorrum.

Eksempel 12.13. Givet $v \in \mathbb{R}^n$ med $v \neq 0$, er linjen L igennem v og origo, mængden

$$L = \{tv : t \in \mathbb{R}\}.$$

Dette mængde indeholder v selv, så er ikke tom.

To vilkårlige elementer i L har formen t_0v og t_1v, så deres sum er $(t_0v) + (t_1v) = (t_0 + t_1)v$, som ligger i L.

For tv i L og s i $\mathbb{R}$ har vi desuden at $s(tv) = (st)v$, som ligger i L.

Så L er et underrum af $\mathbb{R}^n$, og dermed et vektorrum. △

Dette eksempel kan udvides til planer osv. på den følgende måde.

Definition 12.14. Rummet *udspændt* af vektorer $v_0, v_1, \ldots, v_{k-1} \in V$ er

$$\begin{aligned}&\mathrm{Span}\{v_0, v_1, \ldots, v_{k-1}\}\\ &\qquad = \{x_0v_0 + x_1v_1 + \cdots + x_{k-1}v_{k-1} \mid x_0, x_1, \ldots, x_{k-1} \text{ skalarer}\}.\end{aligned}$$

Vektorerne

$$x_0 v_0 + x_1 v_1 + \cdots + x_{k-1} v_{k-1}$$

er netop alle de mulige lineære kombinationer af $v_0, \ldots, v_{k-1}$.

Eksempel 12.15. xy-planen i $\mathbb{R}^3$ består af alle vektorer

$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix}.$$

Men

$$\begin{bmatrix} x \\ y \\ 0 \end{bmatrix} = x \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + y \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = xe_0 + ye_1.$$

Så xy-planen er netop $\operatorname{Span}\{e_0, e_1\}$. △

Definition 12.16. Lad $A \in \mathbb{R}^{m \times n}$. Så er *søjlerummet* $S(A)$ af A rummet udspændt af dens søjler.

Dvs. hvis

$$A = \left[v_0 \mid v_1 \mid \ldots \mid v_{n-1} \right],$$

så er

$$S(A) = \operatorname{Span}\{v_0, v_1, \ldots, v_{n-1}\}.$$

Eksempel 12.17. Matricen

$$A = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

har søjler

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}.$$

Så søjlerummet består af alle vektorer af formen

$$x_0 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + x_1 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} x_0 + 2x_2 \\ x_1 + x_2 \\ 0 \end{bmatrix}. \tag{12.4}$$

Nar x_i varierer får vi alle vektorer af formen

$$\begin{bmatrix} s \\ t \\ 0 \end{bmatrix},$$

dvs. i dette tilfælde er søjlerummet $S(A)$ planen i $\mathbb{R}^3$ udspændt af e_0 og e_1.

Bemærk at

$$Ax = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_0 + 2x_2 \\ x_1 + x_2 \\ 0 \end{bmatrix},$$

som er det samme som resultatet i (12.4). Så søjlerummet $S(A)$ består netop af alle vektorer der kan fås som Ax for $x \in \mathbb{R}^3$. △

Den sidste bemærkning giver anledning til:

Proposition 12.18. *For $A \in \mathbb{R}^{m\times n}$ er søjlerummet*

$$S(A) = \{Ax \mid x \in \mathbb{R}^n\}.$$

Specielt har ligningssystemet

$$Ax = b$$

en løsning hvis og kun hvis b ligger i søjlerummet $S(A)$ af A.

Bevis. Skriv

$$A = \begin{bmatrix} v_0 \mid v_1 \mid \dots \mid v_{n-1} \end{bmatrix}.$$

Så har vi

$$Ax = \begin{bmatrix} v_0 \mid v_1 \mid \dots \mid v_{n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}.$$

Det følger at de eneste vektorer der kan skrives i formen Ax er netop lineære kombinationer af søjlerne af A, dvs. elementer af søjlerummet $S(A)$ for A. □

Eksempel 12.19. Lad os betragte en singulærværdidekomponering i ydre produktform

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{k-1} u_{k-1} v_{k-1}^T,$$

hvor $\sigma_0 \geqslant \sigma_1 \geqslant \cdots \geqslant \sigma_{k-1} \geqslant 0$. Vælg $r \leqslant k$ så at σ_{r-1} er den sidste singulærværdi som er ikke nul. Så er A givet ved

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{r-1} u_{r-1} v_{r-1}^T.$$

Beregner vi Ax, får vi

$$Ax = \sigma_0 \langle v_0, x\rangle u_0 + \sigma_1 \langle v_1, x\rangle u_1 + \cdots + \sigma_{r-1} \langle v_{r-1}, x\rangle u_{r-1},$$

som er en lineær kombination af $u_0, u_1, \ldots, u_{r-1}$. Ved at lade x løbe over $v_0, v_1, \ldots, v_{r-1}$, ser vi at vi får alle sådanne lineære kombinationer på denne måde. Det betyder at

$$S(A) = S(U\Sigma V^T) = \operatorname{Span}\{u_0, u_1, \ldots, u_{r-1}\}.$$

△

Eksempel 12.20. Betragt $v = (1, 0, -1) \in \mathbb{R}^3$. Sæt

$$W = v^\perp = \{w \in \mathbb{R}^3 \mid \langle v, w\rangle = 0\}.$$

Så er W et underrum: (a) $\langle v, 0\rangle = 0$ giver $0 \in W$, så W er ikke tom, (b) $u, w \in W$ siger $\langle v, u\rangle = 0 = \langle v, w\rangle$, men så er $\langle v, u + w\rangle = \langle v, u\rangle + \langle v, w\rangle = 0 + 0 = 0$, som giver $u + w \in W$, (c) for $w \in W$ og $s \in \mathbb{R}$ har vi $\langle v, sw\rangle = s\langle v, w\rangle = 0$, så $sw \in W$. △

Vi vil møde flere eksempler på underrum efterhånden.

Kapitel 13

Indre produkter

13.1 Definitioner og eksempler

Vi har allerede mødt det standard indre produkt på $\mathbb{R}^n$, og set nogle af dens egenskaber i lemma 8.2. Dette produkt kan udvides umiddelbart på den følgende måde.

Definition 13.1. Et *indre produkt* på et vektorrum V, med enten reelle eller komplekse skalarer, er skalarer $\langle u, v\rangle$ for alle $u, v \in V$ således at

(I1) $\langle u, v\rangle = \overline{\langle v, u\rangle}$,
(I2) $\langle su + tw, v\rangle = s\langle u, v\rangle + t\langle w, v\rangle$,
(I3) $\langle v, v\rangle \in \mathbb{R}$ og $\langle v, v\rangle \geqslant 0$,
(I4) for $v \neq 0$ er $\langle v, v\rangle > 0$,

for alle $u, v, w \in V$ og alle s, t skalar.

Bemærk at vi får fra del (I1) og del (I2) at

$$\langle u, sv + tw\rangle = \bar{s}\langle u, v\rangle + \bar{t}\langle u, w\rangle. \tag{13.1}$$

Som tidligere, siger vi at u og v er *ortogonal* hvis $\langle u, v\rangle = 0$.

Bemærkning 13.2. Hvis vores skalarer er reelle tal så har konjugation ingen effekt og kan droppes fra det ovenstående. ◇

Bemærkning 13.3. Sætters $\|v\| = \sqrt{\langle v, v\rangle}$, fås en *norm* på V, som opfylder det der svarer til definition 11.4, dvs.

(N1) $\|v\| \geqslant 0$ for alle $v \in V$, og $\|v\| = 0$ kun for $v = 0$,
(N2) $\|sv\| = |s|\|v\|$ for alle s skalar,

(N3) $\|v + w\| \leqslant \|v\| + \|w\|$.

(Beviset for del (N3) kræver dog Cauchy-Schwarz uligheden nedenfor.) Vi kalder $\|\cdot\|$ *normen induceret* af $\langle \cdot, \cdot \rangle$.

I situationen hvor vores skalarer er reelle kan man definere *vinklen* mellem u og v stort set som i definition 8.5, dvs.

$$\cos\theta = \frac{\langle u, v\rangle}{\|u\|\,\|v\|},$$

når $u \neq 0 \neq v$. Igen er det Cauchy-Schwarz uligheden, som sikrer at dette giver mening. ◇

Eksempel 13.4. På $\mathbb{C}^n$ er det standard indre produkt givet ved

$$\langle u, v\rangle = u_0\overline{v_0} + u_1\overline{v_1} + \cdots + u_{n-1}\overline{v_{n-1}}. \tag{13.2}$$

Så

$$\begin{aligned}\left\langle \begin{bmatrix}1+i\\2-i\end{bmatrix}, \begin{bmatrix}-1+i\\1+i\end{bmatrix}\right\rangle &= (1+i)\overline{(-1+i)} + (2-i)\overline{(1+i)}\\ &= (1+i)(-1-i) + (2-i)(1-i)\\ &= -1 - i - i - i^2 + 2 - 2i - i + i^2\\ &= 1 - 5i.\end{aligned}$$

Bemærk at

$$\langle u, v\rangle = \overline{v}^T u,$$

da $u_j\overline{v_j} = \overline{v_j}u_j$.

Den tilhørende norm er

$$\|v\|_2 = \|v\| = \sqrt{|v_0|^2 + |v_1|^2 + \cdots + |v_{n-1}|^2}.$$

Så

$$\left\|\begin{bmatrix}1+3i\\2-4i\end{bmatrix}\right\|_2 = \sqrt{((1)^2 + 3^2) + (2^2 + (-4)^2)} = \sqrt{1+9+4+16} = \sqrt{30}.$$

I python kan `np.vdot` godt bruges til at beregne disse indre produkter, men man skal passe på rækkefølgen:

> det indre produkt mellem komplekse vektorer `u` og `v` gives som `np.vdot(v,u)`, med `u` og `v` i modsat rækkefølge.

Funktionen `np.linalg.norm` virker, som det skal.

```
>>> import numpy as np
>>> u = np.array([1.0 + 1.0j, 2.0 - 1.0j])[:, np.newaxis]
>>> v = np.array([-1.0 + 1.0j, 1.0 + 1.0j])[:, np.newaxis]
>>> np.vdot(v, u)
(1-5j)
>>> w = np.array([1.0 + 3.0j, 2.0 - 4.0j])[:, np.newaxis]
>>> np.linalg.norm(w)
5.477225575051661
>>> np.vdot(w, w)
(30+0j)
>>> np.sqrt(np.vdot(w, w))
(5.477225575051661+0j)
>>> machine_epsilon = np.finfo(float).eps
>>> np.allclose(np.linalg.norm(w), np.sqrt(np.vdot(w, w)),
...             rtol = machine_epsilon,
...             atol = machine_epsilon)
True
```

△

Eksempel 13.5. På $\mathbb{R}^n$ givet reelle tal $w_0, \ldots, w_{n-1} > 0$ er

$$\langle u, v \rangle \coloneqq w_0 u_0 v_0 + w_1 u_1 v_1 + \cdots + w_{n-1} u_{n-1} v_{n-1}$$

et indre produkt. Dette kan bruges når forskellige koordinatretninger måler forskellige type størrelse, f.eks. med forskellige enheder. Vi kalder dette et *vægtet indre produktet*.

På $\mathbb{C}^n$ er det tilsvarende udtryk

$$\langle u, v \rangle \coloneqq w_0 u_0 \overline{v_0} + w_1 u_1 \overline{v_1} + \cdots + w_{n-1} u_{n-1} \overline{v_{n-1}},$$

hvor igen $w_0, \ldots, w_{n-1}$ er reelle tal som er skarpt større end 0. △

Eksempel 13.6. På $V = \mathbb{R}^{m \times n}$, kan vi definere

$$\langle A, B \rangle = \big\langle (a_{ij}), (b_{ij}) \big\rangle = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} a_{ij} b_{ij}.$$

Dette definerer et indre produkt, og den tilhørende norm kaldes *Frobeniusnormen*

$$\|A\|_F = \sqrt{\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}(a_{ij})^2}.$$

Der skal understreges at $\|A\|_2 \neq \|A\|_F$ generelt. F.eks. har

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

$\|A\|_2 = \sigma_0 = 2$, men $\|A\|_F = \sqrt{2^2 + 1^2} = \sqrt{5}$.

For en general matrix $A \in \mathbb{R}^{m\times n}$ med singulærværdier $\sigma_0, \dots, \sigma_{k-1}$, hvor $k = \min\{m, n\}$, har vi

$$\|A\|_2 = \sigma_0 \leqslant \|A\|_F = \sqrt{\sigma_0^2 + \cdots + \sigma_{k-1}^2} \leqslant \sqrt{k\sigma_0^2} = \sqrt{k}\|A\|_2.$$

Specielt får vi

$$\frac{1}{\sqrt{k}}\|A\|_F \leqslant \|A\|_2 \leqslant \|A\|_F,$$

så $\|A\|_F$ kan bruges til at estimere $\|A\|_2$ uden at beregne singulærværdierne △

Eksempel 13.7. Lad V være vektorrummet, som består af kontinuerte funktioner $f\colon [a, b] \to \mathbb{R}$, for fast a, b med $a < b$. Det *L^2-indre produkt* er givet ved

$$\langle f, g\rangle = \int_a^b f(x)g(x)\,dx.$$

Det væsentlig er at $\langle f, f\rangle = \int_a^b f(x)^2\,dx \geqslant 0$.

Vi skal se at $\langle f, f\rangle = 0$ tvinger $f \equiv 0$. Det er nok at vise at når f er ikke identisk 0, er $\langle f, f\rangle > 0$. Hvis f er ikke identisk 0, så findes der et punkt $x_0 \in [a, b]$ hvor $f(x_0) \neq 0$. Da f er kontinuert er der et interval $[c, d] \subset [a, b]$, $c < d$, omkring x_0 med $f(x)^2 \geqslant f(x_0)^2/2$ for $c \leqslant x \leqslant d$. Det følger at

$$\begin{aligned}\langle f, f\rangle &= \int_a^b f(x)^2\,dx \geqslant \int_c^d f(x)^2\,dx \\ &\geqslant \int_c^d \frac{1}{2}f(x_0)^2\,dx = \frac{1}{2}(d-c)f(x_0)^2 \\ &> 0,\end{aligned}$$

så del (I4) er opfyldt.

F.eks. på $[0,1]$ har vi for funktionerne $f(x) = x$, $g(x) = x^2$ at

$$\langle f, g\rangle = \int_0^1 x \times x^2\, dx = \int_0^1 x^3\, dx = \left[\frac{1}{4}x^4\right]_0^1 = \frac{1}{4} - 0 = \frac{1}{4}.$$

Givet $w\colon [a,b] \to \mathbb{R}$ kontinuert med $w(x) > 0$, for alle x, kan man definere et *vægtet L^2-indre produkt* ved

$$\langle f, g\rangle = \int_a^b f(x)g(x)w(x)\, dx.$$

△

Eksempel 13.8. For V vektorrummet af kontinuerte $f\colon [a,b] \to \mathbb{C}$ har man et *L^2-indre produkt* givet ved

$$\langle f, g\rangle = \int_a^b f(x)\overline{g(x)}\, dx.$$

△

Mange af vores begreber og resultater for det standard indre produkt gælder også for vilkårlige indre produkter. Specielt gælder Pythagoras sætningen og Cauchy-Schwarz ulighed. For det følgende husk at vi siger at u, v er ortogonal hvis $\langle u, v\rangle = 0$. Vi har

Proposition 13.9. *Lad $\langle\,\cdot\,,\,\cdot\,\rangle$ være et indre produkt på et vektorrum V med skalarer $\mathbb{R}$ eller $\mathbb{C}$. Sæt $\|v\| = \sqrt{\langle v, v\rangle}$ til at være den inducerede norm. De følgende udsagn gælder.*

(a) (*Pythagoras*) *For u, v ortogonal, er $\|u+v\|^2 = \|u\|^2 + \|v\|^2$.*

(b) (*Cauchy-Schwarz*) *For alle $u, v \in V$,*

$$|\langle u, v\rangle| \leqslant \|u\|\,\|v\|,$$

med lighed kun hvis u og v er parallelle, dvs. enten $v = 0$ eller $u = sv$ for en skalar s.

Bevis. For Pythagoras sætningen gentager vi beviset for proposition 8.3 med $\|\cdot\|_2$ erstattet af $\|\cdot\|$: $\langle u, v\rangle = 0$ giver $\langle v, u\rangle = \overline{\langle u, v\rangle} = 0$, så for u, v ortogonal har vi

$$\begin{aligned}
\|u+v\|_2^2 &= \langle u+v, u+v\rangle = \langle u, u+v\rangle + \langle v, u+v\rangle && \text{definition 13.1(I2)},\\
&= \langle u, u\rangle + \langle u, v\rangle + \langle v, u\rangle + \langle v, v\rangle && \text{ligning (13.1)},\\
&= \|u\|_2^2 + \|v\|_2^2.
\end{aligned}$$

For Cauchy-Schwarz uligheden, bemærk først at der er intet at vise hvis $v = 0$, da $\langle u, 0\rangle = \langle u, 0 \times 0\rangle = 0\langle u, 0\rangle = 0$, og $\|v\| = 0$.

Hvis $v \neq 0$, betagter vi

$$w = u - tv, \qquad \text{med } t = \frac{\langle u, v\rangle}{\|v\|^2}.$$

Når $u = sv$ for en skalar s, har vi $\langle u, v\rangle = s\langle v, v\rangle = s\|v\|^2$, og så $t = s$ og $w = sv - tv = 0$. Generelt er

$$\begin{aligned}
0 \leqslant \|w\|^2 &= \langle u - tv, u - tv\rangle \\
&= \langle u, u\rangle - \langle tv, u\rangle - \langle u, tv\rangle + \langle tv, tv\rangle \\
&= \|u\|^2 - t\langle v, u\rangle - \bar{t}\langle u, v\rangle + |t|^2\|v\|^2 \\
&= \|u\|^2 - t\overline{\langle u, v\rangle} - \bar{t}\langle u, v\rangle + |t|^2\|v\|^2 \\
&= \|u\|^2 - \frac{1}{\|v\|^2}\langle u, v\rangle\overline{\langle u, v\rangle} - \frac{1}{\|v\|^2}\overline{\langle u, v\rangle}\langle u, v\rangle + \frac{1}{\|v\|^4}|\langle u, v\rangle|^2\|v\|^2 \\
&= \|u\|^2 - \frac{1}{\|v\|^2}|\langle u, v\rangle|^2 - \frac{1}{\|v\|^2}|\langle u, v\rangle|^2 + \frac{1}{\|v\|^2}|\langle u, v\rangle|^2 \\
&= \|u\|^2 - \frac{1}{\|v\|^2}|\langle u, v\rangle|^2.
\end{aligned}$$

Vi har så

$$|\langle u, v\rangle|^2 \leqslant \|u\|^2\|v\|^2.$$

Tages kvadratroden, fås Cauchy-Schwarz uligheden. Vi har ligehed kun når $w = 0$, dvs. kun når $u = tv$, så u er parallel med v. □

13.2 Ortogonale samlinger og tilnærmelse

Med vores nye definitioner, kan vi arbejde med ortogonale og ortonormale samlinger vektorer stort set som førhen. Specielt gælder proposition 8.17: hvis $v_0, \ldots, v_{k-1}$ er ortogonal og $u \in \operatorname{Span}\{v_0, \ldots, v_{k-1}\}$, så har vi

$$u = x_0v_0 + \cdots + x_{k-1}v_{k-1} \quad \text{med } x_j = \frac{\langle u, v_j\rangle}{\|v_j\|^2},\ j = 0, 1, \ldots, n-1.$$

På en tilsvarende måde er projektionen P på $W = \operatorname{Span}\{v_0, v_1, \ldots, v_{k-1}\}$, når $v_0, v_1, \ldots, v_{k-1}$ er ortogonal, med ingen 0, givet ved

$$P(u) = \frac{\langle u, v_0\rangle}{\|v_0\|^2}v_0 + \frac{\langle u, v_1\rangle}{\|v_1\|^2}v_1 + \cdots + \frac{\langle u, v_{k-1}\rangle}{\|v_{k-1}\|^2}v_{k-1}. \tag{13.3}$$

Vektoren $w = P(u)$ er den vektor i W, som er tættest på u, dvs. har $\|u - w\|$ mindst muligt. Beviset er det samme, som vi har givet i $\mathbb{R}^n$, for projektion på $v_0, v_1, \ldots, v_{k-1}$.

Eksempel 13.10. Lad os betragter rummet V af kontinuerte funktioner $[0, \pi] \to \mathbb{R}$ udstyret med det L^2-indre produkt. Vi påstår at

$$1, \cos(x), \cos(2x), \ldots, \cos((k-1)x)$$

er en ortogonal samling i V. Dette gælder da for $n > 0$

$$\langle 1, \cos nx \rangle = \int_0^\pi \cos(nx)\, dx = \left[\frac{1}{n}\sin(nx)\right]_0^\pi = \frac{1}{n}(\sin(n\pi) - \sin(0)) = 0,$$

så 1 er vinkelret på de andre funktioner. Desuden, hvis vi husker det trigonometriske identitet (9.13), har vi for $m \neq n$ at

$$\begin{aligned}\langle \cos(mx), \cos(nx) \rangle &= \int_0^\pi \cos(mx)\cos(nx)\, dx \\ &= \frac{1}{2}\int_0^\pi \cos((m+n)x) + \cos((m-n)x)\, dx \\ &= \frac{1}{2}\left[\frac{1}{m+n}\sin((m+n)x) + \frac{1}{m-n}\sin((m-n)x)\right]_0^\pi = 0.\end{aligned}$$

Så funktionerne er vinkelret på hinanden, som påstået.

Bemærk at

$$\|1\|^2 = \int_0^\pi 1^2\, dx = \pi,$$

og

$$\|\cos(nx)\|^2 = \int_0^\pi (\cos(nx))^2\, dx = \frac{1}{2}\int_0^\pi (\cos(2nx) + 1)\, dx = \frac{\pi}{2}.$$

Lad os beregne projektionen af funktionen $1 - x$ på

$$\operatorname{Span}\{1, \cos(x), \cos(2x), \ldots, \cos((k-1)x)\}.$$

Vi skal bruge (13.3) med $u = 1 - x$, $v_0 = 1$ og $v_n = \cos(nx)$, for $n = 1, \ldots, k-1$. Vi først beregner de enkelte indre produkter, der indgår i (13.3). Vi har

$$\langle 1 - x, 1 \rangle = \int_0^\pi (1-x)1\, dx = \left[x - \frac{1}{2}x^2\right]_0^\pi = \pi - \frac{1}{2}\pi^2 = \frac{\pi}{2}(2 - \pi)$$

og, for $n > 0$,

$$\begin{aligned}
\langle 1-x, \cos(nx)\rangle &= \int_0^\pi \cos(nx) - x\cos(nx)\,dx = 0 - \int_0^\pi x\cos(nx)\,dx \\
&= -\left[\frac{1}{n}x\sin(nx)\right]_0^\pi + \int_0^\pi \frac{1}{n}\sin(nx)\,dx \\
&= 0 + \left[-\frac{1}{n^2}\cos(nx)\right]_0^\pi = -\frac{1}{n^2}(-\cos(n\pi)+1) \\
&= \frac{1}{n^2}((-1)^n - 1) = \begin{cases} -\dfrac{2}{n^2}, & \text{for } n \text{ ulige,} \\ 0, & \text{for } n \text{ lige.} \end{cases}
\end{aligned}$$

Sættes ind i (13.3), fås

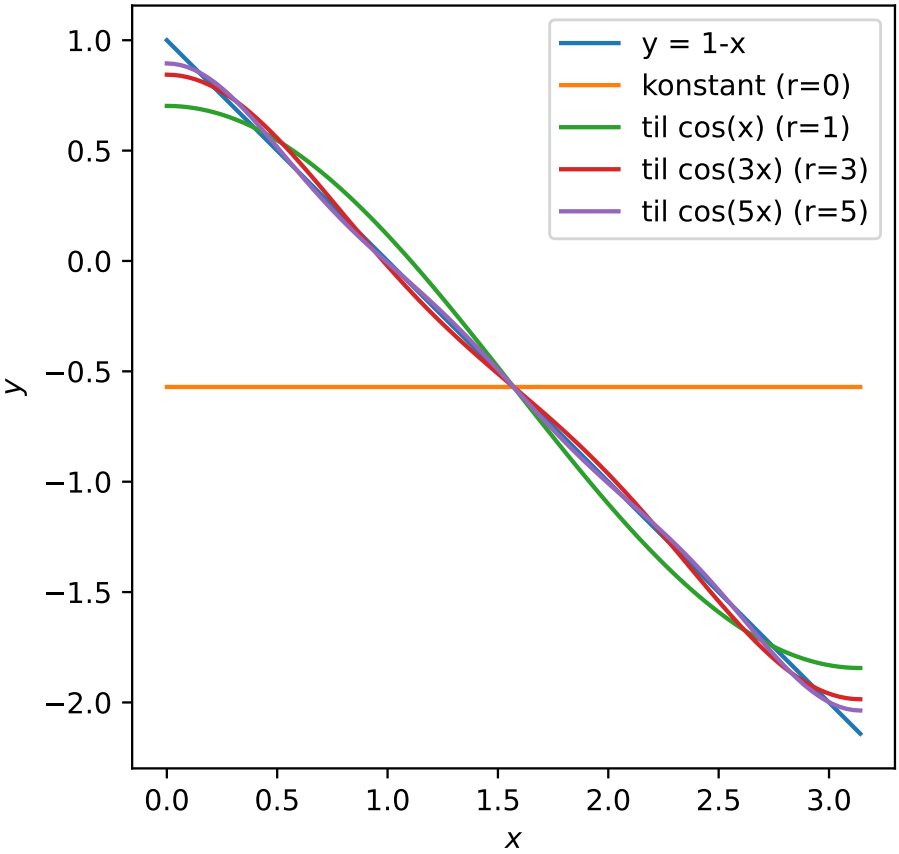
$$\begin{aligned}
&P(1-x) \\
&\quad = \frac{\langle 1-x, 1\rangle}{\|1\|^2}1 + \frac{\langle 1-x, \cos(x)\rangle}{\|\cos(x)\|^2}\cos(x) + \frac{\langle 1-x, \cos(2x)\rangle}{\|\cos(2x)\|^2}\cos(2x) \\
&\qquad + \cdots + \frac{\langle 1-x, \cos((k-1)x)\rangle}{\|\cos((k-1)x)\|^2}\cos((k-1)x) \\
&\quad = \frac{\pi(2-\pi)/2}{\pi}1 + \frac{2/1^2}{\pi/2}\cos(x) + 0 + \frac{2/3^2}{\pi/2}\cos(3x) + 0 \\
&\qquad + \frac{2/5^2}{\pi/2}\cos(5x) + \cdots + \frac{2/r^2}{\pi/2}\cos(rx) \\
&\quad = \frac{1}{2}(2-\pi) + \frac{4}{\pi}\cos(x) + \frac{4}{9\pi}\cos(3x) \\
&\qquad + \frac{4}{25\pi}\cos(5x) + \cdots + \frac{4}{r^2\pi}\cos(rx),
\end{aligned} \tag{13.4}$$

hvor r er det største ulige tal $\leqslant k-1$. Dette kaldes en *Fourier cosinus rækkeudvikling* af funktionen $1-x$. Plot af disse tilnærmelser til funktionen $1-x$ for forskellige r vises i figur 13.1.

Billedfilformatet `jpeg` er baseret på en diskret udgave af Fourier cosinus rækkeudvikling. △

Eksempel 13.11. Fourier rækker generelt kommer fra at betragte kontinuerte funktioner $[-\pi, \pi] \to \mathbb{R}$. Her er

$$1, \cos(x), \sin(x), \cos(2x), \sin(2x), \ldots, \cos((k-1)x), \sin((k-1)x) \tag{13.5}$$



Figur 13.1: Fourier cosinus tilnærmelse for $1 - x$.

ortogonal med

$$\|1\|^2 = 2\pi$$

og

$$\|\cos(nx)\|^2 = \pi = \|\sin(nx)\|^2.$$

Man kan beregne en tilnærmelse til andre kontinuerte funktioner på $[-\pi, \pi]$

ved at projicere på rummet udspændt af (13.5):

$$\begin{aligned} P(f) = a_0 &+ c_1\cos(x) + b_1\sin(x) \\ &+ c_2\cos(2x) + b_2\sin(2x) \\ &+ c_3\cos(3x) + b_3\sin(3x) \\ &+ \cdots + c_{k-1}\cos((k-1)x) + b_{k-1}\sin((k-1)x), \end{aligned}$$

med

$$\begin{aligned} a_0 &= \frac{\langle f, 1\rangle}{\|1\|^2} = \frac{1}{2\pi}\int_{-\pi}^{\pi} f(x)\,dx, \\ c_m &= \frac{\langle f, \cos(mx)\rangle}{\|\cos(mx)\|^2} = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\cos(mx)\,dx \\ b_m &= \frac{\langle f, \sin(mx)\rangle}{\|\sin(mx)\|^2} = \frac{1}{\pi}\int_{-\pi}^{\pi} f(x)\sin(mx)\,dx. \end{aligned}$$

I det næste afsnit vil vi se hvordan, udregning af denne type kan udføres numerisk i python. △

13.3 Numerisk integration

Hvis vi vil udregne eksempler som 13.10 i python, skal vi have styre på hvordan vi kan beregne integraler numerisk.

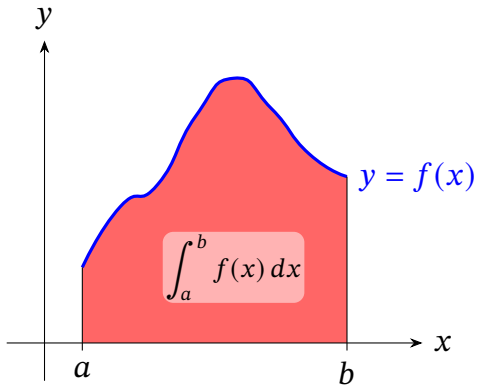
Givet en kontinuert funktion $f\colon [a,b] \to \mathbb{R}$, er integralet $\int_a^b f(x)\,dx$ arealet under dens graf, se figur 13.2. Der er forskellige måde vi kan lave en tilnærmelse til dette areal.

Lad $x_0, x_1, \ldots, x_{n-1}$ være n punkter ligelig fordelt fra $a = x_0$ til $b = x_{n-1}$. Sådan en samling får vi fra `np.linspace(a, b, n)`. For hvert x_j kan vi betragter rektanglet R med venstre side ved x_j, højre side ved x_{j+1}, og højde $f(x_j)$, se figur 13.3. Rektanglet R har areal

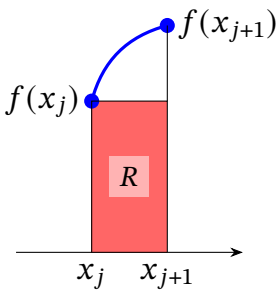
$$hf(x_j)$$

hvor $h = x_{j+1} - x_j$ er dens brede. Vi har sådan et rektangel for hvert venstre start punkt $x_0, x_1, \ldots, x_{n-2}$. Startpunktet x_{n-1} bidrager ikke, da rektanglet til højre fra x_{n-1} ligger uden for området vi arbejder med. En mulig tilnærmelse til integralet er summen af arealerne på disse rektangler:

$$\begin{aligned} &hf(x_0) + hf(x_1) + \cdots + hf(x_{n-2}) \\ &\quad = h\big(f(x_0) + f(x_1) + \cdots + f(x_{n-2})\big). \end{aligned}$$



Figur 13.2: Integral.



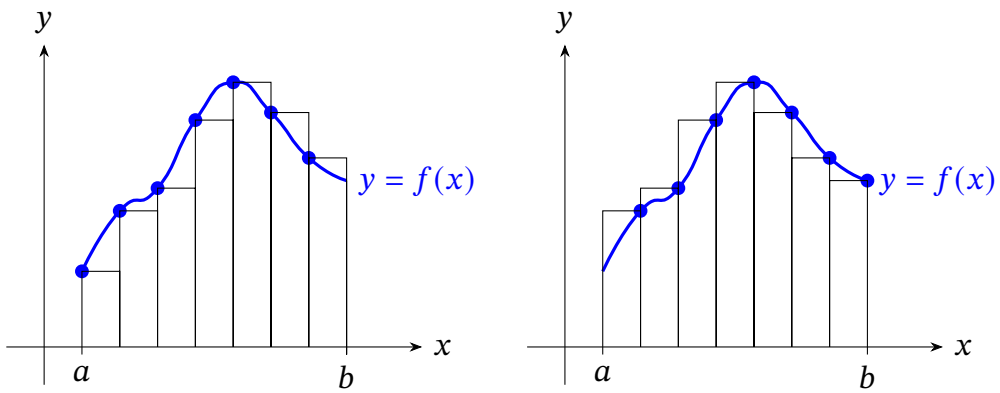
Figur 13.3: Rektanglet fra den venstre funktionsværdi.

Alle rektangler er lige brede, så værdien h kan regnes som

$$h = \frac{1}{n-1}(b-a),$$

da vi har delt $[a, b]$ op i $(n-1)$-delintervaller. Ved brug af `np.linspace()` er det ikke nødvendigt at beregne h, man kan bede `np.linspace()` at returnere denne værdi ved hjælp af `np.linspace(a, b, n, retstep=True)`. F.eks. tager vi funktionen $f(x) = x^3$ på $[0, 2]$, hvor vi ved at $\int_0^2 x^3\,dx = [x^4/4]_0^2 = 4$, kan vi regne i python med $n = 100$:

```
>>> import numpy as np
>>> n = 100
>>> x, h = np.linspace(0, 2, n, retstep=True)
```



Figur 13.4: Venstre- og højre-tilnærmelse til integral.

```
>>> h * np.sum((x**3)[:-1])
3.919600040812163
```

Dette er lidt mindre end 4, da funktionen er voksende og alle rektangler ligger under grafen.

Som alternativ kunne man brug funktionsværdien i den højre endepunkt af intervallet $[x_j, x_{j+1}]$, se figur 13.4, og får tilnærmelsen

$$h\big(f(x_1) + f(x_2) + \cdots + f(x_{n-1})\big)$$

```
>>> h * np.sum((x**3)[1:])
4.081216202428324
```

Denne gang, for funktionen $f(x) = x^3$, får vi et svar der er lidt for stort.

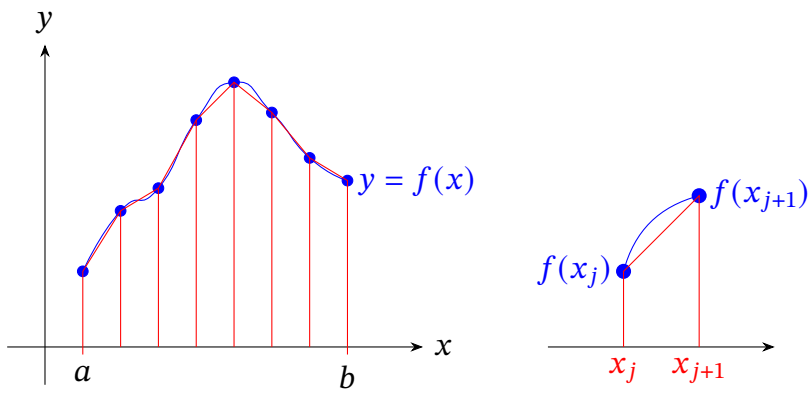
Et kompromis er at erstatte rektanglet med trapezet, som i figur 13.5. Dette trapez har areal

$$h\,\frac{1}{2}\big(f(x_j) + f(x_{j+1})\big).$$

Dette giver *trapez-tilnærmelsen* til integralet

$$\frac{1}{2}h\big(f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-2}) + f(x_{n-1})\big). \tag{13.6}$$

Vi kan implementere denne regel i python som



Figur 13.5: Trapez-tilnærmelse til integral.

```
>>> def trapz(f, h):
...   return h * (f[0] + 2*np.sum(f[1:-1]) + f[-1]) / 2
...
```

Denne funktion `trapz()`, som kan nu anvendes på det overstående eksempel, med det samme `h`, til at få

```
>>> trapz(x**3, h)
4.000408121620244
```

som er væsentligt tættere på den rigtige værdi for integralet.

Generelt kan det vises at denne trapezregel (13.6) på funktioner, som kan differentieres to gange, beregner integraler $\int_a^b f(x)\,dx$ inden for en fejl af

$$\frac{1}{12}h^2(b-a)\max\{|f''(x)| \mid a \leqslant x \leqslant b\}.$$

Trapezformlen (13.6) kommer fra at erstatte grafen af f med linjestykker, dvs. ved polynomier af grad 1. Man kan i stedet for bruge højere ordens polynomier. Andengradspolynomier giver anledning til Simpsons regel

$$\begin{aligned}\int_a^b f(x)\,dx \approx \frac{1}{3}h\big(&f(x_0)+4f(x_1)+2f(x_2)+4f(x_3)+2f(x_4)\\ &+\cdots+2f(x_{n-3})+4f(x_{n-2})+f(x_{n-1})\big)\end{aligned} \tag{13.7}$$

for n ulig. Denne gang er fejlen i beregning af integralet højst

$$\frac{1}{90}h^4(b-a)\max\{|f''''(x)| \mid a \leqslant x \leqslant b\}.$$

Lad os anvender trapezreglen for at beregne eksempel 13.10 numerisk i python.

```
>>> import numpy as np
>>> n = 100
>>> x, h = np.linspace(0, np.pi, 100, retstep=True)
```

Vores ortogonale funktioner er 1, $\cos(x)$, $\cos(2x)$, osv. Den første gemmer vi i

```
>>> konstant = np.ones_like(x)
```

de andre kan bare regnes som `np.cos(m*x)`. Lad os bekræfte at nogle af disse er ortogonal. Indre produktet er $\langle f, g\rangle = \int_0^\pi f(x)g(x)\,dx$, så vi kan bruge vores `trapz()` funktion på $f*g$:

```
>>> def trapz(f, h):
...   return h * (f[0] + 2*np.sum(f[1:-1]) + f[-1]) / 2
...
>>> def indre_produkt(f, g, h):
...     return trapz(f * g, h)
...
>>> indre_produkt(konstant, np.cos(3*x), h)
-4.580029340819416e-17
>>> indre_produkt(np.cos(2*x), np.cos(3*x), h)
-3.523099492938012e-18
>>> indre_produkt(np.cos(5*x), np.cos(3*x), h)
-7.750818884463626e-17
```

som viser at de er ortogonale inden for machine epsilon. Tidligere fandt vi at $\|1\|^2 = \pi$, $\|\cos(mx)\|^2 = \pi/2$. Disse værdier kan også bekræftes i python

```
>>> def norm_sq(f, h):
...     return indre_produkt(f, f, h)
...
```

```
>>> norm_sq(konstant, h) - np.pi
0.0
>>> norm_sq(np.cos(x), h) - np.pi/2
0.0
>>> norm_sq(np.cos(4*x), h) - np.pi/2
0.0
```

med resultater korrekt igen inden for machine epsilon.

I eksempel 13.10 beregnede vi projektionerne af $1 - x$ på de første k basisvektorer. Lad os indføre en funktion, som gør dette for os

```
>>> def proj(f, k, x, h):
...     out = (indre_produkt(f, konstant, h)
...            / norm_sq(konstant, h)
...            * konstant)
...     for m in range(1, k, 2):
...         out += (indre_produkt(f, np.cos(m*x), h)
...                 / norm_sq(np.cos(m*x), h)
...                 * np.cos(m*x))
...     return out
...
```

Plotter man `1-x`, og `proj(1-x, k, x, h)`, for `k` = 1, 2, 4, 6 fås figur 13.1. Vi ser at der fås en bedre tilnærmelse jo større k er.

Bemærk at

```
>>> machine_epsilon = np.finfo(float).eps
>>> np.allclose(proj(1-x, 3, x, h), proj(1-x, 2, x, h),
...             rtol = machine_epsilon,
...             atol = machine_epsilon)
True
```

som bekræfter at $\cos(2x)$ bidrager ikke til denne projektion, se formlen (13.4). Sådanne bidrag falder væk da indre produkterne $\langle 1 - x, \cos(mx)\rangle$ er nul for m lige, som python bekræfter

```
>>> for m in range(2, 7, 2):
...     print(indre_produkt(1-x, np.cos(m*x), h))
```

```
...
-4.227719391525615e-17
-4.227719391525615e-17
1.4092397971752048e-17
```

inden for machine epsilon.

Kapitel 14

Ortogonale samlinger: klassisk Gram-Schmidt

Vi har brugt ortogonale samlinger for at finde tilnærmelser til vektorer og funktioner. Her vil vi undersøge forskellige metoder for at konstruere sådanne samlinger.

14.1 Den klassiske Gram-Schmidt proces

Lad V være et vektorrum med indre produkt $\langle \cdot, \cdot \rangle$. Husk at projektion på linjen udspændt af $v \neq 0$ er givet ved

$$\mathrm{pr}_v(u) = \frac{\langle u, v \rangle}{\|v\|^2} v.$$

Vektoren

$$u - \mathrm{pr}_v(u)$$

er derefter vinkelret på v. Vi har også set at for $v_0, \ldots, v_{k-1}$ ortogonal, med ingen 0, er projektionen til $\mathrm{Span}\{v_0, \ldots, v_{k-1}\}$ givet ved

$$\mathrm{pr}_{v_0,\ldots,v_{k-1}}(u) = \frac{\langle u, v_0 \rangle}{\|v_0\|^2} v_0 + \cdots + \frac{\langle u, v_{k-1} \rangle}{\|v_{k-1}\|^2} v_{k-1}$$

og at $u - \mathrm{pr}_{v_0,\ldots,v_{k-1}}(u)$ er ortogonal på $v_0, \ldots, v_{k-1}$.

Vi kan bruge dette til at opbygge en samling ortogonale vektorer nogenlunde på følgende måde. Input er vektorer $u_0, u_1, \ldots, u_{k-1} \in V$.

KLASSISK GRAM-SCHMIDT PROCES—SKITSE$(u_0, u_1, \ldots, u_{k-1})$

1 Sæt $w_0 = u_0$ og $v_0 = c_0 w_0$, for et valgfrit $c_0 \neq 0$.
2 Sæt $w_1 = u_1 - \mathrm{pr}_{v_0}(u_1)$ og $v_1 = c_1 w_1$, for $c_1 \neq 0$.
3 Sæt $w_2 = u_2 - \mathrm{pr}_{v_0,v_1}(u_2)$ og $v_2 = c_2 w_2$, for $c_2 \neq 0$.
…
4 Sæt $w_r = u_r - \mathrm{pr}_{v_0,\ldots,v_{r-1}}(u_r)$ og $v_r = c_r w_r$, for $c_r \neq 0$.
…
5 **return** $v_0, v_1, \ldots, v_{k-1}$, som er en ortogonal samling,
der udspænder det samme rum som $u_0, u_1, \ldots, u_{k-1}$.

Vælges $c_r = 1/\|w_r\|$ i hvert trin, så bliver $v_0, v_1, \ldots, v_{k-1}$ ortonormal, men ofte er andre valg af c_r mere praktisk.

Eksempel 14.1. I $\mathbb{R}^4$ med det standard indre produkt betragt

$$u_0 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad u_1 = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad u_2 = \begin{bmatrix} -1 \\ 3 \\ 1 \\ -1 \end{bmatrix}.$$

Vi sætter

$$w_0 = u_0 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \qquad \text{og} \qquad v_0 = 1 w_0 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}.$$

Bemærk at $\|v_0\|_2^2 = 1^2 + 1^2 + 0^2 + 0^2 = 2$ og at vi har $\langle u_1, v_0 \rangle = 2 \times 1 + 1 \times 1 + 1 \times 0 + 1 \times 0 = 3$. Så

$$\begin{aligned} w_1 &= u_1 - \mathrm{pr}_{v_0}(u_1) \\ &= u_1 - \frac{\langle u_1, v_0 \rangle}{\|v_0\|_2^2} v_0 = \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix} - \frac{3}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ 2 \\ 2 \end{bmatrix}. \end{aligned}$$

Lad os gange igennem med $c_1 = 2$ for at fjerne brøken og sætte

$$v_1 = 2 w_1 = \begin{bmatrix} 1 \\ -1 \\ 2 \\ 2 \end{bmatrix},$$

som har $\|v_2\|_2^2 = 1+1+4+4 = 10$. Vi regner så at

$$\begin{aligned}
w_2 &= u_2 - \mathrm{pr}_{v_0,v_1}(u_2) \\
&= u_2 - \frac{\langle u_2, v_0\rangle}{\|v_0\|_2^2} v_0 - \frac{\langle u_2, v_1\rangle}{\|v_1\|_2^2} v_1 \\
&= \begin{bmatrix} -1 \\ 3 \\ 1 \\ -1 \end{bmatrix} - \frac{-1+3}{2} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} - \frac{-1-3+2-2}{10} \begin{bmatrix} 1 \\ 1 \\ 2 \\ 2 \end{bmatrix} \\
&= \begin{bmatrix} -1 \\ 3 \\ 1 \\ -1 \end{bmatrix} - \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} + \frac{2}{5} \begin{bmatrix} 1 \\ 1 \\ 2 \\ 2 \end{bmatrix} = \frac{1}{5} \begin{bmatrix} -8 \\ 8 \\ 9 \\ -1 \end{bmatrix}.
\end{aligned}$$

Sættes $v_2 = 5w_2$, fås at

$$v_0 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ -1 \\ 2 \\ 2 \end{bmatrix}, \quad v_2 = \begin{bmatrix} -8 \\ 8 \\ 9 \\ -1 \end{bmatrix}$$

er en ortogonal samling, som udspænder $\mathrm{Span}\{u_0, u_1, u_2\}$ i $\mathbb{R}^4$.

Det tilsvarende regnestykke i python er

```
>>> import numpy as np

>>> u0 = np.array([1.0, 1.0, 0.0, 0.0])[:, np.newaxis]
>>> u1 = np.array([2.0, 1.0, 1.0, 1.0])[:, np.newaxis]
>>> u2 = np.array([-1.0, 3.0, 1.0, -1.0])[:, np.newaxis]

>>> def proj_på(v, u):
...     return np.vdot(v, u) / np.vdot(v, v) * v
...
>>> v0 = u0
>>> v0
array([[1.],
       [1.],
```

```
       [0.],
       [0.]])
>>> w1 = u1 - proj_på(v0, u1)
>>> v1 = 2 * w1
>>> v1
array([[ 1.],
       [-1.],
       [ 2.],
       [ 2.]])
>>> w2 = u2 - proj_på(v0, u2) - proj_på(v1, u2)
>>> v2 = 5 * w2
>>> v2
array([[-8.],
       [ 8.],
       [ 9.],
       [-1.]])
```

△

Ved beskrivelsen af den klassiske Gram-Schmidt proces ovenfor, brugte vi ordet »nogenlunde«. Grunden til dette er at vi har brug for at w_r, og dermed v_r, er ikke nul i hvert trin. Dette er tilfældet hvis og kun hvis $u_0, u_1, \ldots, u_{k-1}$ opfylder:

Definition 14.2. En samling vektorer $u_0, u_1, \ldots, u_{k-1}$ i V er *lineært uafhængig* hvis den eneste løsning på ligningen

$$s_0 u_0 + s_1 u_1 + \cdots + s_{k-1} u_{k-1} = 0 \tag{14.1}$$

med s_i skalarer er $s_0 = 0 = s_1 = \cdots = s_{k-1}$.

Bemærkning 14.3. En måde at tjekke denne betingelse på er at køre algoritmen ovenfor: Hvis $w_r = 0$ for et r, så er $u_r = \mathrm{pr}_{v_0, v_1, \ldots, v_{r-1}}(u_r)$. Men denne projektion er en lineær kombination $s_0 u_0 + s_1 u_1 + s_{r-1} u_{r-1}$ af $u_0, \ldots, u_{r-1}$. Sættes $s_r = -1$ og $s_i = 0$ for $i > r$, fås en løsning til (14.1) med $s_r \neq 0$. Så samlingen er ikke lineært uafhængig.

Omvendt hvis $u_0, u_1, \ldots, u_{k-1}$ er lineært uafhængig, så er ingen u_r en lineær kombination af $u_0, \ldots, u_{r-1}$. Specielt er u_r ikke lige med $\mathrm{pr}_{v_0, v_1, \ldots, v_{r-1}}(u_r)$, så $w_r \neq 0$ for alle r. ◇

Eksempel 14.4. Lad $u_0, u_1, \ldots, u_{k-1} \in \mathbb{R}^n$ og betragt matricen

$$A = \left[u_0 \mid u_1 \mid \ldots \mid u_{k-1}\right] \in \mathbb{R}^{n\times k}$$

hvis søjler er u_r, $r = 0, \ldots, k-1$. Så er $u_0, u_1, \ldots, u_{k-1}$ lineært uafhængig hvis og kun hvis ligningen

$$Ax = 0, \qquad x \in \mathbb{R}^k$$

har kun én løsningen, nemlig $x = 0$. Dette gælder fordi $Ax = x_0u_0 + x_1u_1 + \cdots + x_{k-1}u_{k-1}$, så ligning (14.1) er $Ax = 0$ efter udskiftning af variabler. Reduceres A til echelonform får vi så at $u_0, u_1, \ldots, u_{k-1}$ er lineært uafhængig hvis alle søjler i A indeholder et pivotelement.

F.eks. vektorerne $u_0 = (1, 0, 2)$, $u_1 = (0, 2, 1)$, $u_2 = (1, 2, 3)$ er ikke lineært uafhængig da

$$\begin{bmatrix}1&0&1\\0&2&2\\2&1&3\end{bmatrix} \sim_{R_2\to R_2-2R_0} \begin{bmatrix}1&0&1\\0&2&2\\0&1&1\end{bmatrix} \sim_{R_1\leftrightarrow R_2} \begin{bmatrix}1&0&1\\0&1&1\\0&2&2\end{bmatrix} \sim_{R_2\leftrightarrow R_2-2R_1} \begin{bmatrix}1&0&1\\0&1&1\\0&0&0\end{bmatrix}$$

har kun pivot elementer i de første 2 søjler, og ikke i den sidste. △

Eksempel 14.5. For $a < b$, betragt vektorrummet af differentiable funktioner $f\colon [a, b] \to \mathbb{R}$. Så er polynomierne

$$1,\ x,\ x^2, \ldots,\ x^{k-1}$$

lineært uafhængig i V. For at vise dette, antag at

$$p(x) = s_0 1 + s_1 x + s_2 x^2 + \cdots + s_{k-1}x^{k-1} = 0(x) = 0, \qquad \text{for alle } a \leqslant x \leqslant b.$$

Betragt den $(k-1)$'te afledte $d^{k-1}p/dx^{k-1}$, som er nødvendigvis 0. Vi får

$$0 = \frac{d^{k-1}p}{dx^{k-1}} = (k-1)!\, s_{k-1},$$

som giver $s_{k-1} = 0$. Kikker man derefter på den $(k-2)$'te afledte, får man s_{k-2}, og fortsætter man i på denne måde, ser man at $s_r = 0$ for alle r.

Bruger man den klassiske Gram-Schmidt på $1, x, x^2, \ldots$ for forskellige indre produkter får man nogle kendte ortogonale systemer af polynomier:

(a) *Legendre polynomier* $[a,b] = [-1,1]$, $\langle f,g\rangle = \int_{-1}^{1} f(x)g(x)\,dx$, giver

$$\begin{aligned} P_0(x) &= 1, \\ P_1(x) &= x, \\ P_2(x) &= 3x^2 - 1, \\ P_3(x) &= 5x^3 - 3x. \end{aligned}$$

(b) *Chebychev polynomier* $[a,b] = [-1,1]$,

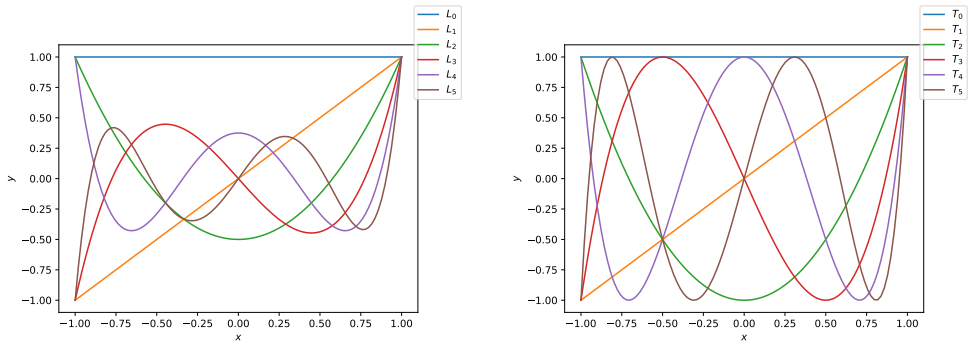
$$\langle f,g\rangle = \int_{-1}^{1} \frac{f(x)g(x)}{\sqrt{1-x^2}}\,dx$$

giver

$$\begin{aligned} T_0(x) &= 1, \\ T_1(x) &= x, \\ T_2(x) &= 2x^2 - 1, \\ T_3(x) &= 4x^3 - 3x, \\ T_4(x) &= 8x^4 - 8x^2 + 1. \end{aligned}$$

Disse og flere andre polynomier af denne type er indbygget i NumPy.

```
import matplotlib.pyplot as plt
import numpy as np

from numpy.polynomial import Polynomial as P
from numpy.polynomial import Legendre as L
from numpy.polynomial import Chebyshev as T

x, h = np.linspace(-1, 1, 100, retstep=True)

fig, ax = plt.subplots()
for i in range(6):
    ax.plot(x, L.basis(i)(x), label=f'$L_{{{i}}}$')

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
fig.legend()
```



Figur 14.1: Legendre og Chebychev polynomier.

```
fig, ax = plt.subplots()
for i in range(6):
    ax.plot(x, T.basis(i)(x), label=f'$T_{{{i}}}$')

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
fig.legend()
```

Vi får koefficienter for de givne polynomier vist via

```
np.polynomial.set_default_printstyle('ascii')
print(T.basis(4).convert(kind=P))
```

```
1.0 + 0.0 x - 8.0 x**2 + 0.0 x**3 + 8.0 x**4
```

som passer med vores udtryk for $T_4(x)$. Vi kan tjekke om et par Legendre funktioner er ortogonale ved

```
def trapz(f, h):
  return h * (f[0] + 2*np.sum(f[1:-1]) + f[-1]) / 2

print(trapz(L.basis(2)(x) * L.basis(3)(x), h))
```

```
1.278438634416847e-16
```

Resultatet er tæt på nul, så vi har ortogonalitet.

For Chebychev polynomier skal vi huske at bruge det rigtige indre produkt, men dette indre produkt har problemer ved $x = \pm 1$:

```
print(trapz(T.basis(2)(x) * T.basis(3)(x)
                / np.sqrt(1-x**2), h))
```

```
nan
```

Resultatet er `nan` da vi deler med 0. Lad os prøve med at smide disse to problematiske x-værdier væk:

```
x_kort = x[1:-1]
print(trapz(T.basis(2)(x_kort) * T.basis(3)(x_kort)
                / np.sqrt(1-x_kort**2), h))
```

```
5.8314744727786e-16
```

som bekræfter det ønskede ortogonalitet. △

14.2 Klassisk Gram-Schmidt i $\mathbb{R}^n$

Vores klassisk Gram-Schmidt starter med $u_0, u_1, \ldots, u_{k-1}$ lineært uafhængig. Lad os indføre lidt andet notation i tilfældet hvor vi konstruere en *ortonormal* samling $v_0, v_1, \ldots, v_{k-1}$, dvs. hvor vi vælger $c_i = 1/\|w_i\|_2$ hver gang. I hvert trin danner vi

$$w_j = u_j - \mathrm{pr}_{v_0,\ldots,v_{j-1}}(u_j) = u_j - r_{0j}v_0 - \cdots - r_{j-1,j}v_{j-1}. \tag{14.2}$$

Da $\langle v_i, w_j\rangle = 0$ og $\|v_i\|_2 = 1$, kan vi tage indre produktet af v_i med (14.2), som dermed giver

$$r_{ij} = \langle v_i, u_j\rangle = v_i^T u_j.$$

For at få en ortonormal samling, sætter vi så

$$v_j = \frac{1}{r_{jj}} w_j, \qquad \text{hvor } r_{jj} = \|w_j\|_2. \tag{14.3}$$

Alt i alt har vi den følgende algoritme

KLASSISK GRAM-SCHMIDT$(u_0, u_1, \ldots, u_{k-1})$

```
for j ∈ {0, 1, ..., k − 1}:
    w_j = u_j
    for i ∈ {0, 1, ..., j − 1}:
        r_ij = v_i^T u_j
        w_j = w_j − r_ij v_i
    r_jj = ‖w_j‖_2
    v_j = w_j / r_jj
return v_0, v_1, ..., v_{k−1}
```

Givet v_i og r_{ij} kan vi omskrive (14.2) og (14.3) til at få

$$\begin{aligned} u_0 &= r_{00}v_0, \\ u_1 &= r_{01}v_0 + r_{11}v_1, \\ u_2 &= r_{02}v_0 + r_{12}v_1 + r_{22}v_2, \\ &\vdots \\ u_{k-1} &= r_{0,k-1}v_0 + r_{1,k-1}v_1 + \cdots + r_{k-1,k-1}v_{k-1}. \end{aligned} \tag{14.4}$$

Dette kan skrives i matrixform, som

$$\begin{aligned} A &= \left[u_0 \mid u_1 \mid u_2 \mid \ldots \mid u_{k-1}\right] \\ &= \left[v_0 \mid v_1 \mid v_2 \mid \ldots \mid v_{k-1}\right] \begin{bmatrix} r_{00} & r_{01} & r_{02} & \ldots & r_{0,k-1} \\ 0 & r_{11} & r_{12} & \ldots & r_{1,k-1} \\ 0 & 0 & r_{22} & \ldots & r_{2,k-1} \\ \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & & r_{k-1,k-1} \end{bmatrix} \\ &= QR. \end{aligned} \tag{14.5}$$

Her er $A \in \mathbb{R}^{n\times k}$, matricen $Q \in \mathbb{R}^{n\times k}$ har ortonormale søjler $v_0, \ldots, v_{k-1}$, og $R \in \mathbb{R}^{k\times k}$ er *øvre triangulær*, dvs. under diagonalen har R kun nultal. En dekomponering af denne form kaldes en *tynd QR-dekomponering* af A. En *fuld QR-dekomponering* fås ved at udvide R med 0-tal til en matrix i $\mathbb{R}^{n\times k}$, og at tilføje søjler til Q, f.eks. ved hjælp af korollar 9.21, så vi får en ortogonal matrix i $\mathbb{R}^{n\times n}$.

Hvis samlingen $u_0, u_1, \ldots, u_{k-1}$ er lineært uafhængig, giver den klassiske Gram-Schmidt proces os en tynd QR-dekomponering med $r_{ii} > 0$ for alle i.

Når A er en $n \times k$-matrix, med $n \geqslant k$, hvis søjler er ikke lineært uafhængig, kan vi køre den klassiske Gram-Schmidt proces med en lille ændring: når et w_j er 0, kan vi bare vælge v_j til at være en tilfældig enhedsvektor vinkelret på $v_0, v_1, \ldots, v_{j-1}$. På denne måde får vi en QR-dekomponering for A med R øvre triangulær, men hvor nogle r_{ii} kan være 0.

Dette giver

Proposition 14.6. *Alle matricer $A \in \mathbb{R}^{n \times k}$, for $n \geqslant k$, tillader en tynd og end fuld QR-dekomponering.*

Hvis søjlerne af A er lineært uafhængig og $r_{ii} > 0$ for alle i, så er denne tynd QR-dekomponering entydig. □

Advarsel 14.7. NumPy har en indbygget funktion `np.linalg.qr`. Den producerer en QR-dekomponering, men den tillader $r_{ii} < 0$, så resultatet skal justeres hvis man vil sammenligne med Gram-Schmidt processen. !

Der er desværre numeriske problemer med den klassiske Gram-Schmidt proces. Vi vil prøve at rette op dette i det næste kapitel.

Kapitel 15

Forbedring af Gram-Schmidt processen

I det forrige kapitel beskrev vi den klassiske Gram-Schmidt proces, som matematisk producerer ortogonale eller ortonormale samlinger fra lineært uafhængige vektorer. Desværre er denne metode ikke så stabilt numerisk. Vi vil give et eksempel nedenfor, som demonstrerer problemet, og derefter se hvordan vi kan forbedre fremgangsmåden for at undgå uhensigtsmæssige fejl.

15.1 Numeriske problemer for klassisk Gram-Schmidt

Lad os betragte et eksempel i python. Vi begynder med de følgende vektorer i $\mathbb{R}^4$:

$$u_0 = \begin{bmatrix} 1 \\ s \\ 0 \\ 0 \end{bmatrix}, \quad u_1 = \begin{bmatrix} 1 \\ 0 \\ s \\ 0 \end{bmatrix}, \quad u_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ s \end{bmatrix}$$

og vælger s til at være tæt på 0, men ikke helt lille.

```
>>> import numpy as np
>>> s = 1e-8
>>> u0 = np.array([1.0,   s, 0.0, 0.0])[:, np.newaxis]
>>> u1 = np.array([1.0, 0.0,   s, 0.0])[:, np.newaxis]
>>> u2 = np.array([1.0, 0.0, 0.0,   s])[:, np.newaxis]
```

```
>>> a = np.hstack([u0, u1, u2])
>>> a
array([[1.e+00, 1.e+00, 1.e+00],
       [1.e-08, 0.e+00, 0.e+00],
       [0.e+00, 1.e-08, 0.e+00],
       [0.e+00, 0.e+00, 1.e-08]])
```

Vi udfører den klassiske Gram-Schmidt proces, med forventningen at det giver en ortonormal samling `v0`, `v1`, `v2`.

```
>>> def proj_på(v, u):
...     return np.vdot(v,u) / np.vdot(v,v) * v
...
>>> v0 = u0 / np.linalg.norm(u0)
>>> w1 = u1 - proj_på(v0, u1)
>>> v1 = w1 / np.linalg.norm(w1)
>>> w2 = u2 - proj_på(v0, u2) - proj_på(v1, u2)
>>> v2 = w2 / np.linalg.norm(w2)
>>> q = np.hstack([v0, v1, v2])
>>> q
array([[ 1.00000000e+00,  0.00000000e+00,  0.00000000e+00],
       [ 1.00000000e-08, -7.07106781e-01, -7.07106781e-01],
       [ 0.00000000e+00,  7.07106781e-01,  0.00000000e+00],
       [ 0.00000000e+00,  0.00000000e+00,  7.07106781e-01]])
```

Så har vi dannet en matrix `q` med søjler `v0`, `v1`, `v2`. Vi kan se om `v0`, `v1`, `v2` er faktisk ortonormal eller ej, ved at beregne Grammatricen for `q`:

```
>>> gram = q.T @ q
>>> gram
array([[ 1.00000000e+00, -7.07106781e-09, -7.07106781e-09],
       [-7.07106781e-09,  1.00000000e+00,  5.00000000e-01],
       [-7.07106781e-09,  5.00000000e-01,  1.00000000e+00]])
```

På diagonalen får vi pæne tal tæt på 1, så `v0`, `v1`, `v2` er enhedsvektorer. Desuden er de andre indre produkter med `v0` af den samme størrelsesorden som s, men de er skuffende langt væk fra machine epsilon. Det værste problem er dog indre

produktet $\langle \mathtt{v1}, \mathtt{v2} \rangle$, som burde være 0, men er tæt på $1/2$. Det vil sige vinklen mellem `v1` og `v2` er

```
>>> np.degrees(np.arccos(np.vdot(v1,v2)
...             / (np.linalg.norm(v1) * np.linalg.norm(v2))))
59.99999999999999
```

omtrent $60°$, så de er meget langt fra at være vinkelret på hinanden. Dette er udtryk for at den klassiske Gram-Schmidt proces er numerisk ustabil.

Lad os bytte rundt på vores operationer ovenfor:

```
>>> v0 = u0 / np.linalg.norm(u0)
>>> w1 = u1 - proj_på(v0, u1)
>>> x2 = u2 - proj_på(v0, u2)
>>> v1 = w1 / np.linalg.norm(w1)
>>> w2 = x2 - proj_på(v1, x2)
>>> v2 = w2 / np.linalg.norm(w2)
```

Her har vi delt dannelsen af w_2 op i to trin

$$\begin{aligned} w_2 &= u_2 - \mathrm{pr}_{v_0}(u_2) - \mathrm{pr}_{v_1}(u_2) \\ &= (u_2 - \mathrm{pr}_{v_0}(u_2)) - \mathrm{pr}_{v_1}(u_2 - \mathrm{pr}_{v_0}(u_2)) \\ &= x_2 - \mathrm{pr}_{v_1}(x_2), \qquad\qquad \text{for } x_2 = u_2 - \mathrm{pr}_{v_0}(u_2). \end{aligned}$$

Denne omskrivning er gyldigt, da $\mathrm{pr}_{v_0}(u_2)$ er parallelt med v_0 og $\mathrm{pr}_{v_1}(v_0) = 0$, så $\mathrm{pr}_{v_1}(\mathrm{pr}_{v_0}(u_2)) = 0$. Vi får nu

```
>>> q = np.hstack([v0, v1, v2])
>>> q
array([[ 1.00000000e+00,  0.00000000e+00,  0.00000000e+00],
       [ 1.00000000e-08, -7.07106781e-01, -4.08248290e-01],
       [ 0.00000000e+00,  7.07106781e-01, -4.08248290e-01],
       [ 0.00000000e+00,  0.00000000e+00,  8.16496581e-01]])
```

og Grammatricen bliver

```
>>> gram = q.T @ q
>>> gram
```

```
array([[ 1.00000000e+00, -7.07106781e-09, -4.08248290e-09],
       [-7.07106781e-09,  1.00000000e+00, -8.65080346e-17],
       [-4.08248290e-09, -8.65080346e-17,  1.00000000e+00]])
```

Dette er en væsentlig forbedring, nu er alle indgange væk fra diagonalen af størrelsesorden højst 10^{-8}, og faktisk er `v2` og `v3` vinkelret på hinanden indenfor machine epsilon.

Hvorfor har denne omskrivning givet et bedre resultat? I beregningen via den klassiske Gram-Schmidt proces er `v0` lige med `u0`, da

```
>>> np.linalg.norm(u0)
1.0
```

Tallet `s` er netop valgt så at `s**2` er mindre end $\epsilon_{\text{machine}}$.

Beregning af `proj_på(v0, u1)` giver så `v0`, og derefter beregnes `w1` til at være $(0, -s, s, 0)$, som er vinkelret på `u2`. Dette medvirker til at `v1` bidrager ikke til udregningen af `w2`. Men `np.vdot(v0, v1)` er kun lidt mindre end `s`, og `proj_på(v0, u2)` er lige med `v0`, og så ortogonalitet mellem `v1` og `v2` bliver ikke sikret.

Til gengæld for den anden version af udregning har vi igen `w1` $= (0, -s, s, 0)$, men sætter `x2` $= (0, -s, 0, s)$. Disse er begge ortogonal på `v0` indenfor machine epsilon og begge indeholder kun led af størrelsesorden `s`. Beregning af `w2` sikre nu en vektor vinkelret på `w1` uden at gøre for stor skade på indre produktet med `v0`.

15.2 Den forbedrede Gram-Schmidt proces

For at implementere den overstående ide, lad os starte med $u_0, u_1, \ldots, u_{k-1}$ lineært uafhængige. Så går vi i gang med at konstruere ortonormale $v_0, v_1, \ldots, v_{k-1}$. I trinnet hvor vi danne v_r vil vi sørge for at vi kun arbejde med vektorer, som er vinkelret på $v_0, \ldots, v_{r-2}$. Dette udmøntes i den følgende.

FORBEDRET GRAM-SCHMIDT PROCES—SKITSE$(u_0, u_1, \ldots, u_{k-1})$

Sæt $w_i^{(0)} = u_i$, $i = 0, \ldots, k-1$, og $v_0 = w_0^{(0)}/\|w_0^{(0)}\|$.

Sæt $w_i^{(1)} = w_i^{(0)} - \mathrm{pr}_{v_0}(w_i^{(0)})$, $i = 1, \ldots, k-1$, og $v_1 = w_1^{(1)}/\|w_1^{(1)}\|$.

Sæt $w_i^{(2)} = w_i^{(1)} - \mathrm{pr}_{v_1}(w_i^{(1)})$, $i = 2, \ldots, k-1$, og $v_2 = w_2^{(2)}/\|w_2^{(2)}\|$.

...

Sæt $w_i^{(r)} = w_i^{(r-1)} - \mathrm{pr}_{v_{r-1}}(w_i^{(r-1)})$, $i = r, \ldots, k-1$, og $v_r = w_r^{(r)}/\|w_r^{(r)}\|$.

...

return $v_0, v_1, \ldots, v_{k-1}$, som er en ortonormal samling,
der udspænder det samme rum som $u_0, u_1, \ldots, u_{k-1}$.

Matematisk er dette ækvivalent med den klassiske Gram-Schmidt proces med $c_i = 1/\|w_i\|_2$.

Eksempel 15.1. Et eksempel, som billedliggør forskellen på klassisk og forbedret Gram-Schmidt er følgende. Betragt $V = \mathbb{R}^5$ og vektorer $u_0, u_1, \ldots, u_4$, som er søjlerne i matricen

$$A = \left[u_0 \mid u_1 \mid u_2 \mid u_3 \mid u_4\right] = \begin{bmatrix} 1 & * & * & * & * \\ 0 & 1 & * & * & * \\ 0 & 0 & 1 & * & * \\ 0 & 0 & 0 & 1 & * \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

hvor $*$ repræsenterer vilkårlige tal. Da $u_0 = e_0$ er en enhedsvektor, ændres denne vektor ikke af det første trin i den klassiske Gram-Schmidt proces Det andet trin derimod sørger for at søjle 1 bliver til e_1, dvs. at indgangen a_{01} sættes til 0. Næste trin giver e_2 i søjle 2, så a_{02} og a_{12} nulstilles, osv.

$$\left[v_0 \mid v_1 \mid v_2 \mid v_3 \mid v_4\right] = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Dvs. den klassiske Gram-Schmidt proces fremstiller vektorerne $v_0, v_1, \ldots, v_4$ søjlevis, én efter én.

For den forbedrede Gram-Schmidt proces, begynder vi på sammen måde, $v_0 = u_0$. Men så beregnes $w_i^{(1)}$, og dette sørger for at der kommer 0-tal i resten

af den første række, dvs. $a_{01}, a_{02}, a_{03}, a_{04}$ nulstilles. Derefter opereres på $A_{[1:,1:]}$, og $*$-indgangene nulstilles rækkevis

$$\left[v_0 \,\middle|\, v_1 \,\middle|\, v_2 \,\middle|\, v_3 \,\middle|\, v_4\right] = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

△

I $\mathbb{R}^n$ fører dette til den følgende algoritme:

FORBEDRET GRAM-SCHMIDT$(u_0, u_1, \ldots, u_{k-1})$

for $i \in \{0, 1, \ldots, k-1\}$**:**
    $w_i = u_i$
for $i \in \{0, 1, \ldots, k-1\}$**:**
    $r_{ii} = \|w_i\|_2$
    $v_i = w_i / r_{ii}$
    for $j \in \{i+1, \ldots, k-1\}$**:**
        $r_{ij} = v_i^T w_j$
        $w_j = w_j - r_{ij} v_i$

Hvis det er nødvendigt kan hukommelsesplads spares ved at gemme v_i i w_i.

15.3 Klassisk kontra forbedret Gram-Schmidt

Lad os kikke på implementering af disse algoritme i python, og giver et eksempel, som viser hvor stærkt den forbedrede Gram-Schmidt proces er i forhold til den klassiske.

Vi begynder med at importere vores standardpakker

```
import matplotlib.pyplot as plt
import numpy as np
```

Den klassiske Gram-Schmidt implementeres på følgende vis

```
def klassisk_gram_schmidt(a):
    _, k = a.shape
```

```
    q = np.empty_like(a)
    r = np.zeros((k, k))
    for j in range(k):
        r[:j, [j]] = q[:, :j].T @ a[:, [j]]
        w = a[:, [j]] - q[:, :j] @ r[:j, [j]]
        r[j, j] = np.linalg.norm(w)
        q[:, [j]] = w / r[j, j]
    return q, r
```

Algoritmen returnerer matricerne `q` og `r` for en tynd QR-dekomponering af `a`. Vores tidligere fremstilling, side 191, af den klassiske Gram-Schmidt proces indebar to **for**-løkker. Ovenfor har vi omskrevet den indre **for**-løkke via matrixprodukter.

Vores input vektorer er søjlerne af `a`; den `j`'te søjle er `a[:, [j]]`. De ortonormale vektorer der produceres af processen bliver til søjlerne i `q`. For at danne matricen `r` har vi brug for at vide hvor mange søjler `a` har. Dette kan gøres ved at sætte `k = a.shape[1]` eller ved at skrive `_, k = a.shape`. Det virker da `a.shape` er et talpar; symbolet _ bruges til værdier der må gerne smides væk.

Ved det `j`'te trin i processen har vi dannet ortonormale søjler `0` til `j-1` af `q`. De nye indgang i `r` gives via indre produkter med `a[:, [j]]`, som kan samles i produktet `q[:, :j].T @ a[:, [j]]`. Projektionen af `a[:, [j]]` på de første `j` søjler af `q` er så `q[:, :j] @ r[:j, [j]]`. Vi overskriver `w` hver gang med den nye w_j, dens længde gemmes i `r[j, j]` og den tilsvarende enhedsvektor bliver til `q[:, [j]]`.

For den forbedrede Gram-Schmidt laver vi tilsvarende omskrivninger til matrixprodukter.

```
def forbedret_gram_schmidt(a):
    _, k = a.shape
    q = np.copy(a)
    r = np.zeros((k, k))
    for i in range(k):
        r[i, i] = np.linalg.norm(q[:, i])
        q[:, i] /= r[i, i]
        r[[i], i+1:] = q[:, [i]].T @ q[:, i+1:]
        q[:, i+1:] -= q[:, [i]] @ r[[i], i+1:]
    return q, r
```

I begyndelsen kopieres a til q; denne kopiering er vigtig, da indgangene i q vil blive overskrevet flere gange i løbet af processen. Ved det i'te trin i processen inderholder søjler 0 til i-1 af q ortonormale vektorer, som er en del af slutresultatet, resten af q indeholder vektorerne $w_j^{(i)}$, som bliver overskrevet gentagende gange. Matricen r dannes rækkevis.

Lad os først bekræfter at disse implementeringer giver de samme resultater, som vi fik for eksemplet i begyndelsen af kapitlet.

```
s = 1e-8
a = np.array([[1.0, 1.0, 1.0],
              [  s, 0.0, 0.0],
              [0.0,   s, 0.0],
              [0.0, 0.0,   s]])
q, r = klassisk_gram_schmidt(a)
print(a - q @ r)
print()
print(q.T @ q)
```

```
[[ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00 -1.02200472e-25 -1.02200472e-25]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00]]

[[ 1.00000000e+00 -7.07106781e-09 -7.07106781e-09]
 [-7.07106781e-09  1.00000000e+00  5.00000000e-01]
 [-7.07106781e-09  5.00000000e-01  1.00000000e+00]]
```

Bemærk at selvom q er langt fra at have ortonormale søjler, vi har trods alt at q @ r er meget tæt på a.

```
q, r = forbedret_gram_schmidt(a)
print(a - q @ r)
print()
print(q.T @ q)
```

```
[[ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00 -1.02200472e-25 -2.08999485e-25]
 [ 0.00000000e+00  0.00000000e+00  8.56217846e-26]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00]]

[[ 1.00000000e+00 -7.07106781e-09 -4.08248290e-09]
 [-7.07106781e-09  1.00000000e+00  1.03018891e-16]
 [-4.08248290e-09  1.03018891e-16  1.00000000e+00]]
```

Igen `q @ r` ligger tæt på `a`, men denne gang er `q` væsentligt tættere på at have ortonormale søjler.

Nu udfører vi et eksperiment. Vi vil danne en tilfældig (100×100)-matrix med bestemte singulærværdier, nemlig

$$(\sigma_0, \ldots, \sigma_{99}) = (1, 1/2, 1/4, 1/8, \ldots, 1/2^{99}).$$

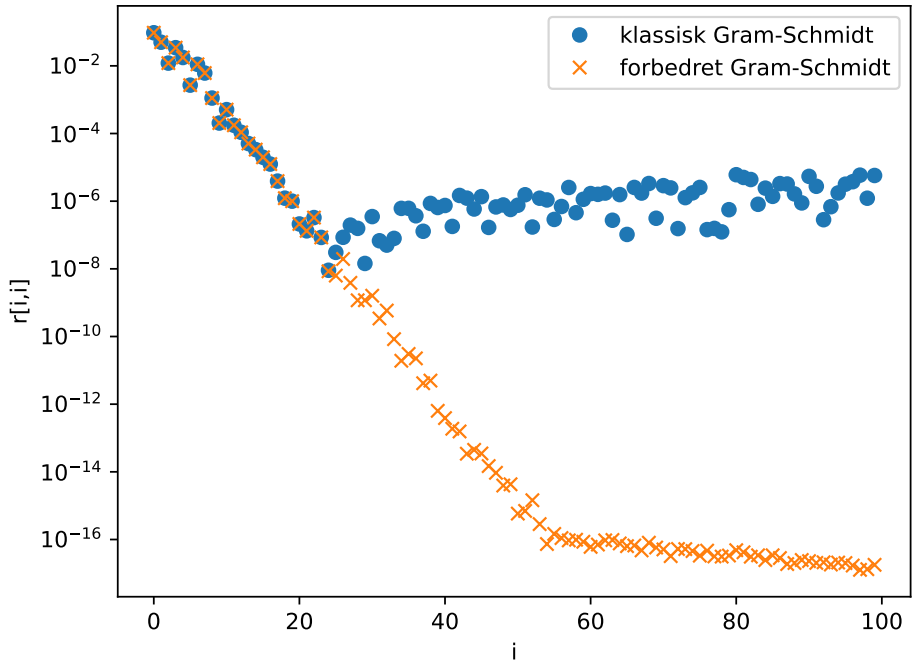
Vi gør dette ved at beregne singulærværdidekomponeringen af en tilfældig matrix og bruger dens `u` og `vt` til at danne en ny matrix `a` med de ønskede singulærværdier:

```
rng = np.random.default_rng()
n = 100
u, _, vt = np.linalg.svd(rng.random((n, n)))
i = np.arange(n)
s = np.array(2.0 ** (-i))
a = u @ np.diag(s) @ vt
print(s[:5])
```

```
[1.     0.5    0.25   0.125  0.0625]
```

Da singulærværdierne aftager hurtigt, burde beregning af QR-dekomponering af `a` giver diagonalindgangene i `r`, som er tæt på singulærværdierne for `a`. Vi kan beregne `r` via de to algoritmer og plotte disse diagonalværdier.

```
qk, rk = klassisk_gram_schmidt(a)
qf, rf = forbedret_gram_schmidt(a)

fig, ax = plt.subplots()
```



Figur 15.1: Klassisk kontra forbedret Gram-Schmidt.

```
ax.set_yscale('log')
ax.plot(i, rk[i,i], 'o', label='klassisk Gram-Schmidt')
ax.plot(i, rf[i,i], 'x', label='forbedret Gram-Schmidt')

ax.set_xlabel('i')
ax.set_ylabel('r[i,i]')
ax.legend()
```

Resultatet visers i figur 15.1. Her ses tydeligt at den forbedrede Gram-Schmidt proces beregner mange flere indgange korrekt end den klassiske. Den forbedrede giver korrekte værdier til omkring indeks `55`, og

```
print(2.**-55)
```

```
2.7755575615628914e-17
```

er tæt på machine epsilon. Til gengæld er den klassiske Gram-Schmidt proces kun korrekt for de første cirka 25 indgange, og værdierne efterfølgende ligger om 10^{-8}, som er tæt på kvadratroden af machine epsilon

```
machine_epsilon = np.finfo(float).eps
print(np.sqrt(machine_epsilon))
```

```
1.4901161193847656e-08
```

Dette er ret typisk for opførelsen af de to algoritmer.

15.4 Flops

Lad os tæller antallet af `float` operationer forbundet med den forbedrede Gram-Schmidt proces. Optællingen for den klassiske version er nogenlunde den samme. Vi kikker på algoritmen på side 198. Løkken i linjerne 1 og 2 tæller ingen flops. Kikker vi på den anden **for**-løkke, som starter på linje 3, har vi først

$$\begin{aligned} r_{ii} &= \|w_i\|_2: \quad 2n+1 \text{ flops} \\ v_i &= w_i/r_{ii}: \quad n \text{ flops} \end{aligned}$$

så efter k omganger bidrager dette med $k(3n+1)$ flops. I den indre **for**-løkke, som starter på linje 6 har vi

$$\begin{aligned} r_{ij} &= v_i^T w_j: \quad 2n \text{ flops} \\ w_j &= w_j - r_{ij} v_i: \quad 2n \text{ flops} \end{aligned}$$

Så denne indre løkke bidrager med $4n(k-i-1)$ flops for hvert $i \in \{0, 1, \ldots, k-1\}$, dvs.

$$4n \sum_{i=0}^{k-1} (k-i-1) = 4n \sum_{j=0}^{k-1} j = 2n(k-1)k \text{ flops.}$$

Det sidste bruger formlen

$$\sum_{j=0}^{k-1} j = \frac{1}{2}(k-1)k, \tag{15.1}$$

som følger af

$$
\begin{aligned}
2\sum_{j=0}^{k-1} j = \quad & 0 + 1 + 2 + \cdots + (k-3) + (k-2) + (k-1) \\
& + (k-1) + (k-2) + (k-3) + \cdots + 2 + 1 + 0 \\
= \quad & (k-1) + (k-1) + (k-1) + \cdots + (k-1) + (k-1) + (k-1) \\
= \quad & (k-1)k.
\end{aligned}
$$

Alt i alt har vi

$$2n(k-1)k + k(3n+1) = 2nk^2 + kn + k \sim 2nk^2 \text{ flops}$$

for den forbedrede Gram-Schmidt proces

Kapitel 16

Mindste kvadraters metode

Det er ikke usædvanligt at vi ønsker at estimere nogle parametre i en model. Nogle gange kan dette gøres ved at udføre konkrete eksperimenter og målinger. For at få en god estimering af parametrene tager man gerne så mange målinger, som muligt. Men typisk er disse målinger forbundet med fejl, så de opnåede datapunkter ligger ikke præcis på den kurve der forventes, og de resulterende ligninger er inkonsistent. Den mindste kvadraters metode forsøge at give et bedste bud på disse parametre.

16.1 Problemformulering

Givet et lineært ligningssystem

$$Ax = b \tag{16.1}$$

med flere ligninger end variabler, er der for typisk b ingen løsning. Lad os tage $A \in \mathbb{R}^{m\times n}$, med $m > n$, så $x \in \mathbb{R}^n$ og $b \in \mathbb{R}^m$. Systemet (16.1) kan skrives

$$x_0 a_0 + x_1 a_1 + \cdots + x_{n-1} a_{n-1} = b,$$

hvor $a_0, a_1, \ldots, a_{n-1}$ er søjlerne af A. Som i proposition 12.18 følger det at systemet har en løsning kun hvis b ligger i søjlerummet $S(A)$ af A. Men for $m > n$ vil $S(A)$ ikke være hele $\mathbb{R}^m$.

For generel $b \in \mathbb{R}^m$, betragter vi *restvektoren*

$$r = b - Ax \in \mathbb{R}^m.$$

I stedet for at løse (16.1), arbejder vi på at finde $x \in \mathbb{R}^n$ således at restvektoren er så lille, som muligt. Lad os måle vektorstørrelse via det standard indre produkt. Så ønsker vi at minimere $\|r\|_2 = \sqrt{r_0^2 + r_1^2 + \cdots + r_{m-1}^2}$. Dette er det samme som at minimere $\|r\|_2^2 = r_0^2 + r_1^2 + \cdots + r_{m-1}^2$. Af denne grund kaldes det følgende den mindste kvadraters problemstilling.

Problemstilling 16.1. Givet $A \in \mathbb{R}^{m\times n}$ og $b \in \mathbb{R}^m$, bestem $x \in \mathbb{R}^n$, som minimerer $\|r\|_2$, hvor $r = b - Ax$ er restvektoren.

Vi har allerede nogle værktøj, der hjælper med at besvare dette. Når x varierer, giver Ax alle vektorer i søjlerummet $S(A)$. Men vektoren i $S(A)$ tættest på b er projektionen Pb af b på søjlerummet af A. Så den mindste kvadraters metode er ækvivalent med at løse

$$Ax = Pb. \tag{16.2}$$

Lad os give to forskellige løsningsmetoder: En via QR-dekomponering, fra f.eks. en Gram-Schmidt proces, og en via singulærværdidekomponering.

16.2 Løsning via QR-dekomponering

Givet en tynd QR-dekomponering af $A = QR$, har vi

$$Ax = QRx = Qy = y_0q_0 + y_1q_1 + \cdots + y_{n-1}q_{n-1}$$

hvor $q_0, q_1, \ldots, q_{n-1}$ er søjlerne af Q og $y = Rx$. Husk at $q_0, \ldots, q_{n-1}$ er en ortonormal samling, så er ortogonal med hver q_i af længde 1. Matricen R er øvre triangulær. Vi antager at $S(Q) = S(A)$, som er det samme som at alle diagonalindgange i R er forskellige fra 0.

Fra definition 8.19 har vi så at projektionen P på søjlerummet af A er givet ved matricen

$$\begin{aligned} P &= q_0q_0^T + q_1q_1^T + \cdots + q_{n-1}q_{n-1}^T \\ &= QQ^T. \end{aligned}$$

Vi ønsker at løse (16.2), som er nu

$$QRx = QQ^Tb. \tag{16.3}$$

Da Q har ortonormale søjler, opfylder dens Grammatrix $Q^TQ = I_n$. Dette kan vi bruge i (16.3) ved at gange Q^T på fra venstre. Vi får dermed at $Rx = I_nRx = Q^TQRx = Q^TQQ^Tb = I_nQ^Tb = Q^Tb$. Omvendt givet ligningen $Rx = Q^Tb$ kan vi gange fra venstre med Q og få $QRx = QQ^Tb$, som er netop (16.3).

Vi har så vist at (16.3) er det samme som

$$Rx = Q^Tb, \tag{16.4}$$

selvom Q er ikke nødvendigvis kvadratisk, så ikke nødvendigvis invertibel. Observér at R er øvre triangulær, så vi kan forvente at løse (16.4) via back substitution.

Dette giver den følgende løsningsmetode for problemstilling 16.1

MINDSTE KVADRATER VIA QR(A, b)

Krav: søjlerne af A er lineært uafhængig

1 Beregn en tynd QR-dekomponering $A = QR$
2 Beregn Q^Tb
3 Løs $Rx = Q^Tb$ via back substitution

Antallet af flops for denne metode er cirka $2mn^2$ hvis vi beregner QR-dekomponering via den forbedrede Gram-Schmidt proces.

16.3 Løsning via singulærværdidekomponering

En alternativ metode er at udnytte singulærværdidekomponeringen. Husk at SVD af $A \in \mathbb{R}^{m\times n}$ er ækvivalent med at skrive

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{k-1} u_{k-1} v_{k-1}^T,$$

hvor $\sigma_0 \geqslant \sigma_1 \geqslant \cdots \geqslant \sigma_{k-1} \geqslant 0$, for $k = \min\{m, n\}$, og samlingerne $u_0, \ldots, u_{k-1}$ i $\mathbb{R}^m$ og $v_0, \ldots, v_{k-1}$ i $\mathbb{R}^n$ er ortonormale.

Lad r være det største tal så at $\sigma_{r-1} > 0$. Så er

$$A = \sigma_0 u_0 v_0^T + \sigma_1 u_1 v_1^T + \cdots + \sigma_{k-1} u_{r-1} v_{r-1}^T,$$

og

$$A = U\Sigma V^T$$

med

$$U = \left[u_0 \;\middle|\; u_1 \;\middle|\; \ldots \;\middle|\; u_{r-1}\right] \in \mathbb{R}^{m\times r}, \quad \Sigma = \operatorname{diag}(\sigma_0, \sigma_1, \ldots, \sigma_{r-1}) \in \mathbb{R}^{r\times r},$$
$$V = \left[v_0 \;\middle|\; v_1 \;\middle|\; \ldots \;\middle|\; v_{r-1}\right] \in \mathbb{R}^{n\times r}.$$

Dette kaldes den *reducerede* singulærværdidekomponering.

Bemærk at $u_0, u_1, \ldots, u_{r-1}$ er nu en ortonormal basis for $S(A)$. Vi kalder tallet r for *rangen* af A, og observerer at den er lige med antallet af pivotsøjler i echelonformen for A.

Projektionen P på $S(A)$ er nu

$$P = UU^T.$$

Så for den mindste kvadraters metode ønsker vi at løse

$$U\Sigma V^T x = UU^T b.$$

Da søjlerne af U er ortonormal, kan vi vise, ligesom i afsnit 16.2, at dette er ækvivalent med

$$\Sigma V^T x = U^T b.$$

Sæt $y = V^T x$. Husk at $\Sigma = \operatorname{diag}(\sigma_0, \sigma_1, \ldots, \sigma_{r-1}) \in \mathbb{R}^{r\times r}$, med alle $\sigma_i > 0$. Det giver at Σ er invertibel med invers

$$\Sigma^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_0}, \frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_{r-1}}\right),$$

og at vi kan nemt løse $\Sigma y = U^T b$ for y, som

$$y = \Sigma^{-1} U^T b.$$

Desuden er VV^T projektionen på $S(V)$. Så ligningen $y = V^T x$ har en løsning givet som

$$x = Vy.$$

Alt i alt har vi

$$x = V\Sigma^{-1} U^T b. \tag{16.5}$$

Det kan bekræftes ved udregningen at

$$\begin{aligned} Ax &= U\Sigma V^T V \Sigma^{-1} U^T b = U\Sigma I_r \Sigma^{-1} U^T b \\ &= U I_r U^T b = UU^T b = Pb, \end{aligned}$$

som ønsket.

Matricen $A^+ = V\Sigma^{-1}U^T$ i (16.5) kaldes *pseudoinversen* til A, og (16.5) er løsningen til $Ax = Pb$, som har $\|r\|_2$ mindst muligt, hvor $r = b - Ax$ er restvektoren.

Vores anden løsningsmetode for problemstilling 16.1 er dermed

MINDSTE KVADRATER VIA SVD(A, b)
1 Beregn en reduceret SVD $A = U\Sigma V^T$
2 Beregn $U^T b$
3 Løs $\Sigma y = U^T b$
4 Sæt $x = Vy$

Antallet af flops for denne metode med $A \in \mathbb{R}^{m\times n}$ er cirka $2mn^2 + 11n^3$. Så generelt er denne metode langsommere end via QR-dekomponering. Men metoden har også nogle fordel. Beregnes sådan en QR-dekomponering via den forbedrede Gram-Schmidt metode er resultatet ikke så præcis, som den fra SVD metoden; vi vil se et eksempel senere. Desuden er SVD metoden gyldig når søjlerne af A er ikke nødvendigvis lineært uafhængig, i modsætning til QR-metoden, og den virker for alle m og n.

16.4 Polynomier

Lad os kikke på et par eksempler hvor vi tilpasser polynomier til given data.

Vi begynder med et simpelt eksempel hvor der er kun 3 datapunkter. Betragt datapunkterne

$$(x_0, y_0) = (0{,}0, 0{,}2), \quad (x_1, y_1) = (1{,}0, 0{,}5), \quad (x_2, y_2) = (2{,}0, 1{,}2).$$

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([0.0, 1.0, 2.0])
y = np.array([0.2, 0.5, 1.2])

fig, ax = plt.subplots()
ax.plot(x, y, 'o')
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Her har vi indlæst punkterne og plottet dem som den første plot i figur 16.1. Da der er kun 3 punkter, findes der et entydig andengradspolynomium igennem disse punkter. Dette polynomium kan findes på følgende måde. Vi danner Vandermondematricen hvis søjler er x-koordinaterne opløftet i potenserne 2, 1 og 0:

```
cols = len(x)
print(f'{cols = }')

a = np.vander(x, cols)
print(a)
```

```
cols = 3
[[0. 0. 1.]
 [1. 1. 1.]
 [4. 2. 1.]]
```

Husk at $p(x) = ax^2 + bx + c$ kan udregnes, som

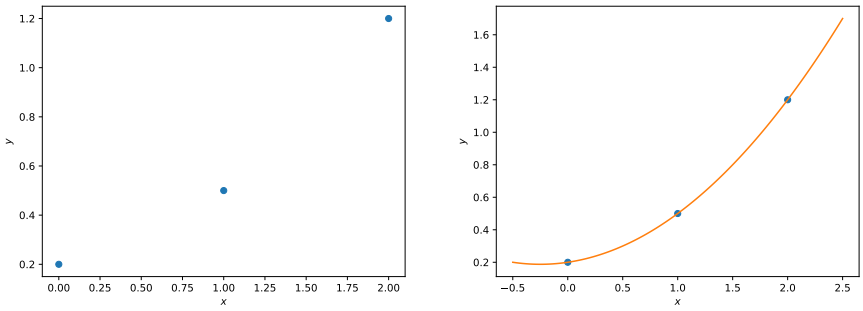
$$p(x) = \begin{bmatrix} x^2 & x & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix},$$

sammenlign med eksempel 4.3. Det følger at bestemmelsen af polynomiet $ax^2 + bx + c$, som har de 3 givne y-værdier i de givne x-punkter, er det samme som at løse det lineære ligningssystem

$$\begin{bmatrix} x_0^2 & x_0 & 1 \\ x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix}.$$

Matricen til venstre er netop Vandermondematricen, som vi har lige konstrueret. Vi kan nu løse systemet i python på følgende vis

```
koeffs = np.linalg.solve(a, y[:, np.newaxis])
print(koeffs)
```



Figur 16.1: Andengradspolynomium igennem 3 datapunkter.

```
[[0.2]
 [0.1]
 [0.2]]
```

Nu kan vi plotte kurven fra andengradspolynomiet

```
t = np.linspace(-0.5, 2.5, 100)

fig, ax = plt.subplots()
ax.plot(x, y, 'o')
ax.plot(t, np.vander(t, cols) @ koeffs)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Resultatet vises til højre i figur 16.1.

Som alternativt kan vi finde en ret linje, dss. polynomium af grad 1, som ligger tæt på den givne punkter, ved hjælp af den mindste kvadraters metode. Dette kaldes også lineær regression. Vi vil gøre det ved brug af QR-metoden via den forbedrede Gram-Schmidt proces.

```
def forbedret_gram_schmidt(a):
    _, k = a.shape
    q = np.copy(a)
    r = np.zeros((k, k))
    for i in range(k):
```

```
        r[i, i] = np.linalg.norm(q[:, i])
        q[:, i] /= r[i, i]
        r[[i], i+1:] = q[:, [i]].T @ q[:, i+1:]
        q[:, i+1:] -= q[:, [i]] @ r[[i], i+1:]
    return q, r
```

Vi vil have et polynomium af grad 1. Den har form $p(x) = ax + b$, så der et 2 ubkendte og antallet af søjler i vores Vandermondematrice er så 2, som er én mere end graden af det ønskede polynomium:

```
cols = 2
a = np.vander(x, cols)
print(a)
```

```
[[0. 1.]
 [1. 1.]
 [2. 1.]]
```

Vi følgende QR-algoritmen fra slutningen af afsnit 16.2. QR-dekomponeringen af a beregnes

```
q, r = forbedret_gram_schmidt(a)
print(q)
print()
print(r)
```

```
[[ 0.          0.91287093]
 [ 0.4472136   0.36514837]
 [ 0.89442719 -0.18257419]]

[[2.23606798 1.34164079]
 [0.         1.09544512]]
```

Vektoren $c = Q^T b$ er så

```
c = q.T @ y[:, np.newaxis]
print(c)
```

```
[[1.29691943]
 [0.14605935]]
```

og koefficienterne til førsteordenspolynomiet findes nu via back substitution

```
def back_subs(r, c):
    _, n = r.shape
    x = np.empty((n, 1))
    for i in reversed(range(n)):
        x[i] = (c[i] - r[[i], i+1:] @ x[i+1:]) / r[i, i]
    return x

koeffs = back_subs(r, c)
print(koeffs)
```

```
[[0.5       ]
 [0.13333333]]
```

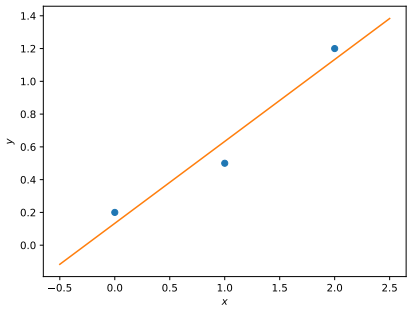
Et plot af resultatet

```
t = np.linspace(-0.5, 2.5, 100)

fig, ax = plt.subplots()
ax.plot(x, y, 'o')
ax.plot(t, np.vander(t, cols) @ koeffs)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

gives i figur 16.2.

Denne samme fremgangsmåde kan anvendes på flere datapunkter. Her vil man ofte gerne bruger polynomier af højere grad. Lad os tage et eksempel og først finde et polynomium, der går igennem alle datapunkter.

```
x = np.array([-2.1, -1.9, -1.5, -0.8, -0.3, 0.1,
              0.5, 1.2, 1.3, 1.7, 2.4])
y = np.array([ 0.1, 0.4, -0.1, -0.6, -0.5, 0.1,
              1.0, 1.3, 0.7, 0.1, 0.2])
```



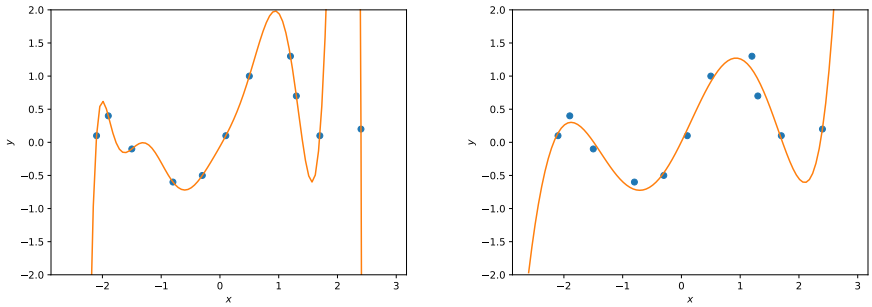
Figur 16.2: Lineær tilnærmelse af data.

```
cols = len(x)
print(f'{cols = }')
```

```
cols = 11
```

```
a = np.vander(x, cols)
koeffs = np.linalg.solve(a, y[:, np.newaxis])

t = np.linspace(x.min()-0.5, x.max()+0.5, 100)

fig, ax = plt.subplots()
ax.set_ylim(-2.0, 2.0)
ax.plot(x, y, 'o')
ax.plot(t, np.vander(t, cols) @ koeffs)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Kikker vi på plottet til venstre i figur 16.3, ses at dette polynomium er en meget dårlig repræsentation af formen af datapunkterne. Den stræber for meget efter at gå igennem hver eneste punkt. Dette kaldes *over fitting*. Reduceres graden af polynomiet, og bruges de mindste kvadraters metode



Figur 16.3: Højere grad.

```
cols = 7
a = np.vander(x, cols)
q, r = forbedret_gram_schmidt(a)
c = q.T @ y[:, np.newaxis]
koeffs = back_subs(r, c)

fig, ax = plt.subplots()
ax.set_ylim(-2.0, 2.0)
ax.plot(x, y, 'o')
ax.plot(t, np.vander(t, cols) @ koeffs)

ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

fås en bedre gengivelse, se plottet til højre i figur 16.3. Afvigelserne af dette polynomium fra de givne datapunkter ligger i restvektoren

```
rest = y[:, np.newaxis] - np.vander(x, cols) @ koeffs
print(rest)
```

```
[[-0.04036569]
 [ 0.09963581]
 [-0.11044335]
 [ 0.11108815]
```

```
 [-0.03960429]
 [-0.09563144]
 [ 0.08603725]
 [ 0.19850296]
 [-0.25445747]
 [ 0.04795903]
 [-0.00272098]]
```

Et godt mål for den samlede afvigelsen er normen

```
print(np.linalg.norm(rest))
```

```
0.40083759520912504
```

Beregningen via SVD i stedet for QR-dekomponering, er givet nedenfor. I dette tilfælde er resultatet meget tæt på den forrige fra QR-metoden, så vi nøjes med at give længden af restvektoren.

```
u, s, vt = np.linalg.svd(a, full_matrices=False)
koeffs_svd = vt.T @ (np.diag(1/s) @ (u.T @ y[:, np.newaxis]))
rest_svd = y[:, np.newaxis] - np.vander(x, cols) @ koeffs_svd
print(np.linalg.norm(rest_svd))
```

```
0.4008375952091249
```

Differencen mellem restvektorerne for de to metoder har længde

```
print(np.linalg.norm(rest_svd - rest))
```

```
3.5891211227079e-14
```

i dette eksempel.

16.5 Løsning via normalligninger

Der findes en tredje løsningsmetode for det mindste kvadraters problem, som burde nævnes, nemlig via normalligningerne. Der er mange tekster, som giver kun denne løsningsmetode, men i det næste kapitel vil vi se at den er numerisk meget upræcist. Så i praksis er metoderne ovenfor altid at foretrække.

Vi antager igen at $A \in \mathbb{R}^{m\times n}$, $m \geqslant n$, har lineært uafhængige søjler. Hvis A har reduceret SVD $A = U\Sigma V^T$, så har vi at $r = \operatorname{rang}(A) = n$ og at

$$\begin{aligned} A^T A &= (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U\Sigma V^T \\ &= V\Sigma I_r \Sigma V^T = V\Sigma^2 V^T \end{aligned}$$

er invertibel med invers $V\Sigma^{-2}V^T$. Dette giver

$$\begin{aligned} (A^T A)^{-1} A^T &= V\Sigma^{-2}V^T V\Sigma U^T = V\Sigma^{-2} I_n \Sigma U^T \\ &= V\Sigma^{-1}U^T = A^+. \end{aligned} \tag{16.6}$$

Vi ved fra (16.5) at løsning af den mindste kvadraters problem $Ax = Pb$ er $x = A^+ b$. For A med lineært uafhængige søjler, viser (16.6) at dette er ækvivalent med at x er løsningen af

$$A^T A x = A^T b. \tag{16.7}$$

(16.7) kaldes *normalligningerne* for det mindste kvadraters problem. Som nævnt ovenfor er denne metode dog numerisk upræcist og anbefales ikke.

Kapitel 17

Mindste kvadrater, konditionstal og Householdertriangulering

Vi forsætter med at studere mindste kvadraters problemer og deres løsningsmetoder. Først vil vi have fokus på konditionstal forbundet til problemerne, som giver et indblik i hvor præcist løsninger kan forventes at være. Derefter vil vi se hvordan Householdermatricer kan bruges til at beregne QR-dekomponeringer og løse mindste kvadraters problemer. Vi vil sammenligne de forskellige fremgangsmåder numerisk.

17.1 Konditionstal for mindste kvadraters problemer

Vi ønsker at studere mindste kvadraters løsninger $x \in \mathbb{R}^n$ til

$$Ax = b$$

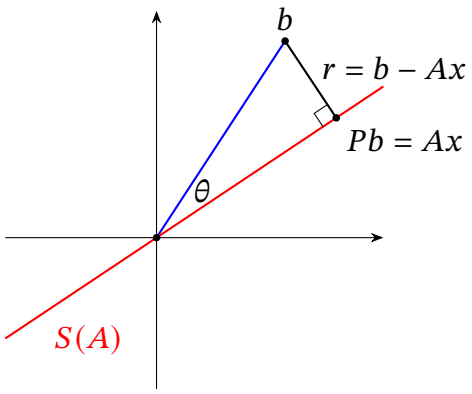
hvor $A \in \mathbb{R}^{m \times n}$, $m > n$, og $b \in \mathbb{R}^m$ er givet. Som vi har set, er dette ækvivalent med at løse

$$Ax = Pb \tag{17.1}$$

hvor P er projektion på søjlerummet $S(A)$.

Her er der flere delproblemer med hvert sit konditionstal. For at beskrive resultaterne er der nogle størrelser der skal indføres.

I figur 17.1 har vi optegnet vektoren b og dens projektion Pb på søjlerummet $S(A)$. Restvektoren $r = b - Ax$ er ortogonal på $S(A)$. Punkterne b, Pb og



Figur 17.1: Vektorer og en vinkel for mindste kvadraters problemstilling.

origo danner en retvinklet trekant. Den spidse vinkel i origo betegnes θ. Vi har

$$\cos\theta = \frac{\|Pb\|_2}{\|b\|_2}.$$

Dette stemmer overens med vores formel i definition 8.5, da $\langle b - Pb, Pb\rangle = 0$, giver

$$\cos\theta = \frac{\langle b, Pb\rangle}{\|b\|_2\|Pb\|_2} = \frac{\langle (b - Pb) + Pb, Pb\rangle}{\|b\|_2\|Pb\|_2} = \frac{\|Pb\|_2^2}{\|b\|_2\|Pb\|_2} = \frac{\|Pb\|_2}{\|b\|_2}.$$

Forbundet til A er dens konditionstal (11.10)

$$\kappa(A) = \frac{\sigma_0}{\sigma_{k-1}}$$

hvor σ_i er singulærværdierne af A, og $k = n$, da $m > n$. For løsningen x til (17.1) er $\|Ax\|_2/\|x\|_2$ højst $\|A\|_2 = \max_{v\neq 0}\|Av\|_2/\|v\|_2 = \sigma_0$. Vi bruger

$$\eta = \|A\|_2\frac{\|x\|_2}{\|Ax\|_2} = \sigma_0\frac{\|x\|_2}{\|Ax\|_2} \geqslant 1,$$

som mål for hvor langt $\|Ax\|_2/\|x\|_2$ er fra at give den maksimale værdi $\|A\|_2$.

Lad os nu giver oplysning om konditionstal for det mindst kvadraters problem. Beviserne gennemgås ikke, men man kan finde nogle detaljer i Trefethen og Bau (1997) og referencerne derinde.

Problemstillingen afhænger af vektoren $b \in \mathbb{R}^m$ og af matricen $A \in \mathbb{R}^{m\times n}$. Hvis vi varierer b mens vi holder A fast, så er

(a) konditionstallet for beregning af Pb fra b

$$\kappa = \frac{1}{\cos\theta},$$

(b) konditionstallet for beregning af løsningen x til $Ax = Pb$, som funktion af b,

$$\kappa = \frac{\kappa(A)}{\eta\cos\theta}. \tag{17.2}$$

Disse er eksakte svar, og er ikke så besværlig at udlede. Derimod når man holder b fast og lad A varierer, er eksakte svar meget vanskelig at få fat i, og vi nøjes med at give øvre grænser:

(a) konditionstallet κ for beregning af Pb, som funktion af A, opfylder

$$\kappa \leqslant \frac{\kappa(A)}{\cos\theta};$$

(b) konditionstallet κ for beregning af løsningen x til $Ax = Pb$, som funktion af A, opfylder

$$\kappa \leqslant \kappa(A) + \frac{\kappa(A)^2|\tan\theta|}{\eta}. \tag{17.3}$$

17.2 Et eksempel

Betragt det følgende eksempel hvor vi vil gerne beregne en tilnærmelse via polynomier til funktionen

$$f(t) = \frac{1}{c}e^{\sin(4t)}, \qquad \text{for } 0 \leqslant t \leqslant 1,$$

hvor c er en given konstant.

Vi arbejder med polynomier af grad 14. Alle funktioner erstattes af deres evaluering i 100 punkter t_i jævnt fordelt over intervallet $[0, 1]$. Vi får så at vi skal løse et mindste kvadraters problemstilling $Ax = b$, hvor A er en Vandermonde matrix med 15 søjler, og b er vektoren af værdierne $f(t_i)$, $i = 0, \ldots, 99$.

Jeg har beregnet løsningen algebraisk, og fundet værdien for konstanten c, der sikre at den korrekte mindste kvadraters løsning $x \in \mathbb{R}^{15}$ har $x_0 = 1{,}0$. Vi har så at $f(t)$ approksimeres af

$$p(t) = t^{14} + x_1 t^{13} + \cdots + x_{13}t + x_{14}.$$

I python kan vi opstille problemet med denne c-værdi på følgende vis

```
import numpy as np

m = 100
cols = 15

t = np.linspace(0, 1, m)

a = np.vander(t, cols)
c = 2006.787453104852
b = np.exp(np.sin(4 * t))[:, np.newaxis] / c
```

Vi vil løse problemet via forskellige metoder, og sammenligne resultaterne for x_0 med den korrekte værdi 1,0.

For at se hvor præcist en løsning vi kan forvente, beregner vi konditionstal; specielt dem givet i (17.2) og (17.3). Først beregner vi `kappa_a`, konditionstallet $\kappa(\mathtt{a})$ for `a`:

```
u, s, vt = np.linalg.svd(a, full_matrices=False)
kappa_a = s[0] / s[-1]
print(f'{kappa_a:e}')
```

```
2.271778e+10
```

Her har vi brugt `print(f'{var:e}')` til at skrive `var` i videnskabelig notation. Udtrykket `f'...'` angiver en streng med formatering. De kaldes »formatted string literals« eller »f-strings« i den officielle dokumentation for python. Elementer i strengen af formen `{var}` erstattes af værdien af `var`. Repræsentationen af værdien kan specificeres ved at skrive `{var:...}` hvor `...` giver formatet. Formen `{var:e}` giver `var` i videnskabelig notation. En lille variation, hvor vi tilføj `' = '` efter variablet, giver følgende

```
print(f'{kappa_a = :e}')
```

```
kappa_a = 2.271778e+10
```

hvor variablets navn vises også.

Vi ser at konditionstallet $\kappa(\mathtt{a})$ er ret stort.

For at beregne $\cos\theta$ og η har vi brug projektionen Pb af b og for en løsning x. Vi satser på at SVD-metoden giver et godt svar.

```
proj_b = u @ (u.T @ b)
cos_theta = np.linalg.norm(proj_b) / np.linalg.norm(b)
print(np.degrees(np.arccos(cos_theta)))
```

```
0.00021462178154016444
```

```
x = vt.T @ (np.diag(1/s) @ (u.T @ b))
eta = s[0] * np.linalg.norm(x) / np.linalg.norm(proj_b)
print(f'{eta = :e}')
```

```
eta = 2.103560e+05
```

Vi kan så samle disse værdier til at få vores oplysning om konditionstal

```
kond_x_b = kappa_a / (eta * cos_theta)
kond_x_a_højst = (kappa_a +
  (kappa_a**2 * np.sqrt(1-cos_theta**2) / (eta * cos_theta)))
print(f'{kond_x_b = :e}')
print(f'{kond_x_a_højst = :e}')
```

```
kond_x_b = 1.079968e+05
kond_x_a_højst = 3.190804e+10
```

Vi ser at den sidste er størrelsesorden 10^{10}. Dette medvirker at vi kan højst forvente vores beregning af x_0 er korrekt inden for $x_0 \times 10^{10} \times \epsilon_{\text{machine}} \approx 10^{-6}$.

Vi har allerede en beregning fra SVD-metoden. Denne har x_0

```
print(x[0,0])
```

```
1.00000020732409
```

som er indenfor 10^{-6} af det korrekte svar 1,0.

```
korrekt = 1.0
print(x[0,0] - korrekt)
```

```
2.0732408989587725e-07
```

Så dette er så godt et svar, som vi kan forvente at beregne.

Lad os nu afprøve beregning via den forbedrede Gram-Schmidt metode fra afsnit 15.3. Vi skal bestemme en QR-dekomponering $A = QR$ via forbedrede Gram-Schmidt, side 199 i afsnit 15.3, og løse derefter ligningen $Rx = Q^T b$ via back substitution, side 213 i afsnit 16.4.

```
q, r = forbedret_gram_schmidt(a)
x_qr_fgs = back_subs(r, q.T @ b)
print(x_qr_fgs[0, 0])
```

```
1.0092114292249557
```

Dette rammer målet mindre godt end SVD-metoden. Vi har en fejl på omtrent 10^{-3}.

```
print(x_qr_fgs[0, 0] - korrekt)
```

```
0.009211429224955658
```

En tredje metode er at bruge normalligninger. Her løser vi ligningen $A^T Ax = A^T b$.

```
print((np.linalg.solve(a.T @ a, a.T @ b))[0,0])
```

```
-0.21909943765371528
```

Dette er en katastrofe! Der er ingen korrekte cifre og selve fortegnet i svaret er forkert.

Bemærk at konditionstallet for $A^T A$ er

```
sn = np.linalg.svd(a.T @ a, compute_uv=False)
kond_at_a = sn[0] / sn[-1]
print(f'{kond_at_a = :e}')
```

```
kond_at_a = 7.879590e+17
```

som er størrelsesorden $1/\epsilon_{\text{machine}}$. Derfor kan man hellere ikke forvente at løsningsmetoden via normalligninger har nogen som helst korrekte cifre.

17.3 Householdertriangulering

Vi vil nu give en alternativ metode for at beregne QR-dekomponeringer. Dette er mere præcist end den forbedrede Gram-Schmidt proces, og er faktisk også et element i computerberegning af SVD-dekomponeringer.

En fuld QR-dekomponeringer $A = QR \in \mathbb{R}^{n\times k}$ med $Q \in \mathbb{R}^{n\times n}$ ortogonal og $R \in \mathbb{R}^{n\times k}$ øvre triangulær er ækvivalent med

$$Q^T A = R = \begin{bmatrix} r_{00} & r_{01} & r_{02} & \dots \\ 0 & r_{11} & r_{12} & \dots \\ 0 & 0 & r_{22} & \dots \\ \vdots & \vdots & & \ddots \\ 0 & 0 & \dots & \dots \end{bmatrix}$$

da $Q^T = Q^{-1}$. Så for at bestemme en QR-dekomponering, er det nok at finde en ortogonal matrix Q^T således at $Q^T A$ er øvre triangulær. Til dette formål kan vi bruge Householdertransformationer.

Husk at en Householdermatrix har formen

$$H = I_n - svv^T$$

hvor $s = 2/\|v\|_2^2$. Proposition 9.17 fortæller os at givet vektoren $a_0 = (a_{00}, a_{10}, \dots, a_{n-1,0})$, findes der en Householdermatrix således at $Ha_0 = (\pm\|a_0\|_2, 0, \dots, 0)$. Beregnes produktet HA, får vi så

$$A = \begin{bmatrix} a_{00} & * & \cdots \\ a_{10} & * & \cdots \\ \vdots & \vdots & \\ a_{n-1,0} & * & \cdots \end{bmatrix} \mapsto HA = \begin{bmatrix} \pm\|a_0\|_2 & * & \cdots \\ 0 & * & \cdots \\ \vdots & \vdots & \\ 0 & * & \cdots \end{bmatrix}.$$

Vi kan så få en øvre triangulær matrix ved at gentage ideen på følgende måde: vi finder Householdermatricer $H_0, H_1, \ldots, H_{k-1}$ således at

$$A \mapsto H_0 A = \begin{bmatrix} r_{00} & r_{01} & \cdots \\ 0 & * & \cdots \\ \vdots & \vdots & \\ 0 & * & \cdots \end{bmatrix} \mapsto H_1 H_0 A = \begin{bmatrix} r_{00} & r_{01} & r_{02} & \cdots \\ 0 & r_{11} & r_{12} & \cdots \\ 0 & 0 & * & \cdots \\ \vdots & \vdots & \vdots & \\ 0 & 0 & * & \cdots \end{bmatrix}$$

$$\mapsto H_2 H_1 H_0 A = \begin{bmatrix} r_{00} & r_{01} & r_{02} & r_{03} & \cdots \\ 0 & r_{11} & r_{12} & r_{13} & \cdots \\ 0 & 0 & r_{22} & r_{23} & \cdots \\ 0 & 0 & 0 & * & \cdots \\ \vdots & \vdots & \vdots & \vdots & \\ 0 & 0 & 0 & * & \cdots \end{bmatrix}$$

$$\cdots \mapsto H_{k-1} \cdots H_1 H_0 A = \begin{bmatrix} r_{00} & r_{01} & r_{02} & r_{03} & \cdots \\ 0 & r_{11} & r_{12} & r_{13} & \cdots \\ 0 & 0 & r_{22} & r_{23} & \cdots \\ 0 & 0 & 0 & r_{33} & \cdots \\ \vdots & \vdots & \vdots & & \ddots \\ 0 & 0 & 0 & \cdots & \cdots \end{bmatrix} = R$$

Vi sætter så

$$Q = (H_{k-1} \ldots H_1 H_0)^T = H_0 H_1 \ldots H_{k-1},$$

hvor den sidste lighed følger af $(AB)^T = B^T A^T$ og at $H_i^T = H_i$ for Householdermatricer H_i.

Kombineret med opskrifterne fra proposition 9.17 kan vi nu bygge algoritmer for at implementere denne metode. Det må bemærkes at det er generelt unødvendigt at danne matricen Q; vi skal blot kende til de enkelte $H_i = I_n - s_i v_i v_i^T$, og disse er bestemt af vektoren v_i samt skalaren s_i.

Lad os først implementere metoden fra proposition 9.17. Vi beregner data for en Householdermatrix, der sender en given vektor x til $\pm\|x\|_2 e_0$. Matematisk er metoden

HOUSE(x)

$u = x/\|x\|_2$

$\varepsilon = \begin{cases} +1 & \text{for } u_0 \geqslant 0 \\ -1 & \text{for } u_0 < 0 \end{cases}$

$s = 1 + |u_0|$

$v = (e_0 + \varepsilon u)/s$

return v og s

Dette kan oversættes til python, som

```
def house(x):
    u = x / np.linalg.norm(x)
    eps = +1 if u[0, 0] >= 0 else -1
    s = 1 + np.abs(u[0, 0])
    v = (eps / s) * u
    v[0, 0] = 1
    return v, s
```

Bemærk at $v = (e_0 + \varepsilon u)/s$ har 0'te indgang 1, så v kan beregnes ved at finde $(\varepsilon/s)u$ og derefter ændre den 0'te indgang til 1. Tallet s kan godt beregnes fra v, men vi behøver ikke at gentage denne udregning senere, så vi vælger at returnere både vektoren `v` og skalaren `s`.

Som nævnt ovenfor er det generelt ikke nødvendigt at beregne Q. I stedet for laver vi en funktion der samler den nødvendige data: matricen R, vektorerne v_i samt skalarene s_i. Matricen R er øvre triangulær, og vektoren v_i har formen

$$v_i = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ (v_i)_1 \\ \vdots \\ (v_i)_{n-1-i} \end{bmatrix},$$

så v_i kan godt gemmes i samme matrix som R, under diagonalen:

$$\begin{bmatrix} r_{00} & r_{01} & r_{02} & \dots \\ (v_0)_1 & r_{11} & r_{12} & \dots \\ (v_0)_2 & (v_1)_1 & r_{22} & \dots \\ \vdots & \vdots & & \ddots \\ (v_0)_{n-1} & (v_1)_{n-2} & (v_2)_{n-3} & \end{bmatrix}. \tag{17.4}$$

HOUSEHOLDER QR DATA(A)

1 Beregn Householder v, s for
2 søjle 0 i A
3 søjle 0 i $(H_0A)_{[1:,1:]}$
4 søjle 0 i $(H_1H_0A)_{[2:,2:]}$
5 ...
6 Gem v data under diagonalen i $R = H_{k-1} \dots H_1H_0A$

Når vi implementerer dette i python må vi gerne huske at beregne $HA = (I_n - svv^T)A$, som $HA = A - (sv)(v^TA)$.

Funktionen `householder_qr_data` gemmer dens output i en matrix vi kalder `data` $\in \mathbb{R}^{n\times k}$ af formen (17.4), og i en vektor vi kalder `s` $\in \mathbb{R}^k$. Matricen `data` overskrives med mellemregninger når vi bevæger os igennem algoritmen. Ved den `j`'te trin, er de første `j-1` rækker af R på plads, og de første `j-1` vektorer v_i og skalarer s_i er beregnede og gemt i deres pladser. Resten af matricen indeholder $(H_{j-1} \dots H_1H_0A)_{[j:,j:]}$. F.eks. for $j = 2$, har vi

$$\texttt{data} = \begin{bmatrix} r_{00} & r_{01} & r_{02} & \dots & r_{0,k-1} \\ (v_0)_1 & r_{11} & r_{12} & \dots & r_{1,k-1} \\ (v_0)_2 & (v_1)_1 & (H_1H_0A)_{22} & \dots & (H_1H_0A)_{2,k-1} \\ \vdots & \vdots & & \ddots & \vdots \\ (v_0)_{n-1} & (v_1)_{n-2} & (H_1H_0A)_{n-1,2} & \dots & (H_1H_0A)_{n-1,k-1} \end{bmatrix}.$$

I begyndelsen af algoritmen er `data` lige med `a` $= A$; vi tager en kopi af `a`, så den oprindelige matrix `a` bliver ikke ændret af funktionen.

```
def householder_qr_data(a):
    data = np.copy(a)
    _, k = a.shape
    s = np.empty(k)
```

```
    for j in range(k):
        v, s[j] = house(data[j:, [j]])
        data[j:, j:] -= (s[j] * v) @ (v.T @ data[j:, j:])
        data[j+1:, [j]] = v[1:]
    return data, s
```

Lad os se på hvor mange flops der bruges. Den største del af arbejdet i denne funktion foregår i trinnet

```
        data[j:, j:] -= (s[j] * v) @ (v.T @ data[j:, j:])
```

Vi har `data[j:, j:]` $\in \mathbb{R}^{n-j,k-j}$, og `v` $\in \mathbb{R}^{n-j}$, så dette trin koster

$$\begin{aligned}
&\text{matrixdifference} + \text{skalar-vektorprodukt} \\
&\quad + \text{ydre produkt} + \text{vektor-matrixprodukt} \\
&\quad = (n-j)(k-j) + (n-j) \\
&\qquad + (n-j)(k-j) + 2(n-j)(k-j) \\
&\quad = 4(n-j)(k-j) + (n-j) \text{ flops.}
\end{aligned}$$

Trinnet udføres for $j = 0, 1, \ldots, k-1$, så i alt har vi

$$\begin{aligned}
&\sum_{j=0}^{k-1} (4(n-j)(k-j) + n - j) \\
&\quad = \sum_{j=0}^{k-1} \left(4j^2 - (4n+4k+1)j + n(4k+1)\right) \\
&\quad = 4\frac{1}{6}(k-1)k(2k-1) - (4n+4k+1)\frac{1}{2}(k-1)k + n(4k+1)k \\
&\quad = 2nk^2 - \frac{2}{3}k^3 + 3nk - \frac{1}{2}k^2 + \frac{7}{6}k \\
&\quad \sim 2nk^2 - \frac{2}{3}k^3 \text{ flops}
\end{aligned}$$

Her havde vi brug for formlen

Lemma 17.1.

$$\sum_{j=0}^{k-1} j^2 = \frac{1}{6}(k-1)k(2k-1).$$

Bevis. Sæt $T_r(k) = \sum_{j=0}^{k-1} j^r$, hvor vi sætter $0^0 = 1$. Vi har $T_0(k) = k$, da det er kun en sum af 1 tal, og ved at $T_1(k) = \frac{1}{2}(k-1)k$, se (15.1).

Bemærk først at $T_r(k+1) = T_r(k) + k^r$, da der er kun en ekstra led. Nu har vi

$$\begin{aligned} T_3(k) + k^3 &= T_3(k+1) \\ &= \sum_{j=0}^{k-1}(j+1)^3 = \sum_{j=0}^{k-1} j^3 + 3j^2 + 3j + 1 \\ &= T_3(k) + 3T_2(k) + 3T_1(k) + T_0(k). \end{aligned}$$

Dette kan løses for $T_2(k)$, den ønskede sum, som

$$\begin{aligned} T_2(k) &= \frac{1}{3}(k^3 - 3T_1(k) - T_0(k)) \\ &= \frac{1}{3}\left(k^3 - \frac{3}{2}(k-1)k - k\right) = \frac{1}{6}k(2k^2 - 3(k-1) - 2) \\ &= \frac{1}{6}k(2(k-1)(k+1) - 3(k-1)) = \frac{1}{6}(k-1)k(2k-1), \end{aligned}$$

som påstået. □

Hvis man vil konstruere Q, er det mest effektiv at beregne det som

$$Q = H_0(H_1(\ldots(H_{k-1}((I_n)_{[:,:k]}))))$$

Grunden til dette ligger i at H_iB ændrer kun delmatricen $B_{[i:,i:]}$, så denne rækkefølge bruger ikke unødvendige flops. Den følgende funktion konstruerer Q og R fra output af `householder_qr_data`.

```
def householder_qr(a):
    data, s = householder_qr_data(a)
    n, k = a.shape
    r = np.triu(data[:k, :k])
    q = np.eye(n, k)
    for j in reversed(range(k)):
        x = data[j+1:, [j]]
        v = np.vstack([[1], x])
        q[j:, j:] -= (s[j] * v) @ (v.T @ q[j:, j:])
    return q, r
```

Lad os teste disse funktioner i et par eksempler. Først for en simpel matrix.

```
a_simpel = np.array([[ 1.0, 2.0],
                     [-1.0, 2.0],
                     [ 0.0, 1.0]])

q, r = householder_qr(a_simpel)
print(f'{q = }')
print()
print(f'{r = }')
```

```
q = array([[-0.70710678, -0.66666667],
       [ 0.70710678, -0.66666667],
       [ 0.        , -0.33333333]])

r = array([[-1.41421356,  0.        ],
       [ 0.        , -3.        ]])
```

Vi tjekker at `q` har ortonormale søjler og at `q @ r` er tæt på `a_simpel`:

```
print(q.T @ q)
print()
print(a_simpel - q @ r)
```

```
[[1.00000000e+00 1.71886984e-17]
 [1.71886984e-17 1.00000000e+00]]

[[ 2.22044605e-16 -4.44089210e-16]
 [-2.22044605e-16 -4.44089210e-16]
 [ 0.00000000e+00  0.00000000e+00]]
```

Nu lad os kikke på vores eksempel fra afsnit 15.1.

```
s = 1e-8
a_s = np.array([[1.0, 1.0, 1.0],
                [  s, 0.0, 0.0],
                [0.0,   s, 0.0],
```

```
                [0.0, 0.0,   s]])

q, r = householder_qr(a_s)
print(f'{q = }')
print()
print(f'{r = }')
```

```
q = array([[-1.00000000e+00,  7.07106781e-09,  4.08248290e-09],
       [-1.00000000e-08, -7.07106781e-01, -4.08248290e-01],
       [ 0.00000000e+00,  7.07106781e-01, -4.08248290e-01],
       [ 0.00000000e+00,  0.00000000e+00,  8.16496581e-01]])

r = array([[-1.00000000e+00, -1.00000000e+00, -1.00000000e+00],
       [ 0.00000000e+00,  1.41421356e-08,  7.07106781e-09],
       [ 0.00000000e+00,  0.00000000e+00,  1.22474487e-08]])
```

Grammatricen er nu

```
print(q.T @ q)
```

```
[[ 1.00000000e+00 -3.54269458e-25  3.39714346e-25]
 [-3.54269458e-25  1.00000000e+00  8.25542809e-18]
 [ 3.39714346e-25  8.25542809e-18  1.00000000e+00]]
```

som er en stor forbedring over beregningerne med forbedret Gram-Schmidt, hvor indre produkterne $\langle \mathtt{v1}, \mathtt{v2} \rangle$ havde størrelsesorden 10^{-16}, og $\langle \mathtt{v0}, \mathtt{v1} \rangle$ og $\langle \mathtt{v0}, \mathtt{v2} \rangle$ var størrelsesorden 10^{-8}. Vi har også at `q @ r` er pænt tæt på `a_s`:

```
print(a_s - q @ r)
```

```
[[ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00 -1.27201051e-24 -1.56873944e-24]
 [ 0.00000000e+00  1.65436123e-24  7.65491762e-25]
 [ 0.00000000e+00  0.00000000e+00 -1.65436123e-24]]
```

Til sammenligning kan vi bruge den indbyggede QR-dekomponering i NumPy fra funktionen `np.linalg.qr`

```
q, r = np.linalg.qr(a_s)
print(q.T @ q)
print()
print(a_s - q @ r)
```

```
[[1.00000000e+00 2.11815366e-25 6.76452457e-26]
 [2.11815366e-25 1.00000000e+00 7.39530654e-17]
 [6.76452457e-26 7.39530654e-17 1.00000000e+00]]

[[ 0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00  3.03798445e-24  3.92242374e-24]
 [ 0.00000000e+00 -1.65436123e-24 -1.86784055e-24]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00]]
```

Vi ser at vores Householder implementering er lige så god, som den indbyggede.

17.4 Householdermetoden for mindste kvadraters problemer

Det overstående kan godt bruges til at løse mindste kvadraters problemer, men det er bedst at bruge `householder_qr_data` i stedet for at beregne Q. Vi skal løse $Rx = Q^T b$, men

$$Q^T b = H_{k-1}(\dots(H_1(H_0 b)))_{[:k]}.$$

Dette fører til følgende python kode

```
def householder_lsq(a, b):
    data, s = householder_qr_data(a)
    _, k = a.shape
    r = np.triu(data[:k, :k])
    c = np.copy(b)
    for j in range(k):
        x = data[j+1:, [j]]
```

metode	opstilling	løsning x fra	~ flops
QR via Gram-Schmidt	$A = QR$	$Rx = Q^T b$	$2mn^2$
QR via Householder	$A = QR$	$Rx = Q^T b$	$2mn^2 - \frac{2}{3}n^3$
reduceret SVD	$A = U\Sigma V^T$	$x = V(\Sigma^{-1}(U^T b))$	$2mn^2 + 11n^3$
normalligning	$A^T Ax = A^T b$	$(A^T A)x = (A^T b)$	$mn^2 + \frac{1}{3}n^3$

Tabel 17.1: Forskellige løsningsmetoder for mindste kvadraters problemer $Ax = Pb$, $A \in \mathbb{R}^{m\times n}$, $m > n$.

```
        v = np.vstack([[1], x])
        c[j:] -= (s[j] * np.vdot(v, c[j:])) * v
    return back_subs(r, c[:k])
```

Lad os afprøve denne metode på vores eksempel fra afsnit 17.2.

```
print(householder_lsq(a, b)[0,0])
```

```
1.000000237959848
```

Dette ligger inden for 10^{-6} af det korrekte svar 1,0

```
print(householder_lsq(a, b)[0,0] - korrekt)
```

```
2.3795984804841908e-07
```

så er lige så godt, som SVD-resultatet. Vi ser at det er ikke selve QR-metoden der er problematisk, men derimod hvordan man udregner data for denne dekomponering. Householdermetoden er meget præcis og bruges som en del af standardmetoder for at udregne SVD-dekomponeringer. Derfor er det til at foretrække at løsning af mindste kvadraters problemer foregår via Householdermetoden, som kræver færre flops.

Tabel 17.1 giver en opsummering af de forskellige metoder vi har. Bortset fra SVD-metoden, forudsætter de andre metoder i tabel 17.1 at søjlerne af A er lineært uafhængig.

Bemærk at det er ofte bedst at løse et reduceret lineært ligningssystem, end at beregne en invers matrix. F.eks. for QR-metoderne, er det bedre at bruge back substitution til at løse $Rx = Q^T b$ end at finde x via $x = R^{-1}(Q^T b)$. Det har vi gjort ovenfor ved at bruge `back_subs` i stedet for `np.linalg.inv`.

Kapitel 18

Lineære afbildninger og matricer

18.1 Lineære afbildninger

Et vektorrum V har to grundlæggende operationer: sum $u + v$ og skalar multiplikation sv. En lineær transformation, eller lineær afbildning, er en funktion, der respekterer disse operationer.

Definition 18.1. En *lineær transformation* er en afbildning $L\colon V \to W$ fra et vektorrum V til et vektorrum W således at
(a) $L(u + v) = L(u) + L(v)$ og
(b) $L(sv) = s\,L(v)$
for alle $u, v \in V$ og alle skalarer s.

Eksempel 18.2. Afbildningen $L\colon \mathbb{R} \to \mathbb{R}$, $L(x) = 5x$ er lineær, da

$$L(x + y) = 5(x + y) = 5x + 5y = L(x) + L(y),$$

og

$$L(sx) = 5(sx) = s\,5x = s\,L(x),$$

for alle $x, y, z \in \mathbb{R}$. △

Eksempel 18.3. Afbildningen $F\colon \mathbb{R} \to \mathbb{R}$, $F(x) = x^2$ er *ikke* lineær. Dette vises bedst ved at finde konkrete elementer hvor den ene af del (a) eller del (b) i definition 18.1 holder ikke. For denne afbildning F kan vi kikke på del (a) ved $u = 1 = v$. Vi har

$$F(1 + 1) = F(2) = 4,$$
$$F(1) + F(1) = 1 + 1 = 2,$$

så

$$F(1+1) = 4 \neq 2 = F(1) + F(1)$$

og del (a) er ikke opfyldt. △

Eksempel 18.4. Lad V være rummet af funktioner $[-1, 2] \to \mathbb{R}$, som er to gange differentiable, og lad W være rummet af en gang differentiable funktioner på det samme interval. Så er afbildningen

$$L\colon V \to W, \quad L(f) = f'$$

lineær. Dette er konsekvens af de almindelige regneregler for den afledte: $(f+g)' = f' + g'$ og $(cf)' = cf'$, for c konstant. △

Eksempel 18.5. Lad V være rummet af kontinuerte funktioner $[0, 3] \to \mathbb{R}$. Så er afbildningerne

$$\begin{aligned} L\colon V \to V, &\qquad L(f) = \int_0^x f(t)\,dt, \\ M\colon V \to \mathbb{R}, &\qquad M(f) = \int_0^3 f(t)\,dt \end{aligned}$$

begge lineære. Igen er dette konsekvens af standard regneregler for integraler:

$$\int_0^x f(t) + g(t)\,dt = \int_0^x f(t)\,dt + \int_0^x g(t)\,dt$$

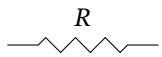
og

$$\int_0^x cf(t)\,dt = c\int_0^x f(t)\,dt$$

for c konstant. △

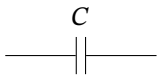
Eksempel 18.6. Forholdet mellem elektrisk spænding V over og elektrisk strøm I igennem flere komponenter er lineær:

(a) Elektrisk modstand R:



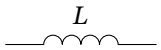
$$\begin{aligned} I &\mapsto V = RI, \\ V &\mapsto I = \frac{1}{R}V; \end{aligned}$$

(b) Elektrisk kondensator, kapacitans C:



$$I(t) \mapsto V(t) - V(t_0) = \frac{1}{C}\int_{t_0}^{t} I(s)\,ds,$$
$$V(t) \mapsto I(t) = C\frac{dV}{dt};$$

(c) Induktionsspole, induktans L:



$$I(t) \mapsto V(t) = L\frac{dI}{dt},$$
$$V(t) \mapsto I(t) - I(t_0) = \frac{1}{L}\int_{t_0}^{t} V(s)\,ds.$$

Dette gør lineær algebra særligt relevant for analyse af elektriske kredsløb. △

Eksempel 18.7. $L\colon \mathbb{R}^n \to \mathbb{R}$

$$L\left(\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{bmatrix}\right) = x_0 + 2x_1 + 3x_2 + \cdots + nx_{n-1}$$

er lineær. △

Eksempel 18.8. Givet en matrix $A \in \mathbb{R}^{m\times n}$, er matrix-vektormultiplikation $L\colon \mathbb{R}^n \to \mathbb{R}^m$

$$L(x) = Ax$$

lineær, da

$$A(x+y) = Ax + Ay \quad \text{og} \quad A(sx) = sAx,$$

for alle $x, y \in \mathbb{R}^n$, $s \in \mathbb{R}$. △

Eksempel 18.9. Lad $\langle\,\cdot\,,\,\cdot\,\rangle$ være et indre produkt på V med reelle skalar. Givet en fast $v \in V$, er $L\colon V \to \mathbb{R}$, $L(u) = \langle u, v\rangle$ lineær, pga. definition 13.1 del (I2). △

Bemærkning 18.10. Der skal noteres at notationen i definition 18.1 gemmer et par væsentlige detaljer. I del (a) er de to plustegn »+« fra forskellige vektorrum: den på den venstre side er sumoperationen i V; den på den højre side af ligningen er sumoperationen i W. Ligeledes er de to skalarmultiplikationer i del (b) fra V og henholdsvis W. ◇

Eksempel 18.11. For V rummet af funktioner $[0,1] \to \mathbb{R}$, er afbildningen $L\colon V \to \mathbb{R}$ givet ved

$$L(f) = f(0)$$

lineær, da

$$L(f+g) = (f+g)(0) = f(0) + g(0)$$

og

$$L(sf) = (sf)(0) = s\,f(0)$$

fra vores definition, eksempel 12.7, af operationerne i V. △

Lad os fremhæve to væsentlige egenskaber af lineære afbildninger.

Proposition 18.12. *For $L\colon V \to W$ lineær gælder*
(a) $L(0) = 0$, *og*
(b) $L(su + tv) = sL(u) + tL(v)$.
Omvendt, hvis $L\colon V \to W$ er en afbildning, som opfylder del (b) for alle $u, v \in V$ og alle s, t skalarer, så er L lineær.

Bevis. For (a) har vi $L(0) = L(0v) = 0\,L(v) = 0$, hvor vi har brugt definition 18.1(b).

Del (b) følger fra brug af begge egenskaber af L. Vi har $L(su + tv) = L(su) + L(tv) = s\,L(u) + t\,L(v)$.

For det omvendte, får vi definition 18.1(a) ved at tage $s = 1 = t$. Desuden gælder definition 18.1(b), da vi kan tage proposition 18.12(b) med $t = 0$ og u vilkårlig. □

Eksempel 18.13. Det følger at $F(x) = x + 1$ er ikke lineær, da $F(0) = 1 \neq 0$. △

Del (b) af proposition 18.12 giver mere generelt

$$\begin{aligned} &L(s_0v_0 + s_1v_1 + \cdots + s_{k-1}v_{k-1}) \\ &\qquad = s_0L(v_0) + s_1L(v_1) + \cdots + s_{k-1}L(v_{k-1}). \end{aligned} \tag{18.1}$$

18.2 Standardmatricen for en lineær transformation

Eksempel 18.8 er faktisk typisk for lineære afbildninger $\mathbb{R}^n \to \mathbb{R}^m$.

Proposition 18.14. *For $L\colon \mathbb{R}^n \to \mathbb{R}^m$ lineær, definér $A = \left[u_0 \mid u_1 \mid \dots \mid u_{n-1}\right] \in \mathbb{R}^{m\times n}$ ved*

$$u_j = L(e_j)$$

hvor $e_j = (0, \dots, 0, 1, 0, \dots, 0)$ er den j'te standard basisvektor, med j'te indgang 1 og alle andre indgange 0.

Så gælder

$$L(x) = Ax \qquad \text{for alle } x \in \mathbb{R}^n.$$

Definition 18.15. Matricen A i proposition 18.14 kaldes *den standard matrixrepræsentation* (SMR) af L.

Bevis for proposition 18.14. For $x \in \mathbb{R}^n$, har vi

$$x = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = x_0 e_0 + x_1 e_1 + \cdots + x_{n-1} e_{n-1}$$

Ved brug af (18.1), får vi

$$\begin{aligned} L(x) &= L(x_0 e_0 + x_1 e_1 + \cdots + x_{n-1} e_{n-1}) \\ &= x_0 L(e_0) + x_1 L(e_1) + \cdots + x_{n-1} L(e_{n-1}) \\ &= x_0 u_0 + x_1 u_1 + \cdots + x_{n-1} u_{n-1} \\ &= \left[u_0 \mid u_1 \mid \dots \mid u_{n-1}\right] x = Ax. \end{aligned}$$

Dvs. $L(x) = Ax$ for alle $x \in \mathbb{R}^n$, som ønsket. □

Eksempel 18.16. $L((x, y)) = (2x - y, y + x, y - 2x)$, $L\colon \mathbb{R}^2 \to \mathbb{R}^3$, har

$$u_0 = L(e_0) = L\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 1 \\ -2 \end{bmatrix}$$

og

$$u_1 = L(e_1) = L\left(\begin{bmatrix}0\\1\end{bmatrix}\right) = \begin{bmatrix}-1\\1\\1\end{bmatrix}$$

Så L har SMR

$$A = \left[u_0 \mid u_1\right] = \begin{bmatrix}2 & -1\\1 & 1\\-2 & 1\end{bmatrix}.$$

△

18.3 Kombinationer af lineære afbildninger

Lineære afbildninger kan kombineres på forskellige vis til at danne nye lineære afbildninger. Dette betyder man kan bygge komplicerede lineære afbildninger op fra simple eksempler.

Proposition 18.17. *Hvis $L_0, L_1 : V \to W$ er lineære og s er en skalar, så er*
(a) $L_0 + L_1$ *og*
(b) sL_0
lineære transformationer.

Bevis. Husk at $(L_0 + L_1)(v)$ defineres til at være $L_0(v) + L_1(v)$. For del (a), har vi

$$\begin{aligned}(L_0 + L_1)(u + v) &= L_0(u + v) + L_1(u + v) = L_0(u) + L_0(v) + L_1(u) + L_1(v)\\ &= (L_0 + L_1)(u) + (L_0 + L_1)(v)\end{aligned}$$

og

$$\begin{aligned}(L_0 + L_1)(sv) &= L_0(sv) + L_1(sv) = sL_0(v) + sL_1(v)\\ &= s(L_0(v) + L_1(v)) = s(L_0 + L_1)(v).\end{aligned}$$

Redegørelsen for del (b) er tilsvarende, med udgangspunkt i $(sL)(v) = s(L(v))$. □

Eksempel 18.18. Afbildningerne $f \mapsto f'$ og $f \mapsto f(0)$ er lineære, så det følger at $f \mapsto 3f' - 2f(0)$ er også lineær. △

Proposition 18.19. *Hvis $L\colon V \to W$ og $M\colon U \to V$ er lineære afbildninger så er*

$$L \circ M\colon U \to W$$

lineær.

Bevis. Her er $(L \circ M)(u) = L(M(u))$. Vi har

$$\begin{aligned}(L \circ M)(u_0 + u_1) &= L(M(u_0 + u_1)) = L(M(u_0) + M(u_1)) \\ &= L(M(u_0)) + L(M(u_1)) = (L \circ M)(u_0) + (L \circ M)(u_1),\end{aligned}$$

og en tilsvarende begrundelse gælder for $(L \circ M)(su) = s(L \circ M)(u)$. □

Eksempel 18.20. Da $f \mapsto f'$ er lineære er $f \mapsto f'' = (f')'$ også lineær. △

18.4 Kerne og billedmængde

Lineære afbildninger bevarer de to fundamental operationer i vektorrum. Ligeledes er underrum også defineret ud fra disse to operationer. Dette fører til at lineære afbildninger sender underrum til underrum og at der er to særlige underrum, som en lineær afbildning giver anledning til.

Definition 18.21. Lad $L\colon V \to W$ være lineær. Så er *kernen* af L

$$\ker L = \{v \in V \mid L(v) = 0\},$$

mængden af alle vektorer i V, som L afbildninger i $0 \in W$.

Billedmængden af L er

$$\operatorname{im} L = L(V) = \{L(v) \mid v \in V\} \subset W$$

mængden af alle værdier af L.

Eksempel 18.22. For $L\colon \mathbb{R}^n \to \mathbb{R}^m$ med SMR A kender vi disse mængder i forvejen, blot med andre navne. Billedmængden er

$$\operatorname{im} L = S(A)$$

søjlerummet af A. Kernen

$$\ker L = N(A) = \{x \in \mathbb{R}^n \mid Ax = 0\}$$

er rummet af løsninger til $Ax = 0$. Dette kaldes *nulrummet* af A. △

Proposition 18.23. *Lad $L\colon V \to W$ være lineær. Så er* $\ker L \subset V$ *et underrum af V og* $\operatorname{im} L \subset W$ *er et underrum af W.*

Bevis. Vi begynder med $\ker L$. Vi skal tjekke at de tre betingelser i definition 12.12 er opfyldt.

Først ser vi at $\ker L$ er ikke tom, da $L(0) = 0$ giver $0 \in \ker L$. Bemærk nu at $u, v \in \ker L$ betyder $L(u) = 0 = L(v)$. Da gælder

$$L(u + v) = L(u) + L(v) = 0 + 0 = 0,$$

som siger at $u + v \in \ker L$. Desuden har vi

$$L(sv) = sL(v) = s0 = 0$$

så $sv \in \ker L$. Dermed er $\ker L$ et underrum af V.

Diskussionen for billedrummet er tilsvarende, men kræver lidt andre overvejelser. Først $\operatorname{im} L$ er ikke tom da $L(0) = 0$ giver $0 \in \operatorname{im} L$. Vi har at $w, y \in \operatorname{im} L$ betyder at der findes $u, v \in V$ med $L(u) = w$ og $L(v) = y$. Da gælder

$$w + y = L(u) + L(v) = L(u + v),$$

som siger $w + y \in \operatorname{im} L$. Desuden har vi

$$sy = sL(v) = L(sv)$$

så $sy \in \operatorname{im} L$, og $\operatorname{im} L$ er et underrum af W. □

Eksempel 18.24. Lad P være projektion på $\operatorname{Span}\{v_0, v_1, \ldots, v_{k-1}\}$ for $v_0, \ldots, v_{k-1}$ ortonormal:

$$P(u) = \langle v_0, u\rangle v_0 + \langle v_1, u\rangle v_1 + \cdots + \langle v_{k-1}, u\rangle v_{k-1}$$

Vi har $\operatorname{im} P = \operatorname{Span}\{v_0, v_1, \ldots, v_{k-1}\}$, da $P(v_i) = v_i$ medfører at $v_i \in \operatorname{im} P$, og da hvert element af $\operatorname{im} P$ er en lineær kombination af $v_0, v_1, \ldots, v_{k-1}$.

Kernen $\ker P$ består af de $v \in V$ med $\langle v_i, v\rangle = 0$, for alle $i = 0, 1, \ldots, k-1$. △

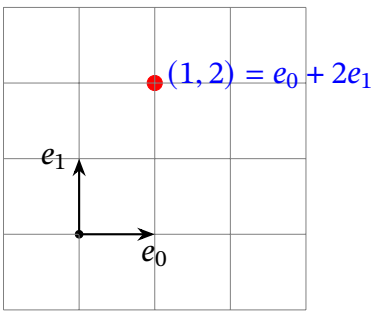
Kapitel 19

Koordinater

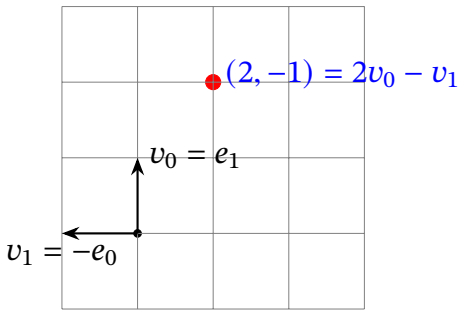
Lad os betragter et punkt i planen, som i figur 19.1. I forhold til det sædvanlige koordinatsystem, har punktet koordinater $(1,2) = e_0 + 2e_1$. Dette siger, at vi kan bevæge os fra origo til punktet ved at gå langs en kopi af standardvektoren e_0 og derefter langs to kopier af e_1. På denne måde er det parret e_0, e_1 af standardvektorer, der bestemmer koordinaterne af vores punkt.

I figur 19.2 kikker vi på det samme punkt, men erstatter parret e_0, e_1 med parret $v_0 = e_1, v_1 = -e_0$. Da $e_0 + 2e_1 = -v_1 + 2v_0 = 2v_0 + (-1)v_1$, kommer vi fra origo til punktet ved at bevæge os langs to kopier af v_0 og derefter langs en kopi af $-v_1$. I forhold til v_0, v_1 har vores punkt koordinater $(2,-1)$.

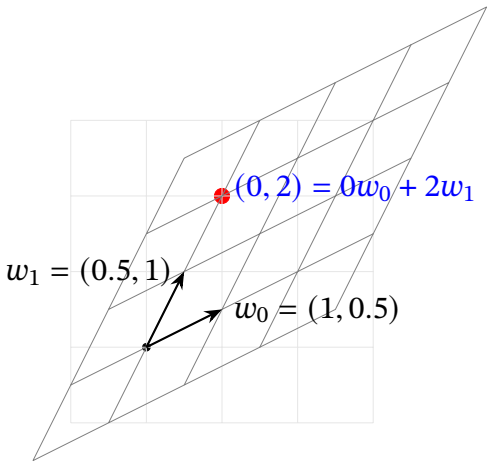
Figur 19.3 viser det samme punktet, men nu arbejder vi ud fra vektorparret w_0, w_1, hvor $w_0 = (1{,}0, 0{,}5)$ og $w_1 = (0{,}5, 1{,}0)$. Da $e_0 + 2e_1 = 2w_1 = 0w_0 + 2w_1$,



Figur 19.1: Standardkoordinater.



Figur 19.2: Drejet ortogonal koordinatsystem.



Figur 19.3: Generel lineært koordinatsystem.

har vores punkt koordinater $(0, 2)$ i koordinatsystemet bestemt af w_0, w_1.

Vi ser at det samme punkt har forskellige beskrivelser afhængig af hvilke vektorpar vi bruger til at danne koordinater. Hvilke par vektorer må vi bruge? Her i planen er det rimelig klart at begge vektor skal være forskellig fra 0 og at de må ikke være parallelle. Dette er faktisk også tilstrækkeligt. I højere dimensioner er der lidt flere krav der skal stilles: disse dækkes af begrebet »basis«, som vi vil nu studere.

19.1 Basis

Lad os lige genkalde et par definitioner vi har mødt før. Først er en samling vektorer $v_0, \dots, v_{n-1}$ er lineært uafhængig (definition 14.2) hvis

$$s_0 v_0 + s_1 v_1 + \cdots + s_{k-1} v_{k-1} = 0 \quad \text{medfører} \quad s_0 = 0 = s_1 = \cdots = s_{k-1}.$$

Rummet udspændt af disse vektorer (definition 12.14) er $\operatorname{Span}\{v_0, v_1, \dots, v_{n-1}\}$, som består af alle lineære kombinationer

$$s_0 v_0 + s_1 v_1 + \cdots + s_{k-1} v_{k-1}$$

for alle skalarer s_i.

Definition 19.1. En samling vektorer $E\colon v_0, v_1, \dots, v_{n-1}$ er en *basis for V* hvis
(a) $v_0, v_1, \dots, v_{n-1}$ er lineært uafhængig, og
(b) $\operatorname{Span}\{v_0, v_1, \dots, v_{n-1}\} = V$.

I $\mathbb{R}^n$ har vi nogle gode værktøj for at tjekke om en samling vektorer er faktisk en basis.

Proposition 19.2. *Lad $v_0, v_1, \dots, v_{k-1}$ være en samling vektorer i $\mathbb{R}^n$, og sæt $A = \left[v_0 \mid v_1 \mid \dots \mid v_{k-1}\right]$. Så er $v_0, v_1, \dots, v_{k-1}$ en basis for $\mathbb{R}^n$ hvis og kun hvis ligningssystemet*

$$Ax = b$$

har en entydig løsning for hvert $b \in \mathbb{R}^n$.

Dette tvinger $k = n$ for en basis af $\mathbb{R}^n$.

Bevis. Husk at $Ax = x_0 v_0 + x_1 v_1 + \cdots + x_{k-1} v_{k-1}$. Fra dette kan vi bemærke to ting:
(i) At betingelsen (a), at samlingen er lineært uafhængig, er ens betydende med at $Ay = 0$ har $y = 0$, som entydig løsning.
(ii) At betingelsen (b), at samlingen udspænder $V = \mathbb{R}^n$, er ens betydende med at for hvert b har $Ax = b$ mindst en løsning.

Antag, som i propositionen, at $Ax = b$ har en entydig løsning for hvert b. At vi har en løsning for hvert b, giver (ii) at samlingen udspænder $\mathbb{R}^n$. Tages $b = 0$, har vi en entydig løsning $x = 0$. Så fra (i) har vi at samlingen er også lineært uafhængig. Dermed er samlingen en basis for $\mathbb{R}^n$.

Omvendt hvis samlingen er en basis, har vi fra (ii) eksistens af løsninger til $Ax = b$ for hvert b, og ved fra (i) at løsningen er entydigt i tilfældet $b = 0$.

Vi mangler så at vise, at for vilkårlig b er løsningen til $Ax = b$ entydig. Antag at vi har to løsninger x og z for samme b: dvs. $Ax = b$ og $Az = b$. Så har vi $A(x - z) = Ax - Az = b - b = 0$. Da $Ay = 0$ har kun løsningen $y = 0$, konkluderer vi at $x - z = 0$. Dette siger at $x = z$, så løsningen på $Ax = b$ er entydig.

For at vise at $k = n$, udelukker vi de andre muligheder.

Hvis $k > n$, så har A flere søjler end pivotelementer. Det betyder at systemet $Ax = b$ med $b = 0$ har frie variabler, og dermed er løsninger ikke entydig. Så samlingen er ikke lineært uafhængig, og dermed ikke en basis.

Hvis $k < n$, er der flere rækker i A end pivotelementer. Vi kan derfor finde b således at systemet $Ax = b$ er ikke konsistent. Så har systemet ingen løsning i dette tilfælde, og dermed at ligger b ikke i $\operatorname{Span}\{v_0, v_1, \ldots, v_{n-1}\}$. Dvs. $\operatorname{Span}\{v_0, v_1, \ldots, v_{n-1}\} \neq V$, så samlingen er ikke en basis. □

Eksempel 19.3. Standard basis for $\mathbb{R}^n$ er $e_0, e_1, \ldots, e_{n-1}$. For at se at dette er en basis, bemærk at den tilsvarende matrix A er I_n. Men systemerne $I_n x = b$, $b \in \mathbb{R}^n$, har altid den entydige løsning $x = b$, så søjlerne af I_n udgør en basis for $\mathbb{R}^n$. △

I tilfældet hvor vi har netop n vektorer i $\mathbb{R}^n$ er der nu mange forskellige måder at afgøre om de udgør en basis.

Proposition 19.4. *Givet $v_0, v_1, \ldots, v_{n-1}$ i $\mathbb{R}^n$ sæt $A = \left[v_0 \mid v_1 \mid \ldots \mid v_{n-1}\right] \in \mathbb{R}^{n\times n}$. Så er udsagnet*

(a) *$v_0, v_1, \ldots, v_{n-1}$ er en basis for $\mathbb{R}^n$*

ækvivalent med hver af de følgende

(b) *hver søjle af A er en pivotsøjle,*

(c) *A er invertibel,*

(d) *Gram-Schmidt processen på $v_0, v_1, \ldots, v_{n-1}$ kan gennemføres til at give en ortogonal samling bestående af netop n vektorer.*

Bevis. Når vi har n vektorer i $\mathbb{R}^n$ er matricen A kvadratisk. Derfor er hvert søjle en pivotsøjle hvis og kun hvis der er ingen frie variabler. Dette er ens betydende med at $Ax = b$ har altid en entydig løsning, så proposition 19.4 giver at samlingen er en basis. Dermed er del (a) ækvivalent med del (b).

Hvis A er invertibel har $Ax = b$ den entydige løsning $x = A^{-1}b$, så $v_0, v_1, \ldots, v_{n-1}$ er en basis. Omvendt, hvis vi har del (a), så er der entydige løsninger $w_0, w_1, \ldots, w_{n-1}$ til ligningerne $Aw_i = e_i$, $i = 0, 1, \ldots, n-1$. Vi kan sætte $B =$

$\left[w_0 \mid w_1 \mid \dots \mid w_{n-1}\right]$ og ser at

$$AB = \left[Aw_0 \mid Aw_1 \mid \dots \mid Aw_{n-1}\right] = \left[e_0 \mid e_1 \mid \dots \mid e_{n-1}\right] = I_n.$$

Det følger fra sætning 7.2 at A er invertibel, med invers $A^{-1} = B$. Vi har så del (c).

Hvis Gram-Schmidt processen kan ikke gennemføres, er det fordi en vektor er en lineær kombination af de forrige. Vektorerne er dermed ikke lineært uafhængig og den første betingelse for basis er ikke opfyldt. Omvendt hvis Gram-Schmidt processen kan gennemføres, er enhver vektor i $\mathbb{R}^n$ en lineær kombination af de ortogonale vektorer konstrueret, og dermed en lineær kombination af de oprindelige vektorer. Det følger at $Ax = b$ har altid en løsning, så hver række indeholder et pivotelement. Men A er kvadratisk, så hvert søjler også indeholder et pivotelement. Dette giver ækvivalens af del (d) med de andre udsagn. □

Eksempel 19.5. For (2×2)-matricer, kan vi tjekke invertibilitet via determinanten som i afsnit 7.1. Så f.eks.

$$\begin{bmatrix} 1 & 0{,}5 \\ 0{,}5 & 1 \end{bmatrix}$$

har determinant $1 \times 1 - 0{,}5 \times 0{,}5 = 1 - 0{,}25 = 0{,}75 \neq 0$, giver at matricen er invertibel, og dermed at søjlerne udgør en basis for $\mathbb{R}^2$. △

19.2 Koordinater

Lad $E\colon v_0, v_1, \dots, v_{n-1}$ være en basis for et vektorrum V. Enhver vektor $b \in V$ kan skrives som

$$b = x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1} \tag{19.1}$$

for nogle skalarer x_i, da definition 19.1(b) sikrer en løsning. Disse skalarer er endda entydig, som konsekvens af definition 19.1(a): hvis $z_0, \dots, z_{n-1}$ er en anden løsning, har vi

$$\begin{aligned} 0 = b - b &= (x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}) - (z_0 v_0 + z_1 v_1 + \cdots + z_{n-1} v_{n-1}) \\ &= (x_0 - z_0) v_0 + (x_1 - z_1) v_1 + \cdots + (x_{n-1} - z_{n-1}) v_{n-1}. \end{aligned}$$

Men lineært uafhængighed giver nu at $x_i - z_i = 0$ for alle i. Dvs. $z_i = x_i$ for alle i, og løsningen er entydig.

Definition 19.6. Lad $E\colon v_0, v_1, \ldots, v_{n-1}$ være en basis for V. En vektor $b \in V$ har *koordinatvektor* $[b]_E$ mht. E givet ved

$$[b]_E = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix},$$

hvor $x_0, x_1, \ldots, x_{n-1}$ er den entydige løsning til (19.1).

Eksempel 19.7. Betragt den følgende basis for $\mathbb{R}^2$, $E\colon v_0, v_1$, hvor

$$v_0 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad v_1 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

Dette er en basis, da matricen $\left[\begin{smallmatrix} 3 & -1 \\ 1 & 2 \end{smallmatrix}\right]$ har determinant $3 \times 2 - (-1) \times 1 = 6 + 1 = 7 \neq 0$.

Lad os bestemme koordinatvektoren for $b = (1, -1)$ med hensyn til basen E. Vi skal løse

$$x_0 \begin{bmatrix} 3 \\ 1 \end{bmatrix} + x_1 \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

som er det samme som at løse det lineære ligningssystem

$$\begin{bmatrix} 3 & -1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Dette kan gøres vi rækkeoperationer på den tilsvarende udvidede matrix

$$\left[\begin{array}{cc|c} 3 & -1 & 1 \\ 1 & 2 & -1 \end{array}\right] \sim_{R_0 \leftrightarrow R_1} \left[\begin{array}{cc|c} 1 & 2 & -1 \\ 3 & -1 & 1 \end{array}\right] \sim_{R_1 \to R_1 - 3R_0} \left[\begin{array}{cc|c} 1 & 2 & -1 \\ 0 & -7 & 4 \end{array}\right]$$
$$\sim_{R_1 \to (-1/7)R_1} \left[\begin{array}{cc|c} 1 & 2 & -1 \\ 0 & 1 & -4/7 \end{array}\right] \sim_{R_0 \to R_0 - 2R_1} \left[\begin{array}{cc|c} 1 & 0 & 1/7 \\ 0 & 1 & -4/7 \end{array}\right],$$

så $x_0 = 1/7$, $x_1 = -4/7$ og

$$[b]_E = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1/7 \\ -4/7 \end{bmatrix}.$$

△

Proposition 19.8. *Lad $E\colon v_0, v_1, \ldots, v_{n-1}$ være en basis for et vektorrum V. Så er afbildningen $L\colon V \to \mathbb{R}^n$ givet ved*

$$L(b) = [b]_E$$

en invertibel lineær afbildning

Bevis. For vilkårlig $b, c \in V$ skriv

$$\begin{aligned} b &= x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}, \\ c &= y_0 v_0 + y_1 v_1 + \cdots + y_{n-1} v_{n-1}. \end{aligned}$$

Så er $[b]_E = (x_0, x_1, \ldots, x_{n-1})$ og $[c]_E = (y_0, y_1, \ldots, y_{n-1})$. Da

$$b + c = (x_0 + y_0)v_0 + (x_1 + y_1)v_1 + \cdots + (x_{n-1} + y_{n-1})v_{n-1}$$

får vi

$$\begin{aligned} L(b+c) = [b+c]_E &= \begin{bmatrix} x_0 + y_0 \\ x_1 + y_1 \\ \vdots \\ x_{n-1} + y_{n-1} \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} + \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} \\ &= [b]_E + [c]_E = L(b) + L(c), \end{aligned}$$

der viser at definition 18.1(a) er opfyldt. Desuden for en skalar s har vi

$$sb = (sx_0)v_0 + (sx_1)v_1 + \cdots + (sx_{n-1})v_{n-1},$$

så

$$L(sb) = [sb]_E = \begin{bmatrix} sx_0 \\ sx_1 \\ \vdots \\ sx_{n-1} \end{bmatrix} = s \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = s[b_E] = sL(b).$$

Dette viser at definition 18.1(b) er også opfyldt, og vi har dermed at L er lineær.

Den inverse til L gives ved

$$L^{-1}\left(\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}\right) = x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1},$$

så L er invertibel. □

19.3 Koordinatskift

Når vi har to forskellige baser for det samme vektorrum, vil man gerne vide hvordan de tilhørende koordinater er relaterede til hinanden. Lad os begynde med at kikke på skift af koordinater i vektorrummet $\mathbb{R}^n$.

Lad $E\colon v_0, v_1, \ldots, v_{n-1}$ være en basis for $\mathbb{R}^n$, og sæt $A = \left[v_0 \mid v_1 \mid \ldots \mid v_{n-1}\right]$. Så er koordinatvektoren $x = [b]_E$ løsningen til systemet

$$Ax = b,$$

dvs.

$$A[b]_E = b. \tag{19.2}$$

Dette er det samme som

$$[b]_E = x = A^{-1}b.$$

Hvis $F\colon w_0, \ldots, w_{n-1}$ er en anden basis, sætter vi $B = \left[w_0 \mid w_1 \mid \ldots \mid w_{n-1}\right]$. Så er koordinatvektoren af b mht. F givet på tilsvarende måde ved

$$[b]_F = B^{-1}b. \tag{19.3}$$

Kombinerer vi ligningerne (19.2) og (19.3), får vi

$$[b]_F = S[b]_E = B^{-1}A[b]_E,$$

som giver relationen mellem de to koordinatvektorer for b. Det er derfor naturligt at kalde matricen $S = B^{-1}A$ for *koordinatskiftsmatricen* fra E til F.

Bemærkning 19.9. Det følger at $A = I_n^{-1}A$ er koordinatskiftsmatricen fra E til standardbasen $e_0, \ldots, e_{n-1}$. Desuden har vi at $A^{-1} = A^{-1}I_n$ er koordinatskiftsmatricen fra standardbasen $e_0, \ldots, e_{n-1}$ til E. ◇

Lad os nu kikke på et vektorrum af polynomier. For at blive konkret, lad V være vektorrummet af polynomier af grad højst 2. Et typisk element i V er så et polynomium

$$p(x) = ax^2 + bx + c,$$

med a, b, c skalarer.

Én basis for V er $E\colon x^2, x, 1$. Vores polynomium p kan skrives, som en lineær kombination af basiselementer fra E via

$$p(x) = ax^2 + bx + c1.$$

Dette giver koordinatvektoren

$$[p]_E = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

af p mht. E.

En anden basis for V er $F\colon 3x^2 - 1, 2x + 2, 3$. Lad os først beregne koordinatskiftet fra F til E. Vi har

$$\begin{aligned} [3x^2 - 1]_E &= (3, 0, -1), \\ [2x + 2]_E &= (0, 2, 2), \\ [3]_E &= (0, 0, 3). \end{aligned}$$

Hvis $[p]_F = (r, s, t)$, betyder dette at

$$p(x) = r(3x^2 - 1) + s(2x + 2) + t(3).$$

Vi har så

$$\begin{aligned} [p]_E &= r[3x^2 - 1]_E + s[2x + 2]_E + t[3]_E \\ &= \left[[3x^2 - 1]_E \;\middle|\; [2x + 2]_E \;\middle|\; [3]_E \right] \begin{bmatrix} r \\ s \\ t \end{bmatrix} \\ &= \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 2 & 3 \end{bmatrix} \begin{bmatrix} r \\ s \\ t \end{bmatrix} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 2 & 3 \end{bmatrix} [p]_F. \end{aligned}$$

Dette betyder at koordinatskiftsmatricen S fra F til E er

$$S = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 2 & 3 \end{bmatrix},$$

dvs. S er matricen hvis søjlerne er koordinatvektorerne mht. E for basiselementerne af F. Koordinatskiftsmatricen den omvendte vej, fra E til F, er nu S^{-1}, som kan beregnes til at være

$$S^{-1} = \begin{bmatrix} \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{2} & 0 \\ \frac{1}{9} & -\frac{1}{3} & \frac{1}{3} \end{bmatrix}.$$

19.4 Baser for nul- og søjlerum af en matrix

Lad os betragte matricen

$$A = \begin{bmatrix} 1 & 1 & 2 & 1 & 0 \\ 1 & 1 & 3 & 2 & 1 \\ 2 & 2 & 5 & 4 & 1 \end{bmatrix}.$$

Denne matrix har reduceret echelonform

$$\begin{bmatrix} 1 & 1 & 0 & 0 & -2 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix},$$

så dens pivotsøjler er søjler 0, 2 og 3, og i ligningssystemet $Ax = b$ er x_1 og x_4 frie variabler. For enhver x er Ax en lineær kombination af pivotsøjlerne. Så vektorerne

$$\begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}, \quad \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}, \quad \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix},$$

som er pivotsøjlerne 0, 2, 3 af A, udspænder $S(A)$. Dette er også en basis for $S(A)$: Betingelsen at en lineær kombination af disse søjler er 0, er det samme som $Ax = 0$ hvor $x \in \mathbb{R}^5$ har formen $x = (x_0, 0, x_2, x_3, 0)$. Men fra echelonformen, ser vi at dette giver $x_0 = 0 = x_2 = x_3$, så $x = 0$, og søjlerne er dermed lineært uafhængig. Dermed har vi at pivotsøjlerne udgør en basis for $S(A)$.

Lad os nu betragter nulrummet $N(A)$ af A. Dette består af alle løsninger $x \in \mathbb{R}^5$ til $Ax = 0$. Fra den reducerede echelonform af A har vi at ligningssystemet $Ax = 0$ er ækvivalent med

$$\begin{aligned} x_0 + x_1 - 2x_4 &= 0, \\ x_2 + x_4 &= 0, \\ x_3 &= 0. \end{aligned}$$

Som vi har bemærket overfor er de frie variabler x_1 og x_4. Så den generelle løsning til $Ax = 0$ er

$$\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -x_1 + 2x_4 \\ x_1 \\ -x_4 \\ 0 \\ x_4 \end{bmatrix} = x_1 \begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + x_4 \begin{bmatrix} 2 \\ 0 \\ -1 \\ 0 \\ 1 \end{bmatrix}.$$

Derfor udspænder disse to vektorer $(-1, 1, 0, 0, 0)$, $(2, 0, -1, 0, 1)$ nulrummet for A. Dette par er også lineært uafhængig: I hver af disse vektorer er der et 1-tal i pladsen der svarer til den tilsvarende frie variabel, og i de andre vektorer er der et 0-tal i den samme plads:

$$\begin{bmatrix} -1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 2 \\ 0 \\ -1 \\ 0 \\ 1 \end{bmatrix}.$$

Så en lineær kombination af disse vektorer kan kun give 0 hvis de tilsvarende frie variabler er 0, dvs. hvis alle koefficienter er 0. De to vektorer udgør derfor en basis for $N(A)$.

Tilsvarende overvejelser for en generel matrix giver

Proposition 19.10. *Lad A være en $(m \times n)$-matrix. Så udgør pivotsøjlerne af A en basis for søjlerummet $S(A)$.*

Hvis A har r pivotsøjler, så er der $s = n - r$ frie variabler $x_{i(0)}, \ldots, x_{i(s-1)}$ for systemet $Ax = 0$. En basis for nulrummet $N(A)$, gives for $j = 0, \ldots, s-1$ af de løsninger til $Ax = 0$ hvor $x_{i(j)} = 1$ og de øvrige $x_{i(k)} = 0$. □

19.5 Dimension

Definition 19.11. Antallet af elementer i en basis for V kaldes *dimensionen*, skrevet $\dim V$, af V.

I eksemplet lige før, afsnit 19.4, har vi så

$$\dim S(A) = 3 \quad \text{og} \quad \dim N(A) = 2.$$

Som direkte konsekvens af proposition 19.10 har vi

Proposition 19.12. *For $A \in \mathbb{R}^{m \times n}$ er $\dim S(A)$ lige med antallet af pivotsøjler i A og $\dim N(A)$ lige med antallet af frie variabler i $Ax = 0$. Det følger at*

$$\dim S(A) + \dim N(A) = n.$$

Bevis. Proposition 19.10 giver

$$\text{antal pivotsøjler} + \text{antal frie variabler} = n$$

og resultater følger. □

Kapitel 20

Lineære transformationer og koordinatskift

Vi indførte lineære transformationer i kapitel 18. I afsnit 18.2 så vi at lineære transformationer $\mathbb{R}^n \to \mathbb{R}^m$ har en standard matrixrepræsentation (SMR) defineret fra de standard koordinator. Vi vil nu bruge vores mere generel definition på koordinater fra kapitel 19 til at definere matrixrepræsentationer for lineære afbildninger i forhold til vilkårlige valg af basis.

20.1 Generelle matrixrepræsentationer

Lad $L\colon V \to W$ være en lineær transformation. Antag at $E\colon v_0, v_1, \ldots, v_{n-1}$ er en basis for V og at $F\colon w_0, w_1, \ldots, w_{m-1}$ er en basis for W.

Definition 20.1. Vi definerer *matricen A af L mht. E og F* til at være matricen $A = \left[a_0 \mid a_1 \mid \ldots \mid a_{n-1}\right] \in \mathbb{R}^{m\times n}$ givet ved

$$a_j = [L(v_j)]_F. \tag{20.1}$$

Vi siger også at A er *matrixrepræsentationen* af L mht. E og F.

Denne matrixrepræsentation er nyttigt da det følgende resultat fortæller os at lineære transformationer er blot multiplikation med en matrix. Mere præcis kan vi erstatte L med matrixmultiplikation ved $A \in \mathbb{R}^{m\times n}$, hvis vi skriver vektorerne ud som koordinatvektorer mht. de givne baser. Vi er glade for matrixmultiplikation, da det er en operation vi kan udføre i python.

Proposition 20.2. *Lad A være matricen af $L\colon V \to W$ mht. baserne E for V og F for W. Da gælder for alle $v \in V$ at*

$$[L(v)]_F = A[v]_E.$$

Bevis. Lad os skrive

$$[v]_E = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}.$$

Så har vi $v = x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}$. Vi kan nu regne

$$\begin{aligned} L(v) &= L(x_0 v_0 + x_1 v_1 + \cdots + x_{n-1} v_{n-1}) \\ &= x_0 L(v_0) + x_1 L(v_1) + \cdots + x_{n-1} L(v_{n-1}). \end{aligned}$$

Skrives $L(v)$ ud i koordinater, får vi så

$$\begin{aligned} [L(v)]_F &= x_0 [L(v_0)]_F + x_1 [L(v_1)]_F + \cdots + x_{n-1} [L(v_{n-1})]_F \\ &= x_0 a_0 + x_1 a_1 + \cdots + x_{n-1} a_{n-1} \\ &= Ax. \end{aligned}$$

Det sidste udtryk er lige med $A[v]_E$, som ønsket. □

Eksempel 20.3. Lad V være vektorrummet af polynomier af grad højst 2 og lad W være vektorrummet af polynomier af grad højst 1. Funktionen $L\colon V \to W$ givet ved $L(p) = p'$, den afledede til p, er en lineær afbildning.

Lad os vælge baserne $E\colon x^2, x, 1$ for V, og $F\colon x, 1$ for W. Vi finder søjlerne a_0, a_1, a_2 af matricen $A \in \mathbb{R}^{2\times 3}$ af L mht. E og F ved

$$\begin{aligned} a_0 &= [L(x^2)]_F = [2x]_F = [2 \times x + 0 \times 1]_F = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \\ a_1 &= [L(x)]_F = [1]_F = [0 \times x + 1 \times 1]_F = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \\ a_2 &= [L(1)]_F = [0]_F = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \end{aligned}$$

Dette giver

$$A = \begin{bmatrix} a_0 \mid a_1 \mid a_2 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Proposition 20.2 siger nu at L svarer til multiplikation med matricen A.

I dette tilfælde har vi for $p(x) = ax^2 + bx + c$ at $L(p(x)) = 2ax + b$ kan udføres, som matrixmultiplikation

$$[L(p(x))]_F = A[p(x)]_E = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 2a \\ b \end{bmatrix},$$

og dermed implementeres f.eks. i python. △

Som konsekvens kan vi nu vise den følgende sætning. Formlen der fremkommer kaldes ofte *rang-nullitetsformlen*, da $\dim \operatorname{im} L$ er *rangen* af L, og $\dim \ker L$ en dens *nullitet*.

Sætning 20.4. *Lad $L\colon V \to W$ være lineær, lad E være en basis for V, lad F være en basis for W og lad A være matricen af L mht. E og F. Da gælder*

(a) $v \in \ker L$ *hvis og kun hvis* $[v]_E \in N(A)$,

(b) $w \in \operatorname{im} L$ *hvis og kun hvis* $[w]_F \in S(A)$.

Det følger at forskellige baser for V har det samme antal elementer og at

$$\dim \operatorname{im} L + \dim \ker L = \dim V.$$

Bevis. For del (a), har vi at $v \in \ker L$ betyder at $L(v) = 0$. For sådan et $v \in \ker L$ har vi nu $A[v]_E = [L(v)]_F = [0]_F = 0$, så $[v]_E \in N(A)$. Omvendt $[v]_E \in N(A)$, giver $[L(v)]_F = A[v]_E = 0$, så $L(v) = 0$ og v ligger i $\ker L$.

Del (b) er tilsvarende: $w \in \operatorname{im} L$ betyder at $w = L(v)$ for et $v \in V$. Så for $w = L(v)$ har vi $[w]_F = [L(v)]_F = A[v]_E$, som siger at $[w]_F$ er en lineær kombination af søjler af A, dvs. $[w]_F \in S(A)$. Omvendt $[w]_F \in S(A)$ betyder $[w]_F = Ax$ for et $x \in \mathbb{R}^n$. Hvis basen E består af $v_0, \ldots, v_{n-1}$, sætter vi $v = x_0 v_0 + \cdots + v_{n-1} x_{n-1}$ og får at $[v]_E = x$. Vi har nu $[L(v)]_F = Ax = [w]_F$, så $L(v) = w$ og $w \in \operatorname{im} L$.

Bemærk nu at

(a) $u_0, \ldots, u_{k-1}$ er en basis for $\ker L$ hvis og kun hvis $[u_0]_E, \ldots, [u_{k-1}]_E$ er en basis for $N(A)$, og

(b) $f_0, \ldots, f_{r-1}$ er en basis for $\operatorname{im} L$ hvis og kun hvis $[f_0]_F, \ldots, [f_{r-1}]_F$ er en basis for $S(A)$.

Ved at bruge proposition 19.12, har vi

$$\dim \operatorname{im} L + \dim \ker L = r + k = \dim S(A) + \dim N(A) = n = \dim V,$$

som er den ønskede relation mellem dimensioner.

For at se at forskellige baser for V har det samme antal elementer, antag nu at E og F er begge baser for V, hvor E har n elementer, og F har m elementer. Lad $L = \mathrm{Id}\colon V \to V$, $\mathrm{Id}(v) = v$ være identitetsafbildningen. Denne afbildning er lineær, så vi kan betragte matricen $A \in \mathbb{R}^{m\times n}$ af $L = \mathrm{Id}$ mht. E og F. Vi har $\ker \mathrm{Id} = \{0\}$ og $\operatorname{im} \mathrm{Id} = V$, så $N(A) = \{0\}$, $S(A) = \mathbb{R}^m$. Vi får nu at

$$n = \dim S(A) + \dim N(A) = m + 0 = m,$$

så både E og F har n elementer. □

Eksempel 20.5. I eksempel 20.3 kan vi godt beregne $\ker L$. Et polynomium $p(x) = ax^2+bx+c$ ligger i $\ker L$ hvis $L(p(x)) = p'(x) = 2ax+b$ er 0-polynomiet. Dette er det samme som $a = 0 = b$, så $\ker L$ består af alle konstante polynomier $p(x) = c$, med c en vilkårlig skalar. Bemærk at disse polynomier har $[p(x)]_E = [c]_E = (0, 0, c)$.

Matricen

$$A = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

af L mht. E og F har $v = (x_0, x_1, x_2) \in N(A)$ kun hvis $Av = 0$. Men

$$Av = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2x_0 \\ x_1 \end{bmatrix},$$

så $Av = 0$ kun for $v = (0, 0, x_2)$, med x_2 vilkårlig. Vektorer af denne form er netop koordinatvektorer for elementer af $\ker L$.

Bemærk at vi har $\dim \ker L = 1 = \dim N(A)$, da der er kun én fri parameter.

Billedmængden $\operatorname{im} L$ består af alle polynomier af formen $2ax + b$, så af alle polynomier af grad højst 1. Vi har dermed at $\dim \operatorname{im} L = 2$, og kan bekræfte at

$$\dim \operatorname{im} L + \dim \ker L = 2 + 1 = 3 = \dim V$$

i overensstemmelse med sætning 20.4. △

20.2 Basisskift

Vi fortsætter med at betragte en lineær afbildning $L\colon V \to W$. Hvis E og E' er to baser for V, har vi en invertibel koordinatskiftsmatrix S fra E til E', som opfylder

$$[v]_{E'} = S[v]_E.$$

Tilsvarende hvis F og F' er baser for W, har vi en koordinatskiftsmatrix T fra F til F' med

$$[w]_{F'} = T[w]_F.$$

Vi kan nu beregne matricen B af L mht. E' og F', givet matricen A af L mht. E og F. Vi har

$$B[v]_{E'} = [L(v)]_{F'} \quad \text{og} \quad A[v]_E = [L(v)]_F,$$

så

$$BS[v]_E = B[v]_{E'} = [L(v)]_{F'} = T[L(v)]_F = TA[v]_E$$

for alle v. Det følger at

$$BS = TA,$$

som er det samme som

$$B = TAS^{-1}. \tag{20.2}$$

Omvendt, hver gang vi har en matrixformel af formen (20.2) med $A, B \in \mathbb{R}^{m\times n}$, og invertibel $S \in \mathbb{R}^{n\times n}$, $T \in \mathbb{R}^{m\times m}$, kan vi betragte dette som sigende at A og B er matrixrepræsentationer for den samme lineære transformation, blot i forhold til forskellige baser.

Vi kender allerede en formel af formen (20.2), nemlig den fulde SVD: $A = U\Sigma V^T$, som er $A = U\Sigma V^{-1}$, da V er ortogonal. Dette siger at A mht. standard baser har samme effekt som Σ mht. baser fra søjlerne af V og U: $Av_i = \sigma_i u_i$. Dette er et godt eksempel på at et nyt valg af baser kan gøre effekten af A mere tydeligt.

20.3 Afbildning på ét vektorrum

Betragt nu en lineær afbildning $L\colon V \to V$, som afbilder vektorrummet V ind i sig selv. Vælger vi en basis $E\colon v_0, \ldots, v_{n-1}$ for V, har vi en *matrix A af L mht. kun E* givet ved (20.1) med $F = E$, dvs. $A = \left[a_0 \mid a_1 \mid \ldots \mid a_{n-1}\right] \in \mathbb{R}^{n\times n}$ med

$$a_j = [L(v_j)]_E.$$

Skifter vi til en anden basis $F\colon w_0, w_1, \ldots, w_{n-1}$ af V, har vi en koordinatskiftsmatrix S fra E til F. Ved brug af (20.2), ser vi nu at matricen B af L mht. kun F er relateret til A via

$$B = SAS^{-1}.$$

Eksempel 20.6. I et lille land bor 500 000 mennesker i storbyen og 700 000 mennesker i landområder. Der observeres at hvert år flyttes 3 % af landbefolkningen til byen, og 2 % af bybefolkningen på landet. Lad os undersøge hvordan befolkningsgrupperne udvikler sig med tid.

Vi starter med befolkningsfordelingen

$$b_0 = \begin{bmatrix} 500\,000 \\ 700\,000 \end{bmatrix},$$

hvor vi skrive antallet i byen øverst, og antallet på landet nederst. Ændringen fra år til år gives ved matricen

$$A = \begin{bmatrix} 0{,}98 & 0{,}03 \\ 0{,}02 & 0{,}97 \end{bmatrix}.$$

Vi kan så beregne befolkningsfordeling over de kommende år, som

$$b_1 = Ab_0, \quad b_2 = Ab_1, \quad b_3 = Ab_2, \ldots$$

I python

```
import numpy as np

b0 = np.array([500000., 700000.])[:, np.newaxis]
a = np.array([[0.98, 0.03],
              [0.02, 0.97]])

b = b0

for i in range(5):
    print(f'b{i} = {b}')
    b = a @ b
```

```
b0 = [[500000.]
 [700000.]]
b1 = [[511000.]
 [689000.]]
b2 = [[521450.]
```

```
 [678550.]]
b3 = [[531377.5]
 [668622.5]]
b4 = [[540808.625]
 [659191.375]]
```

Det ses at bybefolkningen vokser, men det er ikke klart hvad der sker over længere tid.

Lad os prøve at skifte fra standard basen, til basen $F\colon w_0 = (3,2), w_1 = (1,-1)$. Koordinatskiftsmatricen fra F til standardbasen er

$$T = \left[w_0 \mid w_1\right] = \begin{bmatrix} 3 & 1 \\ 2 & -1 \end{bmatrix}.$$

Effekten af A mht. F er givet ved matricen $C = T^{-1}AT$. Vi har at

$$\begin{aligned} C = T^{-1}AT &= \frac{1}{3\times(-1)-1\times 2}\begin{bmatrix} -1 & -1 \\ -2 & 3 \end{bmatrix}\begin{bmatrix} 0{,}98 & 0{,}03 \\ 0{,}02 & 0{,}97 \end{bmatrix}\begin{bmatrix} 3 & 1 \\ 2 & -1 \end{bmatrix} \\ &= -\frac{1}{5}\begin{bmatrix} -1{,}0 & -1{,}0 \\ -1{,}9 & 2{,}85 \end{bmatrix}\begin{bmatrix} 3 & 1 \\ 2 & -1 \end{bmatrix} = -\frac{1}{5}\begin{bmatrix} -5 & 0 \\ 0 & -4{,}75 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 0{,}95 \end{bmatrix} \end{aligned}$$

Bemærk at

$$A = TCT^{-1}$$

og at

$$\begin{aligned} A^2 &= (TCT^{-1})(TCT^{-1}) = TC(T^{-1}T)CT^{-1} = TC^2T^{-1} \\ A^3 &= A^2A = TC^2T^{-1}TCT^{-1} = TC^3T^{-1}. \end{aligned}$$

Så generelt har vi

$$A^k = TC^kT^{-1}.$$

Men matricen C^k er nem at udregne: Det er en diagonalmatrix med indgange de k'te potenser af de tilsvarende indgange i C. Dette giver

$$\begin{aligned} b_k &= A^kb_0 = TC^kT^{-1}b_0 \\ &= T\begin{bmatrix} 1 & 0 \\ 0 & 0{,}95^k \end{bmatrix}T^{-1}b_0 \\ &= \begin{bmatrix} 3 & 0{,}95^k \\ 2 & -0{,}95^k \end{bmatrix}\begin{bmatrix} 240\,000 \\ 220\,000 \end{bmatrix} = \begin{bmatrix} 720\,000 \\ 480\,000 \end{bmatrix} + 0{,}95^k\begin{bmatrix} 240\,000 \\ 220\,000 \end{bmatrix}. \end{aligned} \tag{20.3}$$

Vi kan nu se hvad sker nå k bliver stort. For $k \to \infty$, har vi $(0{,}95)^k \to 0$, da $|0{,}95| < 1$. Sml.

```
for k in range(0, 1000, 57):
    print(0.95**k)
```

```
1.0
0.053733545982740286
0.0028872939638792646
0.00015514454297379496
8.336466433853638e-06
4.479479024570453e-07
2.4069829214547706e-08
1.2933572748966046e-09
6.949667260276838e-11
3.734302652948301e-12
2.006573233156666e-13
1.0782029509155957e-14
5.793566784174943e-16
3.113088872015411e-17
1.6727730405279714e-18
8.988402709189803e-20
4.8297875028563765e-21
2.595216088715975e-22
```

Så fra (20.3) ser vi at

$$b_k \to \begin{bmatrix} 720\,000 \\ 480\,000 \end{bmatrix} \qquad \text{for } k \to \infty.$$

Dvs. efter mange år vil befolkningen blive fordelt med tæt på 720 000 i byen, og 480 000 på landet.

Alternativt kan vi se at

$$A^k = TC^kT^{-1} = T \begin{bmatrix} 1 & 0 \\ 0 & (0{,}95)^k \end{bmatrix} T^{-1} \to T \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} T^{-1}.$$

Så har vi $b_k = A^k b_0 \to T \operatorname{diag}(1,0) T^{-1} b_0$ for $k \to \infty$. I python har vi

```
t = np.array([[3.0, 1.0],
              [2.0, -1.0]])
print(t)
```

```
[[ 3.  1.]
 [ 2. -1.]]
```

og matricen $C = TAT^{-1}$ er

```
c = np.linalg.inv(t) @ (a @ t)
print(c)
```

```
[[ 1.00000000e+00 -2.72004641e-17]
 [ 0.00000000e+00  9.50000000e-01]]
```

Grænsen for b_k når $k \to \infty$ er nu

```
print(t @ np.diag([1.0, 0.0]) @ np.linalg.inv(t) @ b0)
```

```
[[720000.]
 [480000.]]
```

som stemmer overens med grænsen fundet ovenfor. $\triangle$

Kapitel 21

Egenværdier og egenvektorer

21.1 Et første eksempel

Lad os kikke igen på eksempel 20.6. Problemet er styret af matricen

$$A = \begin{bmatrix} 0{,}98 & 0{,}03 \\ 0{,}02 & 0{,}97 \end{bmatrix}$$

og vi valgte en særlig basis $F\colon\ w_0 = (3, 2), w_1 = (1, -1)$. Hvor kommer denne basis fra?

Lad os lige beregne Aw_0 og Aw_1:

$$\begin{aligned} Aw_0 &= \begin{bmatrix} 0{,}98 & 0{,}03 \\ 0{,}02 & 0{,}97 \end{bmatrix} \begin{bmatrix} 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} = w_0 \\ Aw_1 &= \begin{bmatrix} 0{,}98 & 0{,}03 \\ 0{,}02 & 0{,}97 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0{,}95 \\ -0{,}95 \end{bmatrix} = 0{,}95\, w_1. \end{aligned}$$

Vi ser at i begge tilfælde har vi en ligning af formen

$$Aw_i = \lambda_i w_i, \tag{21.1}$$

hvor $\lambda_0 = 1$ og $\lambda_1 = 0{,}95$.

Dette medfør at vi kan beregne effekten af A på b ved at skrive b, som lineær kombination af basisvektorerne w_0, w_1. Hvis $b = x_0 w_0 + x_1 w_1$, har vi

$$\begin{aligned} Ab &= A(x_0 w_0 + x_1 w_1) = x_0\, Aw_0 + x_1\, Aw_1 \\ &= x_0 w_0 + 0{,}95\, x_1 w_1. \end{aligned}$$

Denne fremstilling gør det nemt at gentage multiplikation med A og at se en videreudvikling: Vi har

$$A^k w_i = \lambda_i^k w_i,$$

som fører til

$$\begin{aligned} A^k b &= A^k(x_0 w_0 + x_1 w_1) = x_0 A^k w_0 + x_1 A^k w_1 \\ &= x_0 w_0 + (0{,}95)^k x_1 w_1. \end{aligned}$$

For eksempel, da $(0{,}95)^k \to 0$ for $k \to \infty$, kan vi nu aflæse at $A^k b \to x_0 w_0$. Så for alle startvektorer $b = x_0 w_0 + x_1 w_1$ får vi en fast grænsevektor $x_0 w_0$ for $A^k b$ når $k \to \infty$.

Det væsentlige i disse regnestykker er relationerne (21.1).

21.2 Definitioner og første regnemetoder

Definition 21.1. Lad A være en kvadratisk matrix af størrelse $n \times n$. En skalar λ er en *egenværdi* for A, hvis der findes en vektor v, med $v \neq 0$, således at

$$Av = \lambda v.$$

Vektoren $v \neq 0$ er så en *egenvektor* for A hørende til λ.

I eksemplet ovenfor har vi 2 egenværdier $\lambda_0 = 1$ og $\lambda_1 = 0{,}95$, med $w_0 = (3, 2)$ en egenvektor hørende til λ_0 og $w_1 = (1, -1)$ en egenvektor hørende til λ_1.

Hvis λ er en egenværdi for A og $v \neq 0$ er en tilhørende egenvektor, så kan vi omskrive $Av = \lambda v$ til

$$(A - \lambda I_n)v = 0.$$

Vi ser så at λ er en egenværdi for A hvis og kun hvis $A - \lambda I_n$ er ikke invertibel. For $n = 2$, ved vi at en (2×2)-matrix B er invertibel hvis og kun hvis $\det B \neq 0$. Vi får så

Proposition 21.2. *En skalar λ er en egenværdi for en $(n \times n)$-matrix A hvis og kun hvis $A - \lambda I_n$ er ikke invertibel.*

For $n = 2$, er λ en egenværdi hvis og kun hvis

$$\det(A - \lambda I_2) = 0. \qquad \square$$

Eksempel 21.3. Betragt $A \in \mathbb{R}^{2\times 2}$ givet ved

$$A = \begin{bmatrix} 3 & 1 \\ 2 & 2 \end{bmatrix}.$$

Vi har

$$\begin{aligned} \det(A - \lambda I_2) &= \det \begin{bmatrix} 3-\lambda & 1 \\ 2 & 2-\lambda \end{bmatrix} = (3-\lambda)(2-\lambda) - 1 \times 2 \\ &= \lambda^2 - 5\lambda + 4. \end{aligned}$$

Hvis λ er en egenværdi af A, har vi så $\lambda^2 - 5\lambda + 4 = 0$. Denne andengradsligning har løsninger

$$\frac{-(-5) \pm \sqrt{(-5)^2 - 4 \times 1 \times 4}}{2 \times 1} = \frac{5 \pm \sqrt{25-16}}{2} = \frac{5 \pm 3}{2} = 1 \text{ og } 4.$$

For at finde tilhørende egenvektorer skal vi finde $v \neq 0$ således at $(A - \lambda I_2)v = 0$. Det kan vi godt gøre ved hjælp af rækkeoperationer.

Vi skal løse et lineært ligningssystem af formen $Bx = c$, med $c = 0$. Sædvanligvis ville vi stille en udvidet matrix $\left[B \mid c\right]$ op. Men når $c = 0$, forbliver den sidste søjle 0 under alle rækkeoperationer, og vi kan nøjes med at arbejde blot med B.

For $\lambda_0 = 1$, har vi

$$A - \lambda_0 I_2 = \begin{bmatrix} 3-1 & 1 \\ 2 & 2-1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \sim_{R_1 \to R_1 - R_0} \begin{bmatrix} 2 & 1 \\ 0 & 0 \end{bmatrix}.$$

Hvis $v_0 = (x, y)$, siger dette reduktion at $2x + y = 0$, så en løsning er $v_0 = (1, -2)$.

For $\lambda_1 = 4$, beregnes

$$A - \lambda_1 I_2 = \begin{bmatrix} 3-4 & 1 \\ 2 & 2-4 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \sim_{R_1 \to R_1 + 2R_0} \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}.$$

For $v_1 = (x, y)$ har vi så ligningen $-x + y = 0$, dvs. $y = x$ og en løsning er $v_1 = (1, 1)$. △

Bemærk at for for $A \in \mathbb{R}^{2\times 2}$ er $\det(A - \lambda I_2)$ et andengradspolynomium. Dette kaldes det *karakteristiske polynomium* for $A \in \mathbb{R}^{2\times 2}$.

Når vi skal bestemme en egenvektor v hørende til λ, får vi et system med (mindst) et fri variabel. Dette bekræfter at matricen $A - \lambda I_n$ er ikke invertibel, dvs.

at vi har fat i en korrekt egenværdi λ. Det understreger også at egenvektorerne er ikke entydigt bestemt: Hvis v er en egenvektor for A hørende til λ, så har vi $Av = \lambda v$; hvis $s \neq 0$ er en skalar, har vi nu $A(sv) = \lambda sv$, så sv er også en egenvektor hørende til λ.

For $n > 2$, kan vi godt bestemme egenvektorer, som ovenfor, hvis vi er givet en eller flere egenværdier på forhånd.

Eksempel 21.4. Betragt matricen

$$A = \begin{bmatrix} 3 & -1 & 2 \\ 1 & 2 & -1 \\ 4 & 1 & 1 \end{bmatrix}.$$

Jeg påstår at $\lambda = 1$ er en egenværdi for A.

Dette kan bekræftes ved at kikke på $A - \lambda I_3$ og vise at $(A - \lambda I_3)v = 0$ har en løsning med $v \neq 0$.

Vi har for $\lambda = 1$ at

$$\begin{aligned} A - \lambda I_3 &= \begin{bmatrix} 3-1 & -1 & 2 \\ 1 & 2-1 & -1 \\ 4 & 1 & 1-1 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 2 \\ 1 & 1 & -1 \\ 4 & 1 & 0 \end{bmatrix} \\ &\sim_{R_0 \leftrightarrow R_1} \begin{bmatrix} 1 & 1 & -1 \\ 2 & -1 & 2 \\ 4 & 1 & 0 \end{bmatrix} \sim_{R_1 \to R_1 - 2R_0} \begin{bmatrix} 1 & 1 & -1 \\ 0 & -3 & 4 \\ 4 & 1 & 0 \end{bmatrix} \\ &\sim_{R_2 \to R_2 - 4R_0} \begin{bmatrix} 1 & 1 & -1 \\ 0 & -3 & 4 \\ 0 & -3 & 4 \end{bmatrix} \sim_{R_2 \to R_2 - R_1} \begin{bmatrix} 1 & 1 & -1 \\ 0 & -3 & 4 \\ 0 & 0 & 0 \end{bmatrix}. \end{aligned}$$

Vi ser så at der er én fri variabel, fra den sidste søjle, så $\lambda = 1$ er en egenværdi. En tilhørende egenvektor fås ved at sætte denne frie variabel lige med 1, for eksempel. Det giver $v = (-1/3, 4/3, 1)$. Et alternativ valg er vektoren $3v = (-1, 4, 3)$. Begge er egenvektorer hørende til $\lambda = 1$. △

Senere vil vi begynde at se på metoder for at finde egenværdier generelt, men vi vil erfare i det næste afsnit at det er ikke altid et problem der tillader en løsning. Desuden er de første metoder vi kikker på ikke gode for numeriske beregninger.

Lad os begynde med at bemærke det følgende resultat.

Proposition 21.5. *Hvis $\lambda_0, \lambda_1, \ldots, \lambda_{k-1}$ er forskellige egenværdier for A og $v_0, v_1, \ldots, v_{k-1}$ er tilhørende egenvektorer, så er $v_0, v_1, \ldots, v_{k-1}$ lineært uafhængig.*

Det følger at hvis en $(n \times n)$-matrix A har n forskellige egenværdier, så udgør de tilhørende egenvektorer $v_0, \ldots, v_{n-1}$ en basis.

Bevis. Lad os kikke først på tilfældet $k = 2$. Antag at

$$x_0 v_0 + x_1 v_1 = 0. \tag{21.2}$$

Vi skal vise at $x_0 = 0 = x_1$.

Ganger vi matricen A på denne ligning, får vi fra $Av_i = \lambda_i v_i$ at

$$x_0 \lambda_0 v_0 + x_1 \lambda_1 v_1 = 0. \tag{21.3}$$

Trækker vi λ_1 gange ligning (21.2) fra (21.3), fås

$$x_0(\lambda_0 - \lambda_1)v_0 = 0.$$

Men $v_0 \neq 0$, så $x_0(\lambda_0 - \lambda_1) = 0$. Da vi har antaget at $\lambda_0 \neq \lambda_1$, får vi $x_0 = 0$. Ligning (21.2) er nu $x_1 v_1 = 0$. Men $v_1 \neq 0$, giver $x_1 = 0$. Så $x_0 = 0 = x_1$, og samlingen v_0, v_1 er lineært uafhængig.

Generelt, hvis vi har vist at $v_0, \ldots, v_{k-2}$ er lineært uafhængig, kan vi kikke på ligningen

$$x_0 v_0 + \cdots + x_{k-2} v_{k-2} + x_{k-1} v_{k-1} = 0. \tag{21.4}$$

Vi skal vise at $x_0 = 0 = x_1 = \cdots = x_{k-1}$.

Ganger vi matricen A på denne ligning, får vi

$$x_0 \lambda_0 v_0 + \cdots + x_{k-2} \lambda_{k-2} v_{k-2} + x_{k-1} \lambda_{k-1} v_{k-1} = 0. \tag{21.5}$$

Trækkes λ_{k-1} ganger (21.4) fra (21.5), finder vi

$$x_0(\lambda_0 - \lambda_{k-1})v_0 + \cdots + x_{k-2}(\lambda_{k-2} - \lambda_{k-1})v_{k-2} = 0.$$

Da $v_0, \ldots, v_{k-2}$ er lineært uafhængig, fås $x_i(\lambda_i - \lambda_{k-1}) = 0$ for $i = 0, \ldots, k-2$. Men $\lambda_i \neq \lambda_{k-1}$, så $x_i = 0$, for $i = 0, \ldots, k-2$. Nu er (21.4) blot $x_{k-1} v_{k-1} = 0$, så vi får også $x_{k-1} = 0$. Dermed er $v_0, v_1, \ldots, v_{k-1}$ lineært uafhængig. □

Proposition 21.6. *Hvis en $(n \times n)$-matrix A har en basis $v_0, v_1, \ldots, v_{n-1}$ af egenvektorer, så kan A skrives som*

$$A = V \Lambda V^{-1}, \tag{21.6}$$

hvor $V = \left[v_0 \mid v_1 \mid \ldots \mid v_{n-1} \right]$ *og*

$$\Lambda = \operatorname{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{n-1}).$$

Bevis. Vi bemærker at

$$\begin{aligned} AV &= A\left[v_0 \mid v_1 \mid \dots \mid v_{n-1}\right] = \left[Av_0 \mid Av_1 \mid \dots \mid Av_{n-1}\right] \\ &= \left[\lambda_0 v_0 \mid \lambda_1 v_1 \mid \dots \mid \lambda_{n-1} v_{n-1}\right] = V\Lambda. \end{aligned}$$

Da V er en $(n \times n)$-matrix med lineært uafhængige søjler, er V invertibel og ligning (21.6) følger. □

Definition 21.7. En matrix A, der kan skrives i formen (21.6), kaldes *diagonaliserbar*.

Eksempel 21.8. I eksempel 21.3 fandt vi at

$$A = \begin{bmatrix} 3 & 1 \\ 2 & 2 \end{bmatrix}$$

har egenværdier 1 og 4 med tilhørende egenvektorer $(1, -2)$ hhv. $(1, 1)$. Det følger at

$$A = V\Lambda V^{-1}$$

med

$$V = \begin{bmatrix} 1 & 1 \\ -2 & 1 \end{bmatrix} \quad \text{og} \quad \Lambda = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}.$$

Dette kan bekræftes ved

```
>>> import numpy as np
>>> v = np.array([[ 1.0, 1.0],
...               [-2.0, 1.0]])
>>> lambda_mat = np.diag([1.0, 4.0])
>>> v @ lambda_mat @ np.linalg.inv(v)
array([[3., 1.],
       [2., 2.]])
```

△

21.3 Andre eksempler

Eksempel 21.9. Betragt matricen $A \in \mathbb{R}^{2\times 2}$ givet ved

$$A = \begin{bmatrix} 1 & 2 \\ -2 & 1 \end{bmatrix}.$$

Det karakteristiske polynomium er

$$\det(A - \lambda I_2) = \det \begin{bmatrix} 1-\lambda & 2 \\ -2 & 1-\lambda \end{bmatrix} = (1-\lambda)^2 + 4 = \lambda^2 - 2\lambda + 5.$$

Egenværdier er løsninger til $p(\lambda) = \lambda^2 - 2\lambda + 5 = 0$. Men p har toppunkt i 1 og $p(1) = 4$, så p har ingen rødder, som er reel tal, og dermed har A ingen egenværdier og ingen egenvektorer i $\mathbb{R}^2$.

Arbejder vi dog med komplekse skalarer, dvs. betragter vi A som en matrix i $\mathbb{C}^{2\times 2}$, kan vi finde rødder via den sædvanlige formel

$$\frac{2 \pm \sqrt{4-20}}{2} = \frac{2 \pm \sqrt{-16}}{2} = \frac{2 \pm 4i}{2} = 1 \pm 2i.$$

Lad os finde tilsvarende egenvektorer i $\mathbb{C}^2$ ved hjælp af python.

Først for $\lambda_0 = 1 + 2i$,

```
>>> import numpy as np

>>> a = np.array([[ 1.0, 2.0],
...               [-2.0, 1.0]])
>>> lambda_0 = 1.0 + 2.0j
>>> b = a - lambda_0 * np.eye(2)
>>> b
array([[ 0.-2.j,  2.+0.j],
       [-2.+0.j,  0.-2.j]])
>>> b[0, :] /= -2.0j
>>> b
array([[ 1.+0.j, -0.+1.j],
       [-2.+0.j,  0.-2.j]])
>>> b[1, :] += 2.0 * b[0, :]
>>> b
array([[ 1.+0.j, -0.+1.j],
       [ 0.+0.j,  0.+0.j]])
```

så $v_0 = (-i, 1)$ er en tilhørende egenvektor. For $\lambda_1 = 1 - 2i$, har vi

```
>>> lambda_1 = 1.0 - 2.0j
>>> c = a - lambda_1 * np.eye(2)
```

```
>>> c
array([[ 0.+2.j,  2.+0.j],
       [-2.+0.j,  0.+2.j]])
>>> c[0, :] /= 2.0j
>>> c
array([[ 1.+0.j,  0.-1.j],
       [-2.+0.j,  0.+2.j]])
>>> c[1, :] += 2.0 * c[0, :]
>>> c
array([[1.+0.j, 0.-1.j],
       [0.+0.j, 0.+0.j]])
```

og $v_1 = (i, 1)$ er en tilhørende egenvektor. Det følger at v_0, v_1 er en basis for $\mathbb{C}^2$ og at

$$A = V\Lambda V^{-1}$$

med

$$V = \left[v_0 \mid v_1\right] = \begin{bmatrix} -i & i \\ 1 & 1 \end{bmatrix} \quad \text{og} \quad \Lambda = \begin{bmatrix} 1+2i & 0 \\ 0 & 1-2i \end{bmatrix}.$$

Dette kan bekræftes i python

```
>>> v = np.array([[-1.j, 1.j],
...               [1.0,  1.0]])
>>> lambda_mat = np.diag([1.0+2.0j, 1.0-2.0j])
>>> v @ lambda_mat @ np.linalg.inv(v)
array([[ 1.+0.j,  2.+0.j],
       [-2.+0.j,  1.+0.j]])
```

△

Eksempel 21.10. Betragt matricen

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Det karakteristiske polynomium er

$$\det(A - \lambda I_2) = \det \begin{bmatrix} 1-\lambda & 1 \\ 0 & 1-\lambda \end{bmatrix} = (1-\lambda)^2,$$

så det eneste egenværdi er $\lambda = 1$. Lad os finde de tilhørende egenvektorer. Vi har

$$A - \lambda I_2 = \begin{bmatrix} 1-1 & 1 \\ 0 & 1-1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix},$$

så enhver egenvektor er et multiplum af $v = (1, 0)$. Det følger at der findes ingen basis af $\mathbb{R}^2$ (eller af $\mathbb{C}^2$), som består af egenvektorer til A. $\triangle$

21.4 Determinanter

For (2×2)-matricer har vi at

$$\det A = \det \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix} = a_{00}a_{11} - a_{01}a_{10}$$

og at matricen A er invertibel hvis og kun hvis $\det A \neq 0$. Denne definition kan udvides til $(n \times n)$-matricer på den følgende måde. Først for (1×1)-matricer, sætter vi

$$\det \begin{bmatrix} a_{00} \end{bmatrix} = a_{00}.$$

For $n > 1$, og for $0 \leqslant i, j \leqslant n-1$ definerer vi den *(i, j)'te undermatrix* M_{ij}, til at være den $((n-1) \times (n-1))$-delmatrix af A der fås ved at slette hele række i og hele søjle j fra A. Dvs. at M_{ij} er A fraregnet rækken $A_{[i,:]}$ og søjle $A_{[:,j]}$.

Eksempel 21.11. For den følgende matrix A, bestemmer vi den $(1, 2)$'te undermatrix M_{12} ved

$$A = \begin{bmatrix} 2 & -1 & 3 & 4 & 5 \\ 6 & -2 & -5 & 2 & -1 \\ -1 & 3 & 0 & -6 & 3 \\ 4 & 1 & -3 & -3 & 2 \\ 5 & 2 & 1 & -1 & -2 \end{bmatrix} = \begin{array}{c} \\ A_{[1,:]} \\ \\ \\ \\ \end{array} \begin{array}{c} \begin{bmatrix} 2 & -1 & 3 & 4 & 5 \\ 6 & -2 & -5 & 2 & -1 \\ -1 & 3 & 0 & -6 & 3 \\ 4 & 1 & -3 & -3 & 2 \\ 5 & 2 & 1 & -1 & -2 \end{bmatrix} \\ A_{[:,2]} \end{array}$$

giver

$$M_{12} = \begin{bmatrix} 2 & -1 & 4 & 5 \\ -1 & 3 & -6 & 3 \\ 4 & 1 & -3 & 2 \\ 5 & 2 & -1 & -2 \end{bmatrix}$$

I python kan dette undermatrix fås via `np.delete`:

```
>>> import numpy as np
>>> a = np.array([[2.0, -1.0, 3.0, 4.0, 5.0],
...               [6.0, -2.0, -5.0, 2.0, -1.0],
...               [-1.0, 3.0, 0.0, -6.0, 3.0],
...               [4.0, 1.0, -3.0, -3.0, 2.0],
...               [5.0, 2.0, 1.0, -1.0, -2.0]])

>>> m12 = np.delete(np.delete(a, 1, 0), 2, 1)
>>> m12
array([[ 2., -1.,  4.,  5.],
       [-1.,  3., -6.,  3.],
       [ 4.,  1., -3.,  2.],
       [ 5.,  2., -1., -2.]])
```

Her sletter `np.delete(a, 1, 0)` række `1`, og `np.delete(a, 2, 1)` sletter søjle `2`. Det er den sidste argument til `np.delete` der bestemmer om der er taler om en række eller en søjle, som skal slettes: `0` for rækker, `1` for søjler. △

Bemærk nu at for $n = 2$, har vi $M_{00} = \begin{bmatrix} a_{11} \end{bmatrix}$ og $M_{01} = \begin{bmatrix} a_{10} \end{bmatrix}$, så vi kan skrive

$$\det A = a_{00}a_{11} - a_{01}a_{10} = a_{00}\det(M_{00}) - a_{01}\det(M_{01}).$$

Definition 21.12. For A en $(n \times n)$-matrix, $n > 1$, sætter vi *determinanten* til at være

$$\det A = a_{00}\det(M_{00}) - a_{01}\det(M_{01}) + \cdots + (-1)^{n-1}a_{0,n-1}\det(M_{0,n-1}),$$

hvor fortegnene er skiftevis $+, -, +, -, \cdots$.

Eksempel 21.13. Lad os beregne determinanten af en (3×3)-matrix

$$\begin{aligned}
\det \begin{bmatrix} 1 & -1 & 2 \\ 3 & 2 & -2 \\ -1 & 4 & 5 \end{bmatrix} &= 1 \times \det \begin{bmatrix} 2 & -2 \\ 4 & 5 \end{bmatrix} - (-1) \times \det \begin{bmatrix} 3 & -2 \\ -1 & 5 \end{bmatrix} + 2 \times \det \begin{bmatrix} 3 & 2 \\ -1 & 4 \end{bmatrix} \\
&= 1 \times (10 - (-8)) + 1 \times (15 - 2) + 2 \times (12 - (-2)) \\
&= 1 \times 18 + 1 \times 13 + 2 \times 14 \\
&= 59.
\end{aligned}$$

△

Sætning 21.14. *For en kvadratisk matrix A, er A invertibel hvis og kun hvis* $\det A \neq 0$.

Vi vil ikke give et bevis for dette endnu, det gemmes til afsnit 23.5 især side 314. Men sætningen har den følgende konsekvens:

Proposition 21.15. *For en $(n \times n)$-matrix A, er λ en egenværdi for A hvis og kun hvis*

$$\det(A - \lambda I_n) = 0. \qquad \square$$

Polynomiet $p(\lambda) = \det(A - \lambda I_n)$ kaldes det *karakteristiske polynomium* for A.

21.5 Et andet eksempel

Lad os beregne egenværdier og egenvektorer for den følgende (3×3)-matrix

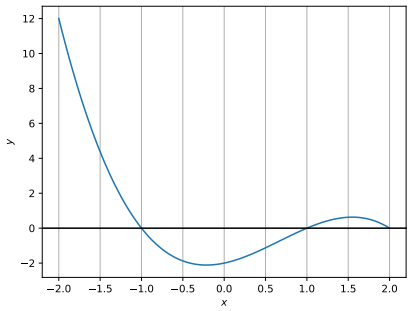
$$A = \begin{bmatrix} -3 & 4 & 2 \\ 1 & 0 & -1 \\ -6 & 6 & 5 \end{bmatrix}.$$

Vi finder først det karakteristiske polynomium

$$\begin{aligned}
\det(A - \lambda I_3) &= \det \begin{bmatrix} -3-\lambda & 4 & 2 \\ 1 & -\lambda & -1 \\ -6 & 6 & 5-\lambda \end{bmatrix} \\
&= (-3-\lambda) \det \begin{bmatrix} -\lambda & -1 \\ 6 & 5-\lambda \end{bmatrix} - 4 \det \begin{bmatrix} 1 & -1 \\ -6 & 5-\lambda \end{bmatrix} + 2 \det \begin{bmatrix} 1 & -\lambda \\ -6 & 6 \end{bmatrix} \\
&= (-3-\lambda)(\lambda^2 - 5\lambda + 6) - 4(5 - \lambda - 6) + 2(6 - 6\lambda) \\
&= -\lambda^3 + 2\lambda^2 + \lambda - 2.
\end{aligned}$$

Nu har vi et tredjegradspolynomium der skal løses. Dette er ikke så nemt: der findes en formel, men den er ikke helt lige til. Vi kan dog plotte funktionen

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-2, 2, 100)
fig, ax = plt.subplots()
```



Figur 21.1: Tredjegradspolynomium.

```
ax.plot(x, -x**3+2*x**2+x-2)
ax.grid(axis='x')
ax.axhline(0, color='black')    # tegn x aksen
ax.set_xlabel('$x$')
ax.set_ylabel('$y$')
```

Vi ser fra plottet i figur 21.1 at der er rødder tæt på $\lambda = -1$, 1 og 2. Vi har faktisk

$$\begin{aligned}\det(A - \lambda I_3) &= -\lambda^3 + 2\lambda^2 + \lambda - 2 \\ &= -(\lambda+1)(\lambda^2 - 3\lambda + 2) = -(\lambda+1)(\lambda-1)(\lambda-2),\end{aligned}$$

så rødderne er netop $\lambda_0 = -1$, $\lambda_1 = 1$ og $\lambda_2 = 2$.

Vi kan nu finde tilsvarende egenvektorer. For $\lambda_0 = -1$,

$$A - \lambda_0 I_3 = \begin{bmatrix} -2 & 4 & 2 \\ 1 & 1 & -1 \\ -6 & 6 & 6 \end{bmatrix} \sim_{R_0 \leftrightarrow R_1} \begin{bmatrix} 1 & 1 & -1 \\ -2 & 4 & 2 \\ -6 & 6 & 6 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 & -1 \\ 0 & 6 & 0 \\ 0 & 12 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

giver $v_0 = (1, 0, 1)$. For $\lambda = 1$,

$$A - \lambda_1 I_3 = \begin{bmatrix} -4 & 4 & 2 \\ 1 & -1 & -1 \\ -6 & 6 & 4 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & -1 \\ -4 & 4 & 2 \\ -6 & 6 & 4 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & -1 \\ 0 & 0 & -2 \\ 0 & 0 & -2 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & -1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

og $v_1 = (1, 1, 0)$. Til sidst for $\lambda_2 = 2$,

$$A - \lambda_2 I_3 = \begin{bmatrix} -5 & 4 & 2 \\ 1 & -2 & -1 \\ -6 & 6 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & -2 & -1 \\ -5 & 4 & 2 \\ -6 & 6 & 3 \end{bmatrix} \sim \begin{bmatrix} 1 & -2 & -1 \\ 0 & -6 & -3 \\ 0 & -6 & -3 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

og $v_2 = (0, 1, -2)$.

Det følger at

$$A = V\Lambda V^{-1}$$

med

$$V = \left[v_0 \mid v_1 \mid v_2\right] = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & -2 \end{bmatrix} \quad \text{og} \quad \Lambda = \begin{bmatrix} \lambda_0 & 0 & 0 \\ 0 & \lambda_1 & 0 \\ 0 & 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

Som det kan ses er denne fremgangsmåde ikke velegnet til generelle numeriske eksempler. Den største forhindring ligger i at bestemme egenværdierne. Vi vil finde numeriske metoder senere, som finder egenværdier og egenvektorer samtidigt. Faktisk er sådanne metode så effektive at en standard tilgang til bestemmelse af rødder af et polynomium $p(x)$ er:

(1) bestem en matrix A, som har karakterisk polynomium lige med $p(x)$,
(2) brug numeriske metoder til at bestemme egenværdierne for A,
(3) observér at disse egenværdier er de ønskede rødder af $p(x)$.

Se f.eks. bemærkningerne i the *NumPy documentation* fra the NumPy community 2023 efter beskrivelsen af funktionen `polynomial.polynomial.polyroots`.

21.6 Gentagende egenværdier

Hvis det karakteristiske polynomium $p(\lambda)$ har en rod, som gentager sig, kan vi ikke bruge proposition 21.5 direkte. Eksempel 21.10 har $p(\lambda) = (1-\lambda)^2$, og viser at man kan generelt ikke forventer at matricen bliver diagonaliserbar. Dog har vi den følgende variation på proposition 21.5, som vi giver for to forskellige egenværdier, men kan godt udvides til flere forskellige egenværdier.

Proposition 21.16. *Antag at λ og μ er to forskellige egenværdier for A. Hvis $v_0, \ldots, v_{r-1}$ er lineært uafhængige egenvektorer hørende til λ, og $w_0, \ldots, w_{s-1}$ er lineært uafhængige egenvektorer hørende til μ, så er samlingen*

$$v_0, \ldots, v_{r-1}, w_0, \ldots, w_{s-1}$$

lineært uafhængig.

Bevis. Vi betragter ligningen

$$x_0 v_0 + \cdots + x_{r-1} v_{r-1} + y_0 w_0 + \cdots + y_{s-1} w_{s-1} = 0 \tag{21.7}$$

og skal vise at $0 = x_0 = \cdots = x_{r-1} = y_0 = \cdots = y_{s-1}$.

Sæt

$$v = x_0 v_0 + \cdots + x_{r-1} v_{r-1} \quad \text{og} \quad w = y_0 w_0 + \cdots + y_{s-1} w_{s-1}.$$

Så har vi

$$\begin{aligned} Av &= A(x_0 v_0 + \cdots + x_{r-1} v_{r-1}) \\ &= x_0 A v_0 + \cdots + x_{r-1} A v_{r-1} = x_0 \lambda v_0 + \ldots x_{r-1} \lambda v_{r-1} \\ &= \lambda(x_0 v_0 + \cdots + x_{r-1} v_{r-1}) = \lambda v, \end{aligned}$$

og tilsvarende

$$Aw = \mu w.$$

Ligningen (21.7) siger

$$v + w = 0. \tag{21.8}$$

Ganges A på (21.8), fås

$$\lambda v + \mu w = 0. \tag{21.9}$$

Trækkes μ gange (21.8) fra (21.9), har vi

$$(\lambda - \mu) v = 0.$$

Men $\lambda \neq \mu$, så vi har $v = 0$. Da $v_0, \ldots, v_{r-1}$ er lineært uafhængig fås $0 = x_0 = \cdots = x_{r-1}$. (21.8) giver nu at $w = 0$, og $w_0, \ldots, w_{s-1}$ lineært uafhængig fører til $0 = y_0 = \cdots = y_{s-1}$, som ønsket. □

Eksempel 21.17. Dette eksempel er mere i stil med eksempel 21.4. Betragt matricen

$$A = \begin{bmatrix} 1 & 2 & -4 \\ 0 & 3 & -4 \\ 0 & 2 & -3 \end{bmatrix}.$$

Der påstås at $\lambda = -1$ og $\mu = 1$ er egenværdier for A og desuden at A har en basis af egenvektorer, så er diagonaliserbar.

Vi begynder med at lade efter egenvektorer med egenværdi $\lambda = -1$.

$$\begin{aligned}
A - \lambda I_3 &= A + I_3 \\
&= \begin{bmatrix} 2 & 2 & -4 \\ 0 & 4 & -4 \\ 0 & 2 & -2 \end{bmatrix} \sim_{R_0 \to (1/2)R_0} \begin{bmatrix} 1 & 1 & -2 \\ 0 & 4 & -4 \\ 0 & 2 & -2 \end{bmatrix} \\
&\sim_{R_2 \to (1/2)R_2} \begin{bmatrix} 1 & 1 & -2 \\ 0 & 4 & -4 \\ 0 & 1 & -1 \end{bmatrix} \sim_{R_1 \leftrightarrow R_2} \begin{bmatrix} 1 & 1 & -2 \\ 0 & 1 & -1 \\ 0 & 4 & -4 \end{bmatrix} \\
&\sim_{R_2 \to R_2 - 4R_1} \begin{bmatrix} 1 & 1 & -2 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \sim_{R_0 \to R_0 - R_1} \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix}.
\end{aligned}$$

Her har vi én fri variabel, så $\lambda = -1$ er en egenværdi, og vi finder at $v_0 = (1, 1, 1)$ er en tilhørende egenvektor.

Nu for $\mu = 1$, har vi

$$\begin{aligned}
A - \mu I_3 = A - I_3 &= \begin{bmatrix} 0 & 2 & -4 \\ 0 & 2 & -4 \\ 0 & 2 & -4 \end{bmatrix} \sim_{R_1 \to R_1 - R_0} \begin{bmatrix} 0 & 2 & -4 \\ 0 & 0 & 0 \\ 0 & 2 & -4 \end{bmatrix} \\
&\sim_{R_2 \to R_2 - R_0} \begin{bmatrix} 0 & 2 & -4 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \sim_{R_0 \to (1/2)R_0} \begin{bmatrix} 0 & 1 & -2 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.
\end{aligned}$$

Vi har nu to frie variabler, så $\mu = 1$ er en egenværdi. Vi finder desuden at $w_0 = (1, 0, 0)$ samt $w_1 = (0, 2, 1)$ er egenvektorer. Bemærk at w_0, w_1 er lineært uafhængig og udgør en basis for $N(A - \mu I_3)$.

Proposition 21.16 giver nu at v_0, w_0, w_1 er lineært uafhængig. Da denne samling består af netop 3 vektorer i $\mathbb{R}^3$, er v_0, w_0, w_1 en basis for $\mathbb{R}^3$, og A er dermed diagonaliserbar.

For at være mere konkret, sætter vi

$$V = \begin{bmatrix} v_0 \mid w_0 \mid w_1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 2 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{og} \quad \Lambda = \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \mu & 0 \\ 0 & 0 & \mu \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

har vi $A = V \Lambda V^{-1}$.

```
import numpy as np

a = np.array([[1.0, 2.0, -4.0],
              [0.0, 3.0, -4.0],
              [0.0, 2.0, -3.0]])

v = np.array([[1.0, 1.0, 0.0],
              [1.0, 0.0, 2.0],
              [1.0, 0.0, 1.0]])
mat_lambda = np.diag([-1.0, 1.0, 1.0])

print(v @ mat_lambda @ np.linalg.inv(v))
```

```
[[ 1.  2. -4.]
 [ 0.  3. -4.]
 [ 0.  2. -3.]]
```

```
machine_epsilon = np.finfo(float).eps
print(np.allclose(a, v @ mat_lambda @ np.linalg.inv(v),
                  rtol = machine_epsilon,
                  atol = machine_epsilon))
```

```
True
```

△

Kapitel 22

Differentialligninger

En væsentlig anvendelse af egenværdier og egenvektorer er til løsninger af lineære differentialligninger. Problemer for generelle lineære differentialligninger kan omskrives til lineære systemer af førsteordensdifferentialligninger, så det er disse systemer vi starter med.

22.1 Førsteordenssystemer

Lad os begynde med en konkret førsteordensdifferentialligning for én ukendt funktion $y(t)$:

$$y'(t) = ay(t), \tag{22.1}$$

hvor a er en given konstant. Funktionen $y(t) = ce^{at}$, for c konstant, har

$$y'(t) = \frac{d}{dt}(ce^{at}) = cae^{at} = ay(t),$$

så er en løsning til (22.1).

To ting skal bemærkes. Hvis $y(t) = ce^{at}$ og en startværdi $y(t_0) = r_0$ er givet, så er konstanten c entydigt bestemt af $ce^{at_0} = r_0$, dvs. $c = r_0e^{-at_0}$. Desuden hvis $z(t)$ er en anden løsning til (22.1), så kan vi betragte $w(t) = z(t)e^{-at}$, som opfylder

$$w'(t) = \frac{d}{dt}(z(t)e^{-at}) = z'(t)e^{-at} - az(t)e^{-at} = (z'(t) - az(t))e^{-at} = 0$$

for alle t. Det følger at $w(t)$ er konstant, så $z(t)e^{-at} = w(t) = c$ for en konstant c. Vi har dermed at $z(t) = ce^{at}$ for en konstant c, så er $z(t)$ nødvendigvis en af de løsninger vi har allerede skrevet ned. Vi har dermed det følgende resultat:

Proposition 22.1. *Alle løsninger $t \mapsto y(t) \in \mathbb{R}$ til differentialligningen (22.1) har formen $y(t) = ce^{at}$ med c konstant. Løsningerne er entydigt bestemt af en begyndelsesværdi $y(t_0) = r_0$.* □

Definition 22.2. Et *førsteordenssystem af lineære differentialligninger* er et system af formen

$$\begin{aligned}
y_0'(t) &= a_{00}y_0(t) + a_{01}y_1(t) + \cdots + a_{0,n-1}y_{n-1}(t),\\
y_1'(t) &= a_{10}y_0(t) + a_{11}y_1(t) + \cdots + a_{1,n-1}y_{n-1}(t),\\
&\vdots\\
y_{n-1}'(t) &= a_{n-1,0}y_0(t) + a_{n-1,1}y_1(t) + \cdots + a_{n-1,n-1}y_{n-1}(t),
\end{aligned}$$

med $A = (a_{ij}) \in \mathbb{R}^{n\times n}$ en kvadratisk matrix af konstanter. I systemet er der n ukendte funktioner $y_i(t)$, $i = 0, 1, \ldots, n-1$.

Systemet kan skrives i vektornotation ved at sætte $y(t) = (y_0(t), \ldots, y_{n-1}(t))$. Vi får

$$y'(t) = Ay(t). \tag{22.2}$$

Dette system er lineær, da for løsninger $y(t)$, $z(t)$ og konstanter a, b har vi at $ay(t) + bz(t)$ er også en løsning, jævnfør

$$(ay(t) + bz(t))' = ay'(t) + bz'(t) = aAy(t) + bAz(t) = A(ay(t) + bz(t)).$$

Hvis $A = \operatorname{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{n-1})$ er en diagonalmatrix, er systemet blot

$$\begin{aligned}
y_0'(t) &= \lambda_0 y_0(t),\\
y_1'(t) &= \lambda_1 y_1(t),\\
&\vdots\\
y_{n-1}'(t) &= \lambda_{n-1} y_{n-1}(t),
\end{aligned}$$

som er n kopier af (22.1). Det følger at løsningen i dette tilfælde er

$$\begin{aligned}
\begin{bmatrix} y_0(t) \\ y_1(t) \\ \vdots \\ y_{n-1}(t) \end{bmatrix} &= \begin{bmatrix} c_0 e^{\lambda_0 t} \\ c_1 e^{\lambda_1 t} \\ \vdots \\ c_{n-1} e^{\lambda_{n-1} t} \end{bmatrix} \\
&= c_0 \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} e^{\lambda_0 t} + c_1 \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} e^{\lambda_1 t} + \cdots + c_{n-1} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix} e^{\lambda_{n-1} t} \\
&= c_0 e_0 e^{\lambda_0 t} + c_1 e_1 e^{\lambda_1 t} + \cdots + c_{n-1} e_{n-1} e^{\lambda_{n-1} t},
\end{aligned}$$

for konstante $c_0, c_1, \ldots, c_{n-1}$. Vektorerne i resultatet er standardbasisvektorerne $e_0, \ldots, e_{n-1}$. Bemærk at for denne diagonalmatrix $A = \operatorname{diag}(\lambda_0, \lambda_1, \ldots, \lambda_{n-1})$ opfylder vektorerne e_i at $Ae_i = \lambda_i e_i$, dvs. de er egenvektorer for A.

Generelt hvis $v \in \mathbb{R}^n$ er en egenvektor for A med $Av = \lambda v$, har vi at $y(t) = cve^{\lambda t}$ løser (22.2):

$$\begin{aligned}
y'(t) &= \frac{d}{dt}(cve^{\lambda t}) = cv\frac{d}{dt}e^{\lambda t} = cv\lambda e^{\lambda t} \\
&= ce^{\lambda t}\lambda v = ce^{\lambda t} Av = A(cve^{\lambda t}) \\
&= Ay(t).
\end{aligned}$$

Fra linearitet følger det, at hvis $v_0, v_1, \ldots, v_{k-1}$ er egenvektorer for A med egenværdier $\lambda_0, \lambda_1, \ldots, \lambda_{k-1}$, så er

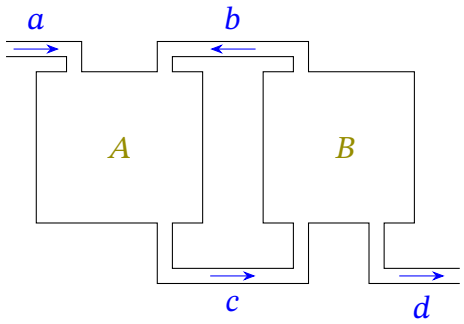
$$y(t) = c_0 v_0 e^{\lambda_0 t} + c_1 v_1 e^{\lambda_1 t} + \cdots + c_{k-1} v_{k-1} e^{\lambda_{k-1} t},$$

med c_i konstant, en løsning til (22.2).

Proposition 22.3. *Hvis A har en basis bestående af egenvektorer $v_0, v_1, \ldots, v_{n-1}$ med egenværdier $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$, så er alle løsninger til (22.2) af formen*

$$y(t) = c_0 v_0 e^{\lambda_0 t} + c_1 v_1 e^{\lambda_1 t} + \cdots + c_{n-1} v_{n-1} e^{\lambda_{n-1} t},$$

med c_i konstant. Givet en vektor af begyndelsesværdier $y(t_0) = r \in \mathbb{R}^n$, har (22.2) en entydig løsning med disse begyndelsesværdier.



Figur 22.1: Beholdere forbudne med rør.

Bevis. Ligningssystemet er $y' = Ay = V\Lambda V^{-1}y$, som er ækvivalent med $(V^{-1}y)' = \Lambda(V^{-1}y)$, da V er en konstant matrix. Vi har desuden $(V^{-1}y)(t_0) = V^{-1}r$. Skrives $z(t) = V^{-1}y(t)$ har vi derfor at systemet er ækvivalent med $z' = \Lambda z$, med begyndelsesvektor $z(t_0) = V^{-1}r$. Da Λ er diagonal, har vi set ovenfor at eksistens og entydighed for begyndelsesværdiproblemet for $z(t)$ følger nu fra tilfældet for $n = 1$, dvs. det følger fra proposition 22.1. Men $z(t)$ bestemmer $y(t) = Vz(t)$, så vi har eksistens og entydighed for $y(t)$. □

Eksempel 22.4. To beholdere A og B er forbundet med rør, som i figur 22.1. Som udgangspunkt indeholder A 200 L vand med 60 g salt, og B inderholder 200 L rent vand. Vandtilstanden udvikler sig ved at der tilføjes 15 L/min rent vand via rør a, der flyder 5 L/min blanding igennem rør b i den anviste retning, og der flyder 20 L/min igennem rør c. Igennem rør d forlader 15 L/min saltvand. Hvor meget salt er der i hver beholder efter tid t?

Vi sætter $y_0(t)$ til at være mængden af salt i A ved tid t min (minutter), og $y_1(t)$ til mængden af salt i B. For beholder A mistes der $20(y_0(t)/200) = y_0(t)/10$ gram salt per minut, og der tilføjes $5(y_1(t)/200) = y_1(t)/40$ gram salt per minut. Så

$$y_0'(t) = -\frac{1}{10}y_0(t) + \frac{1}{40}y_1(t).$$

For beholder B har vi

$$y_1'(t) = \frac{1}{10}y_0(t) - \frac{5+15}{200}y_1(t) = \frac{1}{10}y_0(t) - \frac{1}{10}y_1(t).$$

Så systemet er

$$y'(t) = Ay, \qquad A = \begin{bmatrix} -\frac{1}{10} & \frac{1}{40} \\ \frac{1}{10} & -\frac{1}{10} \end{bmatrix},$$

med begyndelsesværdi

$$y(0) = \begin{bmatrix} 60 \\ 0 \end{bmatrix}.$$

Nu vi vil løse systemet. Det karakteristiske polynomium for A er

$$\det \begin{bmatrix} -\frac{1}{10} - \lambda & \frac{1}{40} \\ \frac{1}{10} & -\frac{1}{10} - \lambda \end{bmatrix} = \left(-\frac{1}{10} - \lambda\right)^2 - \frac{1}{10}\frac{1}{40} = \lambda^2 + \frac{1}{5}\lambda + \frac{3}{400},$$

som har rødder

$$\frac{-(1/5) \pm \sqrt{(1/25) - (3/100)}}{2} = \frac{-(1/5) \pm (1/10)}{2} = -\frac{1}{20}, -\frac{3}{20}$$

Matricen A er en (2×2)-matrix, og der er to forskellige rødder, så A er diagonaliserbar.

For $\lambda_0 = -1/20$ har vi

$$\begin{bmatrix} -\frac{1}{10} - \lambda_0 & \frac{1}{40} \\ \frac{1}{10} & -\frac{1}{10} - \lambda_0 \end{bmatrix} = \begin{bmatrix} -\frac{1}{20} & \frac{1}{40} \\ \frac{1}{10} & -\frac{1}{20} \end{bmatrix} \sim \begin{bmatrix} -2 & 1 \\ 2 & -1 \end{bmatrix} \sim \begin{bmatrix} 2 & -1 \\ 0 & 0 \end{bmatrix}$$

så en egenvektor er $v_0 = (1, 2)$. For $\lambda_1 = -3/20$ har vi

$$\begin{bmatrix} -\frac{1}{10} - \lambda_1 & \frac{1}{40} \\ \frac{1}{10} & -\frac{1}{10} - \lambda_1 \end{bmatrix} = \begin{bmatrix} \frac{1}{20} & \frac{1}{40} \\ \frac{1}{10} & \frac{1}{20} \end{bmatrix} \sim \begin{bmatrix} 2 & 1 \\ 0 & 0 \end{bmatrix}$$

og en egenvektor er $v_1 = (1, -2)$. Proposition 22.3 fortæller os så at løsningen har formen

$$y(t) = c_0 \begin{bmatrix} 1 \\ 2 \end{bmatrix} e^{-t/20} + c_1 \begin{bmatrix} 1 \\ -2 \end{bmatrix} e^{-3t/20}.$$

Konstanterne c_0 og c_1 kan bestemmes ud fra vores begyndelsesdata. Ved $t = 0$ har vi

$$y(0) = c_0 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + c_1 \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix} = \begin{bmatrix} 60 \\ 0 \end{bmatrix},$$

som har løsning $c_0 = c_1 = 30$. Så saltmængderne er givet ved

$$y(t) = \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix} = 30 \begin{bmatrix} 1 \\ 2 \end{bmatrix} e^{-t/20} + 30 \begin{bmatrix} 1 \\ -2 \end{bmatrix} e^{-3t/20} = \begin{bmatrix} 30e^{-t/20} + 30e^{-3t/20} \\ 60e^{-t/20} - 60e^{-3t/20} \end{bmatrix}.$$

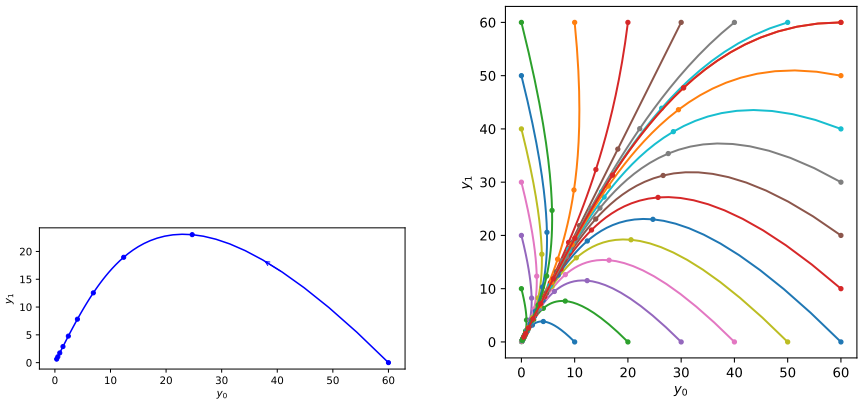
Da $e^{-t} \to 0$ for $t \to \infty$, ser vi at løsningen går mod 0 når t vokser. Men $e^{-t/20}$ går langsommere mod 0 end $e^{-3t/20}$, så forholdet mellem $y_0(t)$ og $y_1(t)$ nærmer sig fordeling fra indgangene i v_0, dvs. 1 til 2.

Vi kan plotte $(y_0(t), y_1(t))$, og får figuren til venstre i figur 22.2. Punkterne bevæger sig langs kurven fra højre til venstre, og vi ser at kurvens tangentretning nærmer sig v_0 efterhånden.

```
import matplotlib.pyplot as plt
import numpy as np

v0 = np.array([1.0, 2.0])[:, np.newaxis]
v1 = np.array([1.0, -2.0])[:, np.newaxis]
lambda0 = -1.0 / 20.0
lambda1 = -3.0 / 20.0

t = np.linspace(0, 100, 100)
løsning = (30 * v0 * np.exp(lambda0 * t)
           + 30 * v1 * np.exp(lambda1 * t))

fig, ax = plt.subplots()
ax.set_aspect('equal')
ax.plot(*løsning, color='b',
        marker='o', markevery=10, markersize=4)
ax.annotate("", xy=løsning[:,5], xytext=løsning[:,4],
            arrowprops=dict(arrowstyle='->', color='b'))
ax.set_xlabel('$y_{0}$')
ax.set_ylabel('$y_{1}$')
```

Vi kan også lave plots med andre startværdier.

```
v = np.hstack([v0, v1])

def sol(r0, r1, v, lambda0, lambda1, t):
```



Figur 22.2: Saltmængden i beholder A mod beholder B.

```
    c = np.linalg.solve(v, np.array([r0, r1])[:, np.newaxis])
    return (c[0] * v[:, [0]] * np.exp(lambda0 * t)
            + c[1] * v[:, [1]] * np.exp(lambda1 * t))

t = np.linspace(0, 100, 100)

marks = dict(marker='o', markevery=10, markersize=3)

fig, ax = plt.subplots()
ax.set_aspect('equal')
for s in np.linspace(10, 60, 6):
    ax.plot(*sol(s, 0, v, lambda0, lambda1, t), **marks)
    ax.plot(*sol(s, 60, v, lambda0, lambda1, t), **marks)
for s in np.linspace(10, 60, 6):
    ax.plot(*sol(0, s, v, lambda0, lambda1, t), **marks)
    ax.plot(*sol(60, s, v, lambda0, lambda1, t), **marks)
ax.set_xlabel('$y_{0}$')
ax.set_ylabel('$y_{1}$')
```

Disse visses til højre i figur 22.2. △

22.2 Komplekse egenværdier

Selvom A har reelle indgange, kan egenværdierne godt være komplekse tal. Det har vi set i det forrige kapitel. Vi vil dog gerne følge metoden ovenfor for at løse differentialligninger. Så vi begynder med at kikke på ligning (22.1) i det komplekse tilfælde. Hvis $\lambda = a + ib$, $a, b \in \mathbb{R}$, har vi brug for funktioner $t \mapsto y(t) \in \mathbb{C}$, der opfylder

$$y'(t) = \lambda y(t) = (a + ib)y(t).$$

Heldigvis findes sådanne funktioner. Betragt

$$y(t) = e^{at}(\cos(bt) + i\sin(bt)).$$

Vi har

$$\begin{aligned}
y'(t) &= \frac{d}{dt}\left(e^{at}(\cos(bt) + i\sin(bt))\right) \\
&= \left(\frac{d}{dt}(e^{at})\right)(\cos(bt) + i\sin(bt)) + e^{at}\frac{d}{dt}(\cos(bt) + i\sin(bt)) \\
&= ae^{at}(\cos(bt) + i\sin(bt)) + e^{at}(-b\sin(bt) + ib\cos(bt)) \\
&= ae^{at}(\cos(bt) + i\sin(bt)) + ibe^{at}(\cos(bt) + i\sin(bt)) \\
&= (a + ib)e^{at}(\cos(bt) + i\sin(bt)) \\
&= (a + ib)y(t),
\end{aligned}$$

som er netop det vi ønsker.

Vi definerer $e^{\lambda t}$ for kompleks $\lambda = a + ib$ til at være

$$e^{\lambda t} = e^{at}(\cos(bt) + i\sin(bt)),$$

og kalder dette *den komplekse eksponentialfunktion*.

Vi har lige vist at

$$\frac{d}{dt}e^{\lambda t} = \lambda e^{\lambda t}.$$

Ved hjælp af standard trigonometriske identiteter, som findes i afsnit 9.6, kan man desuden tjekke at

$$e^{\lambda+\mu} = e^{\lambda}e^{\mu}$$

for komplekse tal λ og μ. Så denne eksponentialfunktion for komplekse tal har mange af de samme egenskaber, som vi kender for eksponentialfunktionen for reelle tal.

Bemærk at vi har nu den smukke *Eulers identitet*

$$e^{i\pi} = -1,$$

da $e^{i\pi} = \cos(\pi) + i\sin(\pi) = -1 + i0 = -1$.

Hvis $A \in \mathbb{R}^{n\times n}$ er en reel matrix, der har en kompleks egenværdi $\lambda = a+ib \in \mathbb{C}$ med egenvektor $v \in \mathbb{C}^n$, har vi $Av = \lambda v$. Det følger at $\overline{\lambda} = a - ib$ er også en egenværdi for A over $\mathbb{C}$ med egenvektor $\overline{v}$, da $\overline{A} = A$ giver

$$A\overline{v} = \overline{A}\overline{v} = \overline{Av} = \overline{\lambda v} = \overline{\lambda}\overline{v}.$$

Så opfylder både $e^{\lambda t}v$ og $e^{\overline{\lambda}t}\overline{v}$ systemet $y' = Ay$. De er dog løsninger med værdier i $\mathbb{C}^n$.

For at få reelle løsninger kan vi skrive $\lambda = a+ib$, med $a, b \in \mathbb{R}$, og tilsvarende skrive $v = w + iz$, med $w, z \in \mathbb{R}^n$. De følgende to lineære kombinationer af løsningerne $e^{\lambda t}v$ og $e^{\overline{\lambda}t}\overline{v}$,

$$\begin{aligned} u_0(t) &= \frac{1}{2}(e^{\lambda t}v + e^{\overline{\lambda}t}\overline{v}) = \mathrm{Re}(e^{\lambda t}v), \\ u_1(t) &= \frac{1}{2i}(e^{\lambda t}v - e^{\overline{\lambda}t}\overline{v}) = \mathrm{Im}(e^{\lambda t}v), \end{aligned}$$

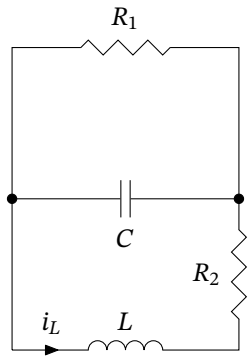
er løsninger til $y' = Ay$ i $\mathbb{R}^n$. Ved at bruge $\lambda = a + ib$ og $v = w + iz$, har vi $e^{\lambda t}v = e^{at}(\cos(bt) + i\sin(bt))(w+iz)$ og $e^{\overline{\lambda}t}\overline{v} = e^{at}(\cos(bt) - i\sin(bt))(w - iz)$, og vi finder at

$$\begin{aligned} u_0(t) &= e^{at}(\cos(bt)w - \sin(bt)z), \\ u_1(t) &= e^{at}(\sin(bt)w + \cos(bt)z). \end{aligned}$$

Eksempel 22.5. Betragt det elektriske kredsløb i figur 22.3 med $R_1 = 5\,\Omega$, $R_2 = 0{,}8\,\Omega$, $L = 0{,}4\,\mathrm{H}$, $C = 0{,}1\,\mathrm{F}$. Ved brug af Kirchhoffs love, eksempel 4.11, og egenskaber af de enkelte komponenter, eksempel 18.6, finder man at den elektriske strøm i_L igennem induktionsspolen L og spændingsfaldet v_C over kondensatoren C opfylder systemet

$$\begin{bmatrix} i_L'(t) \\ v_C'(t) \end{bmatrix} = \begin{bmatrix} -R_2/L & -1/L \\ 1/C & -1/(R_1C) \end{bmatrix} \begin{bmatrix} i_L(t) \\ v_C(t) \end{bmatrix}.$$

Sættes $y(t) = (i_L(t), v_C(t))$, har vi



Figur 22.3: Elektrisk kredsløb.

$$y'(t) = Ay(t), \qquad \text{med} \quad A = \begin{bmatrix} -2{,}0 & -2{,}5 \\ 10{,}0 & -2{,}0 \end{bmatrix}.$$

Det karakteristiske polynomium for A, $p(\lambda) = \det(A - \lambda I_2) = a\lambda^2 + b\lambda + c$, har koefficienter

```
a = 1.0
b = -2 * (-2.0)
c = (-2.0) * (-2.0) - (-2.5) * (10.0)
print(a, b, c)
```

```
1.0 4.0 29.0
```

Den tilsvarende diskriminant Δ er

```
diskriminant = b**2 - 4 * a * c
print(diskriminant)
```

```
-100.0
```

som er negative. Det følger at en kompleks egenværdi er $(-b + i\sqrt{-\Delta})/(2a)$:

```
lambda_c = (-b + 1.0j * np.sqrt(-diskriminant)) / (2 * a)
print(lambda_c)
```

```
(-2+5j)
```

Dvs. $\lambda = -2 + 5i$. Vi finder en tilsvarende egenvektor af formen $v = (q, 1)$ ved at løse ligningen fra den første række i $A - \lambda I_2 = 0$, dvs. $(a_{00} - \lambda)q + a_{01}1 = 0$:

```
a_mat = np.array([[-2.0, -2.5],
                  [10.0, -2.0]])
a_mat_c = a_mat - lambda_c * np.eye(2)

print('A - lambda_c I_2')
print(a_mat_c)

v_c = np.array([-a_mat_c[0,1] / a_mat_c[0,0],
                1.0])[:, np.newaxis]

print('Kompleks egenvektor v_c')
print(v_c)

print('Tjek: (A - lambda_c I_2)v_c = 0')
print(np.all(a_mat_c @ v_c == np.zeros_like(v_c)))
```

```
A - lambda_c I_2
[[ 0. -5.j -2.5+0.j]
 [10. +0.j  0. -5.j]]
Kompleks egenvektor v_c
[[0.+0.5j]
 [1.+0.j ]]
Tjek: (A - lambda_c I_2)v_c = 0
True
```

Dvs. $v = (i/2, 1)$. Dette giver de følgende komplekse løsninger

$$ve^{\lambda t} = \begin{bmatrix} i/2 \\ 1 \end{bmatrix} e^{(-2+5i)t}, \quad \overline{v}e^{\overline{\lambda}t} = \begin{bmatrix} -i/2 \\ 1 \end{bmatrix} e^{(-2-5i)t}.$$

Da

$$v = \begin{bmatrix} i/2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} + i \begin{bmatrix} 1/2 \\ 0 \end{bmatrix} =: w + iz,$$

har vi

$$w = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \text{og} \quad z = \begin{bmatrix} 1/2 \\ 0 \end{bmatrix}$$

og de tilsvarende reelle løsninger er

$$u_0(t) = e^{-2t}\left(\cos(5t)\begin{bmatrix} 0 \\ 1 \end{bmatrix} - \sin(5t)\begin{bmatrix} 1/2 \\ 0 \end{bmatrix}\right) = e^{-2t}\begin{bmatrix} -(1/2)\sin(5t) \\ \cos(5t) \end{bmatrix},$$
$$u_1(t) = e^{-2t}\left(\sin(5t)\begin{bmatrix} 0 \\ 1 \end{bmatrix} + \cos(5t)\begin{bmatrix} 1/2 \\ 0 \end{bmatrix}\right) = e^{-2t}\begin{bmatrix} (1/2)\cos(5t) \\ \sin(5t) \end{bmatrix}.$$

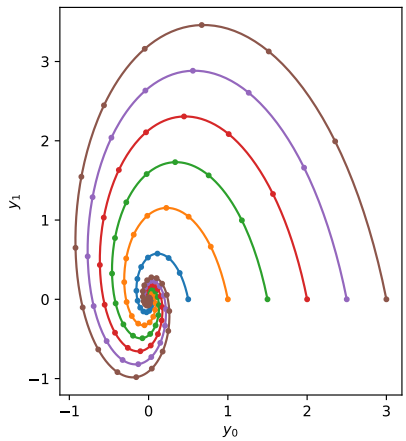
Den generelle løsning til systemet er så $y(t) = c_0 u_0(t) + c_1 u_1(t)$.

Givet begyndelsesværdierne $y(0) = (i_L(0), v_C(0))$, kan vi løse

$$y(0) = c_0 \begin{bmatrix} 0 \\ 1 \end{bmatrix} + c_1 \begin{bmatrix} 1/2 \\ 0 \end{bmatrix}$$

for c_0, c_1. Vi plotter disse løsninger som før med forskellige startværdier

```
def sol_c(r0, r1, v_c, lambda_c, t):
    w = v_c.real
    z = v_c.imag
    v = np.hstack([w, z])
    a = lambda_c.real
    b = lambda_c.imag
    r = np.array([r0, r1])[:, np.newaxis]
    c = np.linalg.solve(v, r)
    return (c[0] * np.exp(a*t)
            * (np.cos(b*t) * w - np.sin(b*t) * z)
            + c[1] * np.exp(a*t)
              * (np.sin(b*t) * w + np.cos(b*t) * z))

t = np.linspace(0, 4, 1000)

marks = dict(marker='o', markevery=20, markersize=3)

fig, ax = plt.subplots()
ax.set_aspect('equal')
for s in np.linspace(0.5, 3, 6):
    ax.plot(*sol_c(s, 0, v_c, lambda_c, t), **marks)
```



Figur 22.4: Løsningen til kredsløb.

```
ax.set_xlabel('$y_{0}$')
ax.set_ylabel('$y_{1}$')
```

△

22.3 Højere ordens lineære differentialligninger

Højere ordens differentiallinger kan omskrives til systemer af førsteordensligninger. Lad os demonstrere dette for lineære differentialligninger af orden 2.

Betragt

$$y''(t) + ay'(t) + by(t) = 0, \qquad y(t) \in \mathbb{R}. \tag{22.3}$$

Vi indfører nye funktioner $y_0(t)$ og $y_1(t)$ givet ved

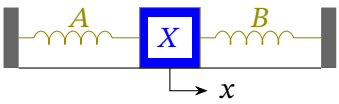
$$y_0(t) = y(t), \quad y_1(t) = y'(t).$$

Ligning (22.3) kan nu omskrives ved hjælp af $y_0(t)$, $y_1(t)$ og den første ordens afledede $y_1'(t)$:

$$y_1'(t) + ay_1(t) + by_0(t) = 0. \tag{22.4}$$

Husker vi definitionen på $y_1(t)$, får vi også en ligning for den første ordens afledede $y_0'(t)$:

$$y_0'(t) = y'(t) = y_1(t). \tag{22.5}$$



Figur 22.5: System med klods og fjedre.

Vi samler ligninger (22.4) og (22.5) som et førsteordenssystem

$$\begin{aligned} y_0'(t) &= y_1(t), \\ y_1'(t) &= -by_0(t) - ay_1(t). \end{aligned}$$

I matrixform er dette system

$$\begin{bmatrix} y_0'(t) \\ y_1'(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -b & -a \end{bmatrix} \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix}. \tag{22.6}$$

Sådan et system kan nu løses ved hjælp af de metoder vi indførte tidligere i kapitlet.

Bemærk at en begyndelsesværdi for systemet (22.6) er en angivelse af $y_0(t_0) = y(t_0)$ og $y_1(t_0) = y'(t_0)$, dvs. en angivelse af y og dens første ordens afledede i tid t_0. Derfor forventer vi at løsninger er entydigt bestemt af sådanne begyndelsesværdier.

Eksempel 22.6. Betragt figur 22.5 hvor en klods X af masse m er forbundet med fjedre A og B med fjederkonstanter k_A og k_B. Klodsen kan bevæge til venstre og til højre. Vi angiver dens position ved et vandret koordinat x_0, som er 0 når X er i dens ligevægtsstilling, og er positiv når X bevæger sig til højre.

Bevægelsesligningerne følger fra Newtons anden lov, som siger

$$\text{masse} \times \text{acceleration} = \text{kraft}.$$

I følge Hooks lov bidrager fjeder A med en ekstra kraft på $-k_A x(t)$: minus fortegnet skyldes at $x(t)$ måles mod højre, men sammentrækningskraften for fjeder A er mod venstre for $x(t)$ positiv. Kraftens størrelse er fjederkonstant k_A ganget på afvigelsen fra ligevægtens længde af fjederen A. Ligeledes bidrager fjeder B med en ekstra kraft $-k_B x(t)$: for $x(t)$ positiv, skubber fjedren klodsen til venstre. I alt får vi

$$mx''(t) = -k_A x(t) - k_B x(t) = -(k_A + k_B)x(t).$$

Lad os tage $m = 1$, $k_A = k_B = 1$. Vi sætter $y_0(t) = x(t)$ og $y_1(t) = x'(t)$ og får

$$\begin{aligned} y_0'(t) &= x'(t) = y_1(t), \\ y_1'(t) &= x''(t) = -2x(t) = -2y_0(t). \end{aligned}$$

Så har vi et førsteordenssystem

$$\begin{bmatrix} y_0'(t) \\ y_1'(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -2 & 0 \end{bmatrix} \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix},$$

dvs.

$$\begin{bmatrix} y_0'(t) \\ y_1'(t) \end{bmatrix} = A \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix} \qquad \text{med } A = \begin{bmatrix} 0 & 1 \\ -2 & 0 \end{bmatrix}.$$

For at løse systemet, beregner vi først det karakteristiske polynomium for koefficientmatricen A:

$$\det(A - \lambda I_2) = \det \begin{bmatrix} -\lambda & 1 \\ -2 & -\lambda \end{bmatrix} = \lambda^2 + 2.$$

Polynomiet har rødder $\pm i\sqrt{2} \in \mathbb{C}$. Nu finder vi en kompleks egenvektor v for $\lambda = i\sqrt{2}$:

$$\begin{bmatrix} -\lambda & 1 \\ -2 & -\lambda \end{bmatrix} = \begin{bmatrix} -i\sqrt{2} & 1 \\ -2 & -i\sqrt{2} \end{bmatrix} \sim \begin{bmatrix} -i\sqrt{2} & 1 \\ 0 & 0 \end{bmatrix},$$

giver en egenvektor $v = (1, i\sqrt{2}) \in \mathbb{C}^2$. Vi får en kompleks løsning $e^{\lambda t}v = e^{i\sqrt{2}t}(1, i\sqrt{2}) = (\cos(\sqrt{2}t) + i\sin(\sqrt{2}t))(1, i\sqrt{2})$. Fra afsnit 22.2 har vi at den generelle løsning er nu en lineær kombination af de reelle og imaginære del af den komplekse løsning:

$$\begin{aligned} \begin{bmatrix} x(t) \\ x'(t) \end{bmatrix} &= \begin{bmatrix} y_0(t) \\ y_1(t) \end{bmatrix} \\ &= c_0\big(\cos(\sqrt{2}t)\operatorname{Re}(v) - \sin(\sqrt{2}t)\operatorname{Im}(v)\big) \\ &\qquad + c_1\big(\sin(\sqrt{2}t)\operatorname{Re}(v) + \cos(\sqrt{2}t)\operatorname{Im}(v)\big) \\ &= \begin{bmatrix} c_0\cos(\sqrt{2}t) + c_1\sin(\sqrt{2}t) \\ -c_0\sin(\sqrt{2}t)\sqrt{2} + c_1\cos(\sqrt{2}t)\sqrt{2} \end{bmatrix}. \end{aligned}$$

Så vi har

$$x(t) = c_0\cos(\sqrt{2}t) + c_1\sin(\sqrt{2}t),$$

dvs. en svingning med svingningstid $2\pi/\sqrt{2} \approx 4{,}44\,\text{s}$. △

Kapitel 23

Matrixfaktorisering fra egenværdier

Egenværdier og egenvektorer giver anledning til forskellige matrixfaktoriseringer. Her vil vi kikke på nogle enkle betingelser der garanterer eksistens af sådanne faktoriseringer.

23.1 Eksistens af egenværdier

Polynomiet

$$p(x) = x^2 + 1$$

har ingen rødder i $\mathbb{R}$. Men tillader vi komplekse tal så har den to rødder $i, -i$ i $\mathbb{C}$. At man har bedre kontrol over eksistens af rødder over komplekse tal er indholdet i:

Sætning 23.1 (Algebraens fundamentalsætning)**.** *Ethvert polynomium* $p(x) = a_0 + a_1 x + \cdots + a_n x^n$ *har en rod i* $\mathbb{C}$. □

Givet en rod z_0, kan vi så skrive $p(x) = (x - z_0)q(x)$ med $q(x)$ er polynomium af grad $n - 1$ og anvender sætningen igen på $q(x)$. Vi får til sidst at $p(x)$ kan faktoriseres, som

$$p(x) = a_n(x - z_0)(x - z_1)\dots(x - z_{n-1})$$

for nogle $z_0, z_1, \dots, z_{n-1} \in \mathbb{C}$.

Egenværdier er netop rødder til det karakteristiske polynomium $p(\lambda) = \det(A - \lambda I_n)$, så vi får som konsekvens af sætning 23.1 at

Proposition 23.2. *Enhver matrix $A \in \mathbb{C}^{n\times n}$ har en egenværdi.* □

Dette siger for enhver $A \in \mathbb{C}^{n\times n}$ findes der en $\lambda \in \mathbb{C}$ og $v \in \mathbb{C}^n$, med $v \neq 0$, således at

$$Av = \lambda v.$$

Bemærk at vi kan altid erstatte v med $v/\|v\|_2$, og får dermed en egenvektor af længde 1, skulle det ønskes.

23.2 Determinant og egenværdier

Lad os notere nogle egenskaber af determinanten og giver nogle simple konsekvenser for bestemmelse af egenværdier. For det første opfører determinanten sig godt i forhold til matrixmultiplikation.

Sætning 23.3. *For A og B to $(n \times n)$-matricer gælder*

$$\det(AB) = \det(A)\det(B).$$ □

Vi giver et bevis for sætningen senere, se afsnit 23.5, side 314. Her er en vigtig anvendelse.

Proposition 23.4. *For $A = VBV^{-1}$ har A og B det samme karakteristiske polynomium, og så de samme egenværdier.*

Bevis. Bemærk at sætning 23.3 giver

$$\det(AB) = \det(A)\det(B) = \det(B)\det(A) = \det(BA).$$

Det følger at

$$\begin{aligned}\det(A - \lambda I_n) &= \det(VBV^{-1} - \lambda I_n) = \det(VBV^{-1} - V\lambda I_n V^{-1})\\ &= \det(V(B - \lambda I_n)V^{-1}) = \det((B - \lambda I_n)V^{-1}V)\\ &= \det(B - \lambda I_n),\end{aligned}$$

dvs. at A og B har det samme karakteristiske polynomium. Da egenværdier kan fås som netop rødder af det karakteristiske polynomium, får vi det sidste påstand. □

Proposition 23.5. *For en øvre triangulær matrix*

$$B = \begin{bmatrix} \lambda_0 & * & \dots & * \\ 0 & \lambda_1 & \dots & * \\ \vdots & & \ddots & \vdots \\ 0 & 0 & & \lambda_{n-1} \end{bmatrix}$$

er

$$\det(B) = \lambda_0 \lambda_1 \dots \lambda_{n-1}$$

og $\lambda_0, \lambda_1, \dots, \lambda_{n-1}$ *er egenværdierne af* B.

Igen giver vi beviset senere, se afsnit 23.5, side 310. Lad os kikke på et eksempel.

Eksempel 23.6. Matricen

$$A = \begin{bmatrix} 0{,}8 & 2{,}6 \\ -8{,}4 & 10{,}2 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 6 & -10 \\ 0 & 5 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}^{-1}$$

har samme egenværdier som

$$B = \begin{bmatrix} 6 & -10 \\ 0 & 5 \end{bmatrix}$$

Men B er øvre triangulær, så egenværdierne i B er blot dens diagonale indgange. Dvs. $\lambda_0 = 6$ og $\lambda_1 = 5$. △

23.3 Unitære matricer og Schurdekomponeringen

Det standard indre produkt i $\mathbb{C}^n$ er

$$\langle u, v \rangle = \overline{v}^T u.$$

Som analog til ortogonale matricer i $\mathbb{R}^{n \times n}$ har vi følgende definition over $\mathbb{C}$.

Definition 23.7. En matrix $U \in \mathbb{C}^{n \times n}$ er *unitær* hvis

$$\overline{U}^T U = I_n.$$

Bemærk at U er unitær hvis og kun hvis U er invertibel med

$$U^{-1} = \overline{U}^T.$$

Det følger fra sætning 7.2 at $U\overline{U}^T = I_n$ også.

For generel U, kan vi skrive $U = \left[u_0 \mid u_1 \mid \dots \mid u_{n-1}\right]$ via dens søjler, og danne en kompleks Grammatrix:

$$\overline{U}^T U = \begin{bmatrix} \overline{u_1}^T u_1 & \overline{u_1}^T u_2 & \dots & \overline{u_1}^T u_{n-1} \\ \overline{u_2}^T u_1 & \overline{u_2}^T u_2 & \dots & \overline{u_2}^T u_{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \overline{u_{n-1}}^T u_1 & \overline{u_{n-1}}^T u_2 & \dots & \overline{u_{n-1}}^T u_{n-1} \end{bmatrix} = \left(\langle u_j, u_i \rangle\right).$$

Dette er den $(n \times n)$-matrix, som har (i, j)-indgang $\langle u_j, u_i \rangle$. Det følger at

Proposition 23.8. *$U \in \mathbb{C}^{n\times n}$ er unitær hvis og kun hvis dens søjler $u_0, u_1, \dots, u_{n-1}$ er ortonormal i $\mathbb{C}^n$.* □

Sætning 23.9 (Schurdekomponering)**.** *For enhver matrix $A \in \mathbb{C}^{n\times n}$ findes der en unitær matrix $U \in \mathbb{C}^{n\times n}$, $\overline{U}^T = U^{-1}$, således at $T = \overline{U}^T AU$ er øvre triangulær, dvs. $A = UT\overline{U}^T$.*

Bevis. Lad os først betragte tilfældet $n = 2$. Fra proposition 23.2 ved vi at matricen A har en egenværdi $\lambda_0 \in \mathbb{C}$ og en egenvektor $u_0 \in \mathbb{C}^2$. Sættes $v_0 = u_0/\|u_0\|_2 = (z, w)$ fås en egenvektor af længde 1, så med $|z|^2 + |w|^2 = 1$.

Vi kan nu finde en ortonormal basis for $\mathbb{C}^2$ ved at bruge v_0 og vektoren $v_1 = (-\overline{w}, \overline{z})$. Vi har $\|v_1\|_2^2 = |w|^2 + |z|^2 = 1$ og $\langle v_0, v_1 \rangle = -wz + zw = 0$, så v_0, v_1 er ortonormal. Det følger at $U = \left[v_0 \mid v_1\right]$ er en unitær matrix.

Vi har at $Av_0 = \lambda_0 v_0$ og U er invertibel. Lad os sætte $d_1 = Av_1$ og $(z_1, b_{00}) = U^{-1}d_1 \in \mathbb{C}^2$. Så har vi

$$\begin{aligned} AU &= A\left[v_0 \mid v_1\right] = \left[Av_0 \mid Av_1\right] \\ &= \left[\lambda_0 v_0 \mid d_1\right] = \left[v_0 \mid v_1\right] \begin{bmatrix} \lambda_0 & z_1 \\ 0 & b_{00} \end{bmatrix}. \\ &= U \begin{bmatrix} \lambda_0 & z_1 \\ 0 & b_{00} \end{bmatrix} \end{aligned}$$

Dermed er

$$\overline{U}^T AU = \begin{bmatrix} \lambda_0 & * \\ 0 & * \end{bmatrix},$$

en øvre triangulær, som ønsket.

For generel n, kan vi begynde på samme måde med en egenværdi λ_0 og en egenvektor v_0 af længde 1. Vi kan så bruge Gram-Schmidt processen på $v_0, e_0, \ldots, e_{n-1}$ til at finde en ortonormal basis $v_0, v_1, \ldots, v_{n-1}$ af $\mathbb{C}^n$. Sættes $U_0 = \left[v_0 \mid v_1 \mid \ldots \mid v_{n-1}\right]$, har vi en unitær matrix U_0.

Nu ser vi at

$$\begin{aligned}
AU_0 &= A\left[v_0 \mid v_1 \mid \ldots \mid v_{n-1}\right] = \left[Av_0 \mid Av_1 \mid \ldots \mid Av_{n-1}\right] \\
&= \left[\lambda_0 v_0 \mid d_1 \mid \ldots \mid d_{n-1}\right] \qquad \text{for } d_i = Av_i \\
&= U_0 U_0^{-1}\left[\lambda_0 v_0 \mid d_1 \mid \ldots \mid d_{n-1}\right] \\
&= U_0 \begin{bmatrix} \lambda_0 & z_1 & \ldots & z_{n-1} \\ 0 & b_{00} & \ldots & b_{0,n-2} \\ \vdots & \vdots & & \vdots \\ 0 & b_{n-2,0} & \ldots & b_{n-2,n-2} \end{bmatrix},
\end{aligned}$$

hvor i det sidste trin har vi sat $U_0^{-1} d_i = (z_i, b_{0i}, b_{1i}, \ldots, b_{n-2,i})$ og brugt at $U_0^{-1} v_0 = (\overline{U_0})^T v_0 = (\overline{v_0}^T v_0, \ldots, \overline{v_{n-1}}^T v_0) = (1, 0, \ldots, 0)$.

Matricen $B = (b_{ij}) \in \mathbb{C}^{n-1\times n-1}$ har mindre størrelse end A, så per induktion kan vi skrive $B = VS\overline{V}^T$ med $V = (v_{ij}) \in \mathbb{C}^{n-1\times n-1}$ unitær og $S = (s_{ij}) \in \mathbb{C}^{n-1\times n-1}$ øvre triangulær. Sættes

$$U_1 = \begin{bmatrix} 1 & 0 & \ldots & 0 \\ 0 & v_{00} & \ldots & v_{0,n-2} \\ \vdots & \vdots & & \vdots \\ 0 & v_{n-2,0} & \ldots & v_{n-2,n-2} \end{bmatrix} \quad \text{og} \quad T = \begin{bmatrix} \lambda_0 & z_1 & \ldots & z_{n-1} \\ 0 & s_{00} & \ldots & s_{0,n-2} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & & s_{n-2,n-2} \end{bmatrix},$$

har vi at U_1 er unitær, T er øvre triangulær og

$$AU_0 = U_0 U_1 T \overline{U_1}^T,$$

så

$$AU_0U_1 = U_0U_1T.$$

Men $U = U_0U_1$ er unitær:

$$\begin{aligned}
\overline{U}^T U &= \overline{U_0U_1}^T U_0U_1 = (\overline{U_0U_1})^T U_0U_1 \\
&= \overline{U_1}^T \overline{U_0}^T U_0U_1 = \overline{U_1}^T I_n U_1 \\
&= I_n.
\end{aligned}$$

Det følger at $AU = UT$, eller

$$T = \overline{U}^T AU,$$

som ønsket. □

23.4 Spektralsætningen

Vi kan få diagonaliseringsresultater hvis vi sætter ekstra krav på vores matrix. Vi giver først et resultat for reelle symmetriske matricer, og derefter en version af samme resultat for visse komplekse matricer.

Sætning 23.10 (Spektralsætningen for reelle symmetriske matricer)**.** *For $A \in \mathbb{R}^{n\times n}$ symmetrisk findes der en ortogonal matrix $V \in \mathbb{R}^{n\times n}$ således at*

$$A = V\Lambda V^T$$

for en diagonal matrix $\Lambda = \operatorname{diag}(\lambda_0, \ldots, \lambda_{n-1}) \in \mathbb{R}^{n\times n}$. Specielt er A diagonaliserbar med reelle egenværdier $\lambda_0, \ldots, \lambda_{n-1}$.

Inden vi går i gang med beviset lad os kikke på et par eksempler.

Eksempel 23.11. Betragt

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix},$$

som er symmetrisk. Dens karakteristiske polynomium er $p(\lambda) = (1-\lambda)^2 - 4 = \lambda^2 - 2\lambda - 3$, så der er to egenværdier $\lambda_0 = 3$, $\lambda_1 = -1$.

Vi kikker efter egenvektorer. Først for λ_0:

$$A - \lambda_0 I_2 = \begin{bmatrix} 1-3 & 2 \\ 2 & 1-3 \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \sim \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix},$$

giver alle egenvektorer $v_0 = (x, y)$ for λ_0 har $x = y$. Vi får en enhedsvektor ved at sætter f.eks. $x = 1 = y$ og dele derefter med normen:

$$v_0 = \frac{1}{\|(1,1)\|_2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix}.$$

For λ_1 har vi

$$A - \lambda_1 I_2 = \begin{bmatrix} 1-(-1) & 2 \\ 2 & 1-(-1) \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

så kan tage

$$v_1 = \frac{1}{\|(1,-1)\|_2}\begin{bmatrix}1\\-1\end{bmatrix} = \begin{bmatrix}1/\sqrt{2}\\-1/\sqrt{2}\end{bmatrix}.$$

Bemærk at $\langle v_0, v_1\rangle = 0$, så

$$V = \begin{bmatrix} v_0 \mid v_1 \end{bmatrix}$$

er en ortogonal matrix, og vi har

$$A = V\Lambda V^T \qquad \text{med } \Lambda = \begin{bmatrix}3 & 0\\0 & -1\end{bmatrix}.$$

△

Eksempel 23.12. Betragt

$$A = \begin{bmatrix}2 & 0 & 0\\0 & 1 & -1\\0 & -1 & 1\end{bmatrix},$$

som er symmetrisk med karakteristisk polynomium

$$\begin{aligned} p(\lambda) = \det(A - \lambda I_3) &= \det\begin{bmatrix}2-\lambda & 0 & 0\\0 & 1-\lambda & -1\\0 & -1 & 1-\lambda\end{bmatrix}\\ &= (2-\lambda)\big((1-\lambda)^2 - (-1)^2\big) = (2-\lambda)(\lambda^2 - 2\lambda)\\ &= \lambda(2-\lambda)^2, \end{aligned}$$

så A har egenværdier 0 og 2.

For $\lambda = 0$, er

$$v_0 = \frac{1}{\|(0,1,1)\|_2}\begin{bmatrix}0\\1\\1\end{bmatrix} = \begin{bmatrix}0\\1/\sqrt{2}\\1/\sqrt{2}\end{bmatrix}$$

en egenvektor.

For $\lambda = 2$, har vi

$$A - 2I_3 = \begin{bmatrix}0 & 0 & 0\\0 & -1 & -1\\0 & -1 & -1\end{bmatrix} \sim \begin{bmatrix}0 & 1 & 1\\0 & 0 & 0\\0 & 0 & 0\end{bmatrix}.$$

Der er to frie variabler (x_0 og x_2), og dermed er den generelle løsning $v = (x_0, -x_2, x_2)$. Bemærk at alle disse vektorer er vinkelret på v_0. Sættes $x_0 = 1$,

$x_2 = 0$, får vi en enhedsvektor

$$v_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

Vi mangler nu kun en enhedsvektor v_2, som er både en egenvektor med egenværdi $\lambda = 2$ og er vinkelret på v_1. Egenvektorerne $v = (x_0, -x_2, x_2)$, som er vinkelret på v_1 har $x_0 = 0$ og så er generelt af formen $v = (0, -x_2, x_2)$. Ved at tage $x_2 = 1$ og dele med normen får vi en enhedsvektor

$$v_2 = \frac{1}{\|(0,-1,1)\|_2} \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix},$$

Nu er v_0, v_1, v_2 en ortonormal samling af egenvektorer.

Sættes

$$V = \left[v_0 \mid v_1 \mid v_2 \right] = \begin{bmatrix} 0 & 1 & 0 \\ 1/\sqrt{2} & 0 & -1/\sqrt{2} \\ 1/\sqrt{2} & 0 & 1/\sqrt{2} \end{bmatrix}$$

fås en ortogonal matrix V med

$$A = V \Lambda V^T, \qquad \text{for } \Lambda = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

Vi bekræfter dette i python

```
import numpy as np

s = 1 / np.sqrt(2.0)
v = np.array([[0.0, 1.0, 0.0],
              [  s,   0,  -s],
              [  s,   0,   s]])
egenværdier = np.array([0.0, 2.0, 2.0])

print(v @ np.diag(egenværdier) @ v.T)
```

```
[[ 2.  0.  0.]
 [ 0.  1. -1.]
 [ 0. -1.  1.]]
```

som er vores oprindelig matrix A. △

Lemma 23.13. *Hvis $A \in \mathbb{R}^{n\times n}$ er symmetrisk, så har A en egenværdi $\lambda \in \mathbb{R}$.*

Bevis. Vi betragter A som en kompleks matrix $A \in \mathbb{C}^{n\times n}$, med $\overline{A} = A = A^T$. Proposition 23.2 giver at der findes en kompleks egenværdi $\lambda \in \mathbb{C}$. Lad $v \in \mathbb{C}^n$ være en tilhørende egenvektor, så $Av = \lambda v$ og $v \neq 0$.

Vi beregner $\overline{v}^T Av$ på to forskellige måder. Først har vi

$$\overline{v}^T Av = \overline{v}^T (Av) = \overline{v}^T \lambda v = \lambda \|v\|_2^2. \tag{23.1}$$

Desuden har vi

$$\overline{v}^T Av = (\overline{v}^T \overline{A}^T)v = (\overline{Av})^T v = (\overline{\lambda v})^T v = \overline{\lambda}\overline{v}^T v = \overline{\lambda}\|v\|_2^2. \tag{23.2}$$

Da vi startede med den samme venstre sider i (23.1) og (23.2) må de to resultater være ens. Dette siger at

$$\lambda\|v\|_2^2 = \overline{\lambda}\|v\|_2^2.$$

Men $v \neq 0$ medfører $\|v\|_2 \neq 0$, så vi kan dele med $\|v\|_2^2$ og få $\lambda = \overline{\lambda}$. Dette siger netop at $\lambda \in \mathbb{R}$.

Husk at λ er en egenværdi for A over $\mathbb{C}$, så $\det(A - \lambda I_n) = 0$. Men alle tal i $A - \lambda I_n$ er reelle, så polynomiet $\det(A - \lambda I_n)$ et udtryk i reelle tal og $\det(A-\lambda I_n) = 0$ siger nu at λ er en egenværdi for den reelle matrix $A \in \mathbb{R}^{n\times n}$. □

Bevis for sætning 23.10. Vælg en egenværdi $\lambda_0 = \lambda \in \mathbb{R}$ for A. Lad $v = v_0 \in \mathbb{R}^n$ være en enhedsvektor med $Av_0 = \lambda_0 v_0$.

Vi kan finde en ortogonal matrix af formen $V_0 = \left[v_0 \;\middle|\; * \;\middle|\; \dots \;\middle|\; *\right] \in \mathbb{R}^{n\times n}$, med v_0, som den 0'te søjle. F.eks. man kan vælge en Householdermatrix. Som i beviset for sætning 23.9, får vi

$$AV_0 = V_0 \begin{bmatrix} \lambda_0 & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & & \vdots \\ 0 & * & \dots & * \end{bmatrix}.$$

Da V_0 er ortogonal er $V^{-1} = V^T$ og det giver

$$A = V_0 \begin{bmatrix} \lambda_0 & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & & \vdots \\ 0 & * & \dots & * \end{bmatrix} V_0^T.$$

Men A er symmetrisk, $A = A^T$, og

$$A^T = V_0 \begin{bmatrix} \lambda_0 & 0 & \dots & 0 \\ * & * & \dots & * \\ \vdots & \vdots & & \vdots \\ * & * & \dots & * \end{bmatrix} V_0^T.$$

Da V_0 er invertibel kan vi sammenligne disse to udtryk, og konkluderer at

$$A = V_0 \begin{bmatrix} \lambda_0 & 0 & \dots & 0 \\ 0 & b_{00} & \dots & b_{0,n-2} \\ \vdots & \vdots & & \vdots \\ 0 & b_{n-2,0} & \dots & b_{n-2,n-2} \end{bmatrix} V_0^T,$$

hvor $B = (b_{ij}) \in \mathbb{R}^{n-1 \times n-1}$ med $B^T = B$. Da B er symmetrisk kan vi finde en ortogonal matrix V_1 således at

$$A = V_0 V_1 \begin{bmatrix} \lambda_0 & 0 & 0 & \dots & 0 \\ 0 & \lambda_1 & 0 & \dots & 0 \\ 0 & 0 & c_{00} & \dots & c_{0,n-3} \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & c_{n-3,0} & \dots & c_{n-3,n-3} \end{bmatrix} V_1^T V_0^T$$

med $C = (c_{ij}) \in \mathbb{R}^{n-2}$ symmetrisk. Forsættes fås

$$A = V_0 V_1 \dots V_{n-1} \begin{bmatrix} \lambda_0 & 0 & \dots & 0 \\ 0 & \lambda_1 & & 0 \\ \vdots & & \ddots & \\ 0 & 0 & & \lambda_{n-1} \end{bmatrix} V_0^T V_1^T \dots V_{n-1}^T = V \Lambda V^T$$

med $V = V_0 V_1 \dots V_{n-1}$ ortogonal. □

Det tilsvarende resultat over komplekse tal bruger det følgende begreb: en matrix $A \in \mathbb{C}^{n\times n}$ er *hermitisk* hvis $\overline{A}^T = A$. Beviserne ovenfor kan udvides til dette tilfælde og vi får

Sætning 23.14 (Spektralsætningen for hermitiske matricer)**.** *Hvis $A \in \mathbb{C}^{n\times n}$ er hermitisk, så er dens egenværdier reelle tal og der findes en unitær matrix $U \in \mathbb{C}^{n\times n}$ således at*

$$A = U\Lambda\overline{U}^T. \qquad \square$$

23.5 Egenskaber af determinanter

I dette afsnit samler vi flere egenskaber af determinantfunktionen og beviser nogle af de tidligere resultater vi har brugt. Et nøgle værktøj er at determinanter opfører sig godt i forhold til rækkeoperationer.

Proposition 23.15. *De følgende relationer gælder for determinanter af $(n \times n)$-matricer A og B, hvor B fås fra A via den pågældende rækkeoperation.*

(I) *Hvis $A \sim_{R_i \leftrightarrow R_j} B$, $i \neq j$, så er* $\det B = -\det A$.

(II) *Hvis $A \sim_{R_i \to sR_i} B$, så er* $\det B = s \det A$.

(III) *Hvis $A \sim_{R_i \to R_i + tR_j} B$, så er* $\det B = \det A$.

Bevis gives senere i dette afsnit. Bemærk at del (II) er gyldigt for alle skalarer s, men giver kun en elementær rækkeoperation hvis $s \neq 0$.

Vi har også

Proposition 23.16. *For A en $(n \times n)$-matrix er* $\det(A^T) = \det A$.

som også beviser i kommende sider.

Propositionen giver at vi kan udvikle determinanter via deres først søjle i stedet for dens første række:

Korollar 23.17. *For A en $(n \times n)$-matrix gælder*

$$\det A = a_{00}\det(M_{00}) - a_{10}\det(M_{10}) + \cdots + (-1)^{n-1}a_{n-1,0}\det(M_{n-1,0}).$$

Bevis. Der er intet at vise for $n = 1$. For $n > 1$, har vi fra definition 21.12

$$\det A = a_{00}\det(M_{00}) - a_{01}\det(M_{01}) + \cdots + (-1)^{n-1}a_{0,n-1}\det(M_{0,n-1}).$$

Lad os anvende denne formel på $B = A^T$. Vi har $b_{ij} = a_{ji}$ og at $M_{ij}(B)$, som fås fra B ved at slette række i og søjle j. Dette er det samme som at slette

række j og søjle i fra og derefter transponere resultatet, dvs. $M_{ij}(B) = M_{ji}^T$, den transponerede af M_{ji} fra A. Dette giver

$$\begin{aligned}
\det(A^T) &= \det B \\
&= b_{00}\det(M_{00}(B)) - b_{01}\det(M_{01}(M)) + \dots \\
&\qquad + (-1)^{n-1} b_{0,n-1}\det(M_{0,n-1}(B)) \\
&= a_{00}\det(M_{00}^T) - a_{10}\det(M_{10}^T) + \dots + (-1)^{n-1} a_{n-1,0}\det(M_{n-1,0}^T).
\end{aligned}$$

Men M_{ij} har størrelse $(n-1)\times(n-1)$, så per induktion er $\det(M_{ij}^T) = \det(M_{ij})$, og resultatet følger. □

Eksempel 23.18. Kombineret med proposition 23.15 giver korollar 23.17 en relativ praktisk måde at regne determinanter. For eksempel,

$$\begin{aligned}
&\det\begin{bmatrix} 2 & 3 & 1 & -1 \\ 1 & 1 & 0 & 2 \\ 0 & 2 & 3 & 1 \\ -1 & 2 & 1 & 2 \end{bmatrix} \\
&=_{R_0 \leftrightarrow R_1} -\det\begin{bmatrix} 1 & 1 & 0 & 2 \\ 2 & 3 & 1 & -1 \\ 0 & 2 & 3 & 1 \\ -1 & 2 & 1 & 2 \end{bmatrix} \\
&=_{\substack{R_1 \to R_1 - 2R_0 \\ R_3 \to R_3 + R_0}} -\det\begin{bmatrix} 1 & 1 & 0 & 2 \\ 0 & 1 & 1 & -5 \\ 0 & 2 & 3 & 1 \\ 0 & 3 & 1 & 4 \end{bmatrix} \\
&= -(1)\times\det\begin{bmatrix} 1 & 1 & -5 \\ 2 & 3 & 1 \\ 3 & 1 & 4 \end{bmatrix} + 0\times\det(M_{10}) - 0\times\det(M_{20}) + 0\times\det(M_{30}) \\
&=_{\substack{R_1 \to R_1 - 2R_0 \\ R_2 \to R_2 - 3R_0}} -\det\begin{bmatrix} 1 & 1 & -5 \\ 0 & 1 & 11 \\ 0 & -2 & 19 \end{bmatrix} = -(1)\times\det\begin{bmatrix} 1 & 11 \\ -2 & 19 \end{bmatrix} \\
&= -(1\times 19 - 11\times(-2)) = -19 - 22 = -41.
\end{aligned}$$

△

Bevis for proposition 23.5. Vi bruger korollar 23.17, til at udvikle via den første søjle:

$$\begin{aligned}\det B &= \lambda_0 \det(M_{00}) - 0 \times \det(M_{10}) + \cdots + (-1)^{n-1} 0 \times \det(M_{n-1,0}) \\ &= \lambda_0 \det(M_{00}) = \lambda_0 \det \begin{bmatrix} \lambda_1 & \dots & * \\ & \ddots & \vdots \\ 0 & & \lambda_{n-1} \end{bmatrix} \\ &= \lambda_0 \lambda_1 \det \begin{bmatrix} \lambda_2 & \dots & * \\ & \ddots & \vdots \\ 0 & & \lambda_{n-1} \end{bmatrix} = \cdots = \lambda_0 \lambda_1 \dots \lambda_{n-1}.\end{aligned}$$

For det karakteristiske polynomium har vi så

$$\begin{aligned}p(\lambda) = \det(B - \lambda I_n) &= \det \begin{bmatrix} \lambda_0 - \lambda & * & \dots & * \\ 0 & \lambda_1 - \lambda & \dots & * \\ \vdots & & \ddots & \vdots \\ 0 & 0 & & \lambda_{n-1} - \lambda \end{bmatrix} \\ &= (\lambda_0 - \lambda)(\lambda_1 - \lambda) \dots (\lambda_{n-1} - \lambda),\end{aligned}$$

som ønsket. □

Bevis for proposition 23.15. Vi skal regne effekten af rækkeoperationer på determinanten.

Vi begynder med at vise del (I) for $R_0 \leftrightarrow R_1$. Vi har fra definitionen at

$$\begin{aligned}\det A &= a_{00} \det(M_{00}) - a_{01} \det(M_{01}) + \cdots + (-1)^{n-1} a_{0,n-1} \det(M_{0,n-1}) \\ &= \sum_{j=0}^{n-1} (-1)^j a_{0j} \det(M_{0j}).\end{aligned} \tag{23.3}$$

Nu skal vi anvende denne definition på hvert M_{0j}. Lad $N_{ia;jb}$ være delmatricen af A af størrelse $(n-2) \times (n-2)$, som fås ved at slette de to rækker i og a samt de to søjler j og b fra A. Den første række i M_{0j} er $a_{10}, \dots, a_{1,j-1}, a_{1,j+1}, a_{1,n-1}$. Så har vi

$$\det(M_{0j}) = \sum_{k=0}^{j-1} (-1)^k a_{1k} \det(N_{01;jk}) - \sum_{\ell=j+1}^{n-1} (-1)^\ell a_{1\ell} \det(N_{01;j\ell}).$$

Bemærk at $N_{01;jb} = N_{01;bj}$, så i udtrykket (23.3) for $\det A$ optræder $\det(N_{01;jk})$ to steder: I leddene $a_{0j}\det(M_{0j})$ og $a_{0k}\det(M_{0k})$. Koefficienten af $\det(N_{01;jk})$ i $\det A$ er så

$$(-1)^j a_{0j}(-1)^k a_{1k} - (-1)^k a_{0k}(-1)^j a_{1j} = (-1)^{j+k}(a_{0j}a_{1k} - a_{0k}a_{1j}). \tag{23.4}$$

Når vi bytter række 0 med række 1 er $N_{01;jk}$ uændret, og (23.4) skifter fortegn. Dette giver del (I) for operationen $R_0 \leftrightarrow R_1$.

For at få del (I) generelt, bemærk først at for tilfældet $R_i \leftrightarrow R_{i+1}$ kan vi skrive $\det A$ som en sum af led af formen $(-1)^r a_{0p}a_{1q}\dots a_{i-1,t}$ gange determinanten af en kvadratisk undermatrix N af $A_{[i:,:]}$. Når vi bytter R_i med R_{i+1} bytter vi den 0'te og 1'te række af N, så $\det N \mapsto -\det N$. Det følger at $\det A \mapsto -\det A$.

For det generel tilfælde $R_i \leftrightarrow R_j$ med $i < j$ fås ved den følgende række af operationer af typen $R_k \leftrightarrow R_{k+1}$: ryk R_i trinvis ned til R_j, derefter ryk R_j trinvis optil R_i, som illustreres for $R_1 \leftrightarrow R_4$ ved

$$\begin{bmatrix} A_{[0,:]} \\ A_{[1,:]} \\ A_{[2,:]} \\ A_{[3,:]} \\ A_{[4,:]} \\ A_{[5,:]} \end{bmatrix} \sim_{R_1\leftrightarrow R_2} \begin{bmatrix} A_{[0,:]} \\ A_{[2,:]} \\ A_{[1,:]} \\ A_{[3,:]} \\ A_{[4,:]} \\ A_{[5,:]} \end{bmatrix} \sim_{R_2\leftrightarrow R_3} \begin{bmatrix} A_{[0,:]} \\ A_{[2,:]} \\ A_{[3,:]} \\ A_{[1,:]} \\ A_{[4,:]} \\ A_{[5,:]} \end{bmatrix}$$

$$\sim_{R_3\leftrightarrow R_4} \begin{bmatrix} A_{[0,:]} \\ A_{[2,:]} \\ A_{[3,:]} \\ A_{[4,:]} \\ A_{[1,:]} \\ A_{[5,:]} \end{bmatrix} \sim_{R_2\leftrightarrow R_3} \begin{bmatrix} A_{[0,:]} \\ A_{[2,:]} \\ A_{[4,:]} \\ A_{[3,:]} \\ A_{[1,:]} \\ A_{[5,:]} \end{bmatrix} \sim_{R_1\leftrightarrow R_2} \begin{bmatrix} A_{[0,:]} \\ A_{[4,:]} \\ A_{[2,:]} \\ A_{[3,:]} \\ A_{[1,:]} \\ A_{[5,:]} \end{bmatrix}.$$

Hver af disse operationer skifter fortegnet på $\det A$. Dette fortegnsskift sker i alt $p = (j-i) + (j-i-1)$ gange, da rækken i flyttes ned $j-i$ gange, og derefter flyttes række j, som er nu i plads $j-1$, op $j-i-1$ gange. Dette p er et ulige tal, så fortegnet skifter et ulige antal gange og vi har $\det A \mapsto -\det A$.

For del (II), har vi for $i = 0$, at

$$\begin{aligned} \det B &= (sa_{00})\det(M_{00}) - (sa_{01})\det(M_{01}) + \dots + (-1)^{n-1}(sa_{n-1,0}) \\ &= s\det A. \end{aligned}$$

For generel i, er $A \sim_{R_i\to sR_i} B$ det samme som

$$A \sim_{R_0\leftrightarrow R_i} A' \sim_{R_0\to sR_0} A'' \sim_{R_0\leftrightarrow R_i} B.$$

Under disse operationer har vi

$$\det A \mapsto -\det A \mapsto -s\det A \mapsto s\det A,$$

så $\det B = s\det A$.

For del (III), kikker vi først på tilfældet $i = 0$. Vi har

$$\begin{aligned}\det B &= (a_{00} + ta_{j0})\det(M_{00}) - (a_{01} + ta_{j1})\det(M_{01}) \\ &\quad + \cdots + (-1)^{n-1}(a_{0,n-1} + ta_{j,n-1})\det(M_{0,n-1}) \\ &= \det A + t\det C,\end{aligned}$$

hvor C er A med række 0 erstattet af række j. Dvs. i C er række 0 og række j ens. Matricen C er uændret under $R_0 \leftrightarrow R_j$, men fra del (I) skifter dens determinant fortegn. Vi har så at $\det C = 0$, og har vist del (III) for $i = 0$.

For general i gør vi lige som i del (II), byt række i med række 0, udføre rækkeoperationen på række 0, og så byt række 0 tilbage til række i. Determinant skifter fortegn i alt to gange under denne følge af operationer, så er uændret til sidst. Dette giver del (III). □

Husk at elementære rækkeoperationer kan udføres som matrixmultiplikation ved en elementær matrix, se proposition 7.13.

Lemma 23.19. *Hvis E er en elementærmatrix svarende til en elementær rækkeoperation, så er* $\det(EA) = \det(E)\det(A)$.

Bevis. Som i beviset overfor kan vi reducere til tilfældene hvor $i = 0$ og $j = 1$. Vores elementære matricer har så formen

$$E = \begin{bmatrix} E' & 0 \\ 0 & I_{n-2} \end{bmatrix},$$

hvor E' er en elementær matrix af størrelse (2×2). Vi har $\det E = \det E'$, og for hver type rækkeoperation har vi de følgende determinanter.

Type (I):

$$E' = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \qquad \det E = \det E' = -1.$$

Type (II):

$$E' = \begin{bmatrix} s & 0 \\ 0 & 1 \end{bmatrix}, \qquad \det E = \det E' = s.$$

Type (III):

$$E' = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}, \qquad \det E = \det E' = 1.$$

Disse skaleringsfaktor er de samme, som giver ændringerne i $\det A$ i proposition 23.15, og vi har resultatet. □

Bevis for sætning 21.14. Vi skal vise at A er invertibel hvis og kun hvis $\det A \neq 0$.

A er en kvadratisk matrix, så den kan rækkereduceres til dens reduceret echelonform F. Køres disse operationer baglæns får vi A via rækkeoperationer fra F. Vi har så nogle elementære matricer $E_0, E_1, \ldots, E_{k-1}$ således at

$$A = E_0 E_1 \ldots E_{k-1} F. \tag{23.5}$$

Fra lemma 23.19 har vi

$$\det A = \det(E_0)\det(E_1)\ldots\det(E_{k-1})\det(F).$$

Men hvert $\det(E_i)$ er forskellig fra 0, så $\det A = 0$ hvis og kun hvis $\det F = 0$.

Hvis F har en nulrække er A ikke invertibel. Vi har $\det F = 0$, så $\det A = 0$.

Hvis F ikke har en nulrække er alle søjler pivotsøjler, A er invertibel og $F = I_n$. Det følge at $\det F \neq 0$ og så $\det A \neq 0$, som ønsket. □

Bevis for sætning 23.3. Vi vil vise at $\det(AB) = \det(A)\det(B)$.

Som i beviset før, skriv

$$A = E_0 E_1 \ldots E_{k-1} F,$$

som produkt af elementære matricer E_i og en reduceret echelonmatrix F.

Vi påstår at $\det(FB) = \det(F)\det(B)$. Hvis $F = I_n$, så er $\det F = 1$, og ligningen gælder. Hvis F er ikke I_n, er dens sidste række nul. Så er den sidste række af FB også nul, og FB er ikke invertibel, og dermed er $\det(FB) = 0 = \det(F)\det(B)$.

Nu har vi

$$\begin{aligned} \det(AB) &= \det(E_0 E_1 \ldots E_{k-1} FB) = \det(E_0 E_1 \ldots E_{k-1})\det(FB) \\ &= \det(E_0 E_1 \ldots E_{k-1})\det(F)\det(B) = \det(A)\det(B), \end{aligned}$$

som ønsket. □

Bevis for proposition 23.16. Skriv igen A i formen (23.5):

$$A = E_0 E_1 \dots E_{k-1} F.$$

Vi har nu

$$A^T = F^T E_{k-1}^T \dots E_1^T.$$

Vi bemærker at for hver type elementær matrix, har vi $\det(E_i^T) = \det(E_i)$ via direkte udregning.

For $F = I_n$ er $F^T = I_n$, så $\det(F^T) = \det F$.

Hvis $F \neq I_n$, er den sidste række af F nul. I F^T får vi så at den sidste søjle er nul. Dette betyder der er højst $n-1$ pivot søjle i F^T. Så F^T er ikke invertibel, og $\det(F^T) = 0 = \det(F)$.

I begge tilfælde har vi $\det(F^T) = \det F$ og så

$$\begin{aligned}\det(A^T) &= \det(F^T E_{k-1} \dots E_1) = \det(F^T)\det(E_{k-1})\dots\det(E_1)\\ &= \det(E_1)\dots\det(E_{k-1})\det(F) = \det(E_1 \dots E_{k-1} F) = \det A,\end{aligned}$$

som ønsket. □

Kapitel 24

Potensmetoden og inverspotensmetoden for egenværdier

For generelle $(n \times n)$-matricer findes der ingen algebraisk formel for rødder af det karakteristiske polynomium. Numeriske metoder for egenværdier og egenvektorer er derfor nødt til at bruge iteration og kan kun beregne tilnærmelser. I dette kapitel kikker vi på to metoder der tillader beregning af en enkelt egenværdi og en tilhørende egenvektor. Den første kan kun finde den egenværdi, som har størst numerisk værdi. Den anden metode finder en egenværdi i nærheden af et første gæt. Vi vil diskutere disse metoder anvendt på symmetriske matricer, men metoderne kan også bruges på andre matricer. Fordelen med symmetriske matricer er at vi har spektralsætningen, sætning 23.10, der garanterer at matricen faktisk har en reel egenværdi.

Vi vil også give en version af den første metode der kan anvendes på vilkårlig matricer til at estimere deres største singulærværdi.

24.1 Potensmetoden

Lad $A \in \mathbb{R}^{n \times n}$ være en kvadratisk matrix. Potensmetoden er den følgende algoritme:

POTENSMETODEN(A)

Begynd med en vilkårlig vektor $w^{(0)} \neq 0$
for $k \in \{1, 2, \ldots\}$:
    $v = Aw^{(k-1)}$
    $w^{(k)} = v/\|v\|_2$
    $\lambda^{(k)} = \langle Aw^{(k)}, w^{(k)} \rangle$

Under visse betingelser konvergerer $w^{(k)}$ til en egenvektor for A, og $\lambda^{(k)}$ konvergerer til dens egenværdi.

Eksempel 24.1. Lad os regne betragte matricen

$$\begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

og arbejde i python.

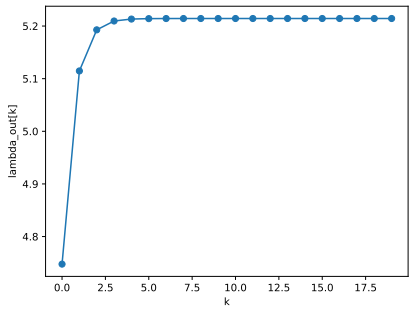
```
import matplotlib.pyplot as plt
import numpy as np

rng = np.random.default_rng()

a = np.array([[2., 1., 1.],
              [1., 3., 1.],
              [1., 1., 4.]])
```

Vi kan lave en funktion, som regner $\lambda^{(k)}$ og $u = w^k$ fra A og $w^{(k-1)}$:

```
def potens_skridt(a, w):
    v = a @ w
    v /= np.linalg.norm(v)
    lambda_ny = np.vdot(v, a @ v)
    return lambda_ny, v
```

Nu kan vi køre potensmetoden og samler resultaterne, som vi får undervejs:

```
m, _ = a.shape
w = rng.standard_normal((m, 1))
```



Figur 24.1: Værdier af λ i løbet af potensmetoden.

```
n = 20
lambda_out = np.zeros(n)

for k in range(n):
    lambda_out[k], w = potens_skridt(a, w)

print(lambda_out)
```

```
[4.7476315  5.11468666 5.19278744 5.20957877 5.21326784
 5.21408574 5.21426765 5.21430814 5.21431716 5.21431917
 5.21431962 5.21431971 5.21431974 5.21431974 5.21431974
 5.21431974 5.21431974 5.21431974 5.21431974 5.21431974]
```

Vi ser at værdierne $\lambda^{(k)}$ = `lambda_out[k]` ser ud til at stabilisere. Det kan vi også se i et plot

```
fig, ax = plt.subplots()
ax.plot(lambda_out, '-o')
ax.set_xlabel('k')
ax.set_ylabel('lambda_out[k]')
```

som giver figur 24.1.

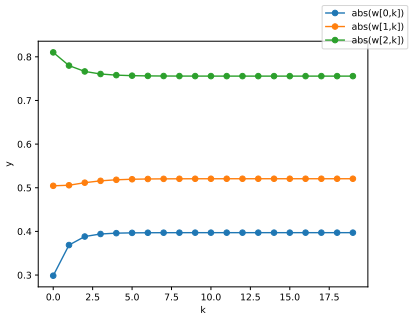
Hvis vi beregner $Aw^{(k)}$ og sammenligner med $\lambda^{(k)}w^{(k)}$ ser vi at $w^{(k)}$ er tæt på at være en egenvektor for A:

```
print('A w :')
print(a @ w)

print('Sidste lambda ganget på w :')
print(lambda_out[-1] * w)

print('Relativ fejl:')
print(np.linalg.norm(a @ w - lambda_out[-1] * w)
      / np.linalg.norm(w))
```

```
A w :
[[-2.07067195]
 [-2.71487445]
 [-3.94092689]]
Sidste lambda ganget på w :
[[-2.07067211]
 [-2.71487496]
 [-3.94092646]]
Relativ fejl:
6.898666300340367e-07
```

Hvis vi starter forfra kan vi også samle og plotte numerisk værdi af komponenterne af w.

```
m, _ = a.shape
w = rng.standard_normal((m, 1))
n = 20
lambda_out = np.zeros(n)
w_out = np.zeros((m, n))

for k in range(n):
    lambda_out[k], w = potens_skridt(a, w)
    w_out[:, [k]] = w

fig, ax = plt.subplots()
for i in range(m):
```



Figur 24.2: Numeriske værdier af komponenterne af w i løbet af potensmetoden.

```
    ax.plot(np.abs(w_out[i, :]), '-o',
            label=f'abs(w[{i},k])')
ax.set_xlabel('k')
ax.set_ylabel('y')
fig.legend()
```

Dette giver figur 24.2, hvor vi ser at hvert komponent $w_i^{(k)}$ af $w^{(k)}$ konvergerer numerisk mod en slutværdi. Vi plotte numeriske værdier, da i tilfældet hvor egenværdien er negative, erfarer man at fortegnet på hver komponent of w skifter i hvert skridt. $\triangle$

For nemheds skyld analyserer vi potensmetoden for A en symmetrisk matrix. Så er A diagonaliserbar via en ortogonal matrix V, pga. sætning 23.10, og specielt eksisterer der n lineært uafhængige egenvektorer. Søjlerne af den ortogonale matrix V giver en ortonormal basis $v_0, \ldots, v_{n-1}$ af egenvektorer for A. Skriv $\lambda_0, \ldots, \lambda_{n-1}$ for de tilhørende egenværdier, så $Av_i = \lambda_i v_i$ for hvert i. Da vi har en basis, kan vi skrive en vilkårlig $w \in \mathbb{R}^n$ som en lineær kombination af $v_0, \ldots, v_{n-1}$:

$$w = s_0 v_0 + s_1 v_1 + \cdots + s_{n-1} v_{n-1}.$$

Da $Av_i = \lambda_i v_i$, kan vi nu beregne Aw som

$$\begin{aligned} Aw &= s_0 A v_0 + s_1 A v_1 + \cdots + s_{n-1} A v_{n-1} \\ &= s_0 \lambda_0 v_0 + s_1 \lambda_1 v_1 + \cdots + s_{n-1} \lambda_{n-1} v_{n-1}. \end{aligned}$$

I potensmetoden er $w^{(k)}$ en enhedsvektor i retningen $A^k w$. Vi ser at

$$\begin{aligned} A^k w &= s_0 \lambda_0^k v_0 + s_1 \lambda_1^k v_1 + \cdots + s_{n-1} \lambda_{n-1}^k v_{n-1} \\ &= \lambda_0^k \left(s_0 v_0 + s_1 \left(\frac{\lambda_1}{\lambda_0} \right)^k v_1 + \cdots + s_{n-1} \left(\frac{\lambda_{n-1}}{\lambda_0} \right)^k v_{n-1} \right), \end{aligned} \tag{24.1}$$

hvis $\lambda_0 \neq 0$.

Antag nu at $|\lambda_0| > |\lambda_1| \geqslant \cdots \geqslant |\lambda_{n-1}| \geqslant 0$ og at $s_0 \neq 0$. For hvert $i > 0$ er $|\lambda_i/\lambda_0| < 1$, så faktoren $(\lambda_i/\lambda_0)^k$ konvergerer mod 0 når $k \to \infty$. Dette betyder at for stort k domineres $A^k w$ af ledet

$$\lambda_0^k s_0 v_0,$$

som er parallel med v_0. Da $w^{(k)}$ og v_0 er enhedsvektorer, har vi dermed at for alle store k ligger $w^{(k)}$ tæt på $\varepsilon_k v_0$, hvor $\varepsilon_k \in \{\pm 1\}$.

Proposition 24.2. *Antag at $A \in \mathbb{R}^{n \times n}$ er symmetrisk, med egenværdier, der opfylder $|\lambda_0| > |\lambda_1| \geqslant \cdots \geqslant |\lambda_{n-1}| \geqslant 0$. Lad v_0 være en enhedsvektor, som er en egenvektor for A med egenværdi λ_0, og lad $w \in \mathbb{R}^n$ opfylder $\langle w, v_0 \rangle \neq 0$. Lad $w^{(0)} = w$, og lad $w^{(k)}$, $\lambda^{(k)}$ være frembragt af potensmetoden.*

Så findes der $\varepsilon_k \in \{\pm 1\}$ og konstanter $K_0, K_1 \geqslant 0$ således at for alle k tilstrækkelig stor gælder

$$\|w^{(k)} - \varepsilon_k v_0\|_2 \leqslant K_0 \left| \frac{\lambda_1}{\lambda_0} \right|^k \quad \text{og} \quad |\lambda^{(k)} - \lambda_0| \leqslant K_1 \left| \frac{\lambda_1}{\lambda_0} \right|^{2k}.$$

□

Dette siger at efter et vis trin, forbedres tilnærmelsen til λ_0 med en faktor der er højst $|\lambda_1/\lambda_0|^2$ for hver iteration af potensmetoden. Med andre ord, har vi kvadratisk konvergens. Vi skriver

$$|\lambda^{(k)} - \lambda_0| \leqslant O\left(\left| \frac{\lambda_1}{\lambda_0} \right|^{2k} \right) \quad \text{for } k \to \infty.$$

Notationen $O(\,\cdot\,)$ kaldes *stor-»O«-notationen*. Det følger at potensmetoden virker godt hvis $|\lambda_1|$ er væsentlig mindre end $|\lambda_0|$.

Bevis. Vores antagelser sikrer at $\lambda_0^k s_0$, for $s_0 = \langle w, v_0 \rangle$, er forskellig fra 0. Lad ε_k være fortegnet af $\lambda_0^k s_0$. Vi har $w^{(k)} = A^k w / \|A^k w\|_2$. Men fra (24.1) har vi

$$\langle w^{(k)}, \varepsilon_k v_0 \rangle = \frac{\langle A^k w, \varepsilon_k v_0 \rangle}{\|A^k w\|_2} = \frac{\langle \lambda_0^k s_0 v_0, \varepsilon_k v_0 \rangle}{\|A^k w\|_2} = \frac{\varepsilon_k \lambda_0^k s_0}{\|A^k w\|_2} > 0,$$

da $\langle v_i, v_0\rangle = 0$, for $i > 0$. Ved at bruge $\langle w^{(k)}, \varepsilon_k v_0\rangle \geqslant 0$, følger det at

$$\begin{aligned}\|w^{(k)} - \varepsilon_k v_0\|_2^2 &= \|w^{(k)}\|_2^2 + \|v_0\|_2^2 - 2\langle w^{(k)}, \varepsilon_k v_0\rangle = 2 - 2\langle w^{(k)}, \varepsilon_k v_0\rangle \\ &\leqslant \big(2 - 2\langle w^{(k)}, \varepsilon_k v_0\rangle\big)\big(1 + \langle w^{(k)}, \varepsilon_k v_0\rangle\big) = 2\big(1 - \langle w^{(k)}, \varepsilon_k v_0\rangle^2\big).\end{aligned}$$

Men

$$1 - \langle w^{(k)}, \varepsilon_k v_0\rangle^2 = 1 - \frac{\langle A^k w, \varepsilon_k v_0\rangle^2}{\|A^k w\|_2^2} = \frac{\|A^k w\|_2^2 - \langle A^k w, \varepsilon_k v_0\rangle^2}{\|A^k w\|_2^2}$$

Da v_i er ortonormal, er

$$\|A^k w\|_2^2 = s_0^2\lambda_0^{2k} + s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}$$

og $\langle A^k w, \varepsilon_k v_0\rangle^2 = s_0^2\lambda_0^{2k}$, som er netop den første led i $\|A^k w\|_2^2$. Det følger at

$$\begin{aligned}1 - \langle w^{(k)}, \varepsilon_k v_0\rangle^2 &= \frac{s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}}{s_0^2\lambda_0^{2k} + s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}} \\ &\leqslant \frac{s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}}{s_0^2\lambda_0^{2k}} \\ &\leqslant \frac{(s_1^2 + \cdots + s_{n-1}^2)\lambda_1^{2k}}{s_0^2\lambda_0^{2k}} = \frac{1 - s_0^2}{s_0^2}\left(\frac{\lambda_1}{\lambda_0}\right)^{2k},\end{aligned}$$

og den første ulighed i proposition 24.2 holder med

$$K_0 = \frac{\sqrt{2(1 - s_0^2)}}{|s_0|}.$$

Tilsvarende har man

$$\begin{aligned}\lambda^{(k)} - \lambda_0 = (w^{(k)})^T A w^{(k)} - \lambda_0 &= \frac{(A^k w)^T A^{k+1} w}{\|A^k w\|_2^2} - \lambda_0 \\ &= \frac{w^T A^{2k+1} w}{w^T A^{2k} w} - \lambda_0 \\ &= \frac{s_0^2\lambda_0^{2k+1} + s_1^2\lambda_1^{2k+1} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k+1}}{s_0^2\lambda_0^{2k} + s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}} - \lambda_0 \\ &= \frac{s_1^2\lambda_1^{2k}(\lambda_1 - \lambda_0) + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}(\lambda_{n-1} - \lambda_0)}{s_0^2\lambda_0^{2k} + s_1^2\lambda_1^{2k} + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}}\end{aligned}$$

så

$$
\begin{aligned}
|\lambda^{(k)} - \lambda_0| &\leqslant \frac{\left|s_1^2\lambda_1^{2k}(\lambda_1 - \lambda_0) + \cdots + s_{n-1}^2\lambda_{n-1}^{2k}(\lambda_{n-1} - \lambda_0)\right|}{s_0^2\lambda_0^2} \\
&\leqslant \frac{(s_1^2 + \cdots + s_{n-1}^2)\lambda_1^{2k}\max_{i=1,\ldots,n-1}|\lambda_i - \lambda_0|}{s_0^2\lambda_0^2} \\
&\leqslant \left(\frac{1 - s_0^2}{s_0^2}\max_{i=1,\ldots,n-1}|\lambda_i - \lambda_0|\right)\left(\frac{\lambda_1}{\lambda_0}\right)^{2k},
\end{aligned}
$$

som er den anden ønskede ulighed. □

24.2 Page rank

En anvendelse af potensmetoden er til rangering af web sider via page rank, som bruges af Google i forbindelse med prioritering af resultater fra internet søgninger.

Lad os antage at vi vil gerne bestemme rangering af n websider. En webside j har links til $k(j)$ andre forskellige sider. Så indfører vi en $(n \times n)$-matrix $M = (m_{ij})$ med

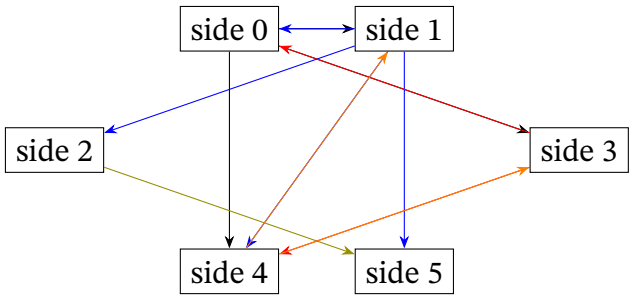
$$
m_{ij} = \begin{cases} \dfrac{1}{k(j)}, & \text{hvis der er en link fra side } j \text{ til side } i, \\ 0, & \text{hvis der er ingen link } j \text{ til } i \text{ og } k(j) \neq 0, \\ \dfrac{1}{n}, & \text{hvis } j \text{ har ingen links til andre sider.} \end{cases}
$$

Vi opstiller en model for hvilket sider en bruger, der læser på nettet, besøger. Brugeren starter på en af siderne. Brugeren går videre til en ny side ved enten (a) at følge en link på den nuværende side eller (b) at springe direkte til en tilfældig side.

Vi antager at tilfældet (a) sker med sandsynlighed $0 < p < 1$, og at tilfældet (b) sker med sandsynlighed $1 - p$. Ofte tages $p = 0{,}85$.

Sandsynligheden for at brugeren går fra side j direkte til side i er nu

$$
a_{ij} = pm_{ij} + (1 - p)\frac{1}{n}. \tag{24.2}
$$



Figur 24.3: En samling websider med links.

Bemærk at valget af værdien $m_{ij} = 1/n$, når side j har ingen links til andre sider, giver $a_{ij} = p(1/n) + (1-p)1/n = 1/n$ i sådan en situation. Dette repræsenterer korrekt at brugeren er tvunget til at vælge en tilfældig side, når der er ingen links at følge.

Eksempel 24.3. For samlingen af 6 websider givet i figur 24.3 har vi

$$M = \begin{bmatrix} 0 & 1/4 & 0 & 1/2 & 0 & 1/6 \\ 1/3 & 0 & 0 & 0 & 1/2 & 1/6 \\ 0 & 1/4 & 0 & 0 & 0 & 1/6 \\ 1/3 & 0 & 0 & 0 & 1/2 & 1/6 \\ 1/3 & 1/4 & 0 & 1/2 & 0 & 1/6 \\ 0 & 1/4 & 1 & 0 & 0 & 1/6 \end{bmatrix}.$$

Vi beregne matricen A i python lidt senere. △

Som eksemplet viser, har hver søjle af M sum 1. Matricer med dette egenskab kaldes *stokastiske matricer*. Formlen (24.2) for $A = (a_{ij})$, giver at A er også en stokastisk matrix. Bemærk desuden at A har at alle indgange a_{ij} opfylder $a_{ij} \geqslant (1-p)/n > 0$.

Proposition 24.4. *Hvis $B \in \mathbb{R}^{n\times n}$ er en stokastisk matrix så er $\lambda = 1$ en egenværdi.*

Bevis. I matricen $B - I_n$ er hvert søljesum lige med $1 - 1 = 0$. Det følger at summen af alle rækker i $B - I_n$ er rækkevektoren 0, så echelonformen for $B - I_n$ indeholder en nulrække. Det følger at $(B - I_n)x = 0$ har en løsning med $x \neq 0$, og dermed $Bx = I_n x = x$. Så x er egenvektor med egenværdi 1. □

Et dybere resultat er

Sætning 24.5 (Perron-Frobenius). *Hvis A er en stokastisk matrix med alle indgange > 0, så har egenværdien $\lambda = 1$ en egenvektor v med positive indgange. Desuden er alle egenvektorer for $\lambda = 1$ proportionelt med v og alle andre egenværdier λ_i for A har $|\lambda_i| < 1 = \lambda$.* □

Bemærkning 24.6. Når vi ved at $|\lambda_i| < 1 = \lambda$ for alle andre egenværdier, kan vi bruge potens metoden til at finde en egenvektor v for $\lambda = 1$ med alle indgange positive: Lad v' være en vilkårlig egenvektor for $\lambda = 1$. Da $v' \neq 0$, findes der $0 \leqslant i \leqslant n-1$ således at $(v')_i \neq 0$. Sæt $w^{(0)} = e_i$, den i'te standard basisvektor. Så har $v^{(1)} = Aw^{(0)} = (a_{0i}, a_{1i}, \ldots, a_{n-1,i})$ alle indgange strengt positive, og det samme gælder for $w^{(1)} = v^{(1)}/\|v^{(1)}\|_2$. Forsættes potens metoden, fås at $w^{(k)}$ har strengt positive indgange, og dermed at grænsen v har positive indgange. Analysen af potensmetoden viser at v er en egenvektor for A med egenværdi $\lambda = 1$. ◇

Page rank er den positive egenvektor v med $\|v\|_2 = 1$ og egenværdi 1 for matricen A af (24.2). Sætningen sikre at vi kan bruge potensmetoden til at bestemme den.

Eksempel 24.7. Vi fortsætter eksempel 24.3. Vi opstiller matricen `m` og definere `a` via (24.2).

```
import numpy as np

m = np.array([[  0., 1/4.,  0., 1/2.,   0., 1/6.],
              [1/3.,   0.,  0.,   0., 1/2., 1/6.],
              [  0., 1/4.,  0.,   0.,   0., 1/6.],
              [1/3.,   0.,  0.,   0., 1/2., 1/6.],
              [1/3., 1/4.,  0., 1/2.,   0., 1/6.],
              [  0., 1/4., 1.0,   0.,   0., 1/6.]])

p = 0.85
n, _ = m.shape
a = p * m + ((1-p)/n) * np.ones_like(m)

print(np.array_str(a, precision=4))
```

```
[[0.025  0.2375 0.025  0.45   0.025  0.1667]
 [0.3083 0.025  0.025  0.025  0.45   0.1667]
```

```
 [0.025  0.2375 0.025  0.025  0.025  0.1667]
 [0.3083 0.025  0.025  0.025  0.45   0.1667]
 [0.3083 0.2375 0.025  0.45   0.025  0.1667]
 [0.025  0.2375 0.875  0.025  0.025  0.1667]]
```

Her har vi brugt `np.array_str(a, precision=4)` for at gøre fremvisningen af matricen `a` mere overskuelig.

Nu vil vi bruge potensmetoden. Men vi skriver den lidt om så der kun udføres én matrix-vektormultiplikation per løkke. Bemærk at da vi ønsker en egenvektor med positive indgange og alle indgange i `a` er positive, kan vi begynde med en vektor `v` med positive indgange. Vi vælger at stoppe når de sidste to kandidater for egenvektorer er næste parallelle med hinanden

Koden nedenfor bruger en **`while`** løkke. Gentagelse af løkken kontrolleres af en test, som vi sætter til altid at give sandt, **`True`**. Løkken afbrydes i stedet med **`break`** kommandoen, når indre produktet mellem `v_ny` og `v` er tilstrækkelig tæt på `1.0`.

```
rng = np.random.default_rng()

# indgange uniform fordelt over [0,1]
v = rng.random((n, 1))
print('v positiv:', np.all(v >= 0))

v /= np.linalg.norm(v)

nøjagtighed = 1e-9

while True:
    v_ny = a @ v
    v_ny /= np.linalg.norm(v_ny)
    if np.vdot(v_ny, v) > 1.0 - nøjagtighed:
        rang_v = v_ny
        break
    else:
        v = v_ny

print('rang_v:')
print(rang_v)
```

```
print('lambda =', np.vdot(rang_v, a @ rang_v))

print('webside med største page rank:', rang_v.argmax())
```

```
v positiv: True
rang_v:
[[0.39645436]
 [0.44277275]
 [0.20828352]
 [0.44277275]
 [0.50878915]
 [0.38533115]]
lambda = 0.999998942253078
webside med største page rank: 4
```

Vi ser at den beregnede λ = `rang_v.T @ (a @ rang_v)` er tæt på 1, og side 4 har fået den største page rank, tæt efterfulgt af sider 1 og 3. Vi kan vise rangeringen af siderne bestemt af `v_ny` ved hjælp af funktionen `np.argsort()`, som vil giver indekserne for ordningen af indgangene i `v_ny` i voksende rækkefølger. Vi er mere interesseret i den omvendte rækkefølge, så vi bruger `np.flip()` bagefter.

```
print('Rangering af websider (højest først)')
print(np.flip(np.argsort(rang_v, axis=0)))
```

```
Rangering af websider (højest først)
[[4]
 [3]
 [1]
 [0]
 [5]
 [2]]
```

△

24.3 Rayleighs kvotient

I potensmetoden brugt vi udtrykket $\langle Aw^{(k)}, w^{(k)}\rangle$ som vores tilnærmelse til egenværdien λ_0. Generelt hvis $v \neq 0$, er *Rayleighs kvotient* tallet

$$r(v) = \frac{\langle Av, v\rangle}{\langle v, v\rangle}.$$

Når v er en egenvektor for A med $Av = \lambda v$, er

$$r(v) = \frac{\langle Av, v\rangle}{\langle v, v\rangle} = \frac{\langle \lambda v, v\rangle}{\langle v, v\rangle} = \lambda$$

den tilhørende egenværdi.

Hvis w er tæt på en egenvektor v, er $r(w)$ en god tilnærmelse til egenværdien λ: Skalaen $r(x)$ er den mindste kvadraters løsning på systemet $xr = Ax$, da systemet har normalligninger $x^T xr = x^T Ax$, dvs. $\langle x, x\rangle r = \langle Ax, x\rangle$.

For A symmetrisk har vi

$$\begin{aligned}
\frac{\partial r(x)}{\partial x_j} &= \frac{\partial}{\partial x_j} \frac{\sum_{k,\ell=0}^{n-1} x_k a_{k\ell} x_\ell}{\sum_{i=0}^{n-1} x_i^2} \\
&= \frac{\sum_{\ell=0}^{n-1} a_{j\ell} x_\ell + \sum_{k=0}^{n-1} x_k a_{kj}}{x^T x} - \frac{x^T Ax\, 2x_j}{(x^T x)^2} \\
&= \frac{1}{x^T x}\big(2(Ax)_j - r(x)2x_j\big) = \frac{2}{x^T x}\big(Ax - r(x)x\big)_j.
\end{aligned}$$

Det følger at x er et kritisk punkt for $r(x)$ hvis og kun hvis x er en egenvektor. Vi får også, at hvis v er egenvektor for A, så gælder

$$r(x) - r(v) = O(\|x - v\|_2^2) \quad \text{for } x \to v.$$

Dette siger at $r(x)$ giver en estimering af λ, som er korrekt til anden orden i $\|x - v\|_2$.

24.4 Potensmetoden for den største singulærværdi

For en vilkårlig matrix $A \in \mathbb{R}^{m\times n}$ ønsker vi at finde singulærværdier for A. Vi har at $A = U\Sigma V^T$, med U og V ortogonale, og Σ diagonal med diagonalindgange

$\sigma_0 \geqslant \sigma_1 \geqslant \cdots \geqslant 0$. Hvis V har søjler $v_0, v_1, \ldots$ og U har søjler $u_0, u_1, \ldots$, så gælder

$$Av_i = \sigma_i u_i$$

for hvert i. Bemærk at $A^T = V\Sigma^T U^T$ er en singulærværdidekomponering af A^T, og dette giver

$$A^T u_i = \sigma_i v_i.$$

Specielt er

$$\begin{aligned}\sigma_i &= \langle \sigma_i u_i, u_i \rangle = \langle Av_i, u_i \rangle \\ &= \langle \sigma_i v_i, v_i \rangle = \langle A^T u_i, v_i \rangle.\end{aligned}$$

For generel $u \in \mathbb{R}^m$ og $v \in \mathbb{R}^n$ forskellig fra nul, spiller nu funktionen $r(u, v) = \langle Av, u \rangle / (\|u\|_2 \|v\|_2) = \langle A^T u, v \rangle / (\|u\|_2 \|v\|_2)$ rollen af Rayleighs kvotient. Dette lader os hen til følgende potensmetode for singulærværdier.

POTENSMETODE FOR SINGULÆRVÆRDI(A)

Begynd med en vilkårlig $v^{(0)}$ af længde 1
for $k \in \{1, 2, \ldots\}$:
    $w = Av^{(k-1)}$
    $u^{(k)} = w / \|w\|_2$
    $\sigma^{(2k-1)} = \langle Av^{(k-1)}, u^{(k)} \rangle$
    $z = A^T u^{(k)}$
    $v^{(k)} = z / \|z\|_2$
    $\sigma^{(2k)} = \langle A^T u^{(k)}, v^{(k)} \rangle$

Dette producerer en følger tal $\sigma^{(1)}, \sigma^{(2)}, \sigma^{(3)}, \ldots$, som vi kan forvente konvergere mod den største singulærværdi σ_0 af A. Bemærk at $\sigma^{(2k-1)} = \|w\|_2$ og $\sigma^{(2k)} = \|z\|_2$, så vi kan bytte lidt om på beregningsrækkefølge når vi implementere denne metode.

Eksempel 24.8. Lad os betragte matricen

$$A = \begin{bmatrix} 5 & 1 & 2 & -1 \\ 1 & 3 & -1 & 2 \\ 0 & -1 & 2 & 1 \end{bmatrix}$$

```
import matplotlib.pyplot as plt
import numpy as np
```

```
rng = np.random.default_rng()

a = np.array([[5., 1., 2., -1.],
              [1., 3., -1., 2.],
              [0., -1., 2., 1.]])
```

Vi laver en funktion der udfør den ene halvdel af algoritmen

```
def sing_skridt_del(a, v):
    w = a @ v
    sigma = np.linalg.norm(w)
    u = w / sigma
    return sigma, u
```

Så lad os køre dette på vores matrix A med en tilfældig startvektor

```
m, n = a.shape

v = rng.standard_normal((n, 1))
v /= np.linalg.norm(v)

len = 20
sigma_out = np.zeros(2*len)

for k in range(len):
    sigma_out[2*k], u = sing_skridt_del(a, v)
    sigma_out[2*k+1], v = sing_skridt_del(a.T, u)

print(sigma_out)
```

```
[4.07965034 5.1996566  5.44882344 5.55811484 5.60960996
 5.63443012 5.64640194 5.65215544 5.65491203 5.65623029
 5.65686006 5.65716076 5.6573043  5.65737281 5.6574055
 5.65742111 5.65742855 5.65743211 5.6574338  5.65743461
 5.657435   5.65743518 5.65743527 5.65743531 5.65743533
 5.65743534 5.65743535 5.65743535 5.65743535 5.65743535
```

```
 5.65743535 5.65743535 5.65743535 5.65743535 5.65743535
 5.65743535 5.65743535 5.65743535 5.65743535 5.65743535]
```

Vi kan sammenligne med σ_0 beregnet af NumPys SVD funktion

```
uu, ssigma, vvt = np.linalg.svd(a, full_matrices=False)
print(f'sigma_0 = {ssigma[0]}')
print('Relativ fejl:',
      np.abs(sigma_out[-1] - ssigma[0])/ssigma[0])
```

```
sigma_0 = 5.657435353201458
Relativ fejl: 4.898185302102051e-14
```

og ser at generelt får vi en god tilnærmelse. △

Vi bemærker at den overstående metode er blot potensmetoden i skjult form. Ligningerne

$$Av_i = \sigma_i u_i \quad \text{og} \quad A^T u_i = \sigma_i u_i$$

kan sammenskrives til

$$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} u_i \\ v_i \end{bmatrix} = \sigma_i \begin{bmatrix} u_i \\ v_i \end{bmatrix},$$

så σ_i er en egenværdi for matricen

$$B = \begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix}.$$

Bemærk at

$$\begin{bmatrix} 0 & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} u_i \\ -v_i \end{bmatrix} = \begin{bmatrix} -Av_i \\ A^T u_i \end{bmatrix} = \sigma_i \begin{bmatrix} -u_i \\ v_i \end{bmatrix} = -\sigma_i \begin{bmatrix} u_i \\ -v_i \end{bmatrix},$$

så $-\sigma_i$ er også en egenværdi for B. Men matricen B er symmetrisk, så diagonaliserbar, og det overstående viser at B's egenværdier er $\sigma_0, -\sigma_0, \sigma_1, -\sigma_1, \ldots$.

24.5 Inverspotensmetoden

Lad os kikke igen på kvadratiske matricer og deres egenværdier. En stor ulempe med potensmetoden er at den kun finde egenværdien, som er numerisk størst. Har man interesse i andre egenværdier, eller hvis man har et godt bud på den største, kan man bruge inverspotensmetoden, som bruger et gæt μ for den ønskede egenværdi.

INVERSPOTENSMETODEN(A, μ)
1 Begynd med et vilkårligt $w^{(0)}$ af længde 1
2 **for** $k \in \{1, 2, \ldots\}$:
3 Løs $(A - \mu I_n)v = w^{(k-1)}$ for $v \neq 0$
4 $w^{(k)} = v/\|v\|_2$
5 $\lambda^{(k)} = \langle Aw^{(k)}, w^{(k)} \rangle$

Under visse antagelser konvergerer $\lambda^{(k)}$ til egenværdien af A, som er numerisk tættest på μ. Desuden vil $w^{(k)}$ konvergerer mod en egenvektor for denne egenværdi.

For at gøre disse udsagn mere præcis, antager vi at $A \in \mathbb{R}^{n\times n}$ er symmetrisk og at $\mu \in \mathbb{R}$ er givet. Lad λ_p være egenværdien af A, som minimerer $|\mu - \lambda_p|$, og lad λ_q være egenværdien af A med $|\mu - \lambda_q|$ næst mindst. Vi antager så at egenværdierne $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$ opfylder

$$|\mu - \lambda_p| < |\mu - \lambda_q| \leqslant |\mu - \lambda_i| \qquad \text{for alle } i \neq p.$$

I tilfældet hvor $\mu \neq \lambda_i$ for alle i, er $A - \mu I_n$ både invertibel og diagonaliserbar, med egenværdier $\lambda_i - \mu$. Den inverse $(A - \mu I_n)^{-1}$ har så egenværdier $1/(\lambda_i - \mu)$, og den største af disse numerisk er $1/|\lambda_p - \mu|$. Den inverspotensmetoden er stort set potensmetoden for $(A - \mu I_n)^{-1}$.

Lad v_p være en egenvektor med egenværdi λ_p og $\|v_p\|_2 = 1$, og antag at begyndelsesvektoren $w^{(0)}$ i inverspotensmetoden opfylder $s_p = \langle w, v_p \rangle \neq 0$. Så har vi:

Proposition 24.9. *For $w^{(k)}$ og $\lambda^{(k)}$ produceret af den inverspotensmetoden, findes der $\varepsilon_k \in \{\pm 1\}$ således at*

$$\|w^{(k)} - \varepsilon_k v_p\|_2 = O\left(\left|\frac{\mu - \lambda_q}{\mu - \lambda_p}\right|^k\right) \quad \text{og} \quad |\lambda^{(k)} - \lambda_0| = O\left(\left|\frac{\mu - \lambda_q}{\mu - \lambda_p}\right|^{2k}\right).$$

□

Vi får igen kvadratisk konvergens for egenværdien, og denne konvergens er hurtig hvis μ ligger væsentlige tættere på λ_p end på λ_q. Dette er en foretrukken metoden for beregning af konkrete egenværdier.

Eksempel 24.10. Lad os kikke igen på eksempel 24.1.

```
import numpy as np

a = np.array([[2., 1., 1.],
              [1., 3., 1.],
              [1., 1., 4.]])
```

Denne gang bruger vi inverspotensmetoden. Når vi implementerer metoden, bemærker vi at i hvert trin løses et lineært ligningssystem $Bv = w$, med den samme koefficientmatrix $B = A - \mu I_n$ hver gang. Vi kan så med fordel først beregne er QR-faktorisering $B = QR$ og så løser $Rv = Q^T w$ hver gang. Løsning af $Bv = w$ kræver $O(n^3)$ flops hver gang, mens løsning af $Rv = Q^T w$ kræver kun $O(n^2)$ flops hver gang, og beregning af QR-dekomponering $O(n^3)$ kun en gang for alle.

Først lad os kikke efter en egenværdi, som er tættest på 1,0

```
def inv_potens_skridt(q, r, w, mu):
    v = back_subs(r, q.T @ w)
    v /= np.linalg.norm(v)
    lambda_ny = np.vdot(v, a @ v)
    return lambda_ny, v

mu = 1.0

rng = np.random.default_rng()

m, _ = a.shape

b = a - mu * np.eye(m)
q, r = householder_qr(b)

w = rng.standard_normal((m, 1))
w /= np.linalg.norm(w)
```

```
n = 20
lambda_out = np.empty(n)

for i in range(n):
    lambda_out[i], w = inv_potens_skridt(q, r, w, mu)

print(lambda_out)
```

```
[2.17182833 1.43502914 1.33078916 1.32516292 1.32488366
 1.32486985 1.32486916 1.32486913 1.32486913 1.32486913
 1.32486913 1.32486913 1.32486913 1.32486913 1.32486913
 1.32486913 1.32486913 1.32486913 1.32486913 1.32486913]
```

Vi ser at $\lambda^{(k)}$ stabiliserer relativt hurtigt, og vi kan nemt tjekker hvor godt et par λ, w vi har fået

```
machine_epsilon = np.finfo(float).eps
print(np.allclose(a @ w, lambda_out[-1] * w,
                  atol = machine_epsilon))
```

```
True
```

Fra eksempel 24.1 ved vi at den største egenværdi ligger tæt på 5,2, dette kan sættes ind i inverspotensmetoden, som værdien for μ

```
mu = 5.2

m, _ = a.shape

b = a - mu * np.eye(m)
q, r = householder_qr(b)

w = rng.standard_normal((m, 1))
w /= np.linalg.norm(w)

n = 20
```

```
lambda_out = np.empty(n)

for i in range(n):
    lambda_out[i], w = inv_potens_skridt(q, r, w, mu)

print(lambda_out)
```

```
[5.21411784 5.21431974 5.21431974 5.21431974 5.21431974
 5.21431974 5.21431974 5.21431974 5.21431974 5.21431974
 5.21431974 5.21431974 5.21431974 5.21431974 5.21431974
 5.21431974 5.21431974 5.21431974 5.21431974 5.21431974]
```

som giver hurtigere konvergens end potensmetoden. △

Bemærkning 24.11. Implementering af den inverse potensmetoden kan forberedes ydeligere ved følge fremgangsmåden i `householder_lsq` fra afsnit 17.4. Dvs. beregn blot QR-data for B, og derefter udregner $Q^T w$ direkte fra denne data, uden at danne Q, inden man løser $Rv = Q^T w$. ◇

Hvis vi bemærker at vi kan ændre på valget af μ undervejs, kan vi få endnu bedre konvergens. Forfines inverspotensmetoden på denne måde, far vi Rayleighkvotientmetoden:

RAYLEIGHKVOTIENTMETODEN(A, w)

$w^{(0)} = w$ en enhedsvektor
$\lambda^{(0)} = \langle Aw, w\rangle$
for $k \in \{1, 2, \ldots\}$:
    Løs $(A - \lambda^{(k-1)} I_n)v = w^{(k-1)}$ for $v \neq 0$
    $w^{(k)} = v/\|v\|_2$
    $\lambda^{(k)} = \langle Aw^{(k)}, w^{(k)}\rangle$

som har tredjeordens konvergens med en faktor der forbedres for hvert skridt:

Proposition 24.12. *For A symmetrisk, konvergerer Rayleighkvotientmetoden til en egenværdi λ_p af A og for stort k gælder*

$$\|w^{(k+1)} - \varepsilon_{k+1} v_p\|_2 = O(\|w^{(k)} - \varepsilon_k v_p\|_2^3) \quad \text{og} \quad |\lambda^{(k+1)} - \lambda_p| = O(|\lambda^{(k)} - \lambda_p|^3),$$

med $\varepsilon_k \in \{\pm 1\}$. □

Eksempel 24.13. For matricen i (24.1), ser vi konvergens af både egenværdien og egenvektoren indenfor machine epsilon ofte efter kun 3 iterationer:

```
def rayleigh_skridt(a, lambda_est, w):
    m, _ = a.shape
    b = a - lambda_est * np.eye(m)
    q, r = householder_qr(b)
    v = back_subs(r, q.T @ w)
    v /= np.linalg.norm(v)
    lambda_ny = np.vdot(v, a @ v)
    return lambda_ny, v

print(a)
m, _ = a.shape
forsøg = 20
for j in range(forsøg):
    w = rng.standard_normal((m, 1))
    w /= np.linalg.norm(w)
    lambda_est = np.vdot(w, a @ w)

    for i in range(20):
        lambda_est, w = rayleigh_skridt(a, lambda_est, w)
        if np.allclose(a @ w, lambda_est * w,
                       atol = machine_epsilon):
            break

    print(f'Egenværdi estimat {lambda_est:<17.16}',
          f'efter {i+1} skridt')
```

```
[[2. 1. 1.]
 [1. 3. 1.]
 [1. 1. 4.]]
Egenværdi estimat 2.46081112718911  efter 5 skridt
Egenværdi estimat 1.324869129433354 efter 4 skridt
Egenværdi estimat 2.460811127189111 efter 7 skridt
Egenværdi estimat 2.46081112718911  efter 4 skridt
Egenværdi estimat 2.460811127189111 efter 3 skridt
```

```
Egenværdi estimat 5.214319743377526 efter 4 skridt
Egenværdi estimat 5.214319743377535 efter 5 skridt
Egenværdi estimat 5.214319743377442 efter 4 skridt
Egenværdi estimat 1.324869129433354 efter 3 skridt
Egenværdi estimat 2.460811127189111 efter 3 skridt
Egenværdi estimat 2.460811127189112 efter 4 skridt
Egenværdi estimat 1.324869129433357 efter 5 skridt
Egenværdi estimat 5.21431974337249  efter 2 skridt
Egenværdi estimat 2.46081112718911  efter 3 skridt
Egenværdi estimat 5.214319743377434 efter 7 skridt
Egenværdi estimat 2.460811127182946 efter 3 skridt
Egenværdi estimat 1.324869129433354 efter 5 skridt
Egenværdi estimat 5.214319743377535 efter 3 skridt
Egenværdi estimat 5.214319743286422 efter 3 skridt
Egenværdi estimat 2.460811127189111 efter 3 skridt
```

Bemærk at forskellige egenværdier kan beregnes på denne måde. Ønsker man en approksimation til en konkret egenværdi, kan man sætte en startværdi for `lambda_est`, som vi gjorde med `mu` i inverspotensmetoden, i stedet for at beregne den fra `w`. △

Potensmetoden på $A \in \mathbb{R}^{n\times n}$ bruger $O(n^2)$ flops per skridt til at estimere den numerisk størst egenværdi. For inverspotensmetoden, løser vi et lineært ligningssystem med den samme koefficientmatrix $A - \mu I_n$ hver gang, og dette koster $O(n^3)$ flops. Hvis vi først beregne en QR-dekomponering af $A - \mu I_n$ i begyndelsen, kan denne bruges til at reducere arbejdet per skridt fra $O(n^3)$ flops til $O(n^2)$ flops. Tilgengæld, kan sådan en forbedring ikke bruges til Rayleighkvotientmetoden, som bruger $O(n^3)$ flops per skridt. Men dette opvejes af at Rayleighkvotientmetoden bruger meget få skridt til at få meget høj nøjagtighed.

Kapitel 25

Hessenberg- og tridiagonalform

Vi begynder nu arbejdet for at beskrive metoder til at bestemme alle egenværdier og en basis af tilhørende egenvektorer for diagonaliserbar $A \in \mathbb{C}^{n\times n}$ og for symmetrisk $A \in \mathbb{R}^{n\times n}$. Overordnet har disse metoder, og deres udvidelse til beregning af Schurform, to trin:

(a) bring A i en særlig form, og så
(b) brug en potensmetode.

Mere konkret for generel kompleks $A \in \mathbb{C}^{n\times n}$, udføres de følgende to trin

$$\underset{A}{\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}} \to \underset{H\,=\,\text{Hessenberg}}{\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{bmatrix}} \to \underset{S\,=\,\text{Schurform}}{\begin{bmatrix} \lambda_0 & * & * & * & * \\ 0 & \lambda_1 & * & * & * \\ 0 & 0 & \lambda_2 & * & * \\ 0 & 0 & 0 & \lambda_3 & * \\ 0 & 0 & 0 & 0 & \lambda_4 \end{bmatrix}}.$$

Hessenbergform er lidt svagere end øvre triangulær, da der er nogle elementer lige under diagonalen, som kan være forskellig fra nul. Mere præcist har en matrix $H = (h_{ij})$ *Hessenbergform* hvis $h_{ij} = 0$ for alle element under underdiagonalen, dvs. for alle $i > j + 1$.

For generel reel $A \in \mathbb{R}^{n\times n}$, kan man også reducere til Hessenbergform, og bagefter til en reel version af Schurformen, med blokke af størrelse højst 2×2 langs diagonalen. I situationen hvor $A = A^T \in \mathbb{R}^{n\times n}$ er reel symmetrisk, eller $A = \overline{A}^T \in \mathbb{C}^{n\times n}$ kompleks hermitisk, er der garanti for at A er diagonaliserbar,

og processerne kan forbedres til

$$\underset{A}{\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \end{bmatrix}} \to \underset{T\,=\,\text{tridiagonal}}{\begin{bmatrix} * & * & 0 & 0 & 0 \\ * & * & * & 0 & 0 \\ 0 & * & * & * & 0 \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{bmatrix}} \to \underset{\Lambda}{\begin{bmatrix} \lambda_0 & 0 & 0 & 0 & 0 \\ 0 & \lambda_1 & 0 & 0 & 0 \\ 0 & 0 & \lambda_2 & 0 & 0 \\ 0 & 0 & 0 & \lambda_3 & 0 \\ 0 & 0 & 0 & 0 & \lambda_4 \end{bmatrix}}$$

Generelt har en *tridiagonal* matrix $T = (t_{ij})$ kun indgange, som er forskellige fra 0, på diagonalen, på overdiagonalen og på underdiagonalen, dvs. at $t_{ij} = 0$ for alle $|i - j| > 1$. I de overstående metoder opnås at T har samme egenskaber som A, så T er symmetrisk i det reelle tilfælde, og er hermitisk i det komplekse tilfælde. I dette kapitel vil vi fokusere på reelle matricer.

Vi ønsker at lave disse reduktioner så at A transformeres til VAV^{-1} for ortogonal (eller unitær) V. Vi har set at Householdermatricer er meget nyttig og er gode eksempler på ortogonale matricer.

Vi kan altid vælger en Householdermatrix $H_0 = H_0^T = H_0^{-1}$ således at H_0A har kun nultal under diagonalen i den første søjle

$$A = \begin{bmatrix} a_{00} & * & \dots & * \\ a_{10} & * & \dots & * \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1,0} & * & \dots & * \end{bmatrix} \mapsto H_0A = \begin{bmatrix} \pm\|a_0\|_2 & * & \dots & * \\ 0 & * & \dots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & * & \dots & * \end{bmatrix}$$

men disse nultal bliver overskrevet når vi beregner

$$H_0AH_0^{-1} = H_0AH_0 = \begin{bmatrix} * & * & \dots & * \\ * & * & \dots & * \\ \vdots & \vdots & \ddots & \vdots \\ * & * & \dots & * \end{bmatrix},$$

da højre multiplikation med H_0 ændrer alle søjler.

Eksempel 25.1. Her er et eksempel i python. Vi starter med en tilfældig symmetrisk matrix $A \in \mathbb{R}^{4\times 4}$.

```
import numpy as np

rng = np.random.default_rng()
```

```
a = rng.standard_normal((4, 4))
a += a.T
print(a)
```

```
[[ 0.11829959 -2.65906079  1.01561289 -4.17350994]
 [-2.65906079 -0.24189403 -0.40064855  1.15895717]
 [ 1.01561289 -0.40064855  2.79536979  0.72724739]
 [-4.17350994  1.15895717  0.72724739 -1.52328329]]
```

Nu beregner vi Householderdata for den 0'te søjle i a. Det gør vi som tidligere ved side 227 i afsnit 17.3, men passer ekstra på tilfældet hvor søjlen er nul.

```
def house(x):
    norm_x = np.linalg.norm(x)
    if norm_x == 0:
        v = np.zeros_like(x)
        v[0, 0] = 1
        s = 0
    else:
        u = x / norm_x
        eps = +1 if u[0, 0] >= 0 else -1
        s = 1 + np.abs(u[0, 0])
        v = (eps / s) * u
        v[0, 0] = 1
    return v, s

v, s = house(a[:, [0]])
h0 = np.eye(4) - s * v @ v.T
print(np.array_str(h0 @ a, precision=6))
```

```
[[-5.053143e+00  9.726972e-01 -1.957860e-01 -6.967105e-01]
 [ 6.459670e-16 -2.109277e+00  2.222305e-01 -6.287493e-01]
 [-1.089796e-16  3.125877e-01  2.557465e+00  1.410052e+00]
 [ 4.003002e-16 -1.771981e+00  1.704883e+00 -4.329165e+00]]
```

Vi ser at denne giver tal tæt på 0 i den 0'te søjle underdiagonalen. Men Householdermatricer opfylder $H_0^T = H_0$, og $H_0 A H_0^T = H_0 A H_0$ har

```
print(h0 @ a @ h0)
```

```
[[ 0.09407222 -1.67390632  0.81506888 -4.85066822]
 [-1.67390632 -1.24858539 -0.10650576  0.72214358]
 [ 0.81506888 -0.10650576  2.71753505  0.75226639]
 [-4.85066822  0.72214358  0.75226639 -0.41452981]]
```

som ikke har nogen nultal i den 0'te søjle. △

Vi har været for ambitiøst med hvor mange nultal vi vil frembringe. Men ved at skrue lidt ned for ambitionerne kan vi godt bruge Householdermatricer til at reducere til Hessenbergform.

25.1 Reduktion til Hessenbergform

Betragt en Householdermatrix $H_0 = H_0^T = H_0^{-1} = I_n - svv^T$ af formen

$$H_0 = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \dots & * \end{bmatrix},$$

dvs. med $v = (0, 1, *, \dots)$. Vælges v korrekt kan vi opnå at

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0,n-1} \\ a_{10} & * & * & \dots & * \\ a_{20} & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1,0} & * & * & \dots & * \end{bmatrix} \mapsto H_0 A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0,n-1} \\ * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \dots & * \end{bmatrix}.$$

Bemærk at den første række er uændret. Vektoren $v = (0, v_{[1:]})$ vælges, som i afsnit 9.4, således at $A_{[1:,0]} = (a_{10}, a_{20}, \dots, a_{n-1,0}) \in \mathbb{R}^{n-1}$ afbildes af $(H_0)_{[1:,1:]} =$

$I_{n-1} - sv_{[1:]}v_{[1:]}^T$ til $(\pm\|A_{[1:,0]}\|_2, 0, \ldots, 0)$. Når vi nu danner H_0AH_0 ved at gange H_0 fra højre på H_0A, bliver den første søjle fastholdt, og vi får

$$H_0AH_0 = \begin{bmatrix} a_{00} & * & * & \ldots & * \\ * & * & * & \ldots & * \\ 0 & * & * & \ldots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \ldots & * \end{bmatrix}.$$

Vælg nu en Householdermatrix H_1 således at

$$\begin{bmatrix} a_{00} & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \end{bmatrix} \mapsto H_1H_0AH_0 = \begin{bmatrix} a_{00} & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & * & * & * \end{bmatrix}$$

og så

$$H_1H_0AH_0H_1 = \begin{bmatrix} a_{00} & * & * & * & * \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & * & * & * \end{bmatrix}.$$

Fortsættes på denne måde får vi den følgende algoritme

HESSENBERG($A \in \mathbb{R}^{n\times n}$)
1 $B = A$
2 **for** $k \in \{0, 1, \ldots, n-3\}$:
3 Beregn Householder H_k for $B_{[k+1:,k]}$
4 $B = H_kBH_k$
5 **return** B

Dette kan implementeres i python på den samme måde som for QR-dekomponering. Nemlig vi begynder med en funktion, der beregner den relevante data for en Hessenbergreduktion, men vi samler ikke Householderprodukterne, i stedet for gemmes data for Householdervektorerne under underdiagonalen og skalaerne s_k gemmes i en separat `np.array`:

```
def hessenberg_data(a):
    data = np.copy(a)
    n, _ = a.shape
    s = np.empty(n-2)
    for j in range(n-2):
        v, s[j] = house(data[j+1:, [j]])
        data[j+1:, j:] -= (s[j] * v) @ (v.T @ data[j+1:, j:])
        data[:, j+1:] -= (s[j] * (data[:, j+1:] @ v)) @ v.T
        data[j+2:, [j]] = v[1:]
    return data, s
```

Selve Hessenbergmatricen er nu blot delen af matricen `data`, der ligger på underdiagonalen og derover

```
def hessenberg(a):
    data, s = hessenberg_data(a)
    return np.triu(data, -1)
```

Ønsker man også en samlet matrix Q således at $A = QHQ^T$, kan den dannes ved at gange Householdermatricerne sammen

```
def hessenberg_qh(a):
    data, s = hessenberg_data(a)
    n, _ = a.shape
    h = np.triu(data, -1)
    q = np.eye(n)
    for j in reversed(range(n-2)):
        x = data[j+2:, [j]]
        v = np.vstack([[1], x])
        q[j+1:, j+1:] -= (s[j] * v) @ (v.T @ q[j+1:, j+1:])
    return q, h
```

Vi tester dette på en matrix:

```
rng = np.random.default_rng()
a = np.round(rng.normal(0.0, 100, (5, 5)))
print(a)
```

```
[[ -86.   71.   14.  -93.  236.]
 [  19. -100.   25.   -3. -154.]
 [ -90.   50.   74.   88.  -70.]
 [  49.   27.  -89.  263.  -38.]
 [ 162.  -80.   43.   43.  -18.]]
```

```
q, h = hessenberg_qh(a)

machine_epsilon = np.finfo(float).eps

print('Tjek q ortogonal:',
      np.allclose(q.T @ q, np.eye(5),
                  rtol = machine_epsilon,
                  atol = machine_epsilon))

print('Tjek dekomponering af a:')
print(q @ h @ q.T)
print(np.allclose(q @ h @ q.T, a,
                  atol = machine_epsilon))

print('Hessenbergmatrix')
print(np.array_str(h, precision=3))
```

```
Tjek q ortogonal: False
Tjek dekomponering af a:
[[ -86.           71.           14.          -93.          236.        ]
 [ -18.63030238  -51.23129139   45.69882232  -25.84487105 -126.49494722]
 [ -88.24880073    4.73358451   55.65674977  108.77894027  -97.36180435]
 [  48.04656929   56.10545609  -75.64343772  248.87498529  -23.69988802]
 [ 158.84784132 -108.23172394   31.19699723   56.13689997  -34.30044368]]
False
Hessenbergmatrix
[[ -86.     161.274 -186.526   41.982  -83.775]
 [ 188.881   53.53    78.775  -32.152  -17.84 ]
 [   0.     161.392  -79.06    -9.191   88.443]
 [   0.       0.    -114.695  199.674 -166.239]
 [   0.       0.       0.      29.999   44.855]]
```

Reduktion til Hessenbergform uden beregning af Q bruger $\sim 10n^3/3$ flops.

25.2 Reduktion til tridiagonalform

Lad os gå videre og reducere fra Hessenbergform til tridiagonalform. Her begynder vi med en reel symmetrisk matrix $A = A^T \in \mathbb{R}^{n\times n}$. I princippet bruger vi den samme procedure, som for Hessenbergreduktion, men der er en væsentlig detalje når Householdermultiplikation skal implementeres.

Først vælger vi en Householdermatrix $H_0 = H_0^T = H_0^{-1}$ således at

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0,n-1} \\ a_{10} & * & * & \dots & * \\ a_{20} & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1,0} & * & * & \dots & * \end{bmatrix} \mapsto H_0A = \begin{bmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0,n-1} \\ * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \dots & * \end{bmatrix}$$

Denne gang når vi beregne $H_0AH_0 = H_0AH_0^T$, med A symmetrisk, får vi igen en symmetrisk matrix. Så

$$H_0AH_0 = \begin{bmatrix} a_{00} & * & 0 & \dots & 0 \\ * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \dots & * \end{bmatrix}$$

og dette begynder at ligne den tridiagonalform vi ønsker.

Vi fortsætter ved at vælge en Householdermatrix H_1 så at

$$\begin{bmatrix} a_{00} & * & 0 & 0 & 0 \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \\ 0 & * & * & * & * \end{bmatrix} \mapsto H_1H_0AH_0 = \begin{bmatrix} a_{00} & * & 0 & 0 & 0 \\ * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & * & * & * \end{bmatrix}$$

og får dermed

$$H_1H_0AH_0H_1 = \begin{bmatrix} a_{00} & * & 0 & 0 & 0 \\ * & * & * & 0 & 0 \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & * & * & * \end{bmatrix},$$

osv.

Det er væsentlig at bemærke at for en symmetrisk matrix A og en Householdermatrix $H = I_n - svv^T$ har vi

$$\begin{aligned}
HAH &= (I_n - svv^T)A(I_n - svv^T) \\
&= A - svv^T A - sAvv^T + s^2 vv^T Avv^T \\
&= A - svv^T A - sAvv^T + s^2 (v^T Av)vv^T \\
&= A - v(sv^T A^T) - (sAv)v^T \\
&\qquad + \frac{s}{2} v(v^T(sAv))v^T + \frac{s}{2}((sAv)^T v)vv^T \\
&= A - vw^T - wv^T,
\end{aligned}$$

hvor

$$w = (sAv) - \frac{s}{2}\big((sAv)^T v\big)v.$$

Vi kan derfor forkorte beregningen af HAH. Vi justerer lidt på vores `house` funktion, så vi trækker mere oplysning ud.

```
import numpy as np

def house_plus(x):
    norm_x = np.linalg.norm(x)
    if norm_x == 0:
        v = np.zeros_like(x)
        v[0, 0] = 1
        s = 0
        eps = 1
    else:
        u = x / norm_x
        eps = +1 if u[0, 0] >= 0 else -1
        s = 1 + np.abs(u[0, 0])
        v = (eps / s) * u
        v[0, 0] = 1
    return v, s, eps, norm_x

def tridiagonal_data(a):
    data = np.copy(a)
    if not np.allclose(a, a.T):
```

```
        raise np.linalg.LinAlgError(
            'In tridiagonal_data() input must ' +
            'be a symmetric matrix')
    n, _ = a.shape
    s = np.empty(n - 2)
    for j in range(n - 2):
        v, s[j], eps, norm = house_plus(data[j+1:, [j]])
        u = s[j] * (data[j+1:, j+1:] @ v)
        w = u - ((s[j]/2) * (u.T @ v)) * v
        v_wT = v @ w.T
        data[j+1, j] = - eps * norm
        data[j, j+1] = data[j+1, j]
        data[j+1:, j+1:] -= v_wT + v_wT.T
        data[j+2:, [j]] = v[1:]
    return data, s

def tridiagonal_qt(a):
    data, s = tridiagonal_data(a)
    n, _ = a.shape
    t = np.tril(np.triu(data, -1), 1)
    q = np.eye(n)
    for j in reversed(range(n-2)):
        x = data[j+2:, [j]]
        v = np.vstack([[1], x])
        q[j+1:, j+1:] -= (s[j] * v) @ (v.T @ q[j+1:, j+1:])
    return q, t
```

Vi afprøver dette på en tilfældig symmetrisk matrix af størrelse (30×30):

```
rng = np.random.default_rng()

n = 30
a = rng.normal(0.0, 5.0, (n, n))
a += a.T
q, t = tridiagonal_qt(a)

machine_epsilon = np.finfo(float).eps
```

```
print('Tjek q ortogonal:',
      np.allclose(q.T @ q, np.eye(n),
                  rtol = 2*machine_epsilon,
                  atol = 2*machine_epsilon))

print('Tjek dekomponering af a:',
      np.allclose(q @ t @ q.T, a,
                  atol = 2*machine_epsilon))
```

```
Tjek q ortogonal: True
Tjek dekomponering af a: True
```

Tridiagonalisering uden beregning af Q bruger $\sim 4n^3/3$ flops.

25.3 Reduktion til bidiagonalform

I forbindelse med beregning af singulærværdidekomponering har vi mere fleksibilitet. Begynder vi med et $A \in \mathbb{R}^{m\times n}$, er dens singulærværdidekomponering $A = U\Sigma V^T$ med U og V ortogonale. Vi har så lov til at ændre A ved at gange forskellige ortogonale matricer på venstre- og højresider af A. Med denne ekstra frihed kan vi opnå en bidiagonalform

$$A = QBP^T, \qquad B = \begin{bmatrix} * & * & 0 & 0 & 0 \\ 0 & * & * & 0 & 0 \\ 0 & 0 & * & * & 0 \\ 0 & 0 & 0 & * & * \\ 0 & 0 & 0 & 0 & * \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Dette giver en *bidiagonal matrix* $B = (b_{ij})$, dvs. med $b_{ij} = 0$ for $i \neq j, j+1$.

Vi arbejder som før, men det er lidt bøvlet at tælle hvor mange Householder multiplikationer man skal bruge fra venstre- hhv. højreside. Nedenfor er antallet fra venstre betegnet k_v, og antallet fra højre betegnet k_h.

BIDIAGONAL($A \in \mathbb{R}^{m \times n}$)

$B = A$
$k_v = n$ **if** $m > n$ **else** $m - 1$
$k_h = m$ **if** $m < n - 1$ **else** $n - 2$
for $k \in \{0, 1, \ldots, \max\{k_v, k_h\}\}$:
    if $k \leqslant k_v$:
        Beregn Householder H_k for $B_{[k:,k]}$
        $B = H_k B$
    if $k \leqslant k_h$:
        Beregn Householder K_k for $B_{[k,k+1:]}$
        $B = BK_k$

```
import numpy as np

def house(x):
    norm_x = np.linalg.norm(x)
    if norm_x == 0:
        v = np.zeros_like(x)
        v[0, 0] = 1
        s = 0
    else:
        u = x / norm_x
        eps = +1 if u[0, 0] >= 0 else -1
        s = 1 + np.abs(u[0, 0])
        v = (eps / s) * u
        v[0, 0] = 1
    return v, s

def bidiagonal_data(a):
    data = np.copy(a)
    m, n = a.shape
    kv = n if m > n else m-1
    kh = m if m < n-1 else n-2
    sv = np.zeros(kv)
    sh = np.zeros(kh)
    for k in range(np.max([kv, kh])):
        if k < kv:
```

```
            v, sv[k] = house(data[k:, [k]])
            data[k:, k:] -= ((sv[k] * v)
                             @ (v.T @ data[k:, k:]))
            data[k+1:, [k]] = v[1:]
        if k < kh:
            v, sh[k] = house(data[[k], k+1:].T)
            data[k:, k+1:] -= (sh[k] * (data[k:, k+1:]
                                        @ v)) @ v.T
            data[[k], k+2:] = v[1:].T
    return data, sv, sh

def bidiagonal(a):
    data, sv, sh = bidiagonal_data(a)
    return np.tril(np.triu(data), 1)

def bidiagonal_qbpt(a):
    data, sv, sh = bidiagonal_data(a)
    m, n = a.shape
    kv = n if m > n else m-1
    kh = m if m < n-1 else n-2
    b = np.tril(np.triu(data), 1)
    q = np.eye(m)
    pt = np.eye(n)
    for k in reversed(range(np.max([kv, kh]))):
        if k < kh:
            x = data[[k], k+2:]
            vt = np.hstack([[[1]], x])
            pt[k+1:, k+1:] -= (sh[k] * (pt[k+1:, k+1:]
                                        @ vt.T)) @ vt
        if k < kv:
            x = data[k+1:, [k]]
            v = np.vstack([[1], x])
            q[k:, k:] -= (sv[k] * v) @ (v.T @ q[k:, k:])
    return q, b, pt
```

Lad os prøve dette på et eksempel.

```
rng = np.random.default_rng()
a = np.round(rng.normal(0.0, 100, (8, 5)))
print(a)
```

```
[[ 118.  112.  -30. -182.   48.]
 [ -62.  190.  240.   53.  133.]
 [  99. -101.  145.  106.  255.]
 [  71.  169.   81.  -26. -156.]
 [-136.   -7.  124.  -57.  -73.]
 [  44.  -71.  -71.   47.  -41.]
 [  81.   45.  111.  -45.  -48.]
 [ 107.   -7.  -88.  -84.  -64.]]
```

```
q, b, pt = bidiagonal_qbpt(a)

machine_epsilon = np.finfo(float).eps

print('Bidiagonal matrix')
print(np.array_str(b, precision=3))

print('Tjek q ortogonal:',
      np.allclose(q.T @ q, np.eye(8),
                  rtol = 2*machine_epsilon,
                  atol = 2*machine_epsilon))

print('Tjek pt ortogonal:',
      np.allclose(pt.T @ pt, np.eye(5),
                  rtol = 2*machine_epsilon,
                  atol = 2*machine_epsilon))

print('Tjek dekomponering af a:')
print(q @ b @ pt)
print(np.allclose(q @ b @ pt, a,
                  atol = machine_epsilon))
```

```
Bidiagonal matrix
[[-266.556  106.559    0.       0.       0.   ]
 [   0.    -251.913  222.494    0.       0.   ]
 [   0.       0.    -337.561   14.173    0.   ]
 [   0.       0.       0.     245.664  237.435]
 [   0.       0.       0.       0.     226.036]
 [   0.       0.       0.       0.       0.   ]
 [   0.       0.       0.       0.       0.   ]
 [   0.       0.       0.       0.       0.   ]]
Tjek q ortogonal: True
Tjek pt ortogonal: True
Tjek dekomponering af a:
[[ 118.  112.  -30. -182.   48.]
 [ -62.  190.  240.   53.  133.]
 [  99. -101.  145.  106.  255.]
 [  71.  169.   81.  -26. -156.]
 [-136.   -7.  124.  -57.  -73.]
 [  44.  -71.  -71.   47.  -41.]
 [  81.   45.  111.  -45.  -48.]
 [ 107.   -7.  -88.  -84.  -64.]]
True
```

Kapitel 26

Egendekomponering via QR-metoden

Vi vil nu indføre QR-metoden for beregningen af samtlige egenværdier og egenvektorer af symmetriske reelle matricer.

26.1 QR-metoden: første version

Vi begynder med en idé til hvordan QR-faktorisering kan bruges i forbindelse med beregning af egenværdier. Ideen er meget enkel, men det er ikke oplagt hvorfor det skulle virke.

FØRSTE QR-METODE($A \in \mathbb{R}^{n\times n}$)
1 $A^{(0)} = A$
2 **for** $k \in \{1, 2, \ldots\}$:
3 Beregn QR-faktorisering $A^{(k-1)} = Q^{(k)}R^{(k)}$
4 Sæt $A^{(k)} = R^{(k)}Q^{(k)}$

Her har vi skrevet $A^{(k)}$ osv. for matricerne dannet ved den k'te iteration. Under visse antagelser konvergerer $A^{(k)}$ til en diagonalmatrix når $A \in \mathbb{R}^{n\times n}$ er reel symmetrisk. (Den kan også bruges til at producere en Schurform for $A \in \mathbb{R}^{n\times n}$ eller for $A \in \mathbb{C}^{n\times n}$.) I det reelle tilfælde vil vi se hvorfor og hvornår dette virker, ved at relatere metoden til en potensmetode.

Lad os først bemærke at, da Q er ortogonal, har vi

$$RQ = Q^T QRQ = Q^T AQ.$$

Så matricerne $A = QR$ og $A^{(1)} = RQ = Q^T AQ = Q^{-1}AQ$ har i hver fald de samme egenværdier. Det, der er ikke oplaget, er hvorfor $A^{(1)}$ er tættere på diagonalform end A.

Vi kan dog kikke på et eksempel. Betragt matricen

```
a = np.array([[1., 2., 1.],
              [2., 1., 1.],
              [1., 1., 3.]])
```

Denne matrix er symmetrisk

```
print(np.all(a.T == a))
```

```
True
```

Lad os prøve et første skridt af QR-metoden ovenfor

```
q, r = householder_qr(a)
a1 = r @ q
print(a1)
```

```
[[ 3.66666667 -1.40705294  0.86164044]
 [-1.40705294 -0.57575758 -0.25979437]
 [ 0.86164044 -0.25979437  1.90909091]]
```

Vi ser allerede at indgangene væk fra diagonalen er blevet mindre. Dette forstærkes med den næste iteration

```
q1, r1 = householder_qr(a1)
a2 = r1 @ q1
print(a2)
```

```
[[ 4.34020619  0.38439645  0.35812246]
 [ 0.38439645 -0.97233054  0.02577822]
 [ 0.35812246  0.02577822  1.63212435]]
```

Regner vi flere iterationer får vi

```
b = np.copy(a)
for i in range(10):
    q, r = householder_qr(b)
    b = r @ q

print(b)
```

```
[[ 4.41421356e+00  2.72594426e-06  1.01264816e-04]
 [ 2.72594427e-06 -1.00000000e+00  5.09658517e-11]
 [ 1.01264816e-04  5.09658574e-11  1.58578644e+00]]
```

som er rimelig tæt på en diagonalmatrix, så indgangene på diagonalen nærmere sig egenværdier for a. Dog er konvergens generelt ret langsomt. For eksempel, betragt denne matrix

```
a = np.array([[ 1.,  2., -1.],
              [ 2., -1.,  1.],
              [-1.,  1.,  3.]])
print(np.all(a == a.T))
```

```
True
```

```
b = np.copy(a)
for i in range(30):
    q, r = householder_qr(b)
    b = r @ q
    if i % 5 == 0:
        print('iteration:', i, ' b[0,1]:', b[0,1])

print('b:', b)
```

```
iteration: 0  b[0,1]: -2.0283702113484408
iteration: 5  b[0,1]: 1.408989123757998
```

```
iteration: 10  b[0,1]: -0.400209115321721
iteration: 15  b[0,1]: 0.09838867659570023
iteration: 20  b[0,1]: -0.0238333832643744
iteration: 25  b[0,1]: 0.005761847932806483
b: [[ 3.42362157e+00  1.84994907e-03 -1.09531882e-05]
 [ 1.84994907e-03 -2.57699406e+00  2.11786853e-02]
 [-1.09531882e-05  2.11786853e-02  2.15337249e+00]]
```

Her tog det omtrent 10 iterationer at få `b[0,1]` reduceret med en faktor 10. Efter `30` iterationer har `b[0,1]` størrelsesorden 10^{-3} og man kan derfor kun forvente at diagonalindgangene giver egenværdierne inden for cirka 3 decimaler.

26.2 Ortogonal iteration

Vi kan udvide potensmetoden fra afsnit 24.1 til at beregne flere egenværdier og egenvektorer. Givet en symmetrisk matrix $A \in \mathbb{R}^{n\times n}$, lad os sige at vi ønsker at beregne de »første« m egenværdier og egenvektorer, hvor $m \leqslant n$.

Metoden begynder med en vilkårlig matrix

$$W = \left[w_0 \mid w_1 \mid \ldots \mid w_{m-1}\right] \in \mathbb{R}^{n\times m}$$

med ortonormal søjler. I princippet vil vi gerne bare beregne $A^k W$ og håber at søjlerne konvergerer mod m egenvektorer. Men vi ved at dette kan ikke forventes, da fra potensmetoden ved vi at hver søjle $A^k w_i$ forventes $(A^k w_i)/\|A^k w_i\|_2$ at nærmere sig retningen for egenvektoren hørende til egenværdien, som er numerisk størst.

Vi kan ændre på denne forventning ved, at vælge en ortonormal basis for søjlerummet af $A^k W$. Dette kan gøres ved at beregne QR-dekomponeringen $A^k W = Q^{[k]} R^{[k]}$. Så udgør søjlerne af $Q^{[k]}$ denne ortonormal basis. (Vi bruger $Q^{[k]}$ her for at skelne mellem disse matricer og dem, der optræder i den faktiske ortogonal iteration nedenfor.)

Søjlerne af $Q^{[k]}$ fås ved at anvende en version af Gram-Schmidt processen på $A^k w_0, A^k w_1, \ldots, A^k w_{m-1}$. Så den første søjle af $Q^{[k]}$ er netop vektoren vi får via potensmetoden på w_0. Den næste søjle af $Q^{[k]}$ er nødvendigvis vinkelret på w_0, så må konvergerer mod en anden egenvektor.

Lad $v_0, v_1, \ldots, v_{n-1}$ være en ortonormal basis for A med egenværdier $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$. Hvis

$$|\lambda_0| > |\lambda_1| > \cdots > |\lambda_{m-1}| > |\lambda_m| \geqslant \cdots \geqslant |\lambda_{n-1}| \tag{26.1}$$

så kan analysen af potensmetoden udvides til at vise at for generelle w_j vil søjlerne $q_j^{[k]}$ af $Q^{[k]}$ konvergerer mod $\pm v_j$, for $j = 0, 1, \ldots, m-1$.

Processen kan gøres lidt mere stabil ved at vælge at arbejde med en ortonormal basis for søjlerummet af $A^k W$ i hvert trin. Dette giver følgende algoritme

ORTOGONAL ITERATION(A, m)
1 $W^{(0)} \in \mathbb{R}^{n\times m}$ tilfældig med ortonormale søjler
2 **for** $k \in \{1, 2, \ldots\}$:
3 $\quad V^{(k)} = AW^{(k-1)}$
4 $\quad Q^{(k)}R^{(k)} = V^{(k)}$
5 $\quad W^{(k)} = Q^{(k)}$

Formelt findes der $\varepsilon_i^{(k)} \in \{\pm 1\}$ således at søjlerne $q_i^{(k)}$ af $Q^{(k)}$ afviger fra $\varepsilon_i^{(k)} v_i$ til

$$O\left(\left(\max_{i=0,1,\ldots,m-1}\left|\frac{\lambda_{i+1}}{\lambda_i}\right|\right)^k\right)$$

når $k \to \infty$. Dette sker hvis egenværdierne opfylder (26.1) og alle delmatricer

$$\left((W^{(0)})^T\left[v_0 \mid v_1 \mid \ldots \mid v_{m-1}\right]\right)_{[:i,:i]}, \qquad \text{for } i = 1, \ldots, m$$

er invertibel. Vi har så lineært konvergens til egenvektorerne.

Lad os kikke på et eksempel. Betragt matricen

```
a = np.array([[1., 2., 1., 1., 0.],
              [2., 2., 1., 2., 1.],
              [1., 1., 3., 1., 0.],
              [1., 2., 1., 4., 1.],
              [0., 1., 0., 1., 5.]])
```

som er symmetrisk af størrelse 5×5:

```
print(np.all(a.T == a), a.shape)
```

```
True (5, 5)
```

Vi vil gerne finde de første 3 egenværdier.

Vi begynder med en tilfældig matrix med ortonormale søjler, fundet ved at beregne QR-dekomponering af en tilfældig (5×3)-matrix

```
rng = np.random.default_rng()
m, _ = a.shape
q, r = householder_qr(rng.random((m, 3)))
w = q
print(w)
```

```
[[-0.52754611  0.62260756  0.36765461]
 [-0.38337495 -0.76092052  0.47385931]
 [-0.38679939  0.00647102 -0.26668421]
 [-0.45760885 -0.16499043 -0.74072192]
 [-0.4644342   0.07807729  0.1431713 ]]
```

Så laver vi ortogonal iteration

```
for i in range(10):
    v = a @ w
    q, r = householder_qr(v)
    w = q

print(w)
```

```
[[-0.29664068 -0.25177549 -0.0734324 ]
 [-0.48175175 -0.16344993  0.04745289]
 [-0.31746352 -0.38079936 -0.77808808]
 [-0.60611705 -0.19473721  0.57878687]
 [-0.46017235  0.85262095 -0.2279048 ]]
```

Vi kan beregne de tilsvarende Rayleighkvotienter, som diagonal indgangene i `q.T @ (a @ q)`

```
lambdaer = np.diag(q.T @ (a @ q))
print(lambdaer)
```

```
[7.36231414 4.58209091 2.29104599]
```

som er vores kandidater for de første 3 egenværdier. Hvis vi sammenligner Aq_i med $\lambda_i q_i$

```
print((a @ q) - q * lambdaer)
```

```
[[ 0.00023715 -0.00055371 -0.00959083]
 [ 0.00015291  0.00083877 -0.0090949 ]
 [ 0.0003661  -0.00750343  0.0011787 ]
 [ 0.00017585  0.006501    0.00460065]
 [-0.00079715 -0.00186909  0.00885614]]
```

ser vi at de stemmer overens indenfor to eller tre decimaler, så vi han tilnærmelser til tilhørende egenvektorer. Dog er konvergens ikke specielt hurtig.

Bemærkning 26.1. Det samme ide kan bruges for at beregne singulærværdier.

```
rng = np.random.default_rng()

a = rng.normal(0.0, 100, (7, 11))
m, n = a.shape

r = 3

v = rng.standard_normal((n, r))
v, _ = householder_qr(v)

for i in range(30):
    u = a @ v
    u, _ = householder_qr(u)
    v = a.T @ u
    v, _ = householder_qr(v)

print(f'Approks til første {r} singulærværdier')
print(np.abs(np.diag(u.T @ a @ v)))
```

```
Approks til første 3 singulærværdier
[557.48089797 461.37934645 374.71157103]
```

hvor de korrekte værdier er

```
s = np.linalg.svd(a, compute_uv=False)
print(s[:r])
```

```
[557.48089798 461.37934646 374.71157102]
```

◇

26.3 Ækvivalens af metoderne

Vi påstår at QR-metoden ovenfor og ortogonal iteration med $m = n$ og $W^{(0)} = I_n$, producerer den samme oplysning, nemlig en QR-faktorisering af A^k, når A er invertibel.

Da vi ved at ortogonal iteration konvergerer mod en diagonalisering af A, følger det at det samme gælder for QR-metoden ovenfor.

For at se ækvivalensen, først lad os betragte QR-metoden. Her har vi $A^{(k-1)} = Q^{(k)}R^{(k)}$ og $A^{(k)} = R^{(k)}Q^{(k)}$. Disse to relationer kan bruges til at give

$$R^{(k)}Q^{(k)} = Q^{(k+1)}R^{(k+1)},$$

som viser hvordan vi kan bytte et RQ-produkt ud med et QR-produkt, ved at forøge indekserne. Vi har nu

$$A = Q^{(1)}R^{(1)}$$

og dermed

$$A^2 = Q^{(1)}R^{(1)}Q^{(1)}R^{(1)} = Q^{(1)}Q^{(2)}R^{(2)}R^{(1)}.$$

Dette kan bruges til at regne A^3, som

$$\begin{aligned} A^3 = AA^2 &= Q^{(1)}R^{(1)}Q^{(1)}Q^{(2)}R^{(2)}R^{(1)} \\ &= Q^{(1)}Q^{(2)}R^{(2)}Q^{(2)}R^{(2)}R^{(1)} = Q^{(1)}Q^{(2)}Q^{(3)}R^{(3)}R^{(2)}R^{(1)}. \end{aligned}$$

Generelt får vi på samme måde

$$A^k = Q^{(1)}Q^{(2)}\dots Q^{(k)}R^{(k)}\dots R^{(2)}R^{(1)}. \tag{26.2}$$

Dette er en QR-dekomponering $A = QR$ med

$$Q = Q^{(1)}Q^{(2)}\dots Q^{(k)} \quad \text{og} \quad R = R^{(k)}\dots R^{(2)}R^{(1)},$$

da produkter af ortogonale matricer er ortogonal, og produkter af øvre triangulære matricer er øvre triangulær.

For ortogonal iteration gælder generelt at

$$AW^{(k-1)} = W^{(k)}R^{(k)}.$$

I tilfældet hvor $m = n$ og $W^{(0)} = I_n$ har vi at hvert $W^{(k)}$ er en ortogonal matrix. Vi kan regne

$$A = AI_n = AW^{(0)} = W^{(1)}R^{(1)}$$

og

$$A^2 = AW^{(1)}R^{(1)} = W^{(2)}R^{(2)}R^{(1)}.$$

Tilsvarende har vi

$$A^3 = AW^{(2)}R^{(2)}R^{(1)} = W^{(3)}R^{(3)}R^{(2)}R^{(1)}$$

og generelt

$$A^k = W^{(k)}R^{(k)}\dots R^{(2)}R^{(1)}. \tag{26.3}$$

Dette er igen en QR-dekomponering af A^k, da $W^{(k)}$ er ortogonal og faktoren $R^{(k)}\dots R^{(2)}R^{(1)}$ er øvre triangulær. For A invertibel, er A^k også invertibel, så søjlerne af A^k er lineært uafhængig. Proposition 14.6 giver så at QR-dekomponeringen af A^k er entydigt, så (26.2) og (26.3) giver den samme QR-dekomponering af A^k, og vores påstand er vist.

Bemærk at vi har specielt at

$$W^{(k)} = Q = Q^{(1)}Q^{(2)}\dots Q^{(k)}$$

og at

$$R = R^{(k)}\dots R^{(2)}R^{(1)}.$$

Desuden har vi

$$A^{(k)} = Q^TAQ$$

da

$$\begin{aligned}
A^{(k)} &= R^{(k)}Q^{(k)} = (Q^{(k)})^TQ^{(k)}R^{(k)}Q^{(k)} \\
&= (Q^{(k)})^TR^{(k-1)}Q^{(k-1)}Q^{(k)} = (Q^{(k)})^T(Q^{(k-1)})^TQ^{(k-1)}R^{(k-1)}Q^{(k-1)}Q^{(k)} \\
&= \cdots = (Q^{(k)})^T\dots(Q^{(2)})^T(Q^{(1)})^TQ^{(1)}R^{(1)}Q^{(1)}Q^{(2)}\dots Q^{(k)} \\
&= (Q^{(k)})^T\dots(Q^{(2)})^T(Q^{(1)})^TAQ^{(1)}Q^{(2)}\dots Q^{(k)} \\
&= Q^TAQ.
\end{aligned}$$

Det følger at for $A^{(k)}$ i QR-metoden har vi at

$$A \text{ symmetrisk} \implies A^{(k)} \text{ symmetrisk}$$

og at $A^{(k)}$ og A har de samme egenværdier.

Konvergensresultaterne for ortogonal iteration giver nu:

Proposition 26.2. *Hvis $A \in \mathbb{R}^{n\times n}$ er symmetrisk med egenværdier der opfylder*

$$|\lambda_0| > |\lambda_1| > \cdots > |\lambda_{n-1}| > 0$$

og alle delmatricer $A_{[:i,:i]}$, $i = 1, \ldots, n$, er invertible, så konvergerer $A^{(k)}$ mod en diagonalmatrix D og Q mod en basis af egenvektorer med rate

$$O\left(\max_{j=0,\ldots,n-2}\left|\frac{\lambda_{j+1}}{\lambda_j}\right|\right).$$

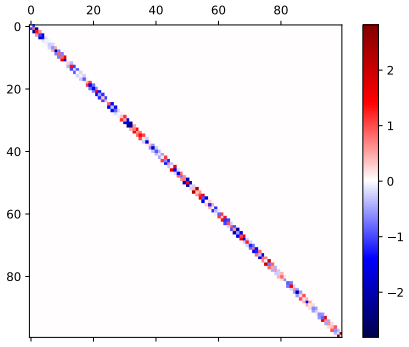
Igen er dette kun lineær konvergens.

26.4 QR-dekomponering af tridiagonale matricer

Som noteret i kapitel 25, er det første trin i en typisk beregning af egenværdier for en symmetrisk matrix, at vi reducerer til tridiagonalform. Det viser sig at QR-dekomponering af tridiagonale matricer har pæne egenskaber, som er relevant når vi skal anvende QR-metoden på dem.

Lad os lave et eksperiment i python. Først definerer vi en funktion, som giver et tydeligt billede af en matrix: En heatmap med farveskala `seismic`, som er hvid i midten af skalaen.

```
import matplotlib.pyplot as plt
import numpy as np

machine_epsilon = np.finfo(float).eps

def heat_map(a):
    fig, ax = plt.subplots()
    ax.set_aspect('equal')
    r = np.max(np.abs(a))
```



Figur 26.1: En tridiagonal matrix.

```
    if r < 20 * machine_epsilon:
        r = 1
    im = ax.matshow(a, cmap='seismic', clim = (-r, r))
    fig.colorbar(im)
```

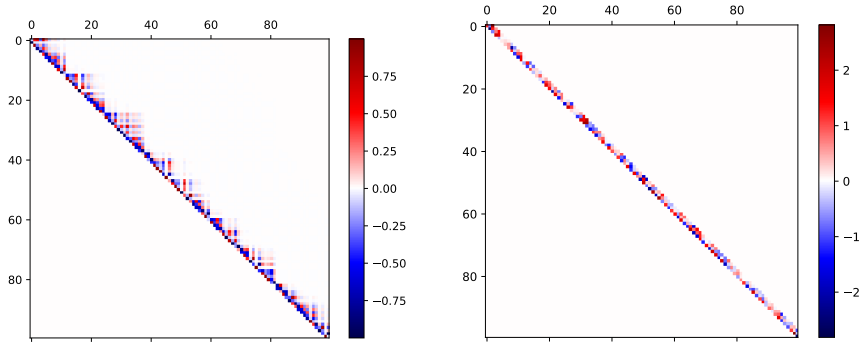
Lad os kikke på en tilfældig symmetrisk tridiagonal matrix `t`

```
n = 100
d = rng.standard_normal(n)
u = rng.standard_normal(n-1)
t = np.diag(d) + np.diag(u, 1) + np.diag(u, -1)

heat_map(t)
```

Billedet vises i figur 26.1. Vi kan nu beregne en QR-dekomponering af `t` og plotte faktorerne på samme måde

```
q, r = householder_qr(t)

heat_map(q)
```

```
heat_map(r)
```



Figur 26.2: Q og R faktorer for en tridiagonal matrix.

I figur 26.2 ser vi at begge faktorer har næsten en øvretriangulærform: R er naturligvis øvre triangulær, og Q har nultal under underdiagonalen. Vi kan bekræfte denne form for Q, ved

```
print(np.all(np.tril(q, -2) == 0))
```

```
True
```

Dette betyder at når vi danne RQ, som trin i QR-metoden, får vi en matrix som igen har nultal i alle indgangene under underdiagonalen. Men for `t`

```
t1 = r @ q
heat_map(t1)
```

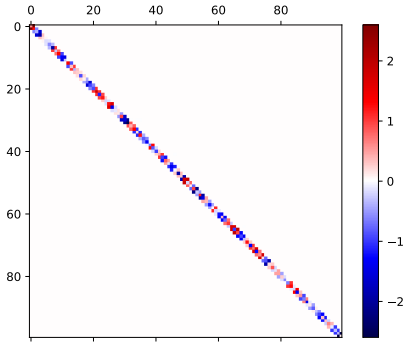
viser i figur 26.3 at RQ er faktisk symmetrisk og tridiagonal. Dette gælder da

$$RQ = Q^T QRQ = Q^T TQ$$

er symmetrisk.

26.5 Praktisk QR-metode

Konvergens af QR-metoden kan forbedres med tre teknikker. Først reducerer vi A til triangulærform $T^{(0)}$. Derefter indføres nogle estimeringer $\mu^{(k)}$ for den



Figur 26.3: Resultat af et trin af QR-metoden på en symmetrisk tridiagonal matrix.

mindste egenværdi af $T^{(k-1)}$, som bruges på en måde der ligner inverspotens-metoden.

Det sidste forbedring er at dele problemet i underproblemer, når en indgang på underdiagonalen af $T^{(k)}$ bliver 0.

Samlet giver disse tiltag følgende algoritme

PRAKTISK QR-METODE($A \in \mathbb{R}^{n\times n}$ symmetrisk)

1 Beregn tridiagonalform $A = (Q^{(0)})^T T^{(0)} Q^{(0)}$
2 **for** $k \in \{1, 2, \ldots\}$:
3 Vælg $\mu^{(k)} \in \mathbb{R}$
4 $Q^{(k)} R^{(k)} = T^{(k-1)} - \mu^{(k)} I_n$
5 $T^{(k)} = R^{(k)} Q^{(k)} + \mu^{(k)} I_n$
6 Hvis der findes r med $(T^{(k)})_{r,r-1} = 0$
7 Sæt $T_0 = (T^{(k)})_{[:r,:r]}$ og $T_1 = (T^{(k)})_{[r:,r:]}$
8 Anvend QR-metoden på T_0 og på T_1

Kernen af algoritmen er QR-faktorisering af $T^{(k-1)} - \mu^{(k)} I_n$ til $Q^{(k)} R^{(k)}$. Matricen $T^{(k)} = R^{(k)} Q^{(k)} + \mu^{(k)} I_n$ har igen samme egenværdier som $T^{(k-1)}$ da

$$\begin{aligned} T^{(k)} &= R^{(k)} Q^{(k)} + \mu^{(k)} I_n = (Q^{(k)})^T Q^{(k)} R^{(k)} Q^{(k)} + \mu^{(k)} (Q^{(k)})^T Q^{(k)} \\ &= (Q^{(k)})^T \big(Q^{(k)} R^{(k)} + \mu^{(k)} I_n\big) Q^{(k)} = (Q^{(k)})^T T^{(k-1)} Q^{(k)}. \end{aligned}$$

I trin 3 er et simpelt valg $\mu^{(k)} = (T^{(k-1)})_{n-1,n-1}$, som er Rayleighs kvotient for den sidste søjle af Q.

En første python version af dette algoritme uden trin 6 er dermed

```
def forskudt_qr_trin(a, mu):
    mu_eye = mu * np.eye(*(a.shape))
    q, r = householder_qr(a - mu_eye)
    return r @ q + mu_eye

def færdig(u):
    return np.allclose(u, np.zeros_like(u),
                       atol = machine_epsilon)

def semi_praktisk_qr_metode(a):
    i_max = 40
    q, t = tridiagonal_qt(a)
    for i in range(i_max):
        mu = t[-1,-1]
        t = forskudt_qr_trin(t, mu)
        if færdig(np.diag(t,-1)):
            break
    return t, i+1
```

Til orientering har vi valgt at denne funktion giver os den resultaterne tridiagonal matrix samt antallet af iterationer der er kørt. Vi stopper tidligt hvis vi har stort set fået en diagonalmatrix. Ellers bruger vi `i_max = 40` iterationer. Lad os prøve dette på en tilfældig matrix

```
n = 10
a = rng.normal(0.0, 5.0, (n, n))
a += a.T

t, i = semi_praktisk_qr_metode(a)

print(np.diag(t))
print('iterationer:', i)
```

```
[ 36.71640111 -37.85081249  28.65841514  17.35963072
 -21.83041807 -14.75029293   5.49992383  -7.81082502
```

```
  -2.68125148  -3.6800243 ]
iterationer: 40
```

Diagonalværdierne fra `t` kan sammenlignes med egenværdier beregnet af NumPys indbygget funktion

```
numpy_eig = np.linalg.eigvals(a)
print(numpy_eig)
```

```
[-37.86009717  36.71651065  28.66759028  17.39145013
 -21.86223748 -14.75029315   5.49992406  -7.81082502
  -2.68125148  -3.6800243 ]
```

Nogle egenværdier er tæt på det rigtige, andre er langt fra.

```
print(np.sort(np.diag(t)) - np.sort(numpy_eig))
```

```
[ 9.28468114e-03  3.18194115e-02  2.27789444e-07
  9.76996262e-15  3.99680289e-15  4.44089210e-15
 -2.27789432e-07 -3.18194115e-02 -9.17513946e-03
 -1.09541679e-04]
```

Egenværdierne, der beregnes bedst, er dem, der har relativt stor afstand fra nabo egenværdier. Ofte er dette den sidste, takket være $\mu^{(k)}$.

Valget $\mu^{(k)} = (T^{(k-1)})_{n-1,n-1}$ har dog problemer hvis $T^{(k-1)}$ har to sidste egenværdier med $|\lambda_{n-2}| = |\lambda_{n-1}|$. Så generelt foretrækkes det at bruge *Wilkinsons forskydning*, som er bestemt af den nederste (2×2)-blok af $T^{(k-1)}$. Hvis vi skriver den blok som

$$X = T^{(k-1)}_{[-2:,-2:]} = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$$

så er Wilkinsons forskydning

$$\mu^{(k)} = c - \frac{\varepsilon b^2}{|\delta| + \sqrt{\delta^2 + b^2}},$$

hvor

$$\delta = \frac{1}{2}(a - c) \quad \text{og} \quad \varepsilon = \begin{cases} 1 & \text{hvis } \delta \geqslant 0, \\ -1 & \text{ellers.} \end{cases}$$

Trin 6 tillader os at dele problemet i to på den følgende måde. Når der er et r så at indgangen $(T^{(k)})_{r,r-1} = 0$, har vi også $(T^{(k)})_{r-1,r} = 0$ og så at

$$T^{(k)} = \begin{bmatrix} T_0 & 0 \\ 0 & T_1 \end{bmatrix},$$

hvor T_i er som givet i algoritmen. Vi kan derefter regne videre med T_0 og T_1 hver for sig. I praksis er det nok at finde en indgang $(T^{(k)})_{r,r-1}$, som er tilstrækkelig tæt på 0.

Sættes begge disse betragtninger i spil, fås noget i retning af det følgende

```
def wilkinson_forskydning(x):
    a, b, c = x[np.triu_indices_from(x)]
    delta = (a - c) / 2
    eps = 1 if delta >=0 else -1
    nævner = np.abs(delta) + np.sqrt(delta**2 + b**2)
    return c - eps * b**2 / nævner

def færdig_eller(u):
    z = np.argwhere(u < machine_epsilon)[:,0]
    færdig = (z.shape == u.shape)
    delings_indeks = -1
    if z.shape != (0,):
       delings_indeks = z[0] + 1
    return færdig, delings_indeks

def praktisk_qr_metode(a):
    trin = 0
    if a.shape == (1,1):
        return a, trin
    q, t = tridiagonal_qt(a)
    for i in range(20):
        trin += 1
        mu = wilkinson_forskydning(t[-2:,-2:])
        t = forskudt_qr_trin(t, mu)
        færdig, r = færdig_eller(np.abs(np.diag(t, -1)))
        if færdig:
            break
        if r > 0:
```

```
            t[:r, :r], j = praktisk_qr_metode(t[:r, :r])
            t[r:, r:], k = praktisk_qr_metode(t[r:, r:])
            trin += j + k
            break
    return t, trin
```

På vores eksempel giver dette

```
t, trin = praktisk_qr_metode(a)

print(np.diag(t))
print('samlet antal trin:', trin)

print('afvigelser')
print(np.sort(np.diag(t)) - np.sort(numpy_eig))
```

```
[-37.86009717  36.71651065  28.66759028  17.39145013
 -21.86223748 -14.75029315   5.49992406  -7.81082502
  -2.68125148  -3.6800243 ]
samlet antal trin: 20
afvigelser
[ 1.42108547e-14 -3.90798505e-14 -7.10542736e-15
  0.00000000e+00  4.44089210e-15  7.10542736e-15
 -1.15463195e-14 -1.06581410e-14  0.00000000e+00
  0.00000000e+00]
```

med alle afvigelser indenfor en rimelig faktor ganger machine epsilon.

Generelt kan det vises at den praktiske QR-metode har de følgende egenskaber:

- konvergens er tredje ordens,
- egenværdier beregnes indenfor $\|A\|_2\epsilon_{\text{machine}} = \sigma_0\epsilon_{\text{machine}}$,
- kan implementeres med $\sim 4n^3/3$ flops

Specielt er den præcist og hurtigt. Korrekt implementeret ligger det største arbejde i tridiagonalisering af a, da QR-metoden på en tridiagonal matrix kan omskrives til $O(n^2)$.

Kapitel 27

Principalkomponent analyse

Vi vil nu kikke på en god anvendelse af singulærværdier og egenværdier, som kan bruges i forbindelse med undersøgning af forskellige datasæt. Vi begynder med et par statistiske begreber og se hvorfor vores værktøj bliver relevante.

27.1 Principalkomponent analyse

Relationerne mellem $(m \times n)$-matricer og symmetriske matricer bruges i en del forskellige sammenhæng. Et eksempel er principalkomponent analyse.

Betragt datasættet givet i tabel 27.1. For en given række

$$x = \begin{bmatrix} x_0 & x_1 & \dots & x_{n-1} \end{bmatrix} \in \mathbb{R}^{1\times n}$$

kan vi beregne dens *middelværdien*

$$\bar{x} = \frac{1}{n}\sum_{i=0}^{n-1} x_i \in \mathbb{R}$$

Person	0	1	2	3	4	5
Vægt (kg)	66	54	56	61	56	63
Højde (m)	1,78	1,54	1,63	1,72	1,52	1,76

Tabel 27.1: Vægt og højde for nogle personer.

og dens *varians*

$$\operatorname{var}(x) = \frac{1}{n-1}\sum_{i=0}^{n-1}(x_i - \bar{x})^2 \in \mathbb{R}.$$

Middelværdien er den gennemsnitlige værdi af $x_0, x_1, \ldots, x_{n-1}$. Variansen måler hvor tæt disse værdier ligger på middelværdien. Er variansen 0, så er alle x_i lige med middelværdien; er variansen stort, så er der stor spredning i x_i værdierne.

Ved at sætte $1_n = \begin{bmatrix} 1 & 1 & \ldots & 1 \end{bmatrix} \in \mathbb{R}^{1\times n}$ kan variansen skrives

$$\operatorname{var}(x) = \frac{1}{n-1}(x - \bar{x}1_n)(x - \bar{x}1_n)^T \in \mathbb{R}.$$

For to rækker kan vi også beregne deres indbyrdes *kovarians*

$$\operatorname{cov}(x, y) = \frac{1}{n-1}(x - \bar{x}1_n)(y - \bar{y}1_n)^T.$$

Kovarians kan betragtes, som et mål for hvor meget y_i-værdier følger de tilsvarende x_i-værdier. Er $\operatorname{cov}(x, y) = 0$, betragtes variablerne x og y, som værende uafhængig af hinanden.

Har man flere datarækker kan man udføre disse operationer i matrix form. Lad os antage at tabellen har m rækker og n søjler. F.eks. har vi n personer hvor vi har taget m forskellige mål. Så kan vi betragte vores data givet ved n søjlevektorer $v_0, \ldots, v_{n-1} \in \mathbb{R}^m$. Hver vektor består af de forskellige målinger for en enkelt person.

Vektoren af middelværdier er så

$$\bar{v} = \frac{1}{n}(v_0 + \cdots + v_{n-1}) \in \mathbb{R}^m.$$

For tabel 27.1 er $\bar{v} \in \mathbb{R}^2$, med den 0'te indgang middelvægten for gruppen og 1'te indgang middelhøjden.

Sæt

$$w_i = v_i - \bar{v}$$

og saml disse vektorer i en matrix

$$W = \begin{bmatrix} w_0 \mid w_1 \mid \ldots \mid w_{n-1} \end{bmatrix} \in \mathbb{R}^{m\times n}.$$

Hver række i W har middelværdi 0, så er centreret omkring sin middelværdi. Nu kan vi danne *kovariansmatricen*

$$C = \frac{1}{n-1}WW^T \in \mathbb{R}^{m\times m}.$$

For $i = j$ er c_{ii} variansen af række i; for $i \neq j$ er c_{ij} kovariansen mellem række i og række j.

Kovariansmatricen er en symmetrisk matrix. Dens egenvektorer giver forskellige lineære kombinationer af datarække. Den tilhørende egenværdier er variansen for denne kombination. Oftest er man interesseret i kombinationerne, der giver den største varians, da det beskriver bedst hvordan datapunkterne adskiller sig mest. Dette svarer til den (numerisk) største egenværdi.

Det er det mest præcist at beregne egenværdier af C fra singulærværdierne af W. Hvis $W = U\Sigma V^T$ er en SVD for W, så har vi

$$C = \frac{1}{n-1} U\Sigma\Sigma^T U^T = U\left(\frac{1}{n-1}\Sigma\Sigma^T\right)U^T.$$

Egenværdierne af C er dermed $\frac{1}{n-1}\sigma_i^2$, hvor σ_i er singulærværdierne for W.

Lad os nu betragte $U^T W$. Dette har kovariansmatrix

$$\begin{aligned}\frac{1}{n-1}(U^T W)(U^T W)^T &= \frac{1}{n-1}(\Sigma V)(\Sigma V)^T = \frac{1}{n-1}\Sigma V V^T \Sigma^T \\ &= \frac{1}{n-1}\Sigma\Sigma^T,\end{aligned}$$

som er diagonal. Dette er udtryk for at rækkerne i $U^T W$ er uafhængig af hinanden. Det vil sige disse lineær kombinationer af målinger er uafhængige af hinanden.

Eksempel 27.1. For datasættet i tabel 27.1 har vi

```
import matplotlib.pyplot as plt
import numpy as np

v = np.array([[66, 54, 56, 61, 56, 63],
              [1.78, 1.54, 1.63, 1.72, 1.52, 1.76]])

fig, ax = plt.subplots()
ax.plot(*v, 'o')
ax.set_xlabel('Vægt (kg)')
ax.set_ylabel('Højde (m)')
```

Middelværdien af en række kan beregnes via `np.mean`

```
print(np.mean(v[0]))
print(np.mean(v[1]))
```

```
59.333333333333336
1.6583333333333332
```

Vi kan beregne middelværdierne for alle rækker på en gang ved at bruge `np.mean` på `v` og angive `axis=1`

```
print(np.mean(v, axis=1))
```

```
[59.33333333  1.65833333]
```

Resultatet kan fås som søjlevektor ved hjælp af `keepdims=True`

```
v_bar = np.mean(v, axis=1, keepdims=True)
print(v_bar)
```

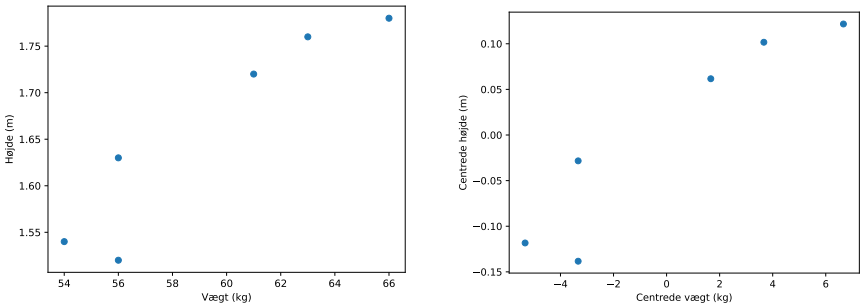
```
[[59.33333333]
 [ 1.65833333]]
```

så den centrede matrix `w` er blot

```
w = v - v_bar
print(np.array_str(w, precision=4))
```

```
[[ 6.6667 -5.3333 -3.3333  1.6667 -3.3333  3.6667]
 [ 0.1217 -0.1183 -0.0283  0.0617 -0.1383  0.1017]]
```

```
fig, ax = plt.subplots()
ax.plot(*w, 'o')
ax.set_xlabel('Centrede vægt (kg)')
ax.set_ylabel('Centrede højde (m)')
```



Figur 27.1: Plot af data og den centrede version.

Se figur 27.1 for plots af den oprindelige datamatrix `v` og den centrede matrix `w`.

Kovariansmatricen er nu

```
m, n = w.shape
c = (1 / (n - 1)) * w @ w.T
print(c)
```

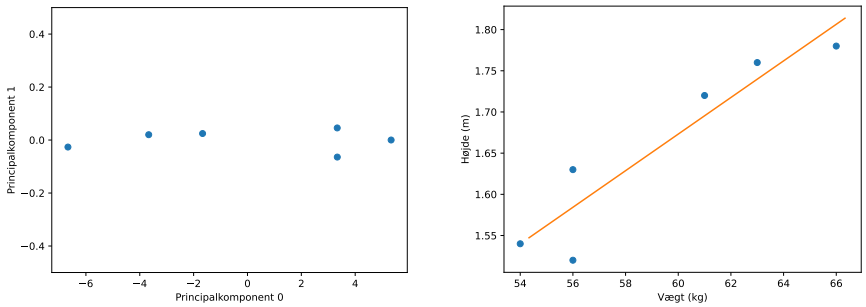
```
[[2.22666667e+01 4.94666667e-01]
 [4.94666667e-01 1.25766667e-02]]
```

som viser større variation i vægterne end i højderne. Beregner vi SVD for `w`, får vi følgende singulærværdier

```
u, s, _ = np.linalg.svd(w, full_matrices=False)
print(s)
```

```
[10.5540648   0.08906689]
```

Så den første retning har en relativt stort variation. En plot af datasættet i de nye målekombinationer er givet i figur 27.2: til venstre plotte via de to nye målekombinationer mod hinanden; til højre tegner vi den første principalretning, svarende til den største singulærværdi og dens målekombination, ovenpå den oprindelige datasæt.



Figur 27.2: Principale komponenter, uden normalisering.

```
fig, ax = plt.subplots()
ax.set_ylim(-.5, .5)
ax.plot*(u.T @ w), 'o')
ax.set_xlabel('Principalkomponent 0')
ax.set_ylabel('Principalkomponent 1')
```

```
fig, ax = plt.subplots()
ax.plot(*v, 'o')
ax.plot(*(u[:, [0]] * np.linspace(-7, 5, 2) + v_bar))
ax.set_xlabel('Vægt (kg)')
ax.set_ylabel('Højde (m)')
```

En del af forklaringen på variationsstørrelsen er de forskellige enheder, og så numeriske størrelser, brugt for de forskellige målinger (vægt og højde). En standard måde at kompensere for dette, er at sørge for at dataet justeres så hver række har middelværdi 0 og varians 1. Disse skaleringsfaktorer et givet via kvadratroden af de diagonale indgange i kovariansmatricen c:

```
skalering = np.sqrt(np.diag(c))[:, np.newaxis]
w_ny = w / skalering
c_ny = (1 / (n - 1)) * w_ny @ w_ny.T
print(c_ny)
```

```
[[1.         0.93476462]
 [0.93476462 1.        ]]
```

Vi kan beregne principale komponenter for det normaliserede data vi singulærværdidekomponeringen

```
u_ny, s_ny, _ = np.linalg.svd(w_ny)
print(s_ny)
```

```
[3.11027701 0.57111898]
```

De nye plots er

```
fig, ax = plt.subplots()
ax.set_ylim(-.5, .5)
ax.plot(*(u_ny.T @ w_ny), 'o')
ax.set_xlabel('Principalkomponent 0')
ax.set_ylabel('Principalkomponent 1')
```

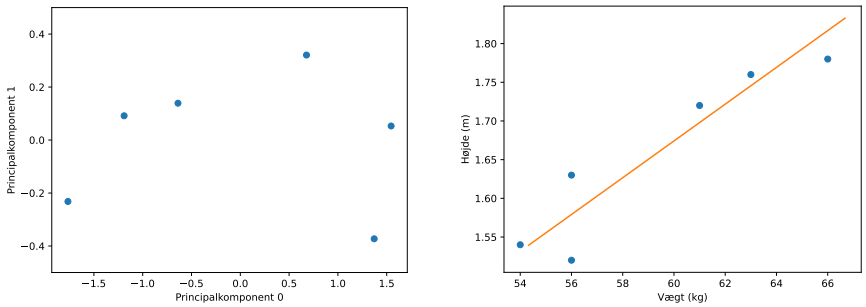
```
fig, ax = plt.subplots()
ax.plot(*v, 'o')
ax.plot(*((u_ny * skalering)[:, [0]]
          * np.linspace(-2.2, 1.5, 2) + v_bar))
ax.set_xlabel('Vægt (kg)')
ax.set_ylabel('Højde (m)')
```

Se figur 27.3. △

27.2 Singulærværdidekomponering og symmetriske matricer

Lad os kikker nærmere på relationen mellem singulærværdier og egenværdier af relaterede matricer, og dermed forklarer hvorfor vi beregnede singulærværdier for W i stedet for egenværdier for $C = (1/(n-1))WW^T$.

Der er (mindst) tre måder man kan danne en ny symmetrisk matrix fra en vilkårlig matrix $A \in \mathbb{R}^{m \times n}$:



Figur 27.3: Principale komponenter, normaliserede.

(a) $A^T A \in \mathbb{R}^{n\times n}$,

(b) $AA^T \in \mathbb{R}^{m\times m}$,

(c) $\begin{bmatrix} 0_{m\times m} & A \\ A^T & 0_{n\times n} \end{bmatrix} \in \mathbb{R}^{(m+n)\times(m+n)}$.

Hvis man bruger QR-metoden til at beregne egenværdier for disse tre symmetriske matricer, vil det koster hhv. (a) $\sim 4n^3/3$ flops, (b) $\sim 4m^3/3$ flops og (c) $\sim 4(m+n)^3/3$ flops, Da $4(m+n)^3/3 > (4m^3/3) + (4n^3/3)$, virker den sidste metode umiddelbart til at være ret bekostelig.

Eksempel 27.2. For matricen

$$A = \begin{bmatrix} 1 & -1 \\ 2 & 1 \\ 4 & 1 \\ 3 & -1 \end{bmatrix}$$

er disse tre matricer givet ved

(a) $A^T A = \begin{bmatrix} 30 & 2 \\ 2 & 4 \end{bmatrix}$,

(b) $AA^T = \begin{bmatrix} 2 & 1 & 3 & 4 \\ 1 & 5 & 9 & 5 \\ 3 & 9 & 17 & 11 \\ 4 & 5 & 11 & 10 \end{bmatrix}$ og

(c) $\begin{bmatrix} 0_{m\times m} & A \\ A^T & 0_{n\times n} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 0 & 4 & 1 \\ 0 & 0 & 0 & 0 & 3 & -1 \\ 1 & 2 & 4 & 3 & 0 & 0 \\ -1 & 1 & 1 & -1 & 0 & 0 \end{bmatrix}.$

△

Hvad siger SVD om disse symmetriske matricer? Lad os skrive

$$A = U\Sigma V^T \in \mathbb{R}^{m\times n}$$

med $U \in \mathbb{R}^{m\times m}$ og $V \in \mathbb{R}^{n\times n}$ ortogonale, og $\Sigma = \operatorname{diag}_{m\times n}(\sigma_0, \sigma_1, \ldots, \sigma_{k-1})$, hvor $k = \min\{m, n\}$.

For (a) har vi

$$A^T A = V\Sigma^T U^T U\Sigma V^T = V\Sigma^T \Sigma V^T = V D_n V^T$$

hvor

$$D_n = \operatorname{diag}(\sigma_0^2, \sigma_1^2, \ldots, \sigma_{k-1}^2, 0, \ldots, 0) \in \mathbb{R}^{n\times n}.$$

Tilsvarende for (b) er

$$AA^T = U\Sigma V^T V\Sigma^T U^T = U\Sigma\Sigma^T U^T = U D_m U^T$$

med $D_m \in \mathbb{R}^{m\times m}$, som ovenfor. Vi ser at i begge tilfælder giver egenværdierne af den symmetriske matrix singulærværdierne for A i anden potens. Da singulærværdier er positiver, er de entydigt bestemt af disse egenværdier. Det betyder at vi kunne beregne singulærværdier af A ved at forme enten $A^T A$ eller AA^T og bruge metoden fra det forrige kapitel til at beregne egenværdier af den symmetriske matrix.

Der er dog flere problemer med at beregne singulærværdier af via egenværdierne af $A^T A$ (eller AA^T). De stammer fra tre kilder:

(i) QR-metoden beregner egenværdier af $A^T A$ indenfor $\|A^T A\|_2 \epsilon_{\text{machine}} = \sigma_0^2 \epsilon_{\text{machine}}$,

(ii) for $m \geqslant n$ har $A^T A$ har konditionstal $\kappa(A^T A) = \sigma_0^2/\sigma_{n-1}^2 = \kappa(A)^2$,

(iii) selve floatberegning af $A^T A$ kan have betydningsfulde fejl.

Punkterne (i) og (ii) peger begge i retning af en halvering af det forventede antal korrekte cifre i hver beregnede singulærværdi. Punkt del (iii) kan også være ret alvorligt.

Eksempel 27.3. For (iii) betragt det følgende eksempel.

```
>>> import numpy as np
>>> s = 1e-9
>>> a = np.array([[1.0, 1.0],
...               [  s, 0.0],
...               [0.0,   s]])
```

Vi har

```
>>> a.T @ a
array([[1., 1.],
       [1., 1.]])
```

og alle indgange i matricen `a.T @ a` er præcis lige med `1.0`, som float:

```
>>> np.all(a.T @ a == np.ones((a.T @ a).shape))
True
```

Dette betyder at de beregnede egenværdier af `a.T @ a` er nødvendigvis `2.0` og `0.0`, med egenvektorerne hhv. `[[1.0], [1.0]]` og `[[1.0], [-1.0]]`. Dette ville give singulærværdier `np.sqrt(2.0)` og `0.0` for `a`. Men de korrekte singulærværdier for `a` er $\sqrt{2+s^2}$ og s; ingen af disse er 0.

```
>>> singv = np.linalg.svd(a, compute_uv=False)
>>> singv
array([1.41421356e+00, 1.00000000e-09])

>>> machine_epsilon = np.finfo(float).eps

>>> np.allclose(singv,
...             [np.sqrt(2.0 + s**2), s],
...             atol = machine_epsilon)
True
```

Disse korrekte singulærværdier giver de følgende korrekte egenværdier for $A^T A$

```
>>> singv**2
array([2.e+00, 1.e-18])
```

som er igen ikke nul.

Problemet stammer fra dannelsen af matricen `a.T @ a`, hvor f.eks. den øverste venstre indgang er

```
>>> (1.0)**2 + (s)**2 + (0.0)**2
1.0
```

men `s**2` er mindre end machine epsilon, så dens sum med `1.0` giver blot `1.0`.

For denne matrix kunne problemet delvis forbigås ved at betragte i stedet den større matrix `a @ a.T`

```
>>> a @ a.T
array([[2.e+00, 1.e-09, 1.e-09],
       [1.e-09, 1.e-18, 0.e+00],
       [1.e-09, 0.e+00, 1.e-18]])
```

Men beregning af dens egenværdier giver en indgang der er ikke positiv

```
>>> evals = np.linalg.eigvals(a @ a.T)
>>> evals
array([ 2.00000000e+00, -1.92592994e-34,  1.00000000e-18])
```

Men ser vi bort fra det kan vi få fat i singulærværdierne

```
>>> singv_fra_evals = np.flip(np.sort(
...                    np.sqrt(np.abs(evals))))[:2]
>>> singv_fra_evals
array([1.41421356e+00, 1.00000000e-09])

>>> np.all(singv == singv_fra_evals)
False

>>> np.allclose(singv, singv_fra_evals,
...             atol = machine_epsilon)
True
```

Men generelt kan AA^T være væsentlige større end A^TA og matricen AA^T kan have de samme underliggende problemer i forhold til præcision af floatberegninger. $\triangle$

Indtil videre har vi kun kikket på de symmetriske matrice af type (a) og (b). For den tredje symmetrisk matrix (c), som er af størrelse $(m+n)\times(m+n)$, har vi

$$\begin{bmatrix} 0_{m\times m} & A \\ A^T & 0_{n\times n} \end{bmatrix} = \begin{bmatrix} 0 & U\Sigma V^T \\ V\Sigma^T U^T & 0 \end{bmatrix} = \begin{bmatrix} U & 0 \\ 0 & V \end{bmatrix} \begin{bmatrix} 0 & \Sigma V^T \\ \Sigma^T U^T & 0 \end{bmatrix}$$
$$= \begin{bmatrix} U & 0 \\ 0 & V \end{bmatrix} \begin{bmatrix} 0 & \Sigma \\ \Sigma^T & 0 \end{bmatrix} \begin{bmatrix} U^T & 0 \\ 0 & V^T \end{bmatrix}.$$

Her er de ydre matricer ortogonale med

$$\begin{bmatrix} U & 0 \\ 0 & V \end{bmatrix} \begin{bmatrix} U^T & 0 \\ 0 & V^T \end{bmatrix} = \begin{bmatrix} UU^T & 0 \\ 0 & VV^T \end{bmatrix} \begin{bmatrix} I_m & 0 \\ 0 & I_n \end{bmatrix} = I_{m+n}.$$

Desuden er matricen i midten symmetrisk, og vi kan beregne dens egenværdier. For hver singulærværdi σ har vi en delmatrix af formen

$$\begin{bmatrix} 0 & \sigma \\ \sigma & 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \sigma & 0 \\ 0 & -\sigma \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Vi ser så at matricen

$$\begin{bmatrix} 0 & \Sigma \\ \Sigma^T & 0 \end{bmatrix}$$

er diagonaliserbar med egenværdier

$$\sigma_0, -\sigma_0, \ldots, \sigma_{k-1}, -\sigma_{k-1}, 0, \ldots, 0.$$

Så den symmetriske matrice af type (c) har samme 2-norm og konditionstal, som A. Det betyder at vi kan forvente en præcise beregning af dens egenværdier, og dermed singulærværdierne for A, via QR-metoden. Selvfølgelig er ulempen at matricen er en $((m+n)\times(m+n))$-matrix, som kan være meget stort.

I praksis beregnes SVD via QR-metoden ved at bruge denne matrix som abstrakt objekt, og skriver algoritmen om til kun at arbejde med matricen A selv. Tridiagonalisering erstattes af bidiagonalisering. Her er mange nultal udenfor en kvadratisk blok, så man kan nu arbejde med en bidiagonal kvadratisk matrix B af den overkommelig størrelse $(k\times k)$. Det største teoretiske arbejde kræves for at se hvordan forskydnings operationen i QR-metoden kan nu oversættes til operationer på B.

Kapitel 28

LU-dekomponering

Her genbesøger vi lineære ligningssystemer, med en metode der er baseret på systematisk brug af elementære rækkeoperationer. Metoden er relativ hurtigt, så er ofte valgt af nogen i anvendelser, men det er desværre ret svært at holde styr på hvor præcist resultatet bliver.

28.1 LU-dekomponering og Gausseliminering

Lad os betragte det følgende lineære ligningssystem

$$\begin{aligned} 2x + 3y - 4z &= 7, \\ 3x - 4y + z &= -2, \\ x + y + 2z &= 3, \end{aligned} \tag{28.1}$$

som i afsnit 6.1. Vi har tidligere løst systemet ved hjælp af elementære rækkeoperationer. Husk at disse er givet ved

(I)	$R_i \leftrightarrow R_j$	`a[ [i,j], :] = a[ [j,i], :]`
(II)	$R_i \to sR_i \ (s \neq 0)$	`a[i, :] *= s`
(III)	$R_i \to R_i + tR_j \ (j \neq i)$	`a[i, :] += t * a[j, :]`

og at de kan realiseres via matrixprodukter. F.eks. for $n = 2$ er operationerne $R_0 \leftrightarrow R_1$, $R_1 \to sR_1$ og $R_1 \to R_1 + tR_0$ realiseret via venstre multiplikation ved hhv.

$$\text{(I)} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad \text{(II)} \begin{bmatrix} 1 & 0 \\ 0 & s \end{bmatrix} \quad \text{(III)} \begin{bmatrix} 1 & 0 \\ t & 1 \end{bmatrix}.$$

For $n = 3$, bemærk at kombinationen (a) $R_1 \to R_1 + tR_0$ efterfulgt af (b) $R_2 \to R_2 + rR_0$ kan realiseres, som $A \mapsto L_b L_a A = LA$, hvor

$$L = L_b L_a = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ r & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ t & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ t & 1 & 0 \\ r & 0 & 1 \end{bmatrix}.$$

Operationen $R_i \to R_i + tR_j$ har invers $R_i \to R_i - tR_j$, så

$$L^{-1} = L_a^{-1} L_b^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -t & 0 & 0 \\ -r & 0 & 0 \end{bmatrix}.$$

Vi kan reducere systemet (28.1) til øvretriangulærform udelukkende ved brug af rækkeoperation (III) $R_i \to R_i + tR_j$ med $j < i$. Arbejdes der systematisk ved at indføre nultal under diagonalen i hver søjle får vi

$$A = \begin{bmatrix} 2 & 3 & -4 \\ 3 & -4 & 1 \\ 1 & 1 & 2 \end{bmatrix} \sim_{\substack{R_1 \to R_1 - \frac{3}{2}R_0 \\ R_2 \to R_2 - \frac{1}{2}R_0}} L_1 A = \begin{bmatrix} 2 & 3 & -4 \\ 0 & -17/2 & 7 \\ 0 & -1/2 & 4 \end{bmatrix}$$

$$\sim_{R_2 \to R_2 - \frac{1}{17}R_1} L_2 L_1 A = \begin{bmatrix} 2 & 3 & -4 \\ 0 & -17/2 & 7 \\ 0 & 0 & 61/7 \end{bmatrix} = U$$

med slutmatricen U øvre triangulær, og med

$$L_1 = \begin{bmatrix} 1 & 0 & 0 \\ -3/2 & 1 & 0 \\ -1/2 & 0 & 1 \end{bmatrix}, \qquad L_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1/171 & \end{bmatrix}.$$

Dette giver os en faktorisering $A = LU$, hvor

$$L = L_1^{-1} L_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 3/2 & 1 & 0 \\ 1/2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1/17 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 3/2 & 1 & 0 \\ 1/2 & 1/17 & 1 \end{bmatrix}$$

er nedre triangulær og U er øvre triangulær. Faktoriseringen $A = LU$ kaldes en *LU-dekomponering* af A.

Denne fremgangsmåde virker generelt, hvis ikke vi får nultal i pivotindgangene. Lad os arbejde med kvadratiske $(n \times n)$-matricer. Vi betragte en matrix X

af formen.

$$X = \begin{bmatrix} x_{00} & \dots & x_{0,i-1} & x_{0i} & x_{0,i+1} & \dots & x_{0,n-1} \\ & \ddots & \vdots & \vdots & \vdots & & \vdots \\ 0 & & x_{i-1,i-1} & x_{i-1,i} & x_{i-1,i+1} & \dots & x_{i-1,n-1} \\ 0 & \dots & 0 & x_{ii} & x_{i,i+1} & \dots & x_{i,n-1} \\ 0 & \dots & 0 & x_{i+1,i} & x_{i+1,i+1} & \dots & x_{i+1,n-1} \\ \vdots & & \vdots & \vdots & \vdots & & \vdots \\ 0 & \dots & 0 & x_{n-1,i} & x_{n-1,i+1} & \dots & x_{n-1,n-1} \end{bmatrix}$$

Hvis x_{ii} er forskellig fra 0, kan vi bruge flere rækkeoperationer af type (III) til at nulstille indgangene $x_{i+1,i}, \dots, x_{n-1,i}$ ved at danne produktet $L_i X$, hvor

$$L_i = \begin{bmatrix} 1 & & 0 & 0 & 0 & \dots & 0 \\ & \ddots & & \vdots & \vdots & & \vdots \\ 0 & & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ 0 & \dots & 0 & -x_{i+1,i}/x_{ii} & 1 & & 0 \\ \vdots & & \vdots & \vdots & & \ddots & \\ 0 & \dots & 0 & -x_{n-1,i}/x_{ii} & 0 & & 1 \end{bmatrix}.$$

Bemærk at matricen L_i kan skrives ved hjælp af et ydre produkt, som

$$L_i = I_n - w_i e_i^T, \quad \text{hvor } w_i = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ x_{i+1,i}/x_{ii} \\ \vdots \\ x_{n-1,i}/x_{ii} \end{bmatrix}.$$

Denne matrix har invers

$$L_i^{-1} = I_n + w_i e_i^T, \tag{28.2}$$

da $e_i^T w_i = 0$ giver $(I_n - w_i e_i^T)(I_n + w_i e_i^T) = I_n - w_i e_i^T + w_i e_i^T - w_i e_i^T w_i e_i^T = I_n$.

Desuden er produktet af to af disse inverse nem at beregne i den ene rækkefølge:

$$L_i^{-1} L_{i+1}^{-1} = I_n + w_i e_i^T + w_{i+1} e_{i+1}^T \tag{28.3}$$

da det manglende led $w_i e_i^T w_{i+1} e_{i+1}^T$ indeholder $e_i^T w_{i+1}$, som er 0. Matrices $L_i^{-1} L_{i+1}^{-1}$ er så en nedre triangulær matrix med ikke nul indgange fra vektorerne w_i og w_{i+1} sat ind under diagonalen i deres respektiver søjler. (Produktet $L_{i+1}^{-1} L_i^{-1}$ har ikke så simpel en form da $e_{i+1}^T w_i = x_{i+1,i}/x_{ii}$ er generelt forskellige fra 0.)

Ligninger (28.2) og (28.3) betyder at vi kan nemt implementere denne metode. Vi får

GAUSSELIMINERING UDEN OMBYTNING$(A \in \mathbb{R}^{n\times n})$

$U = A, L = I_n$
for $i \in \{0, 1, \ldots, n-2\}$:
    $L_{[i+1:,[i]]} = U_{[i+1:,[i]]}/U_{[i,i]}$
    $U_{[i+1:,i:]} = U_{[i+1:,i:]} - L_{[i+1:,[i]]} U_{[[i],i:]}$
return L, U

som bruger $\sim 2n^3/3$ flops.

Via en LU-dekomponering af A, kan vi løse et lineært ligningssystem $Ax = b$ på følgende måde

Beregn $A = LU$
Løs $Ly = b$ for y via forward substitution
Løs $Ux = y$ for x via back substitution

Her kan back substitution realiseres via

BACK SUBSTITUTION$(U \in \mathbb{R}^{n\times n}, b \in \mathbb{R}^{n\times 1})$

for $i \in \{n-1, \ldots, 1, 0\}$:
    $x_i = (b_i - U_{[[i],i+1:]} x_{[i+1:]})/U_{[i,i]}$
return x

og en tilsvarende algoritme giver forward substitution. Back substitution bruger $\sim n^2$ flops, så løsning af $Ax = b$ via LU-dekomponering domineres af selve beregning af L og U, og i alt bruge Gausseliminering uden ombytning $\sim 2n^3/3$ flops.

Dette er dobbelt så hurtigt, som løsning via QR-faktorisering: At løse $Ax = b$, er et særtilfælde af en mindste kvadraters problem. Tabel 17.1 med $m = n$ fortæller at QR-faktorisering via Householder metoden bruger $\sim 2n^3 - (2n^3/3) = 4n^3/3$ flops.

Så LU-metoden er hurtigere, men desværre er den *ikke* stabil. Den kan hellere ikke anvendes hvis vi får $U_{[i,i]} = 0$ i et trin undervejs. F.eks. for

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

fejler LU-dekomponering i det første trin da $A_{[0,0]} = 0$, selvom A er invertibel.

Det sidste problem kan delvis undgås hvis vi også tillader rækkeombytning, så rækkeoperationer af type (I), udover dem af type (III). En implementering af dette er

GAUSSELIMINERING MED OMBYTNING($A \in \mathbb{R}^{n \times n}$)

$U = A, L = I_n, P = I_n$
for $i \in \{0, 1, \ldots, n-2\}$:
    Vælg $j \geqslant i$ så at $|U_{[j,i]}|$ er størst
    Byt række i i P med række j
    Byt række i i $U_{[:,i:]}$ med række j
    Byt række i i $L_{[:,:i]}$ med række j
    $L_{[i+1:,[i]]} = U_{[i+1:,[i]]}/U_{[i,i]}$
    $U_{[i+1:,i:]} = U_{[i+1:,i:]} - L_{[i+1:,[i]]}U_{[[i],i:]}$
return L, U, P

som giver matricer L, U og P således at $PA = LU$. Matricen P har netop én ettal i hver række og i hver søjle, og indeholder oplysning om hvordan man bytter rækkerne i A.

Det kan vises at Gausseliminering med ombytning beregner løsninger indenfor en fejl der er

$$O(\max_{i,j} |U_{[i,j]}| \epsilon_{\text{machine}} / \max_{i,j} |A_{[i,j]}|).$$

Problemet er at denne fejl kan være ret stor.

Betragt for eksempel den følgende matrix

```
import numpy as np

def eksempel_mat(n):
    a = np.eye(n, dtype=float)
    a -= np.tri(n, k=-1)
    a[:, -1] = 1
```

```
    return a

a = eksempel_mat(10)
print(a)
```

```
[[ 1.  0.  0.  0.  0.  0.  0.  0.  0.  1.]
 [-1.  1.  0.  0.  0.  0.  0.  0.  0.  1.]
 [-1. -1.  1.  0.  0.  0.  0.  0.  0.  1.]
 [-1. -1. -1.  1.  0.  0.  0.  0.  0.  1.]
 [-1. -1. -1. -1.  1.  0.  0.  0.  0.  1.]
 [-1. -1. -1. -1. -1.  1.  0.  0.  0.  1.]
 [-1. -1. -1. -1. -1. -1.  1.  0.  0.  1.]
 [-1. -1. -1. -1. -1. -1. -1.  1.  0.  1.]
 [-1. -1. -1. -1. -1. -1. -1. -1.  1.  1.]
 [-1. -1. -1. -1. -1. -1. -1. -1. -1.  1.]]
```

Her er indgangene på diagonalen størst i deres søjle, så der sker ikke noget ombytning i Gausselimineringen.

```
def gauss_uden_ombytning(a):
    n, _ = a.shape
    u = np.copy(a)
    l = np.eye(n)
    for i in range(n-1):
        l[i+1:, [i]] = u[i+1:, [i]] / u[i,i]
        u[i+1:, i:] -= l[i+1:, [i]] @ u[[i], i:]
    return l, u

l, u = gauss_uden_ombytning(a)

print('L =')
print(l)
print('U =')
print(u)
```

```
L =
[[ 1.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

```
 [-1.  1.  0.  0.  0.  0.  0.  0.  0.  0.]
 [-1. -1.  1.  0.  0.  0.  0.  0.  0.  0.]
 [-1. -1. -1.  1.  0.  0.  0.  0.  0.  0.]
 [-1. -1. -1. -1.  1.  0.  0.  0.  0.  0.]
 [-1. -1. -1. -1. -1.  1.  0.  0.  0.  0.]
 [-1. -1. -1. -1. -1. -1.  1.  0.  0.  0.]
 [-1. -1. -1. -1. -1. -1. -1.  1.  0.  0.]
 [-1. -1. -1. -1. -1. -1. -1. -1.  1.  0.]
 [-1. -1. -1. -1. -1. -1. -1. -1. -1.  1.]]
U =
[[  1.   0.   0.   0.   0.   0.   0.   0.   0.   1.]
 [  0.   1.   0.   0.   0.   0.   0.   0.   0.   2.]
 [  0.   0.   1.   0.   0.   0.   0.   0.   0.   4.]
 [  0.   0.   0.   1.   0.   0.   0.   0.   0.   8.]
 [  0.   0.   0.   0.   1.   0.   0.   0.   0.  16.]
 [  0.   0.   0.   0.   0.   1.   0.   0.   0.  32.]
 [  0.   0.   0.   0.   0.   0.   1.   0.   0.  64.]
 [  0.   0.   0.   0.   0.   0.   0.   1.   0. 128.]
 [  0.   0.   0.   0.   0.   0.   0.   0.   1. 256.]
 [  0.   0.   0.   0.   0.   0.   0.   0.   0. 512.]]
```

Vi ser at `u` har én indgang der er `2.**(10 - 1) = 512.`, så fejlen har også denne størrelsesorden

```
print(np.max(np.abs(u)) / np.max(np.abs(a)))
```

```
512.0
```

ganget med machine epsilon

```
machine_epsilon = np.finfo(float).eps
print(np.max(np.abs(u)) * machine_epsilon
      / np.max(np.abs(a)))
```

```
1.1368683772161603e-13
```

For generel n har den tilsvarende matrix a fejlfaktoren kontrolleret af 2^{n-1}. Så for stort n vil fejlen være 2^{n-1} ganget med machine epsilon. F.eks. for $n = 100$ har vi

```
print(2**(99) * machine_epsilon)
```

```
140737488355328.0
```

som er meget stort.

Det kan vises at 2^{n-1} er den størst mulige fejlfaktor for Gausseliminering på en $(n \times n)$-matrix.

Lad og kikke på løsning til $Ax = b$ for denne matrix a og et tilfældigt valgt b.

```
rng = np.random.default_rng()

b = rng.normal(0.0, 5.0, (a.shape[0], 1))
print(b)
```

```
[[-1.50081042]
 [ 1.13821699]
 [-1.96702259]
 [-2.6122975 ]
 [ 7.43389828]
 [ 5.71860251]
 [ 5.57034344]
 [ 4.22238866]
 [-1.7594919 ]
 [-7.84860793]]
```

Først bruger vi LU-metoden

```
def back_subs(u, b):
    _, n = u.shape
    x = np.empty((n, 1))
    for i in reversed(range(n)):
        x[i] = (b[i] - u[[i], i+1:] @ x[i+1:]) / u[i, i]
    return x
```

```
def forward_subs(l, b):
    _, n = l.shape
    y = np.empty((n, 1))
    for j in range(n):
        y[j] = (b[j] - l[[j], :j] @ y[:j] ) / l[j, j]
    return y

x_lu = back_subs(u, forward_subs(l, b))
print(x_lu)
```

```
[[-0.98872172]
 [ 0.66158397]
 [-1.78207164]
 [-4.20941819]
 [ 1.62735941]
 [ 1.53942305]
 [ 2.93058704]
 [ 4.51321929]
 [ 3.04455801]
 [-0.5120887 ]]
```

Vi ser hvor tæt den er på at være en løsning ved at kikke på forskellen med Ax og b:

```
print(a @ x_lu - b)
```

```
[[ 0.00000000e+00]
 [ 0.00000000e+00]
 [-2.22044605e-16]
 [ 4.44089210e-16]
 [-8.88178420e-16]
 [ 0.00000000e+00]
 [ 8.88178420e-16]
 [ 7.10542736e-15]
 [ 1.55431223e-14]
 [-2.13162821e-14]]
```

Dette er ikke så dårligt, og er lidt bedre end, men i tråd med, vores forventning ovenfor, som ville give en absolut fejl på

```
print(np.max(np.abs(u)) / np.max(np.abs(a)), np.max(np.abs(b)))
print(np.max(np.abs(b))
      * np.max(np.abs(u)) * machine_epsilon / np.max(np.abs(a)))
```

```
512.0 7.84860793246199
8.922834163583945e-13
```

Hvis vi bruger QR-metoden, har vi ikke det samme problem med matricer med store indgange

```
q, r = householder_qr(a)
print(np.max(np.abs(r)) / np.max(np.abs(a)))
```

```
3.16227766016838
```

som giver en relativfejl på

```
print(np.max(np.abs(r)) * machine_epsilon / np.max(np.abs(a)))
```

```
7.021666937153403e-16
```

Det bekræftes af at løsningen

```
x_qr = back_subs(r, q.T @ b)
```

er lidt mere præcist.

```
print(a @ x_qr - b)
```

```
[[ 8.88178420e-16]
 [-4.44089210e-16]
 [-1.55431223e-15]
 [-8.88178420e-16]
 [-1.77635684e-15]
 [ 0.00000000e+00]
 [ 0.00000000e+00]
 [ 1.77635684e-15]
 [-8.88178420e-16]
 [-1.77635684e-15]]
```

For større n er der dog en meget tydelig forskel

```
a = eksempel_mat(200)

b = rng.normal(0.0, 5.0, (a.shape[0], 1))
print('Største indgang i b:', np.max(np.abs(b)))

l, u = gauss_uden_ombytning(a)
x_lu = back_subs(u, forward_subs(l, b))
print('Afvigelse Ax fra b' )
print('LU metoden:', np.max(np.abs(a @ x_lu - b)))

q, r = householder_qr(a)
x_qr = back_subs(r, q.T @ b)
print('QR metoden:', np.max(np.abs(a @ x_qr - b)))
```

```
Største indgang i b: 13.52850138565218
Afvigelse Ax fra b
LU metoden: 1073758213.1638513
QR metoden: 1.176836406102666e-13
```

Her ser vi at fejlen i LU-metoden er af samme størrelsesorden, som selve indgangene i b, hvor QR-metoden har kun en fejl på størrelse 10^{-13}.

Så er det meget fristende at afskrive LU-metoden. Men det mystiske er at disse stor fejl viser sig at være atypisk. Lad os først implementere Gausseliminering med ombytning, og bekræfter at det virker i et tilfældig eksempel.

```
def gauss(a):
    n, _ = a.shape
    u = np.copy(a)
    l = np.eye(n)
    p = np.eye(n)
    for i in range(n-1):
        j = np.argmax(np.abs(u[i:, i])) + i
        p[[i,j], :] = p[[j,i], :]
        u[[i,j], i:] = u[[j,i], i:]
        l[[i,j], :i] = l[[j,i], :i]
        l[i+1:, [i]] = u[i+1:, [i]] / u[i,i]
        u[i+1:, i:] -= l[i+1:, [i]] @ u[[i], i:]
    return l, u, p

a = rng.normal(0.0, 5.0, (10, 10))
l, u, p = gauss(a)
print(np.max(np.abs(p @ a - l @ u)))
```

```
3.552713678800501e-15
```

Vi definerer tilvækstraten for en matrix $A = LU$ til at være faktoren

$$\max_{i,j} |U_{[i,j]}| / \max_{i,j} |A_{[i,j]}|,$$

som styre den relative fejl.

```
def tilvækst(a):
    l, u, p = gauss(a)
    return np.max(np.abs(u)) / np.max(np.abs(a))
```

Nu kører vi et eksperiment hvor vi tager tilfældige matricer og beregner denne tilvækstrate. Vi samler på den største tilvækstrate, som opstår undervejs.

```
n = 200
max = 0
for i in range(1000):
```

```
    t = tilvækst(rng.normal(0.0, 5.0, (n, n)))
    if t > max:
        max = t

print(t)
```

```
6.98397086810283
```

Vi ser at dette er væsentlig mindre end raten `2^199` for det værste tilfælde, nemlig matricen `eksempel_mat(200)`.

Desværre findes der ikke en god forklaring for dette, endnu, men det medfører at i praksis kan LU-dekomponering med ombytning alligevel være brugbar.

Appendiks A

Det græske alfabet

α	A	alpha	ν	N	nu
β	B	beta	ξ	Ξ	xi
γ	Γ	gamma	o	O	omicron
δ	Δ	delta	π	Π	pi
ε	E	epsilon	ρ	R	rho
ζ	Z	zeta	σ	Σ	sigma
η	H	eta	τ	T	tau
θ	Θ	theta	υ	Y	ypsilon
ι	I	jota	φ	Φ	phi
κ	K	kappa	χ	X	chi
λ	Λ	lambda	ψ	Ψ	psi
μ	M	mu	ω	Ω	omega

Appendiks B

Python

Programmeringssproget python og dens dokumentation kan findes ved

https://www.python.org

Vær opmærksom på at der er nogle væsentlige forskel mellem python version `2.*` og `3.*`. Vi bruger python 3, mere præcist

```
>>> import sys
>>> print(sys.version)
3.12.3 (main, Apr 12 2024, 17:29:56) [Clang 13.1.6
↪  (clang-1316.0.21.2.5)]
```

Efterhånden har de fleste computer en version af python 3 installeret. Men det kan være en god ide at få fat i nyere version med de ekstra pakker der kræves for dette kursus. Dette gøres nemmest ved hente en standard distribution, som f.eks. anaconda fra

https://www.anaconda.com

som har installationspakker for Windows, Mac and Linux. En standard alternativ er at bruge Visual Studio Code fra

https://code.visualstudio.com/

Udover python selv bruger vi vi to bestemte pakker. Den første er NumPy, som findes ved

https://numpy.org

som er designet til behandling af videnskabelig data i vektor og matrix form. Den anden pakker er Matplotlib

https://matplotlib.org

som tillader os at lave forskellige type grafiske fremvisning af vores data og resultater. Versionerne brugt i disse noter er

```
>>> import numpy
>>> print(numpy.__version__)
1.26.4
>>> import matplotlib
>>> print(matplotlib.__version__)
3.8.4
```

Installationsvejledning findes ved de overnævnte hjemmeside. Begge pakker er inkluderet i `anaconda` distribution, men skal muligvis opdateres til nyere version. Se hjemmesiden for `anaconda` for hvordan det kan gøres.

I kurset holder jeg mig til kun at bruge disse to ekstra pakker for at give en konsistent og overskuelig tilgang til materialet i notesættet.

For at skrive python filer er det nyttigt at have `jupyter notebook` eller `jupyter lab`, som kan hentes fra

https://jupyter.org

I begge opsætninger kan man oprette notebooks, som kombinerer kode og almindelig tekst. Bemærk at disse notebooks arbejdes med i en browser, men kræver ikke at man har en aktiv forbindelse til internettet. Igen er `jupyter` inkluderet som standard i distributionen `anaconda`.

Bibliografi

Golub, G. H. og C. F. Van Loan (2013). *Matrix computations*. 4. udg. Johns Hopkins Studies in the Mathematical Sciences. Johns Hopkins University Press, Baltimore, MD. ISBN: 978-1-4214-0794-4.

IEEE Computer Society (2019). *IEEE standard for floating-point arithmetic*. IEEE Std 754-2019, s. 1–84. ISBN: 978-1-5044-5924-2. DOI: 10.1109/IEEESTD.2019.8766229.

Leon, S. J. (2015). *Linear algebra with applications*. 9. udg. global edition. Harlow: Prentice Hall. ISBN: 978-1-292-07059-9.

Poore, G. M. (jul. 2015). »PythonTeX: reproducible documents with LaTeX, Python, and more«. I: *Comput. Sci. Discov.* 8.1, 014010. DOI: 10.1088/1749-4699/8/1/014010.

Strassen, V. (1969). »Gaussian elimination is not optimal«. I: *Numer. Math.* 13, s. 354–356. ISSN: 0029-599X,0945-3245. DOI: 10.1007/BF02165411.

The NumPy community (16. sep. 2023). *NumPy documentation*. Vers. release 1.26. URL: https://numpy.org/doc/1.26/.

Trefethen, L. N. og D. Bau III (1997). *Numerical linear algebra*. Society for Industrial og Applied Mathematics (SIAM), Philadelphia, PA. ISBN: 0-89871-361-7. DOI: 10.1137/1.9780898719574.

Williams, V. V., Y. Xu, Z. Xu og R. Zhou (16. jul. 2023). *New bounds for matrix multiplication: from alpha to omega*. arXiv: 2307.07970 [cs.DS].

Python indeks

- (differens)
    array, 35
    float, 2, 3
(kommentar), 10
() (tupel), 33
* (produkt)
    float, 2, 3
    skalar-array, 22, 33
* (unpack), 62
** (potens)
    float, 3
** (unpack dictionary), 288
*=, 68
+ (sum)
    array, 34
    float, 2, 3
+=, 68
/ (kvotient)
    float, 2, 3
// (floor divide), 6
/=, 199
: (indeksering), 29
-=, 199
= (sæt et variabel), 10
@ (matrixprodukt), 42
[] (indeksering), 28, 29, 68
_, 199

A
all, 95, 356
allclose, 95
annotate, 288
arccos, 88
arctan2, 19
argsort, 328
argwhere, 92, 370
array, 22, 28
array_str, 326
arrowstyle, 288
axhline, 277

B
back_subs, 213, 392
bidiagonal, 350
bidiagonal_data, 350
bidiagonal_qbpt, 350
break, 327

C
Chebyshev, 188
clim, 31
cmap, 30
color
    plot, 23
colorbar, 31
complex, 158
compute_uv, 224, 362, 382

conj, 161
convert
    polynomial, 189
copy, 199

D
def, 117
default_rng, 52
degrees, 19, 88
delete, 275
diag, 128
dtype, 29, 89
    float, 30

E
e
    i float, 6
eksempel_mat, 389
empty, 89
empty_like, 91
enumerate, 135
exp, 7, 103
eye, 49, 230

F
f'{var:e}' (videnskabelig notation), 222
f'{var = }', 210
False, 131
figsize, 24
finfo (float information), 6
flip, 328
float, 6
    dtype, 30
færdig, 367
færdig_eller, 370
for, 59, 62
forbedret_gram_schmidt, 199, 211
forskudt_qr_trin, 367
forward_subs, 392
from, 188
full_matrices, 131

G
gauss, 395
gauss_uden_ombytning, 390
grid, 277

H
heat_map, 364
hessenberg, 344
hessenberg_data, 343
hessenberg_qh, 344
house, 227, 341
house_plus, 347
house_transformation, 117
householder_lsq, 233
householder_qr, 230
householder_qr_data, 228
hstack, 68

I
identitetsmatrix, *se* eye
if...:...else:, 327
if...else, 116
imag
    complex, 158
import, 6
imread, 135
imshow, 135
indeksering, 28, 29
indexksering, 68
inv
    linalg.inv, 80
inv_potens_skridt, 334

J
j, 158

K
keepdims, 376
klassisk_gram_schmidt, 198

L
label
    plot, 102
legend, 102
Legendre, 188
linalg
    linalg.inv, 80
    linalg.norm, 87, 169
    linalg.qr, 192
    linalg.solve, 210
    linalg.svd, 127, 131, 137
linewidth (printoption), 134
linspace, 51
    retstep, 177

M
marker
    plot, 23
markersize, 131
markevery, 288
matplotlib, 22
matshow, 30, 50, 135
max, 92
mean, 375

N
nbytes
    ndarray, 139
ndarray, 28
ndim, 28
ndindex, 135
newaxis, 38
nextafter, 7
norm
    linalg.norm, 87, 169
normal, *se* rng
np, *se* numpy
numpy, 6

O
ones, 80, 101
ones_like, 180

P
pdf, 23, 24
pi, 19
plot, 22
polynomial
    Chebyshev, 188
    convert, 189
    Legendre, 188
    Polynomial, 188
potens_skridt, 318
praktisk_qr_metode, 370
precision, 326
print, 52, 60
printoption
    linewidth, 134
    set_printoptions, 134
proj_på, 185

Q
qr
    householder_qr, 230
    linalg.qr, 192

R
random, 52
    default_rng, *se* default_rng
range, 60
flat, 135
rayleigh_skridt, 337
real
    complex, 158

reversed, 213, 230
rng, *se også* default_rng
    normal, 152
    random, 52
    standard_normal, 52
round, 134

S
savefig, 23
seismic, 364
semi_praktisk_qr_metode, 367
set_aspect, 22
set_axis_off, 135
set_matplotlib_formats, 24
set_printoptions, 134
set_xlabel, 137
set_xlim, 24
set_ylabel, 137
set_ylim, 24
seterr, 9
shape
    ndarray, 28
show
    billede, 135
    matrix, 30
    plot, 22
shrink, 31
sing_skridt_del, 331
solve
    linalg.solve, 210
sqrt, 10
standard_normal, *se* rng
subplots, 22, 135
sum, 177
svd
    linalg.svd, 127, 131, 137
svdapprox, 134, 138
svg, 24

T
T (transponering), 36
text
    subplot, 132
tight_layout, 135
tilvækst, 396
trapz, 178
tri, 389
tridiagonal_data, 347
tridiagonal_qt, 347
triu, 80
triu_indices_from, 370
True, 327

V
vander, 210
vdot, 117, 169

W
while, 327
wilkinson_forskydning, 370

Z
zeros, 33

Indeks

1-norm, 147
2-norm, 87
    kompleks, 168
    matrix, 147
∞-norm, 147
$\perp$, *se* vinkelret
$\langle \cdot , \cdot \rangle$, *se* indre produkt
$\| \cdot \|_2$, *se* 2-norm

A
afbildning
    lineær, 237
afstand, 87, 94
akse, 28
algebraens fundamentalsætning, 299
andengradsligning, 10, 12

B
back substitution, 67, 388
basis, 247
    for nulrum, 255
    for søjlerum, 255
    ortonormal, 111
    standard for $\mathbb{R}^n$, 248
bidiagonal matrix, 349
billedmængde, 243
BLAS, 59
bundne variabler, 72

C
$\mathbb{C}$, 157
cache, 59
Cauchy-Schwarz ulighed, 92, 171
Chebychev polynomier, 188
cosine similarity, 92
cosinus, 88, 120

D
determinant, 78, 276
diagonaliserbar, 272
differens
    float, 3
differentialligninger, 283
    førsteordenssystem, 284
dimension, 255

E
e_0, *se* standardvektorer
echelonform, 70
egenvektor, 268
egenværdi, 268
    inverspotensmetode, 333
    potensmetode, 317
    Rayleighs kvotient, 329
egenværdig
    Rayleighkvotientmetoden, 336
eksponent, 6
eksponentialfunktion, 7, 283

kompleks, 290
elektrisk
induktionsspole, 239
kondensator, 239
kredsløb, 46, 239, 291
modstand, 238
elementær
matrix, 82
elementære
operationer på ligninger, 66
rækkeoperationer, 66
enhedsvektor, 14, 87
epsilon
machine, 7
Eulers identitet, 291

F

F1, *se* float, aksiomer
F2, *se* float, aksiomer
farvelægning
af en matrix, 30
farveskala, 31
grænser, 31
fejl, 4
relativ, 4
figur
pdf, 24
størrelse, 24
svg, 24
figurer, 22
float, 6
aksiomer, 9
flop, 57
flydende-komma repræsentation, 5
førsteordenssystem
af differentialligninger, 284
fordeling
standard normal-, 52
uniform-, 52
for-løkke, 57, 59
Fourier cosinus række, 174
Fourier række, 174
frie variabler, 72
Frobeniusnorm, 170

G

Gausseliminering
med ombytning, 389
uden ombytning, 388
Grammatrix, 110
Gram-Schmidt
forbedret, 196, 198, 199
klassisk, 183, 191, 198

H

heatmap plot, 30
hermitisk
matrix, 309
Hessenbergform, 339
Hooks lov, 296
Householder
matrix, 112
vektor, 112

I

identitetsmatrix, 17, 49, 77
imaginær del, 157
indeksering, 28, 29
matrix, 27
negative, 29
indgang, 27
indre produkt, 15, 85, 167
L^2-, 170, 171
over $\mathbb{C}$, 168
vægtet, 169
L^2-, 171
induceret norm, 168

induktionsspole, 239
integration
numerisk, 176
invers matrix, 77, 78, 81
2×2, 78
af produkt, 79
via rækkeoperationer, 83
inverspotensmetoden, 333
invertibel, 77, 81

K
karakteristisk polynomium, 277
$n = 2$, 269
kerne, 243
Kirchhoffs love, 46
kompleks tal, 157
eksponentialfunktion, 290
imaginær del, 157
konjugerede, 161
numerisk værdi, 161
reel del, 157
kondensator
elektrisk, 239
konditionstal, 144, 145
absolut, 143
for en invertibel matrix, 149, 151
for en matrix, 153
mindste kvadraters metode, 220
relativ, 144
konjugerede
kompleks tal, 161
koordinatskiftsmatrix, 252
koordinatvektor, 250
kovarians, 374
kovariansmatrix, 374
kredsløb
elektrisk, 46
kvadratisk matrix, 77
kvotient
float, 3

L
L^2-indre produkt, 170, 171
Legendre polynomier, 188
ligningssystem
lineært, 45, 78, 81
lineær
afbildning, 237
standard matrixrepræsentation, 241
kombination, 47, 163
ligningssystem, 45, 78, 81
regression, 211
transformation, 237
lineært
uafhængig, 186
linje
ret, 41
længde, 87
LU-dekomponering, 386

M
machine epsilon, 7
maksimumsnorm, 147
mantisse, 6
matplotlib, 22
matrix, 27
$(m \times n)$-, 27
af en lineær transformation, 257, 261
bidiagonal, 349
diagonaliserbar, 272
difference, 35
elementær, 82

Gram, 110
hermitisk, 309
Hessenbergform, 339
Householder, 112
identitets-, 17, 49, 77
invers, 77, 78, 81
via rækkeoperationer, 83
koordinatskift, 252
kvadratisk, 77
lighed, 27
nul-, 33
øvre triangulær, 191
ortogonal, 107
produkt, *se* matrixprodukt
projektion, 95, 100
pseudoinvers, 208
rang, 208
repræsentation
generel, 257
standard, 241
rotation, 107, 121
singulær, 81
søjlerum, 163
spejling, 107
stokastisk, 325
sum, 33, 35
symmetrisk, 95
tridiagonal, 340, 364
udvidet, 66
under, 275
unitær, 301
Vandermonde, 210
matrixprodukt, 42, 58
Strassen, 59
matrixtransformation, 16
matrix-vektorprodukt, 47
middelværdi, 373
mindste kvadraters metode, 205, 206
konditionstal, 220
via QR, 207
via SVD, 209
modstand
elektrisk, 238
multiplikation
med en skalar, 33, 35, 37, 155

N

$N(A)$, 243
Newtons anden lov, 296
nøjagtighed
af float, 7
norm, 14, 146, 167
∞-, 147
1-, 147
2-, 87, 168
2- for matrix, 147
Frobenius-, 170
induceret af indre produkt, 167
maksimums-, 147
operator, 147
p-, 147
normalfordeling
standard, 52
normalligninger, 217
nulelement, 155, 156
nulfunktion, 157
nullitet, 259
nulmatrix, 33
nulpolynomium, 156
nulrum, 243
basis for, 255
numerisk værdi, 161

O

Ohms lov, 46

øvre triangulær
matrix, 191
one hot vector, 97
operatornorm, 147
ortogonal
iteration, 359
matrix, 107
samling vektorer, 96
vektorer, 85, 167
ortogonale polynomier
Chebychev, 188
Legendre, 188
ortonormal
basis, 111
samling vektorer, 96
over fitting, 214
overflow, 7

P
page rank, 326
parallel, 88, 89
Parsevals identitet, 98
Perron-Frobenius sætning, 326
π, 19
pil, 37
pil (linjedeling $\hookrightarrow$), 25
pil (vektor), 13
pivotelement, 70
plan, 41
plangeometri, 13
plot
heatmap, 30
p-norm, 147
polynomium, 40
nul-, 156
potens
float, 3
potensmetoden, 317
for singulærværdi, 330
prikprodukt, 15
principalkomponent analyse, 373
produkt
float, 3
indre, 85, 167
matrix, 42, 58
matrix-vektor, 47
række-søjle, 39, 57
skalar-vektor, 57
ydre, 50
projektion, 20, 100
matrix, 95, 100
på en linje, 93
pseudoinvers, 208
Pythagoras sætning, 87, 171

Q
QR-dekomponering, 191
fuld, 191
tynd, 191, 199, 206
QR-metoden, 355, 366

R
$\mathbb{R}^{m\times n}$, 27
RAM, 59
randian, 19
rang, 208, 259
rang-nullitetsformlen, 259
Rayleighkvotientmetoden, 336
Rayleighs kvotient, 329
reel del, 157
regression
lineær, 211
relativ fejl, 4
restvektoren, 205
ret linje, 41
række

af en matrix, 29
rækkeoperationer, 66
i python, 68
række-søjleprodukt, 57
rækkevektor, 16, 38
rotation, 18, 107

S

$S(A)$, 163
Simpsons regel, 179
singulær matrix, 81
singulærvektor, 129
højre-, 129
venstr-, 129
singulærværdi, 125, 129
potensmetode, 330
singulærværdidekomponering, 125, 127, 129
forkortet, 130
reduceret, 208
tynd, 125, 130, 134, 137
ydre produkt, 128, 130
sinus, 120
skalar, 27, 155
skalarmultiplikation, *se* multiplikation, med en skalar
skalar-vektorprodukt, 57
skalering, 17
skævvridning, 20
SMR, *se* standard matrixrepræsentation
søjle
af en matrix, 29
søjlerum, 163, 243
basis for, 255
søjlevektor, 36
Span, 162
speciel ortogonal matrix
2×2, 121
spejling, 19, 107
spektralsætning, 304
hermitisk, 309
SSD, 59
standard basis
for $\mathbb{R}^n$, 248
standard matrixrepræsentation, 241
standardvektorer, 14, 97, 114, 149, 163, 245, 285
stokastisk matrix, 325
stor-»O«-notationen, 322
Strassen
matrixprodukt, 59
sum
float, 3
i vektorrum, 155
matrix, 33, 35
vektor, 37
SVD, *se* singulærværdidekomponering
sviningstid, 297
symmetrisk matrix, 95

T

tegning, *se* figurer
tilfældighedsgenerator, 52
transformation
lineær, 237
transponering
af produkt, 108
matrix, 36
vektor, 15
trapezregel, 178
trekantsulighed, 146
tridiagonal matrix, 340, 364
trigonometriske identiter, 119

tupel, 33

U
udspændt, 162
udvidet matrix, 66
underflow, 7
undermatrix, 275
underrum, 162
uniformfordeling, 52
unitær
    matrix, 301

V
Vandermode
    vektor, 40
Vandermonde
    matrix, 210
varians, 374
vektor, 14, 36, 155
    enheds-, 87
    parallelle, 88, 89
    række, 16, 38
    søjle, 36
    standard, 14, 97, 114, 149, 163, 245, 285
    sum, 37
vektorrum, 155
vinkel, 14, 19, 88, 168
    cos, 88, 120
    sin, 120
vinkelret, 85
vægtet indre produkt, 169

W
Wilkinsons forskydning, 369

Y
ydre produkt, 50